SYSTEM DESIGN INTERVIEW SERIES · VOLUME I

The System Design Interview Playbook

A growing compendium of real interview problems, solved end-to-end: rigorous definitions, back-of-the-envelope mathematics, protocol design, and production-grade Rust implementations.

IN THIS VOLUME

Providence Salumu

First Edition · September 2026 · A living document — chapters are added incrementally

The System Design Interview Playbook

Volume I · First Edition

Written and typeset by **Providence Salumu**
Composed in Typst · Body set in Noto Serif · Display set in Noto Sans · Code set in Noto Sans Mono

REVISION HISTORY

v0.1	Chapter 1 — Real-Time Collaborative Text Editor	2026-09-05
v0.2	Chapter 2 — Maps & Navigation Service (Google Maps)	2026-09-05
v0.3	Chapter 3 — Distributed Rate Limiter	2026-09-05
v0.4	Chapter 4 — Distributed Logging & Metrics Platform	2026-09-05
v0.5	Chapter 5 — Hierarchical Comment System (Reddit)	2026-09-05
v0.6	Chapter 6 — Top-K Leaderboard (Gaming)	2026-09-05
v0.7	Chapter 7 — Recommendation Engine (YouTube / TikTok)	2026-09-05
v0.8	Chapter 8 — Database Indexes: How B-Trees Work	2026-09-05
v0.9	Chapter 9 — Conflict-Free Replication: How CRDTs Work	2026-09-06
v0.10	Chapter 10 — Distributed Job Scheduler	2026-09-06
v0.11	Chapter 11 — ACID Database Transactions	2026-09-06
v0.12	Chapter 12 — Ride-Hailing Marketplace (Uber / Lyft)	2026-09-06

Contents

FRONT MATTER

Preface

This book exists because system design interviews reward a very specific skill that ordinary engineering work rarely exercises in full: *taking an ambiguous, enormous problem and shrinking it, in real time and out loud, into a coherent, defensible architecture.* Knowing how Netflix or Google Docs works is not enough; you must be able to **derive** such a design from first principles while an interviewer watches you think.

> **DEFINITION — System design interview**
>
> An interview format, common for senior engineering roles, in which the candidate is asked to design a large-scale software system (for example, "design YouTube" or "design a URL shortener") within 45–60 minutes. There is no single correct answer. The interviewer evaluates how you clarify ambiguity, structure the problem, justify trade-offs, and reason about scale, failure, and consistency.

Each chapter of this book is a complete, self-contained walkthrough of one such problem. Every chapter follows the same battle-tested structure, which mirrors the natural arc of a strong interview performance:

THE CHAPTER FRAMEWORK

1. **Problem statement** — the prompt exactly as an interviewer would pose it.
2. **Clarifying questions & scope** — shrinking an ambiguous prompt into a concrete target.
3. **Functional requirements** — what the system must *do*.
4. **Non-functional requirements** — how *well* it must do it (scale, latency, availability).
5. **Back-of-the-envelope estimation** — arithmetic that sizes the problem before designing.
6. **The core challenge** — the one or two genuinely hard ideas this problem tests.
7. **API & protocol design** — the contract between clients and servers.
8. **Data model & storage** — what we persist, where, and why.
9. **High-level architecture** — the boxes and the arrows, with every box justified.
10. **Deep dives** — the mechanisms, including complete Rust reference implementations.
11. **Trade-offs & alternatives** — what we gave up, and what we would do differently.
12. **Wrap-up** — likely follow-up questions, common pitfalls, and an interview checklist.

Conventions used in this book

Three conventions deserve explicit mention.

Terms are defined before they are used. System design is full of overloaded vocabulary — "consistency", "snapshot", "version vector" — and interviews are lost when a candidate uses a term they cannot define. Whenever a new concept appears, it is introduced in a **Definition** box before the surrounding text relies on it.

All code is written in Rust. Rust's explicit ownership and type system make concurrency logic — the hardest part of most designs — precise and auditable. The listings are not toys: they are faithful, compilable sketches of the real algorithms, with tests.

Numbers are always justified. Every estimate states its assumptions. A number without assumptions is decoration; a number with assumptions is engineering.

How this book grows

This is a living document. New chapters are appended over time, each tackling one new problem. The framework above is fixed, so you always know where to find the requirements, the math, the architecture, and the code — regardless of the problem.

Providence Salumu — September 2026

CHAPTER 1

Designing a Real-Time Collaborative Text Editor

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"12: Design Google Docs / Real Time Text Editor"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 47 min). The talk walks the problem in mock-interview form — both a high-level design (HLD) and a low-level design (LLD) of the concurrency machinery. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol specifications, and Rust reference implementations.

1.1 The Problem Statement

The interviewer looks up and says:

> *"Design a real-time collaborative text editor — something like Google Docs. Multiple people should be able to edit the same document at the same time and see each other's changes as they happen."*

This is one of the richest prompts in the system design canon. It looks like a CRUD application — until you realize that its heart is a distributed-state synchronization problem with no trivially correct answer. The prompt deliberately leaves almost everything open: scale, feature scope, consistency guarantees, offline behavior. How you close those gaps is most of the grade.

DEFINITION — High-level design (HLD) / low-level design (LLD)

HLD is the architecture of the whole system: the major services, data stores, and network paths, and the justification for each. *LLD* is the internal mechanics of the interesting components: data structures, algorithms, and protocols. A strong interview performance does HLD first, then drills into LLD for the one or two components that carry the real difficulty — in this problem, the concurrency-control core.

1.2 Scope & Clarifying Questions

Never design the prompt you were handed; design the prompt you **negotiated**. The series this chapter draws on runs its interviews as a dialogue, and that is exactly how a real interview should feel. A strong opening exchange looks like this:

Speaker	Dialogue
Candidate	"Are we building plain text editing, or rich text — formatting, embedded images, tables?"

Speaker	Dialogue
Interviewer	"Plain text is fine. Assume a document is an ordered sequence of characters."
Candidate	"How many collaborators can edit one document simultaneously?"
Interviewer	"Up to 50 concurrent editors per document."
Candidate	"And the total scale — how many users and documents are we designing for?"
Interviewer	"10 million daily active users. Documents themselves can be up to a few hundred kilobytes of text."
Candidate	"Do users need to see each other's cursors and selections live — presence?"
Interviewer	"Yes, cursors and names, like Google Docs shows."
Candidate	"Is offline editing in scope, or can we assume clients are connected while editing?"
Interviewer	"Assume connected for the core design; if time remains, sketch how offline would work."
Candidate	"Do we need version history — the ability to view and restore older revisions?"
Interviewer	"Yes, keep a full history of changes."

From this exchange we can freeze the scope. Everything later in the chapter traces back to these decisions, so state them explicitly and get a nod from the interviewer before moving on.

AGREED SCOPE

Plain-text documents up to 500 KB; up to **50 concurrent editors per document**; **10M daily active users**; live cursors and presence; full version history; connected editing for the core design (offline sketched as an extension).

INTERVIEW TIP — Negotiate scope in both directions

Candidates think clarifying questions only **shrink** scope. They also **expand** it deliberately: asking "do we need presence?" signals that you know the feature exists and costs something. Offer it, let the interviewer decide, and note the cost either way.

1.3 Functional Requirements

DEFINITION — Functional requirement (FR)

A statement of what the system must **do** — an observable behavior a user can verify, such as "a user can create a document." Functional requirements define the feature set; they say nothing about how fast, how available, or how consistent the system must be. Those qualities are *non-functional requirements*, defined next.

Our scoped functional requirements:

1. **FR-1 — Document management.** Users can create, rename, open, and delete documents.
2. **FR-2 — Real-time collaborative editing.** Multiple users can edit one document concurrently; every connected editor sees every other editor's changes within a fraction of a second.
3. **FR-3 — Presence.** Each editor sees who else is in the document, with a live cursor (and selection) per collaborator, labeled with their name.

4. **FR-4 — Persistence.** A document is never lost because a client disconnects; reopening it later yields its latest committed state.
5. **FR-5 — Version history.** Users can browse the full history of a document and view or restore any earlier revision.
6. **FR-6 — Sharing & access control.** A document has an owner; the owner grants view or edit access to others via a shareable link or explicit invite.

Out of scope (say so explicitly): rich text, comments and suggestions, end-to-end encryption, and offline-first editing as a core flow.

1.4 Non-Functional Requirements

DEFINITION — Non-functional requirement (NFR)

A statement of how **well** the system must perform its functions: scale, latency, availability, durability, consistency. NFRs are where design decisions actually come from — two systems with identical functional requirements (a chat app and a payment ledger) can have opposite architectures because their NFRs differ.

Three qualities dominate this problem, and they deserve precise vocabulary:

DEFINITION — Latency

The time between a cause and its observable effect. Two latencies matter here: *edit latency* (my keystroke appears in my own document — must feel instant, under 16 ms, so it must be applied locally without a network round trip) and *propagation latency* (my keystroke appears on a collaborator's screen — our target: under 150 ms for users in the same region).

DEFINITION — Availability

The fraction of time the system is able to serve requests. Editing must remain available even when individual servers fail, so no single machine may be indispensable. We will aim for the classic "three nines" (99.9% ≈ 8.8 hours of downtime per year) for the editing path.

DEFINITION — Consistency / convergence

In a system where many actors mutate shared state concurrently, *consistency* describes what guarantees observers get about the state they see. For a collaborative editor the crucial guarantee is **convergence**: once the same set of edits has been delivered to every replica, every replica must hold the **identical document**. If two users could permanently end up looking at different text, the product is broken no matter how fast it is. How convergence is achieved — without locking the document — is the core challenge of this chapter and gets its own section (Section 1.6).

The remaining NFRs, stated as targets:

Quality	Target
Propagation latency	p50 < 150 ms within a region; p95 < 400 ms cross-region
Edit (local echo) latency	< 16 ms — edits apply locally, synchronously, before any server contact
Editors per document	Up to 50 concurrent, no degradation
System scale	10M DAU; 1M simultaneous connections at peak

Quality	Target
Durability	No committed edit is ever lost (history is the product's memory)
Availability	99.9% for editing; reading a document should survive almost anything

KEY INSIGHT — Consistency is the NFR that shapes this design

Most interview problems are dominated by throughput (design Twitter) or storage (design Dropbox). This one is dominated by **correctness under concurrency**. The moment two users type at the same time, you need a mathematical answer to "what is the document now?" — Section 1.6 is where the interview is won or lost.

1.5 Back-of-the-Envelope Estimation

DEFINITION — Back-of-the-envelope estimation

Rapid, approximate arithmetic — using round numbers and stated assumptions — that sizes a system **before** designing it. Its purpose is not precision; it is to discover which constraints bite. A design for 500 messages per second and a design for 500,000 messages per second are different systems, and you cannot know which one you owe the interviewer until you count.

DEFINITION — DAU / QPS

DAU (daily active users) counts distinct users active in a day. *QPS* (queries per second) is the request rate a system handles. Interview estimates usually convert DAU into QPS via an activity model ("each user does X, Y times a day").

Assumptions (state them, write them down, invite correction):

- 10M DAU; each user actively edits on 3 documents per day, 20 minutes per session.
- At peak, 5% of DAU are simultaneously connected: **500k concurrent connections** (we design for 1M to leave headroom).
- An active typist produces 3 keystroke-operations per second in bursts; averaged over a session (reading, thinking, pausing), 0.5 ops/sec per connected user.
- One operation (a character insert/delete plus metadata) ≈ 250 bytes on the wire.
- Average document size: 20 KB of text (500 KB worst case per our scope).

Derived numbers:

Quantity	Estimate	How
Peak edit operations	≈ 250k ops/sec	500k conns × 0.5 ops/s
Edit ingress bandwidth	≈ 60 MB/s	250k ops/s × 250 B
Operations stored per day	≈ 2.2B	25k ops/s avg × 86,400 s
Operation log growth	≈ 550 GB/day	2.2B ops × 250 B
New documents per day	≈ 3M	10M users × 0.3
Doc-open QPS (REST)	≈ 600 avg / 6k peak	10M × 5 opens/day ÷ 86,400, ×10 peak factor

Quantity	Estimate	How
Connections per WebSocket gateway	50k	commodity servers hold tens of thousands of idle-ish connections
Gateways needed	20–40	1M conns ÷ 25–50k each, plus redundancy

KEY INSIGHT — What the math tells us

Two facts jump out. First, the **system-wide** write rate (hundreds of thousands of ops/sec) is large but utterly **sharded by document**: each document sees at most 50 editors producing maybe 150 ops/sec in a frantic burst — trivial for one thread. Second, the connection tier is where the scale lives: a million mostly- idle WebSocket connections is a capacity problem, not a correctness problem. Conclusion: put the hard correctness logic **per document** (where throughput is tiny) and make the connection tier **stateless and horizontal** (where throughput is huge). This observation drives the entire architecture in Section 1.8.

1.6 The Core Challenge: Concurrent Editing

Everything else in this chapter is standard distributed-systems plumbing. This section is the intellectual center of the problem — and of the source talk.

1.6.1 Why naive approaches fail

Suppose two users, Alice and Bob, are editing the document `"GO"` — a two-character document: `G` at index 0, `O` at index 1. They type **at the same time**:

- **Alice** inserts the character `A` at index 0 (she is turning `"GO"` into `"AGO"`).
- **Bob** deletes the character at index 1 (he is turning `"GO"` into `"G"`).

What should the document be when both edits are delivered? Both users' intentions are clear, and they do not conflict in spirit: the answer is `"AG"` — Alice's `A` before the `G`, and the `O` gone. Now examine two naive strategies:

Naive strategy 1 — lock the document. Only one editor at a time; everyone else waits. This is **correct** but useless: it defeats the entire purpose of the product. Real-time collaboration means no global mutual exclusion.

DEFINITION — Mutual exclusion / locking

A concurrency-control technique in which a resource may be modified by only one actor at a time; others block until the lock is released. Locks serialize work, guaranteeing correctness at the cost of parallelism — fatal when the resource is a shared document with 50 simultaneous editors.

Naive strategy 2 — last-writer-wins. Let each edit carry a timestamp; the edit with the latest timestamp prevails, earlier conflicting edits are discarded.

DEFINITION — Last-writer-wins (LWW)

A conflict-resolution rule: when two writes to the same datum race, keep the one with the later timestamp and drop the other. Simple and fast, but it **loses data by design** — one of Alice's or Bob's edits would silently vanish. LWW is acceptable for overwritten fields (a profile picture), never for merged text.

Applying LWW to our example: if Bob's delete "wins", Alice's `A` disappears. If Alice's insert "wins", Bob's delete is lost and the `O` resurrects. Neither user did anything wrong, yet one of their intentions

was destroyed. And naive **position-based** application is worse still: if Bob's `delete index 1` is applied to Alice's already-updated `"AGO"`, it deletes the `G` instead of the `O`, producing `"AO"` — a document **neither** user intended.

1.6.2 What "correct" means

We need guarantees that can be stated precisely and proven. Three of them:

> **DEFINITION — Causality**
>
> Event A *causally precedes* event B if A happened first and could have influenced B — for example, B is an edit made by a user who had already seen A applied. Two events with no causal order either way are **concurrent**. Alice's and Bob's edits above are concurrent: neither saw the other's. Concurrency — not time — is what creates conflicts; timestamps cannot detect it, which is why Section 1.7 introduces version vectors.

> **DEFINITION — Convergence**
>
> A replica set converges if, once the **same set** of edits has been delivered to every replica, every replica holds the **identical** document — regardless of the order in which the edits arrived. Convergence is a property of the algorithm, not of luck: it must hold for every possible interleaving.

> **DEFINITION — Intention preservation**
>
> The effect of an edit, as observed by its author at the moment they made it, must survive concurrency with other edits. Alice meant "`A` before `G`" and Bob meant "`O` is gone"; the converged document `"AG"` honors both. An algorithm can converge and still be bad (converging to the empty string is trivial), so convergence alone is not enough — we want convergence **with** intention preservation.

There are exactly two families of algorithms in wide production use that deliver these properties: **Operational Transformation** and **Conflict-free Replicated Data Types**. A candidate who can define, contrast, and implement one of them owns this interview.

1.6.3 Approach A — Operational Transformation (OT)

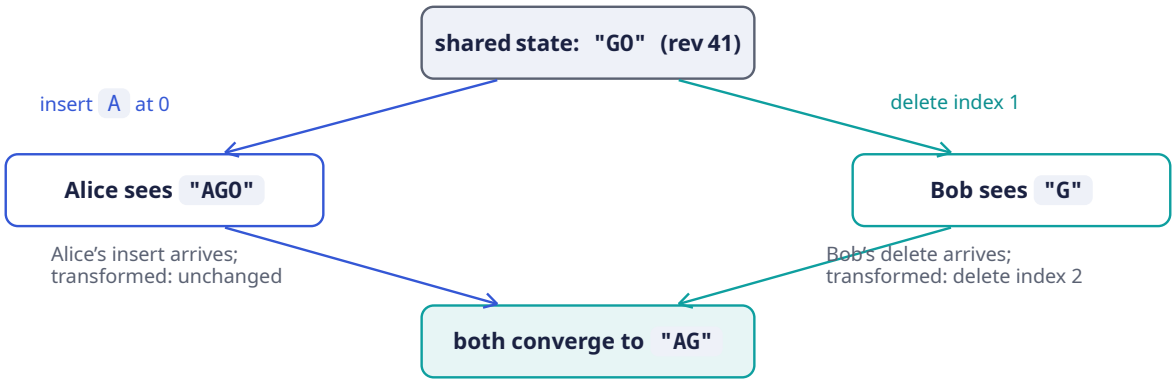
> **DEFINITION — Operational Transformation (OT)**
>
> A concurrency-control technique in which every edit is expressed as an **operation** — for plain text, `insert(position, char)` or `delete(position)` — and, when two operations are concurrent, one is **transformed** against the other: its position is adjusted so that it applies correctly to a document the other operation has already modified. OT powers Google Docs. Its lineage: the Jupiter system (1995), then Google Wave, then Docs.

The transformation rules are a small, complete matrix. To apply operation `a` after concurrent operation `b` has already been applied, transform `a` against `b`:

Pair	Rule for transforming `a` against already-applied `b`
`insert(p_a)` vs `insert(p_b)`	If `p_b < p_a`, shift `a` right by 1. If `p_b == p_a`, break the tie deterministically (e.g., by client ID) so every replica shifts the same way.
`insert(p_a)` vs `delete(p_b)`	If `p_b < p_a`, shift `a` left by 1. Otherwise unchanged.
`delete(p_a)` vs `insert(p_b)`	If `p_b <= p_a`, shift `a` right by 1. Otherwise unchanged.

Pair	Rule for transforming `a` against already-applied `b`
`delete(p_a)` vs `delete(p_b)`	If `p_b < p_a`, shift `a` left by 1. If `p_b == p_a`, both deletes target the same character: `a` becomes a no-op.

The whole of OT is this matrix plus a discipline: **every replica applies the same operations in the same total order, transforming as needed.** Our running example, drawn as the classic OT convergence diamond — both paths from the concurrent state must land on the same document:



Walking the diamond: on Alice's side, Bob's `delete(1)` is transformed against her already-applied `insert(0, A)` — since the insert sits at or before the delete position, the delete shifts right to `delete(2)`, which removes the `O` from `"AGO"` and yields `"AG"`. On Bob's side, Alice's `insert(0, A)` is transformed against his already-applied `delete(1)` — the insert position is before the delete, so it applies unchanged to `"G"`, yielding `"AG"`. Same operations, same converged document, both intentions preserved.

> **NOTE — Where the total order comes from**
>
> OT requires every replica to transform against the **same** concurrent history. The standard way to arrange that is a **single sequencer** per document: the server assigns every operation the next integer **revision** (42, 43, 44, …), and that assignment *is* the total order. Clients send operations tagged with the revision they were based on; the server transforms them across any operations committed in between. We make this precise in the deep dives (Sections 1.9 and 1.10).

1.6.4 Approach B — Conflict-free Replicated Data Types (CRDTs)

> **DEFINITION — Eventual consistency**
>
> A consistency model in which replicas that have stopped receiving new writes will **eventually** reach the same state. It promises convergence but says nothing about **when**, and nothing about what intermediate states look like.

> **DEFINITION — Conflict-free Replicated Data Type (CRDT)**
>
> A data structure whose concurrent updates are merged by rules that are commutative, associative, and idempotent — so replicas that apply the same updates in **any** order, even with duplicates, provably converge. This upgraded guarantee is called **strong eventual consistency**: convergence as soon as updates are delivered, with no central ordering and no transformation step.

For text, the canonical CRDT trick is to stop addressing characters by **integer positions** (which shift under concurrency — the very problem OT transforms away) and instead give every character a

unique, immutable, orderable identifier. In the RGA family of algorithms, an insert is recorded as "insert this new character, with a fresh globally-unique ID, immediately after character ID X." Concurrent inserts after the same character interleave deterministically by ID order; deletes merely mark a character as a tombstone (present in structure, invisible in text). Because identifiers never shift, no transformation is ever needed — merging is just set union.

1.6.5 Choosing between them

Criterion	OT	CRDT
Message size	Tiny: position + char	Larger: globally-unique IDs per character
Memory / metadata	None beyond the text	ID + tombstone per character; tombstone growth needs garbage collection
Needs central sequencer	Yes (classic form)	No — tolerates any delivery order
Algorithmic subtlety	Transformation matrix + history; easy to get subtly wrong	Merge rules; conceptually cleaner, harder to keep compact
Offline / peer-to-peer	Awkward (needs the ordering authority)	Natural fit
Used in production by	Google Docs, older Etherpad	Figma (variants), many local-first tools

Our decision: classic OT behind a per-document sequencer — the architecture the source talk describes, and the one Google Docs runs on. The sequencer is not a bottleneck (Section 1.5 showed a document peaks at 150 ops/sec) and it collapses the hard problem into "transform against a linear log," which we can implement exactly. We keep version vectors (next section) for reconnect and catch-up.

> **INTERVIEW TIP — Name the trade-off out loud**
>
> Saying "I'll use OT because Google Docs does" earns no points. Saying "I'll use OT with a per-document sequencer, accepting a single point of ordering per document — which I mitigate with failover — in exchange for small messages and a linear history that makes version history trivial" is a senior answer. Every choice in this chapter is presented as benefit-purchased-at-cost.

1.7 Version Vectors: Tracking What Each Replica Has Seen

The OT diamond handles the moment of concurrency. But a real system must also answer a bookkeeping question constantly: **which operations has this client already received?** On reconnect, on history fetch, on offline sync, the server and client must compare "what you have seen" against "what exists." Timestamps cannot express this (clock skew makes "when" unreliable across machines). We need a **logical** notion of time.

> **DEFINITION — Logical clock / Lamport timestamp**
>
> A counter, maintained by each participant, that ticks on every local event and is carried with every message; receivers merge it by taking the maximum and ticking on. Logical clocks order events by causality rather than by wall-clock time, which is exactly what distributed state needs — wall clocks on different machines cannot be trusted to agree.

DEFINITION — Version vector

A map from participant ID to a counter: `{Alice: 3, Bob: 1}` means "I have seen 3 events from Alice and 1 from Bob." Two version vectors compare pointwise:

- V_1 **dominates** V_2 (V_2 happened-before V_1) if every entry of V_1 is ≥ the corresponding entry of V_2, and at least one is strictly greater — V_1 has seen everything V_2 has, and more.
- If neither dominates, the two states are **concurrent** — each has seen something the other has not.

A version vector is a Lamport clock generalized to many writers: it captures exactly which causal history a replica has absorbed.

In our design the server already assigns a single integer revision per document (the sequencer), so the **steady-state** sync marker is just `rev = 47`. Version vectors earn their place at the edges of the design: when a client reconnects after a network blip during which a **snapshot** was taken, when replicas in different regions compare state, and in the offline extension. The comparison logic is the same everywhere: if the server's knowledge dominates the client's, send exactly the difference.

COMMON PITFALL — The snapshot / version-vector trap

The source talk's wry description — a client "never received those writes on your local copy since your version vector was more up to date than the document snapshot" — points at a real and common bug. Scenario: a snapshot of the document is written to one store, but the version metadata recording **which revisions the snapshot covers** is written to another, and the two writes are not atomic. A reconnecting client compares its version vector against **stale** snapshot metadata, concludes it is up to date, and never receives the missing operations. The cure is atomicity between a snapshot and its version metadata — Section 1.11 shows how, and defines the two-phase commit protocol the talk name- drops for exactly this purpose.

1.8 API & Protocol Design

The system has two distinct planes, and separating them is itself a design decision worth naming:

- The **control plane** — create/rename/share/open documents, fetch history. These are classic request/response operations; plain HTTPS REST is correct and simple.
- The **data plane** — the live editing session: join, send operations, receive operations, presence. This needs a persistent, bidirectional, low-latency channel per client.

DEFINITION — REST

An API style over HTTP in which resources (nouns: `/documents/{id}`) are manipulated with a fixed set of methods (verbs: `GET`, `POST`, `PATCH`, `DELETE`). Stateless by design: each request carries everything the server needs. Ideal for our control plane.

DEFINITION — WebSocket

A protocol (RFC 6455) that upgrades an HTTP connection into a long-lived, full-duplex channel: after an initial handshake, **either** side may send a message at **any** time, with a few bytes of framing overhead. This is what makes sub-150 ms propagation possible — the server can push an edit the instant it is committed, with no client polling and no new connection setup.

Why not the alternatives?

Mechanism	Behavior	Verdict
Short polling	Client re-asks "anything new?" every few seconds	Latency of seconds; wasted requests — rejected
Long polling	Server holds the request open until data exists	Near-real-time but a new HTTP request per message burst — heavy at our scale
Server-Sent Events	Server → client push over HTTP; client sends via separate requests	Workable, but asymmetric; WebSocket is cleaner for a two-way edit stream
WebSocket	One persistent, bidirectional connection	**Chosen for the data plane**

1.8.1 Control-plane endpoints

Method	Path	Purpose
`POST`	`/documents`	Create a document; returns its `doc_id`
`GET`	`/documents/{id}`	Fetch metadata + the latest committed revision number
`PATCH`	`/documents/{id}`	Rename, change settings
`DELETE`	`/documents/{id}`	Delete (owner only)
`POST`	`/documents/{id}/share`	Grant view/edit access to a user or link
`GET`	`/documents/{id}/history?from=&to=`	List revisions for the version-history UI
`GET`	`/documents/{id}/snapshot?rev=`	Fetch the document as of any revision (view/restore)

1.8.2 Data-plane protocol (WebSocket messages)

After the REST `GET`, the client opens a WebSocket to the gateway and speaks a small JSON protocol. Every message has a `type`; the important ones:

Message	Direction	Payload & semantics
`join`	client → server	`{doc_id, auth_token, last_seen_rev}` — enter the session; `last_seen_rev` enables catch-up
`joined`	server → client	`{doc_id, rev, snapshot, collaborators[]}` — current state, possibly after catch-up
`op`	client → server	`{doc_id, base_rev, op, client_seq}` — one edit, based on `base_rev`, idempotent via `client_seq`
`ack`	server → client	`{client_seq, rev}` — "your edit is committed as revision `rev`"
`op` (broadcast)	server → client	`{rev, author, op}` — a committed edit to apply (transform locally if needed)
`presence`	both	`{cursor, selection, name}` — ephemeral, never persisted
`sync`	server → client	`{rev}` / snapshot — sent when the client is too far behind to patch with ops

Example committed-operation broadcast:

```
{
  "type": "op",
  "doc_id": "d_8f31c2",
  "rev": 47,
  "author": "alice@example.com",
  "op": { "kind": "insert", "pos": 0, "ch": "A" }
}
```

Two protocol details deserve emphasis because interviewers probe them. First, `base_rev` is how the server detects concurrency: if the document is at revision 52 and your operation says `base_rev: 49`, the server transforms it across revisions 50–52 before committing (Section 1.10). Second, `client_seq` is an **idempotency key**:

> **DEFINITION — Idempotency / idempotency key**
>
> An operation is *idempotent* if performing it twice has the same effect as performing it once. Networks retry; clients reconnect and resend. A unique client-supplied key per operation (here, `(client_id, client_seq)`) lets the server recognize a duplicate delivery and answer with the original result instead of applying the edit twice — the difference between at-least-once **delivery** and effectively exactly-once **effect**.

1.9 Data Model & Storage

Four persistent entities, each with a clear job:

Entity	Contents	Store
`Document`	`doc_id`, owner, title, ACL, current `rev`	Metadata DB (relational or document store; sharded by `doc_id`)
`Operation`	`doc_id`, `rev`, author, transformed op payload, `client_seq`, timestamp	Operation log — append-only store, ordered by `rev` per document
`Snapshot`	`doc_id`, `base_rev`, full document text	Blob/object store; pointer + `base_rev` in metadata DB
`Session`	who is connected, cursors	Ephemeral presence cache — **not** persisted

The two storage ideas that interviewers reward:

> **DEFINITION — Operation log (event log)**
>
> Storing every change as an immutable, ordered, append-only record instead of overwriting state. Current state is always derivable by replaying the log. For us the log **is** the product's memory: version history (FR-5) is free, auditing is free, and recovery is replay. Append-only logs are also the fastest possible write pattern — sequential appends, no random updates.

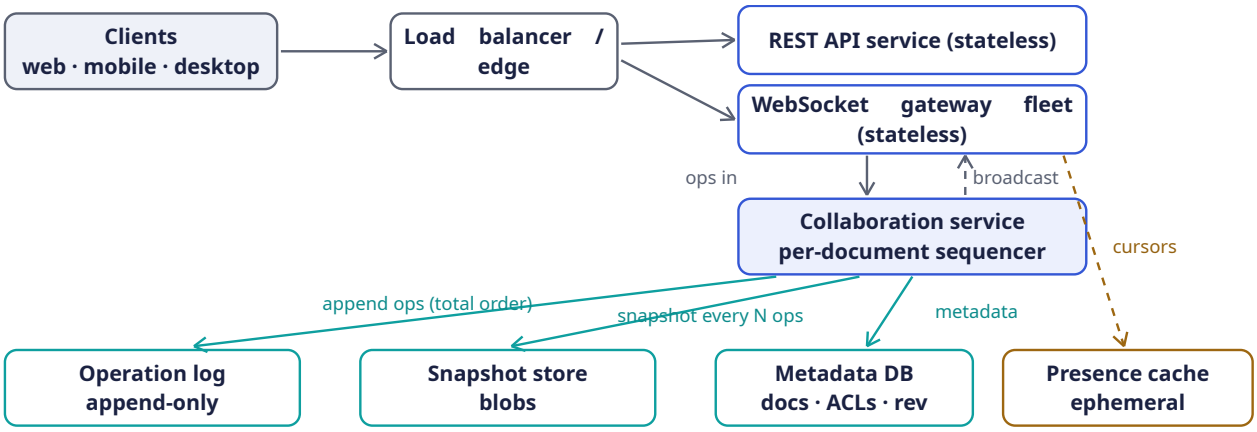
> **DEFINITION — Snapshot**
>
> A materialized copy of the full document as of a specific revision. Replaying 2 billion operations to open a document is absurd; instead store a snapshot every *N* operations (say 500) plus the handful

of operations since, and reconstruct state as `snapshot + tail of the log`. Snapshots are a read optimization over the log, never the source of truth — the log is.

> **NOTE — Why a sharded relational/document store + blob store, not one database**
>
> The metadata (small, hot, transactional) wants indexes and conditional updates; snapshots (large, immutable, cold) want cheap object storage; the operation log (append-only, ordered per key) wants a log-structured store. Choosing one engine per access pattern — and **saying why** — is the answer this section exists to elicit. In an interview, name concrete instincts: PostgreSQL-class for metadata, an S3-class blob store for snapshots, a Kafka/Cassandra-class append store for the log — while stressing the **properties** matter more than the brands.

1.10 High-Level Architecture

The estimation (Section 1.5) gave us the organizing principle: **a thin, stateless, horizontally scaled connection tier in front of a small, stateful, correctness- critical core.** Here is the whole system at a glance:



REST tier reads/writes Metadata DB directly; gateway ↔ collaboration routing is per-document (sticky)

Component responsibilities, and — more importantly — the reason each exists:

Component	Responsibility	Why it is shaped this way
WebSocket gateway	Hold 50k client connections each; terminate TLS; forward messages	Stateless: any client can reconnect to any gateway. Scale by adding boxes — this tier absorbs the 1M-connection problem
Collaboration service	Owns each active document: sequences ops, transforms, commits, broadcasts	Stateful but **sharded by document**: a document's whole session lives on one node, so its sequencer is just an in-memory counter. Throughput per doc is tiny (Section 1.5), so one node hosts thousands of docs
Operation log	Durable, ordered, append-only record per document	Source of truth; powers history (FR-5), replay, recovery

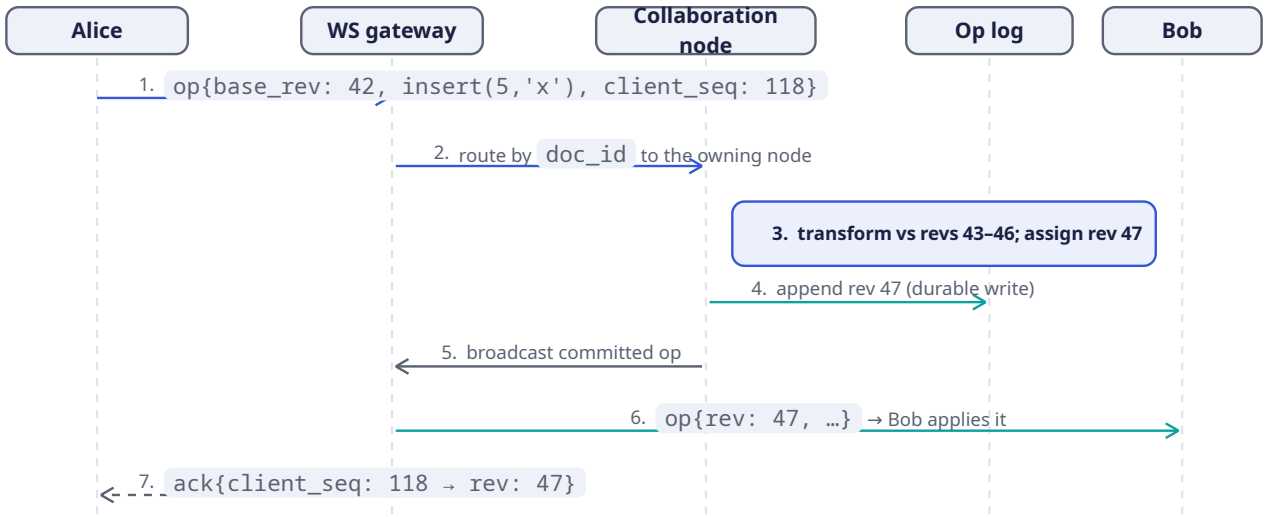
Component	Responsibility	Why it is shaped this way
Snapshot store	Immutable document snapshots every N ops	Makes open/reconnect O(1) instead of O(history)
Metadata DB	Documents, ACLs, current revision counters	Small, hot, transactional reads/writes
Presence cache	Live cursors and collaborator lists	Ephemeral by nature — losing it costs nothing; it rebuilds in seconds
REST API service	Control plane (Section 1.8)	Stateless request/response work; ordinary horizontal scaling

KEY INSIGHT — The one routing rule that makes it all work

All traffic for a given document flows through the single collaboration node that owns it. Gateways look up `doc_id → owner` (a cheap routing-table read) and forward. Because every operation on a document passes through one owner, the owner's revision counter is a **total order** — which is exactly what OT needs (Section 1.6). Ownership can migrate on failure; the invariant is that there is exactly one owner at a time.

1.11 Deep Dive: The Edit Pipeline, Step by Step

Follow one keystroke from Alice's keyboard to Bob's screen. The document `d_8f31c2` is at revision 46; Alice believes it is at revision 42 (she has not yet received four operations from other editors).



In words:

1. **Send.** Alice's keystroke is applied to her local document **immediately** (local echo — this is how edit latency stays under 16 ms) and sent as `op{base_rev: 42, …}` with her next `client_seq`.
2. **Route.** The gateway does no correctness work; it reads `doc_id`, finds the owning collaboration node, forwards.
3. **Sequence & transform.** The owner sees `base_rev: 42` but the document is at 46: Alice's operation is **concurrent** with revisions 43–46. It transforms her operation against each of them, in order (Section 1.13 implements this loop), then stamps it with the next revision, 47.

4. **Commit.** The transformed operation is appended to the operation log. Once the append is durable, the edit is **committed** — it can never be reordered or lost.
5. **Broadcast.** The committed operation goes to the gateways of every connected editor of the document.
6. **Apply.** Bob's client is not behind (its `base_rev` matches), so it applies the operation verbatim. If Bob had a **pending** unacknowledged local edit, he would first transform it against this incoming operation — the client runs the same transform matrix as the server, keeping his local echo intact.
7. **Acknowledge.** Alice learns her operation is committed as revision 47; her pending buffer clears. Her screen never flickered — the local echo and the committed result are identical by construction.

COMMON PITFALL — Order matters: commit before broadcast

If you broadcast before the log append is durable, a node crash between the two steps shows users edits that no longer exist — and on recovery, the "committed" history disagrees with what people saw. Durability first, **then** visibility. This is the write-path equivalent of "don't acknowledge what you can't guarantee."

1.12 Deep Dive: Snapshots, Catch-Up & the Two-Phase Lesson

Two housekeeping jobs remain: making **open/reconnect** fast, and keeping a snapshot and its metadata **atomic**. Both live at the seam between the operation log and the snapshot store.

1.12.1 Catch-up: what to send a rejoining client

When a client sends `join{last_seen_rev}`, the owner compares it to the current revision `R`:

- **Close behind** (`R - last_seen_rev ≤ 500`): send the missing operations from the log. The client transforms any pending local edits across them and is current.
- **Far behind** (offline for hours): replaying thousands of ops is slower than sending state. Send `latest snapshot + ops since the snapshot's base_rev`.
- **Offline edits** (the extension): the client presents its **version vector** (Section 1.7) instead of a scalar revision; the server computes the set difference — ops the client missed — and the client rebases its offline ops on top. Vectors, not timestamps, because they compare **causal** histories.

1.12.2 The atomicity requirement

A snapshot is useless without its metadata (`base_rev`: which revisions it covers), and dangerous with **stale** metadata — that is exactly the failure the source talk jokes about: a client whose version vector is "more up to date than the document snapshot" concludes it needs nothing and **silently misses writes**. Two writes must succeed or fail **together**: the snapshot blob, and the `{snapshot_pointer, base_rev}` record in the metadata DB.

DEFINITION — Two-phase commit (2PC)

A protocol for committing one atomic operation across **multiple** stores. Phase 1 (*prepare*): a coordinator asks every participant "can you commit?" Each participant does everything short of committing and votes yes/no. Phase 2 (*commit*): if all voted yes, the coordinator tells everyone to commit; if any voted no, everyone aborts. 2PC guarantees atomicity but is **blocking** (a crashed coordinator can leave participants waiting) and slow (two round trips) — so we use the minimal version of the idea, not the maximal one.

In practice, prefer the cheapest construction that delivers atomicity:

1. **One store, one transaction.** If the snapshot blob and its metadata live in the same database, a single transaction suffices — no 2PC at all.

2. **Immutable blob + conditional pointer update** (our choice). Write the snapshot to the blob store under a content-addressed name; only when that write is durable, update the metadata row with a conditional compare-and-swap (`UPDATE … WHERE base_rev = <old>`). A crash between the steps leaves an **orphan blob** — garbage-collectable, never incorrect. Atomicity without a coordinator.
3. **Full 2PC** is reserved for when two genuinely independent systems must commit together — worth naming in the interview precisely so you can argue you rarely need it.

1.13 Deep Dive: Rust Reference Implementation

The concurrency core is small enough to show in full. Three pieces: the wire protocol types, the OT transform matrix with tests, and the version vector.

1.13.1 Protocol types

The data-plane messages from Section 1.8, as Rust types (serde for JSON):

```rust
use serde::{Deserialize, Serialize};

/// One edit to a plain-text document.
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum Op {
    Insert { pos: usize, ch: char },
    Delete { pos: usize },
}

/// Data-plane messages (client ⇄ server), serialized as JSON.
#[derive(Clone, Debug, Serialize, Deserialize)]
#[serde(tag = "type", rename_all = "snake_case")]
pub enum Msg {
    Join    { doc_id: String, auth: String, last_seen_rev: u64 },
    Joined  { doc_id: String, rev: u64, snapshot: String },
    Op      { doc_id: String, base_rev: u64, client_seq: u64, op: Op },
    Ack     { client_seq: u64, rev: u64 },
    /// Server → clients: a committed operation.
    Bcast   { doc_id: String, rev: u64, author: String, op: Op },
}
```

The `#[serde(tag = "type")]` attribute makes the JSON look exactly like the protocol table: a `type` field distinguishes the variants.

1.13.2 The OT transform matrix

This is Section 1.6's table, made executable. `transform(a, b, a_wins_ties)` returns the operation `a` adjusted to apply **after** the concurrent operation `b` has been applied:

```rust
use crate::Op;

/// Transform `a` so it applies correctly *after* concurrent op `b`.
/// `a_wins_ties` breaks same-position insert/insert ties identically
/// on every replica (e.g., by client id) — determinism is what matters.
pub fn transform(a: &Op, b: &Op, a_wins_ties: bool) -> Option<Op> {
    match (a, b) {
        (Op::Insert { pos: pa, ch: ca }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa > *pb || (*pa == *pb && !a_wins_ties) { pa + 1 } else
{ *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
        }
        (Op::Insert { pos: pa, ch: ca }, Op::Delete { pos: pb }) => {
            let shifted = if *pa > *pb { pa - 1 } else { *pa };
            Some(Op::Insert { pos: shifted, ch: *ca })
```

```
        }
        (Op::Delete { pos: pa }, Op::Insert { pos: pb, .. }) => {
            let shifted = if *pa >= *pb { pa + 1 } else { *pa };
            Some(Op::Delete { pos: shifted })
        }
        (Op::Delete { pos: pa }, Op::Delete { pos: pb }) => {
            if pa == pb { None }                        // both deleted the same char
            else { Some(Op::Delete { pos: if *pa > *pb { pa - 1 } else { *pa } }) }
        }
    }
}

/// Apply an operation to a document buffer.
pub fn apply(doc: &mut String, op: &Op) {
    match op {
        Op::Insert { pos, ch } => {
            let byte = doc.char_indices().nth(*pos).map_or(doc.len(), |(i, _)| i);
            doc.insert(byte, *ch);
        }
        Op::Delete { pos } => {
            if let Some((byte, c)) = doc.char_indices().nth(*pos) {
                doc.drain(byte..byte + c.len_utf8());
            }
        }
    }
}
```

Note the discipline the code makes explicit: character positions, not byte offsets (Rust strings are UTF-8; mixing the two is a classic production bug), and `None` for the delete/delete collision — an operation transformed into a no-op.

The convergence diamond from Section 1.6, as a test:

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn go_diamond_converges() {
        let alice_op = Op::Insert { pos: 0, ch: 'A' };  // "GO" -> "AGO"
        let bob_op   = Op::Delete { pos: 1 };           // "GO" -> "G"

        // Alice's side: apply hers, transform Bob's against it, apply.
        let mut a_doc = String::from("GO");
        apply(&mut a_doc, &alice_op);
        let bob_at_alice = transform(&bob_op, &alice_op, false).unwrap();
        apply(&mut a_doc, &bob_at_alice);

        // Bob's side: apply his, transform Alice's against it, apply.
        let mut b_doc = String::from("GO");
        apply(&mut b_doc, &bob_op);
        let alice_at_bob = transform(&alice_op, &bob_op, true).unwrap();
        apply(&mut b_doc, &alice_at_bob);

        assert_eq!(a_doc, b_doc);          // convergence
        assert_eq!(a_doc, "AG");           // intention preservation
    }
}
```

1.13.3 The server-side session: transforming against history

The owner node keeps a per-document session. When an operation arrives based on an old revision, transform it across everything committed since (step 3 of the pipeline):

```
pub struct DocSession {
    pub rev: u64,
    pub doc: String,
    pub log: Vec<(u64, Op)>,        // (revision, committed op); the op log in memory
    pub since_snapshot: usize,      // ops committed since last snapshot
}

impl DocSession {
    pub fn commit(&mut self, base_rev: u64, mut op: Op) -> Result<u64, &'static str> {
        if base_rev > self.rev { return Err("base_rev from the future"); }
        // Transform across every revision the client had not seen.
        for (_, past) in self.log.iter().skip(base_rev as usize) {
            op = transform(&op, past, false).ok_or("op became a no-op")?;
        }
        self.rev += 1;
        apply(&mut self.doc, &op);
        self.log.push((self.rev, op.clone()));
        self.since_snapshot += 1;
        if self.since_snapshot >= 500 { self.snapshot(); }   // Section 1.9 policy
        Ok(self.rev)                                          // -> ack + broadcast
    }

    fn snapshot(&mut self) { /* write blob, then conditional pointer update */
        self.since_snapshot = 0;
    }
}
```

This is deliberately unglamorous — a counter and a loop. That is the payoff of the architecture: because **all** operations for a document funnel through one owner, the "distributed consensus" needed here is no consensus at all.

1.13.4 Version vector

Section 1.7's definition, implemented with its three-way comparison:

```
use std::collections::HashMap;

#[derive(Clone, Debug, Default, PartialEq, Eq)]
pub struct VersionVector(pub HashMap<String, u64>);

#[derive(Debug, PartialEq, Eq)]
pub enum Ordering { Before, After, Equal, Concurrent }

impl VersionVector {
    pub fn tick(&mut self, who: &str) { *self.0.entry(who.into()).or_insert(0) += 1; }

    /// Pointwise comparison: Before = self is causally dominated by other.
    pub fn compare(&self, other: &Self) -> Ordering {
        let mut less = false;
        let mut greater = false;
        for id in self.0.keys().chain(other.0.keys()) {
            let a = self.0.get(id).copied().unwrap_or(0);
            let b = other.0.get(id).copied().unwrap_or(0);
            less |= a < b;
            greater |= a > b;
        }
```

```
        match (less, greater) {
            (false, false) => Ordering::Equal,
            (true, false)  => Ordering::Before,
            (false, true)  => Ordering::After,
            (true, true)   => Ordering::Concurrent,
        }
    }

    /// Element-wise maximum: the merged causal history of two replicas.
    pub fn merge(&mut self, other: &Self) {
        for (id, &n) in &other.0 {
            let e = self.0.entry(id.clone()).or_insert(0);
            *e = (*e).max(n);
        }
    }
}
```

The catch-up decision from Section 1.12 is now three lines: if the client's vector is `Before` the server's, send the ops it lacks; if `Equal`, send nothing (and — per the pitfall — verify the **snapshot metadata** said the same thing); if `Concurrent`, the client has offline edits: send the diff and rebase.

1.14 Deep Dive: Presence & Live Cursors

Presence — "who is here, and where is their cursor" — looks trivial and hides one elegant detail: **cursor positions are text positions**, so they must be transformed by the same OT matrix as edits. If Alice's cursor sits at position 10 and Bob inserts three characters at position 4, Alice's cursor must render at 13. Clients transform remote cursors against every incoming operation; the server simply relays.

Design rules that keep presence cheap:

- **Never persist it.** Presence flows through the ephemeral presence cache and broadcasts; a client heartbeat every few seconds refreshes it. If presence data is lost, it rebuilds within one heartbeat — this is why the architecture draws it as a dashed, separate path.
- **Don't sequence it.** Presence messages carry no `rev` and bypass the operation log entirely; a stale cursor for 100 ms is invisible to users, while a stale **edit** is a correctness bug.
- **Throttle it.** Cursor moves fire dozens of events per second per editor; cap updates (e.g., 10/sec/client) — 50 editors × 10 updates = 500 tiny messages/sec, which the same per-document owner trivially fans out.

1.15 Scaling & Sharding

> **DEFINITION — Sharding (horizontal partitioning)**
>
> Splitting data or work across many machines by a key, so that each machine owns a disjoint slice. Sharding is the fundamental answer to "one machine is not enough": pick the right key and load grows by adding machines rather than by growing them.

Every tier shards by its natural key:

- **Gateways**: stateless, so any consistent routing works; connections balance across the fleet, and reconnects land anywhere.
- **Collaboration nodes**: sharded **by document**. A routing service maps `doc_id → owner` with a lease (a time-limited ownership grant the node must renew; on crash the lease expires and ownership

moves — this is how we keep "exactly one sequencer per document" without human intervention).
- **Operation log / snapshots / metadata**: sharded by `doc_id` as well, so all of a document's data is local to its shard.

> **KEY INSIGHT — The 50-editor cap is a load-shedding decision**
>
> Why can a document never become a "hot shard" that melts its owner? Because we capped concurrency at 50 editors — a product decision with an architectural dividend. A live-blog with a million **writers** would need a different design (tree of sequencers, or CRDTs with no central order). Naming which NFR protects you from which failure mode is exactly the reasoning interviewers listen for.

1.16 Failure Modes & Recovery

A senior answer enumerates what breaks **before** being asked. The playbook:

Failure	Handling
Client disconnects	Its edits are already durable in the log (commit-before-broadcast). On reconnect: `join` with `last_seen_rev`, catch up per Section 1.12. Presence entry expires on its own.
WebSocket gateway dies	Clients reconnect to any other gateway (they are stateless). In-flight unacked ops are resent and deduplicated by `client_seq` (idempotency).
Collaboration node dies	Lease expires; routing service assigns a new owner, which rebuilds the session state as `latest snapshot + log tail` and resumes sequencing. Clients see a reconnect, not data loss.
Duplicate message delivery	Impossible to prevent on real networks — so we make it harmless: `client_seq` idempotency on the write path, revision numbers on the read path (already have rev 47? ignore the re-broadcast).
Op-log replica loss	The log is replicated (3× is standard); a lost replica is replaced from its peers.
Network partition	A client that cannot reach the server keeps editing locally and rebases on reconnect (version-vector diff, Section 1.12) — or shows "offline, changes will sync" if we choose availability over freshness. A partitioned **server-side** minority refuses ownership transfers: correctness beats liveness there.

1.17 Trade-offs & Alternatives

The design is a bundle of purchased benefits and accepted costs. Saying the costs out loud is what distinguishes a design from a wish list:

Decision	Benefit purchased	Cost accepted
Per-document sequencer + OT	Small messages, total order, free version history, simple reasoning	Single point of **ordering** per doc (mitigated by lease failover, seconds of unavailability on crash);

Decision	Benefit purchased	Cost accepted
		OT matrix must be exactly right
CRDT instead	No sequencer; offline and peer-to-peer for free	Per-character IDs and tombstones; heavier storage and GC; harder to bolt on a clean linear history
WebSocket data plane	Sub-150 ms push; bidirectional	1M long-lived connections to manage; load balancers must tolerate them
Op log as source of truth	History, audit, replay, recovery all free	Storage growth (550 GB/day at our scale) and the snapshot machinery to keep reads fast
Single-region write ownership per doc	No cross-region consensus on the hot path	Editors far from the owner's region see higher propagation latency

NOTE — PACELC, briefly

A refinement of the CAP theorem: **if** there is a Partition, choose Availability or Consistency; **else** (normal operation), choose Latency or Consistency. Our design picks **consistency** on both branches for the edit path (single owner, total order) and **availability/latency** for presence (ephemeral, best-effort) — different subsystems may legitimately make different PACELC choices.

1.18 Observability & SLOs

DEFINITION — SLI / SLO

A *Service Level Indicator* is a measurable quantity (e.g., propagation latency at p95). A *Service Level Objective* is its target ("p95 < 400 ms"). SLOs turn "the system feels slow" into an engineering signal with an error budget.

What we instrument, at minimum: per-document ops/sec and transform rates; end-to- end propagation latency (client-side measured, tagged by region); ack latency; WebSocket connections and message rates per gateway; catch-up size distribution (reconnect storms reveal themselves here); snapshot lag (`current_rev - snapshot.base_rev`); lease-failover count. Alerts fire on SLO burn, not on individual machine failures — the failure table above exists precisely so that single failures are non-events.

1.19 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *"Rich text?"* — Operations gain attributes (bold ranges, styles); the transform matrix grows cases, the architecture does not change.
- *"Comments / suggestions?"* — Anchored to character ranges that transform like cursors; stored in metadata DB, off the hot path.

- *"End-to-end encryption?"* — Server cannot read content, so server-side OT is impossible; this pushes you toward client-side CRDTs. A great example of a requirement that **re-architects** the core.
- *"5,000 simultaneous editors?"* — Our 50-editor cap is what licenses the single sequencer; beyond it, shard the document itself or move to CRDTs.
- *"Undo?"* — Per-user inverse operations transformed through the log — easy to describe, subtly hard to make intuitive; mention it, don't build it live.

If you remember five things:

1. The product is a distributed-state synchronizer wearing a text editor's clothes.
2. Convergence + intention preservation are the correctness bar; LWW and locks both fail it.
3. A per-document total order (sequencer) reduces "distributed consensus" to a counter and a transform loop.
4. Commit durably, then broadcast — never show a user an edit you cannot guarantee.
5. Snapshots and their version metadata commit atomically, or clients silently miss writes (Section 1.12).

1.20 Summary & Further Reading

We designed a real-time collaborative plain-text editor for 10M DAU: a stateless WebSocket gateway tier holding a million connections; a collaboration service that owns each document and imposes a total order on its operations; operational transformation to merge concurrent edits; an append-only operation log as the source of truth with periodic snapshots; version vectors for catch-up and offline diffing; and idempotent protocols that survive the network's misbehavior.

Primary source for this chapter:

- **"12: Design Google Docs/Real Time Text Editor"** — Systems Design Interview Questions With Ex-Google SWE (Jordan has no life) — the mock-interview walkthrough this chapter expands.

Foundations worth reading:

- Ellis & Gibbs, *Concurrency Control in Groupware Systems* (1989) — the original OT paper.
- Nichols et al., *Jupiter: high-latency collaboration* (1995) — OT with a central server, the direct ancestor of our design.
- Shapiro et al., *Conflict-free Replicated Data Types* (2011) — the CRDT formulation.
- Google Wave's OT whitepapers, and Figma's engineering blog on multiplayer — production war stories for both families.

1.21 Chapter 1 Glossary

A one-glance index of every term this chapter defined. Later chapters assume it.

Term	Meaning in one line
System design interview	Open-ended architecture design under a 45–60 minute clock
HLD / LLD	Whole-system architecture / internal mechanics of the hard components
Functional requirement	What the system must do
Non-functional requirement	How well it must do it: scale, latency, availability, consistency
DAU / QPS	Daily active users / queries per second
Back-of-the-envelope estimation	Approximate arithmetic that sizes the problem before designing

Term	Meaning in one line
Latency	Cause → observable effect; here: local echo < 16 ms, propagation < 150 ms
Availability	Fraction of time the system serves requests
Mutual exclusion (lock)	One writer at a time — correct, but kills real-time collaboration
Last-writer-wins	Latest timestamp overwrites — simple, but silently loses edits
Causality / concurrency	"Could A have influenced B?" If neither way: concurrent
Convergence	Same delivered edits ⇒ identical document on every replica
Intention preservation	Each author's observed edit effect survives concurrency
Operational Transformation	Adjust concurrent operations' positions so all replicas converge
CRDT	Data type whose merge rules converge without a central order
(Strong) eventual consistency	Replicas converge (immediately upon delivery, for SEC)
Logical clock / Lamport	Causality-tracking counters instead of wall-clock time
Version vector	Per-participant counters recording exactly what a replica has seen
REST	Stateless resource-oriented HTTPS API — our control plane
WebSocket	Persistent full-duplex client↔server channel — our data plane
Idempotency key	Client-supplied unique key making retried operations safe
Operation log	Immutable append-only record of all edits; the source of truth
Snapshot	Materialized document at a revision; read optimization over the log
Two-phase commit	Prepare-then-commit protocol for atomic writes across stores
Sharding	Partitioning work/data by key across machines — here, by document
Presence	Ephemeral who's-here-and-where state; never persisted
SLI / SLO	A measured indicator / its target, defining "healthy"

CHAPTER 2

Designing a Maps & Navigation Service

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"13: Google Maps"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*, 2024, 35 min). The talk designs a mapping and navigation product: how the map itself is served, how the road network becomes a graph, how routes and ETAs are computed, and how live traffic feeds back into both. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol specifications, and Rust reference implementations.

2.1 The Problem Statement

The interviewer looks up and says:

> *"Design Google Maps. Users should be able to look at a map, search for places, and get directions from A to B — with an estimated time of arrival that accounts for current traffic."*

Where Chapter 1's problem hid its difficulty behind a familiar CRUD interface, this one hides its difficulty behind a familiar *picture*. A map feels like content — something you serve, like an image. Directions feel like a lookup — something you query, like a database. Neither is true. The map is a planet-sized rendering problem, directions are a graph algorithm running at planetary scale, and "current traffic" means the system's answers change **under you every few minutes**, driven by a firehose of GPS signals from millions of moving phones. Three genuinely hard subsystems — geospatial serving, large-scale graph search, and real-time stream processing — share one interview.

DEFINITION — ETA (estimated time of arrival)

The system's prediction of how long a journey from A to B will take, usually expressed as a duration or an arrival clock time. An ETA is a **prediction**, not a measurement: it must be computed before the journey happens, from a model of the road network plus everything currently known about conditions on it. ETA accuracy is the quality bar this whole chapter is judged against — Section 2.4 makes it a formal requirement and Section 2.18 shows how to measure it.

2.2 Scope & Clarifying Questions

Never design the prompt you were handed; design the prompt you **negotiated**. Google Maps is a dozen products wearing one icon, so the first job is to shrink it. A strong opening exchange looks like this:

Speaker	Dialogue
Candidate	"The full product is enormous — map rendering, place search, directions, turn-by-turn navigation, live traffic, Street View, satellite imagery, transit schedules, offline maps. Which parts are we designing?"
Interviewer	"Focus on five: render the map, search for places, compute directions with an ETA, show live traffic, and run a turn-by-turn navigation session. Skip Street View, satellite, transit, and offline."
Candidate	"What scale — how many users, and how many actively navigating at once?"
Interviewer	"One billion monthly users, a few hundred million daily. Assume up to 15 million people in an active navigation session at peak."
Candidate	"Latency expectations?"
Interviewer	"The map must feel instant while panning. A route request should come back within a second or two."
Candidate	"How accurate does the ETA need to be?"
Interviewer	"Within about ten percent of actual travel time on typical trips."
Candidate	"Do drivers' phones report GPS positions back to us during navigation? That would be our live traffic signal."
Interviewer	"Yes — anonymized location updates, with user consent, every few seconds while navigating."
Candidate	"Availability target? People use this while driving; a dead navigation app mid-highway is a safety problem."
Interviewer	"Four nines for the navigation path."

AGREED SCOPE

Five features: **map rendering**, **place search**, **directions with ETA**, **live traffic**, **turn-by-turn navigation**. Scale: **1B MAU**, 200M DAU, **15M concurrent navigation sessions** at peak. Latency: map feels instant, routes within 1–2 s. ETA accurate to ±10%. Navigation availability **99.99%**. Phones send anonymized GPS updates every few seconds while navigating.

INTERVIEW TIP — Ask where the data comes from

Most candidates ask about users and features; few ask **"what data do we get to see?"** That single question unlocks this entire problem: the answer (GPS updates from phones) is what makes live traffic possible at all. In a real interview, the questions that reveal **inputs** are worth as much as the questions that reveal **scale**.

2.3 Functional Requirements

Chapter 1 defined functional and non-functional requirements; recall that FRs are what the system must **do**. Our scoped functional requirements:

1. **FR-1 — Map rendering.** The user can view an interactive map of the world and pan and zoom it smoothly, from a whole-planet view down to individual streets.
2. **FR-2 — Place search.** The user can find places by name, address, or category ("coffee near me") and see them on the map.

3. **FR-3 — Directions.** The user can request a route between two points and receives a good route: distance, step-by-step instructions, and a path drawn on the map.
4. **FR-4 — ETA.** Every route comes with a travel-time estimate that reflects **current** conditions, not just speed limits.
5. **FR-5 — Turn-by-turn navigation.** The user can start a navigation session and receives timely spoken/visual instructions, a live ETA that updates en route, and an automatic new route if they stray from the planned path.
6. **FR-6 — Traffic overlay.** The map shows current congestion (green/yellow/red) on roads, fresh to within a few minutes.

Out of scope, stated explicitly: Street View and satellite imagery, public transit timetables, offline maps, business-owner tooling, and social features.

2.4 Non-Functional Requirements

> **DEFINITION — Freshness**
>
> The age of the information behind an answer, measured from when reality changed to when the system's output reflects it. A traffic overlay that shows speeds from an hour ago is **stale**; our target is that any road's displayed condition reflects measurements from the last few minutes. Freshness is the NFR that makes this problem a **streaming** system rather than a static one.

Four qualities dominate, stated as targets:

Quality	Target
Map latency	Tiles visible within 100 ms while panning (p95); panning must never stutter
Route latency	Directions within 1 s for typical trips; 2 s for cross-country
ETA accuracy	Within ±10% of realized travel time on typical trips
Traffic freshness	Displayed speeds reflect the last 2–5 minutes of reality
Availability	99.99% on the navigation path (4.4 s of allowed downtime per day); 99.9% elsewhere
Scale	1B MAU; 200M DAU; 15M concurrent navigation sessions at peak

> **KEY INSIGHT — Different subsystems, different NFRs**
>
> Notice that "the system" has no single latency or availability number. Tile serving is a massive, cacheable **read** workload where 100 ms matters and a stale tile is harmless. Routing is a **compute** workload where one second is generous but the algorithm must be exact. The traffic pipeline is a **write** workload where a few minutes of lag degrade quality, not correctness. Saying "it depends which plane you mean" — and then naming the planes — is a senior answer. Chapter 1 drew the same lesson with its control plane / data plane split.

2.5 Back-of-the-Envelope Estimation

Chapter 1 defined back-of-the-envelope estimation; the discipline is identical — state assumptions, write them down, invite correction.

Assumptions:

- 1B MAU, 200M DAU. Each daily user views 60 map tiles worth of panning, searches 2 places, and requests 3 routes per day.

- 15M concurrent navigation sessions at peak; each phone uploads one GPS update every 5 seconds while navigating; one update ≈ 150 bytes on the wire.
- The world's routable road network is on the order of **300 million directed edges** (two directions per road piece, worldwide).
- A raster map tile averages 20 KB; the world is mapped at zoom levels 0–20.

Derived numbers:

Quantity	Estimate	How
Map tile requests	≈ 140k QPS avg, 700k peak	200M users × 60 tiles/day ÷ 86,400 s, ×5 peak factor
Place search QPS	≈ 5k avg, 25k peak	200M × 2/day ÷ 86,400, ×5
Route request QPS	≈ 7k avg, 20k peak	200M × 3/day ÷ 86,400, ×3
GPS update ingress	≈ 3M updates/s	15M navigators ÷ 5 s per update
Ingress bandwidth	≈ 450 MB/s	3M updates/s × 150 B
Road graph size	≈ 20 GB	300M edges × 64 B per adjacency record
Naive tile storage	≈ 29 PB	$\sum_{z=0}^{20} 4^z \approx 1.47$ trillion tiles × 20 KB

KEY INSIGHT — What the math tells us

Three conclusions drive everything that follows. First, tile traffic is enormous but **the content barely changes** — a perfect caching workload; Section 2.6 exists to make 95%+ of those 700k QPS vanish into a CDN. Second, the road graph (20 GB) fits in the RAM of a handful of machines, so routing is an **in-memory** problem — but 20k route QPS × 3 CPU-seconds for a naive shortest-path search would need **60,000 CPU cores**, which no fleet absorbs; Section 2.8's whole purpose is to shave those 3 seconds down to 1 millisecond, at which point 20 cores suffice. Third, 3M GPS updates per second is far too much for any request/response design — it demands a streaming pipeline, which Section 2.12 builds.

2.6 Core Challenge I: Serving the Map

A user opens the app and sees their city. They drag the map; new streets slide in. They pinch; the view dives from the whole country to one neighborhood. Each of those moments requires the **right piece of the planet, rendered, in tens of milliseconds**. Rendering the world on the fly for every gesture of 200 million daily users is impossible; the entire solution rests on one idea: **precompute the map as tiny, independently addressable squares, and cache them everywhere.**

DEFINITION — Map tile

A small, fixed-size square of the map — conventionally 256×256 pixels (as an image) or the equivalent bundle of geometry (as data) — covering a specific rectangular patch of the Earth's surface. A screen full of map is assembled from a grid of tiles, like mosaic pieces. Tiles are the unit of storage, caching, and transfer for every mainstream map service.

DEFINITION — Zoom level

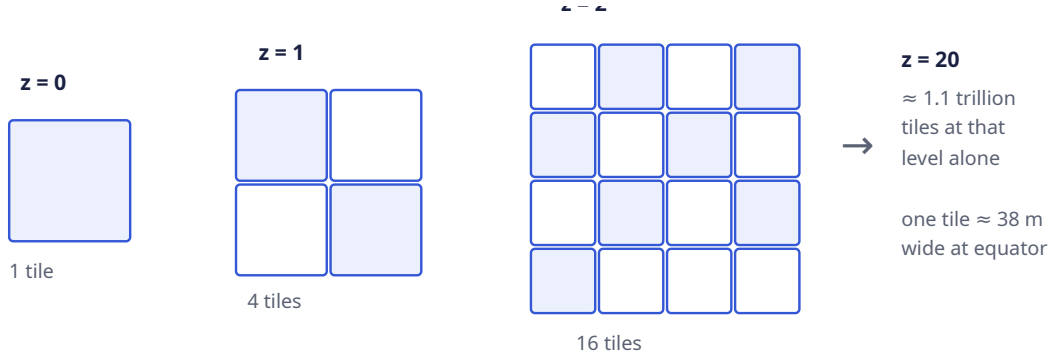
The map's scale, expressed as an integer z from 0 upward. At zoom 0 the **entire world** is one tile. Each step down splits every tile into four children, so zoom z has 2^z tiles per axis and 4^z tiles in total. Zoom 5

is roughly a country; zoom 10 a city; zoom 15 a neighborhood; zoom 20 shows a patch about 38 meters wide at the equator — individual buildings.

DEFINITION — Slippy-map tiling scheme

The near-universal convention for addressing tiles: every tile is identified by three integers **(z, x, y)** — its zoom level and its grid coordinates, counted from the top-left of the world. The Earth's surface is first flattened with the Web Mercator projection (which maps the sphere to a square and caps latitude near ±85.05°), then cut into the $2^z \times 2^z$ grid. Given a latitude/longitude and a zoom, **anyone can compute which tile contains it** with a closed-form formula — Section 2.14 implements it. The addressing is deterministic, so the client never asks the server "which tiles do I need?"; it already knows.

The pyramid shape is the whole storage story. Four zooms, drawn:



Our estimate said storing **all** of that naively costs 29 PB. Two observations make it tractable:

1. **Most tiles are uniform.** Roughly 70% of the planet is ocean; enormous areas are desert, ice, or forest. A solid-blue 256×256 square compresses to almost nothing — and, crucially, **it is the same square everywhere**. Storing tiles content-addressed (Section 2.10) means every identical tile in the world is stored **once**; the trillions collapse to the few percent of tiles that actually contain roads and labels.
2. **Nobody looks at most of the world, most of the time.** Demand is violently skewed toward populated areas and common zooms. That skew is what caching feeds on.

DEFINITION — Content Delivery Network (CDN)

A geographically distributed fleet of **edge caches**: servers in hundreds of cities, each holding copies of popular content close to users. A request is routed to the nearest edge location; on a **cache hit** the content is served locally (single-digit milliseconds, zero load on our infrastructure); on a **miss** the edge fetches from our origin once and caches it for the next requester. CDNs turn a read-heavy, mostly-static workload into someone else's bandwidth.

DEFINITION — TTL (time to live) / cache hit ratio

A cached entry's *TTL* is how long an edge may serve it before revalidating with the origin — the dial between freshness and load. The *hit ratio* is the fraction of requests served from cache. For base map tiles we choose long TTLs (days; roads move slowly) and expect hit ratios above 95%; the **traffic overlay** tiles of Section 2.12 get TTLs of a minute or two, because stale traffic is worthless.

COMMON PITFALL — Serving tiles from your own fleet

> The single most common junior mistake on this problem: drawing a "tile service" that renders or reads tiles on demand per request. At 700k peak QPS that fleet is vast, slow for distant users, and pointless — the content is static and cacheable. The correct shape is: **pre-render tiles offline, push them to object storage, put a CDN in front, and let the origin see a single-digit percentage of traffic.** Rendering is a batch job, not a request path.

One design fork is worth naming here and deciding later (Section 2.17): tiles can be **raster** (pre-rendered images) or **vector** (compact geometry the client renders on its GPU). We will serve vector tiles to modern clients — roughly half the bytes, restyleable without re-rendering, and one tile set serves every zoom-adjacent gesture smoothly — while keeping raster fallbacks for old clients. Either way, the tiling, addressing, and CDN story is identical.

Place search (FR-2) rides on the same geospatial ideas: a **spatial index** lets us find "all places inside this patch of Earth" without scanning 200M records. Section 2.7 defines the indexing structure once, because routing needs it too.

2.7 Core Challenge II: Modeling the Road Network

Directions require a mathematical object we can search. That object is a graph, built from the road network:

> **DEFINITION — Graph / weighted directed graph**
>
> A *graph* is a set of **nodes** connected by **edges**. It is *directed* when edges have a direction (a one-way street is traversable only along its arrow) and *weighted* when every edge carries a number measuring the **cost** of crossing it. We model intersections as nodes and stretches of road between them as edges, and we call one such directed road piece a **segment**.

> **DEFINITION — Segment**
>
> The atomic unit of the road network: one directed piece of road between two nodes, carrying metadata — geometry (the polyline of its shape), length, road class (residential, arterial, highway), speed limit, and turn restrictions at its far end. Segments are also the unit of **traffic**: when we say "this road is slow," we mean "this segment's current crossing time is high," and every GPS update we receive is attributed to exactly one segment (Section 2.12).

The crucial choice is the edge **weight**. Distance is stable but wrong for drivers; what users minimize is **time**. So an edge's weight is its **expected crossing time**: `length / current expected speed`. "Current expected speed" is where live traffic enters — the weight is `length / speed_limit` on an empty road, and degrades as Section 2.12's pipeline reports slower observed speeds. Routing and traffic are thereby one mechanism: **traffic is just edge weights that move.**

> **DEFINITION — Adjacency list**
>
> The standard memory layout for sparse graphs: for every node, a list of its outgoing edges. Routing graphs are extremely sparse (an intersection has 3–6 outgoing segments), so an adjacency list stores 300M small records — the 20 GB from Section 2.5 — instead of the quadratically-sized matrix a dense representation would need. Our routing engine holds adjacency lists **in RAM**; there is no database on the per-request path.

A subtlety interviewers enjoy: a graph edge cannot express "you may enter this intersection from Main St but not turn left onto 1st Ave." Turn restrictions are handled either by splitting nodes (one

node per incoming direction, with only legal turns as edges) or by per-edge metadata the search consults. Either way, the model — nodes, directed edges, time weights — survives intact.

2.8 Core Challenge III: Shortest Paths at Planetary Scale

DEFINITION — Shortest path problem

Given a weighted graph and two nodes, find the path between them whose total edge weight is minimal. With time-weighted segments, the shortest path is the **fastest route**, and its total weight is the route's ETA. This is the single computation at the heart of FR-3 and FR-4.

The canonical algorithm, and the one the interviewer expects you to derive:

DEFINITION — Dijkstra's algorithm

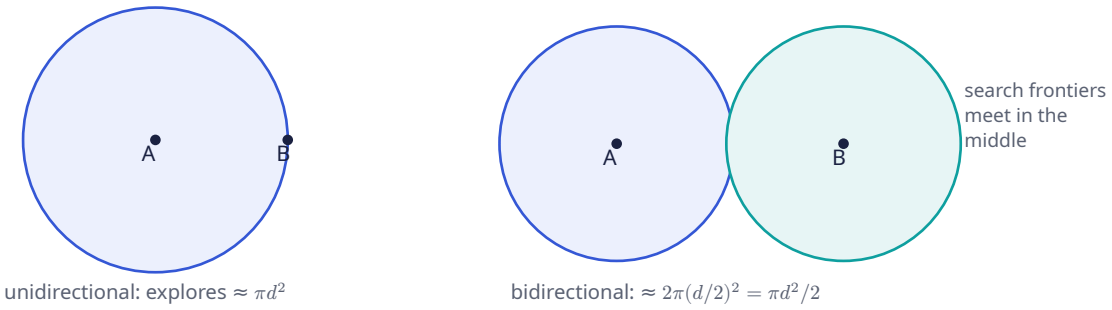
Explores the graph outward from the source in order of increasing distance from it. Maintain for each node the best known distance (initially ∞, 0 for the source); repeatedly take the not-yet-finalized node with the smallest best-known distance, declare it final, and **relax** its edges — for each outgoing edge, check whether reaching its neighbor through this node beats the neighbor's best-known distance, and if so update it. With a min-priority queue selecting the next node, the running time is $O((V + E) \log V)$. It is provably exact for non-negative weights — and crossing times are always non-negative.

DEFINITION — Priority queue (min-heap)

A data structure supporting "insert with a priority" and "remove the minimum-priority element," both in $O(\log n)$. A binary heap inside an array is the standard implementation. Dijkstra's algorithm spends its life asking "which unfinished node is currently closest?" — exactly this operation.

Dijkstra is correct — and unusable here. A cross-country query on 100M nodes takes seconds of CPU; Section 2.5 showed that times 20k QPS is 60,000 cores. Three refinements, each a layer of the same idea — **search less of the graph**:

Refinement 1 — bidirectional Dijkstra. Run two searches simultaneously: one forward from the origin, one **backward** from the destination over reversed edges. Stop when the frontiers meet; the route is the two half-paths joined. Why it helps is geometric:



A forward search from A must explore roughly every node within distance d of A — a ball whose node count grows like the **area** πd^2. Two half-searches each explore a ball of radius $d/2$; together that is $2 \cdot \pi(d/2)^2 = \pi d^2/2$ — about **half** the work in a uniform graph, and better in road networks, where the frontiers approach each other along highways. Same exact answer, half the CPU, and trivially parallelizable across two threads.

Refinement 2 — A* search. Dijkstra explores equally in **all** directions — including straight away from the destination, which is obviously wasted. A* steers the search by reordering the priority queue.

DEFINITION — Heuristic / admissible heuristic

In A*, each node's queue priority is `known distance from source + h(node)`, where the *heuristic* `h` estimates the remaining distance to the destination. The heuristic is *admissible* if it never **overestimates** the true remaining cost. Admissibility preserves Dijkstra's exactness guarantee while pulling the search frontier toward the goal — nodes in the right direction get popped first.

For a road network, the admissible heuristic is the **straight-line distance to the destination divided by the fastest speed anywhere in the network**: no legal route can beat the crow flying at top speed, so this `h` never overestimates. Computing it needs distances between points on a sphere:

DEFINITION — Haversine distance

The great-circle distance between two latitude/longitude points on a sphere — the length of the shortest path over the Earth's surface ("as the crow flies"). A closed-form trigonometric formula computes it in microseconds; Section 2.14 implements it. Haversine is our heuristic's yardstick and our fallback whenever "how far apart are these coordinates?" appears.

Refinement 3 — hierarchy: mega-segments, then contraction hierarchies. Even A* explores every side street near the origin and destination. But observe how **you** drive cross-country: local streets for a minute, then a highway for three hundred miles, then local streets again. The middle of a long route almost never touches small roads. Turn that observation into a mechanism: precompute and **aggregate**.

DEFINITION — Mega-segment

A precomputed synthetic edge that summarizes a chain of ordinary segments — for example, one edge meaning "highway, exit 12 to exit 47, 52 km, 31 minutes at current speeds." A long-distance search can then hop between on-ramps and off-ramps on mega-segments instead of expanding thousands of small segments. The source talk builds its long-range routing this way: segments roll up into mega-segments, which roll up further, so a query quickly climbs from street level to highway level, crosses the country on a handful of edges, and descends again. Mega-segment weights are sums of member segment weights — so when traffic updates a segment (Section 2.12), the change **bubbles up** to every mega-segment containing it.

The literature's formal, optimal version of the same idea:

DEFINITION — Contraction hierarchy (CH)

A preprocessed form of the road graph. Offline, order all nodes by importance (highway junctions outrank cul-de-sacs) and **contract** them in that order: removing a low-importance node, and — whenever the shortest path between two of its neighbors ran through it — inserting a **shortcut edge** with the summed weight, so all shortest-path distances are preserved. At query time, run bidirectional Dijkstra with one rule: only follow edges that go **up** in importance. The two upward searches meet at the most important node on the route. Preprocessing takes hours and adds 30–100% more edges; queries drop from seconds to **microseconds-to-milliseconds** — the roughly 1000× speedup Section 2.5's arithmetic demands. Production engines (OSRM, Valhalla, and the systems the source talk gestures at with mega-segments) all run variants of this playbook.

The full ladder, with the numbers that justify it:

Algorithm	Preprocessing	Query time	Verdict
Dijkstra	none	seconds (continental)	Teaches the principle; too slow to serve
Bidirectional Dijkstra	none	2× faster, still seconds	Free win; still too slow
A* + haversine	none	often 5–10× faster	Great for short trips; long trips still expand too much
Mega-segments / CH	hours, offline	≈ 0.1–1 ms	**Production answer**: the 1000× that makes 20k QPS fit on 20 cores

KEY INSIGHT — Precomputation is the theme of the whole chapter

Tiles are pre-rendered so reads are cache hits (Section 2.6); the graph is pre-contracted so queries touch only highways-in-the-middle (this section); traffic speeds are pre-aggregated per segment so ETAs are table lookups (Section 2.12). A maps system is a machine for moving work **off** the request path and into batch and stream pipelines. When an interviewer asks "how does it answer so fast?", the honest one-word answer is: **earlier**.

2.9 API & Protocol Design

Chapter 1 split its API into a control plane and a data plane; the same split applies, with a twist: our heaviest "API" is not an API at all.

Tiles are plain GETs — and mostly not ours. The client computes the (z, x, y) address of every tile in view (Section 2.6) and issues ordinary `GET /v1/tiles/{z}/{x}/{y}` requests against a tile hostname. Nearly all of these terminate at a CDN edge and never reach us. This is why the tile endpoint has no interesting request parameters: all the intelligence is in the **addressing scheme**.

DEFINITION — Polyline encoding

A compact ASCII encoding of a sequence of coordinates — Google's variant encodes latitude/longitude deltas as variable-length base-64-ish text, so a 500-point route is a few hundred bytes instead of a JSON array of floats. Routes are transmitted as encoded polylines and decoded client-side; the format exists because a route crosses a route-length's worth of geography but must fit in one small response.

The remaining request/response endpoints (REST, as defined in Chapter 1):

Method	Path	Purpose
`GET`	`/v1/tiles/{z}/{x}/{y}`	Map tiles; served by CDN edge, origin on miss
`GET`	`/v1/search?q=&near=lat,lng`	Place search: text + spatial filter, ranked results
`GET`	`/v1/search/autocomplete?q=`	Prefix suggestions as the user types
`POST`	`/v1/routes`	Directions: origin, destination, mode, departure time → candidate routes with distance, ETA, ETA-in-traffic, steps, encoded polyline
`POST`	`/v1/geocode`	Address → coordinates (and reverse: coordinates → address)

A route request and its response:

```
POST /v1/routes
{
  "origin":      { "lat": 37.7749, "lng": -122.4194 },
  "destination": { "lat": 37.3861, "lng": -122.0839 },
  "mode": "driving",
  "depart_at": "now"
}

200 OK
{
  "routes": [{
    "route_id": "rt_9c31",
    "distance_m": 56300,
    "eta_seconds": 2820,
    "eta_in_traffic_seconds": 3450,
    "polyline": "a~l~Fjk~uOwIb@_...",
    "steps": [
      { "instruction": "Head north on Market St",
        "segment_ids": ["seg_7712", "seg_7713"],
        "distance_m": 400, "eta_seconds": 61 }
    ]
  }]
}
```

Navigation (FR-5) is a session, not a request: the phone streams its position up and the server streams guidance down. That is a bidirectional, long-lived, low-latency channel — the WebSocket data plane from Chapter 1, reused verbatim.

Message	Direction	Payload & semantics
`nav_start`	client → server	`{route_id, auth}` — begin the session on the chosen route
`location_update`	client → server	`{lat, lng, speed_mps, heading_deg, t}` — one GPS fix, every 5 s; drives guidance **and** the traffic pipeline
`instruction`	server → client	`{text, voice, distance_to_maneuver_m}` — the next turn, delivered early enough to speak
`eta_update`	server → client	`{eta_seconds, remaining_m}` — refreshed as segment weights move
`reroute`	server → client	`{reason, new_route}` — deviation or a newly faster alternative
`nav_end`	either	Arrived, cancelled, or the connection dropped

Two protocol notes. First, `location_update` is **one stream feeding two masters**: live guidance for this user, and (anonymized, aggregated) the traffic pipeline for everyone else — Section 2.12 draws that fork. Second, navigation is the one place we push **down** to a moving phone, which raises the question "which gateway holds this user's connection?" — answered by the connection manager in Section 2.13.

2.10 Data Model & Storage

DEFINITION — Polyglot persistence

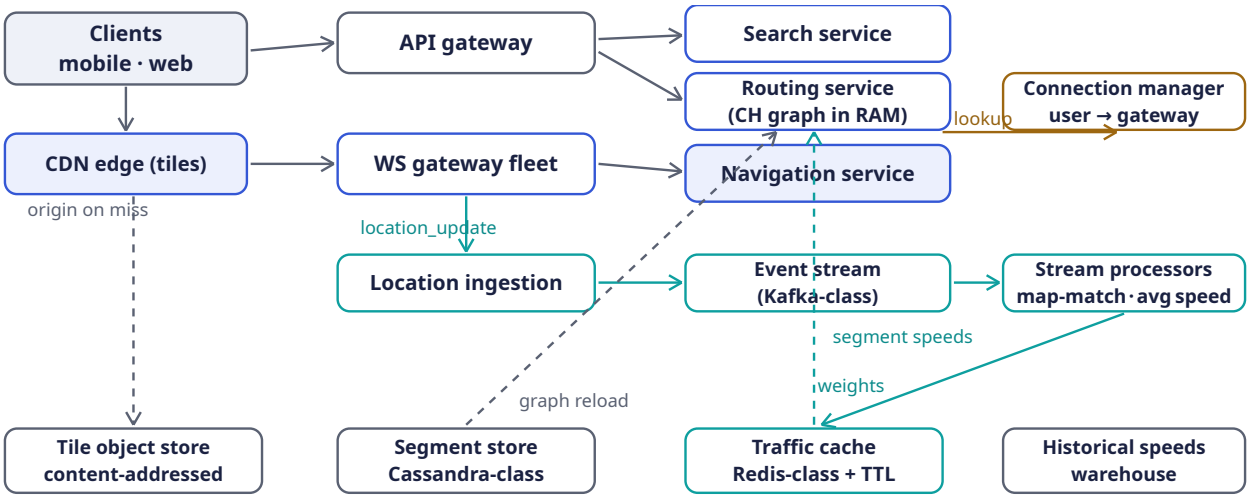
Using different storage engines for different access patterns within one system, instead of forcing every entity into one database. The cost is operational complexity; the benefit is that each workload gets the structure it actually needs. A maps system is the canonical case — our six entities below want six different shapes.

Entity	Contents	Store
`Tile`	`(z,x,y)` → rendered image or vector geometry	Object store behind the CDN; **content-addressed** so identical tiles are stored once
`Place`	name, category, address, location, ratings, hours	Text-inverted index × spatial index; 200M records
`Segment`	geometry, length, road class, speed limit, turn restrictions	Wide-column store (Cassandra-class), sharded by geography; read-heavy, batch-written
`Road graph`	adjacency lists + current weights + shortcuts	**In RAM** on routing nodes; rebuilt/swapped from the segment store + traffic cache
`Traffic`	`segment_id` → current average speed, sample count, updated-at	In-memory key-value cache (Redis-class) with a TTL — absence means "no fresh data," triggering the historical fallback
`Historical speeds`	`segment_id × (day-of-week, time bucket)` → average speed	Analytics warehouse, batch-computed nightly from the GPS archive
`Nav session`	route, progress along it, last fix, ETA	In-memory on the navigation node; ephemeral like Chapter 1's presence data

The ideas interviewers reward here: tiles are **content-addressed** (the 29 PB problem of Section 2.5 collapses because duplicate oceans are stored once); the routing graph lives **in memory**, with stores acting only as its source of truth during reloads; and the traffic cache's TTL **is** the freshness guarantee — a segment whose entry has expired simply stops claiming live data.

2.11 High-Level Architecture

Section 2.4 promised separate planes; here they are. The **read plane** (tiles, search, routes) is stateless and cache-fronted. The **streaming plane** (GPS in, traffic out) is a pipeline. Navigation sits astride both.



Read plane: stateless, cache-fronted. Streaming plane: GPS in → traffic out. Dashed: control/reload paths.

Component responsibilities, and the reason each exists:

Component	Responsibility	Why it is shaped this way
CDN edge	Serve 95%+ of tile requests	Tiles are static and skewed — the textbook cache workload (Section 2.6)
API gateway	Auth, rate limiting, routing for REST endpoints	Stateless; also the paid-API front door, where per-customer quotas are enforced
Search service	Text × spatial place lookup	Read-heavy over a slowly-changing index; replicas scale it
Routing service	Compute routes + ETAs on the CH graph	Holds the graph **in RAM**; scales by region shards and replicas (Section 2.15)
WS gateway fleet	Hold millions of navigation connections	Stateless connection holders, exactly as in Chapter 1
Navigation service	Per-session guidance: progress, instructions, ETA refresh, reroutes	Stateful per session but sessions are small and independent
Connection manager	Registry: user → which gateway	So the navigation service can push `reroute` to the right socket (Section 2.13)
Location ingestion	Absorb 3M updates/s, validate, anonymize	A buffer that decouples phone bursts from processing (Section 2.12)
Event stream	Durable, replayable pipe of GPS updates	Chapter 2 defines it below; lets many consumers read the same firehose

Component	Responsibility	Why it is shaped this way
Stream processors	Map-match pings to segments; average speeds; write traffic cache; bubble weights up mega-segments	The batch-quality work of traffic, done continuously

2.12 Deep Dive: The Live Traffic Loop

This is the subsystem that turns "a map" into "**current** conditions," and it is the heart of the source talk. Follow one GPS update until it changes someone's ETA.

DEFINITION — Event streaming platform (Kafka-class)

A distributed, durable, append-only log organized into **topics**; producers append events, and any number of independent **consumers** read them at their own pace. Topics are **partitioned** (we partition by geographic key, e.g. geohash prefix) so hundreds of consumer instances process the stream in parallel, each owning a disjoint slice of the world. It absorbs our 3M updates/second, decouples ingestion from processing, and — because it is replayable — lets us rebuild derived state after failures.

DEFINITION — Stream processing

Computing over data **as it arrives**, event by event or in small windows, rather than in scheduled batch runs over stored data. Here: every GPS update is map-matched and folded into a per-segment running average within seconds of leaving the phone.

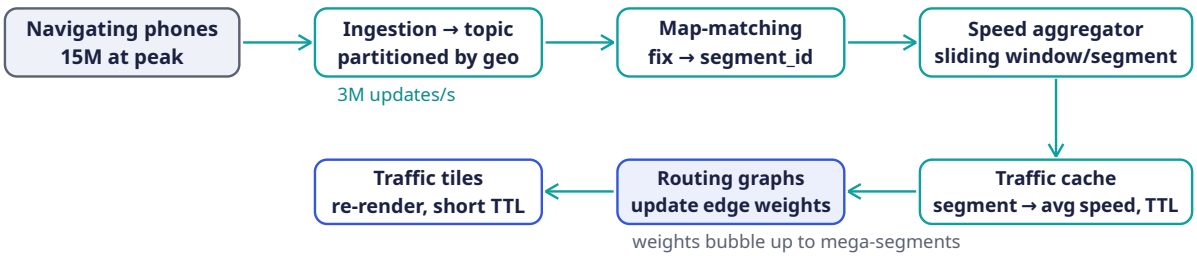
DEFINITION — Map-matching

Snapping a noisy GPS fix to the road segment the vehicle is most likely actually on. Raw GPS is wrong by 5–50 meters — enough to place a car on the wrong street or inside a building. The production-standard approach models the trace as a **hidden Markov model** (hidden states = candidate segments near each fix; emission scores = closeness of fix to segment; transition scores = plausibility of the implied movement) and solves it with the Viterbi algorithm. Map-matching is why the traffic pipeline's first stage is a **processor**, not a database write.

DEFINITION — Sliding window

A moving time interval over a stream — "the last 15 minutes," recomputed as time advances. A per-segment sliding-window average of observed speeds is our definition of "current speed": old enough data falls out of the window and stops influencing the present. Section 2.14 implements the window.

The loop, end to end:



Every few minutes, reality → graph. ETAs and reroutes on new routes then use fresh weights automatically.

Three design details carry the interview:

1. **Speed, not position, is the aggregate.** Per segment and window, we average the speeds of map-matched vehicles. One slow vehicle is noise (a delivery van at the curb); the **average** over dozens of vehicles in 15 minutes is signal. This is also why privacy and accuracy align: nobody's individual trace is stored in the traffic cache — only segment statistics.
2. **Sparse segments fall back to history.** At 3 a.m. on a rural road, the window holds too few samples. Below a minimum count we blend toward the **historical** speed for that segment, day-of-week, and time-of-day — Tuesday-8 a.m. traffic is remarkably similar to last Tuesday-8 a.m. Live data where it exists, patterns where it does not, speed limits as the floor of last resort.
3. **Third-party feeds are scored, not trusted.** We also ingest incident and flow feeds from external providers. The talk's discipline: validate each feed against our own observed reality — if a provider claims congestion on a corridor where our measured speeds never dropped, that claim is wrong, and a provider that is wrong often gets **down-weighted or ignored** at the organization level. Every external signal is guilty until statistically innocent.

The same machinery paints the traffic overlay (FR-6): segment speeds become green/yellow/red classes, rendered into a **separate tile layer** with a TTL of a minute or two — short-lived tiles on top of the long-lived base map, each layer cached at its own freshness.

2.13 Deep Dive: The Turn-by-Turn Navigation Session

A navigation session is a small state machine per user, held in the navigation service: `ON_ROUTE → OFF_ROUTE_SUSPECTED → REROUTING → ON_ROUTE → ARRIVED`. Each `location_update` advances it:

1. **Project.** Map-match the fix onto the planned route's polyline: how far along the route is the user, and how far **off** it?
2. **Guide.** From progress along the route, emit the next `instruction` early enough to be spoken ("in 300 meters, turn right").
3. **Refresh.** Recompute the ETA over the remaining segments using **current** weights from the traffic cache; push `eta_update` when it moves by more than a small threshold (users notice a jumping ETA; hysteresis is a feature).
4. **Reroute.** If the fix sits beyond a distance threshold from the polyline **and** heading disagrees with the route, **sustained** for a few consecutive updates (one bad fix must not trigger a reroute — GPS noise is constant), recompute from the current position and push `reroute` with the new route. Section 2.14 implements this decision.

The push path needs one piece of plumbing: the navigation service knows **what** to send but not **where the user's socket lives**. The **connection manager** — a replicated key-value registry mapping `user_id → gateway_id` — answers that: gateways register each connection on connect and remove it on drop; the navigation service looks the user up and hands the message to that gateway. If the registry entry is stale (a gateway died), the client's reconnect registers a fresh one within seconds, and at worst one push is retried. This is Chapter 1's stateless-gateway story, completed with its directory.

> **COMMON PITFALL — Rerouting on a single GPS fix**
>
> GPS in a downtown canyon can teleport a user two blocks sideways for one update. A reroute fired on that ghost fix is a terrible user experience — and it also **pollutes the traffic pipeline** with a phantom slow-down on a road the user never touched. Both loops therefore consume **map-matched, sustained** signals, never raw single fixes. The same discipline, twice.

> **INTERVIEW TIP — Navigation availability is a degradation story, not an uptime story**

> Four nines on the navigation path does not mean "the server never dies"; it means **the phone copes when it does**. The client caches its route, its upcoming tiles, and its step list; if the session drops, guidance continues locally from the last known state while a new session is established. Saying "the client is the final fallback layer of the server" is exactly the kind of answer the 99.99% requirement is fishing for.

2.14 Deep Dive: Rust Reference Implementations

Four pieces of this chapter are small enough — and interview-critical enough — to show in full: tile addressing, the routing core, the traffic window, and the reroute decision.

2.14.1 Tile addressing: lat/lng → tile → quadkey

The slippy-map formulas of Section 2.6, plus the quadkey encoding that makes tiles prefix-searchable:

```rust
/// Web-Mercator slippy-map addressing (Section 2.6).

/// Which tile (x, y) at `zoom` contains this lat/lng?
pub fn lat_lng_to_tile(lat: f64, lng: f64, zoom: u8) -> (u32, u32) {
    let n = 2f64.powi(zoom as i32);              // tiles per axis: 2^z
    let x = ((lng + 180.0) / 360.0 * n) as u32;
    let lat = lat.to_radians();
    let y = ((1.0 - lat.tan().asinh() / std::f64::consts::PI) / 2.0 * n) as u32;
    let max = n as u32 - 1;
    (x.min(max), y.min(max))                     // clamp the antimeridian edge
}

/// The same address as a quadkey: one digit per zoom level, '0'..='3',
/// interleaving y/x bits from the most significant down. Neighboring tiles
/// share prefixes — a quadkey IS a quadtree path — so a plain sorted
/// key-value store can answer "all tiles in this area" as a range scan.
pub fn quadkey(x: u32, y: u32, zoom: u8) -> String {
    let mut key = String::with_capacity(zoom as usize);
    for level in (0..zoom).rev() {
        let mask = 1u32 << level;
        let mut digit = 0u8;
        if x & mask != 0 { digit += 1; }
        if y & mask != 0 { digit += 2; }
        key.push((b'0' + digit) as char);
    }
    key
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn whole_world_is_tile_zero() {
        assert_eq!(lat_lng_to_tile(0.0, 0.0, 0), (0, 0));
    }

    #[test]
    fn san_francisco_zoom_16() {
        // Verified against the slippy-map formulas.
        assert_eq!(lat_lng_to_tile(37.7749, -122.4194, 16), (10482, 25331));
        assert_eq!(quadkey(10482, 25331, 16), "0230102033330032");
    }
```

```
    #[test]
    fn bing_canonical_quadkey() {
        // The documentation example: tile (3, 5) at level 3 -> "213".
        assert_eq!(quadkey(3, 5, 3), "213");
    }
}
```

The quadkey is the bridge to place search: index every place under the quadkey of its location, and "places near me" becomes "places whose key shares my tile's prefix," widened one ring of neighbors at a time.

2.14.2 The routing core: haversine, Dijkstra, A*

Section 2.8's algorithm ladder, executable. Weights are integer **milliseconds** of expected crossing time — exact, and exactly what the traffic loop updates.

```
use std::cmp::Reverse;
use std::collections::BinaryHeap;

pub type NodeId = u32;

/// One directed segment leaving a node (Section 2.7).
#[derive(Clone, Copy, Debug)]
pub struct Edge {
    pub to: NodeId,
    pub millis: u32, // expected crossing time — the weight traffic moves
}

/// In-memory adjacency-list road graph (Section 2.7).
pub struct RoadGraph {
    pub adj: Vec<Vec<Edge>>,
    pub lat: Vec<f64>, // node coordinates, for haversine and A*
    pub lng: Vec<f64>,
}

/// Great-circle distance in meters (Section 2.8).
pub fn haversine_m(a: (f64, f64), b: (f64, f64)) -> f64 {
    const R: f64 = 6_371_000.0; // Earth radius, meters
    let (la1, la2) = (a.0.to_radians(), b.0.to_radians());
    let dla = (b.0 - a.0).to_radians();
    let dlo = (b.1 - a.1).to_radians();
    let h = (dla / 2.0).sin().powi(2) + la1.cos() * la2.cos() * (dlo / 2.0).sin().powi(2);
    2.0 * R * h.sqrt().asin()
}

/// Dijkstra: exact shortest path by total crossing time.
pub fn dijkstra(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>)>
{
    let mut dist = vec![u64::MAX; g.adj.len()];
    let mut prev = vec![u32::MAX; g.adj.len()];
    let mut heap = BinaryHeap::new();
    dist[source as usize] = 0;
    heap.push(Reverse((0u64, source)));
    while let Some(Reverse((d, u))) = heap.pop() {
        if u == target { break; }             // popped => finalized => optimal
        if d > dist[u as usize] { continue; } // stale heap entry
        for e in &g.adj[u as usize] {
            let nd = d + e.millis as u64;     // relax the edge
            if nd < dist[e.to as usize] {
                dist[e.to as usize] = nd;
```

```
                prev[e.to as usize] = u;
                heap.push(Reverse((nd, e.to)));
            }
        }
    }
    if dist[target as usize] == u64::MAX { return None; }
    let mut path = vec![target];              // walk predecessors back
    while *path.last().unwrap() != source {
        path.push(prev[*path.last().unwrap() as usize]);
    }
    path.reverse();
    Some((dist[target as usize], path))
}

/// A*: the same search, re-prioritized by an admissible heuristic —
/// remaining straight-line distance at the fastest plausible speed, so it
/// never overestimates (and, on a road graph, is *consistent*, which is what
/// licenses the early exit when the target is popped).
pub fn astar(g: &RoadGraph, source: NodeId, target: NodeId) -> Option<(u64, Vec<NodeId>)> {
    const MAX_SPEED_MPS: f64 = 70.0; // ~250 km/h: faster than any legal road
    let t = target as usize;
    let h = |n: NodeId| -> u64 {
        let i = n as usize;
        (haversine_m((g.lat[i], g.lng[i]), (g.lat[t], g.lng[t])) / MAX_SPEED_MPS * 1000.0)
as u64
    };
    let mut dist = vec![u64::MAX; g.adj.len()];
    let mut prev = vec![u32::MAX; g.adj.len()];
    let mut heap = BinaryHeap::new();
    dist[source as usize] = 0;
    heap.push(Reverse((h(source), 0u64, source))); // (priority, dist, node)
    while let Some(Reverse((_, d, u))) = heap.pop() {
        if u == target { break; }
        if d > dist[u as usize] { continue; }
        for e in &g.adj[u as usize] {
            let nd = d + e.millis as u64;
            if nd < dist[e.to as usize] {
                dist[e.to as usize] = nd;
                prev[e.to as usize] = u;
                heap.push(Reverse((nd + h(e.to), nd, e.to)));
            }
        }
    }
    if dist[t] == u64::MAX { return None; }
    let mut path = vec![target];
    while *path.last().unwrap() != source {
        path.push(prev[*path.last().unwrap() as usize]);
    }
    path.reverse();
    Some((dist[t], path))
}
```

And the test that demonstrates the chapter's slogan — **traffic is just edge weights that move**:

```
#[cfg(test)]
mod tests {
    use super::*;

    /// A tiny city: two ways from node 0 to node 4, edges ~1.1–1.6 km.
    ///   0 -> 1 -> 2 -> 4 :  60 s +  60 s +  60 s = 180 s
```

```
    ///   0 -> 3 -> 4      :  30 s + 120 s        = 150 s   (normally fastest)
    /// (Times are consistent with the A* speed cap of 70 m/s, so the
    /// haversine heuristic stays admissible on this graph.)
    fn tiny_city() -> RoadGraph {
        let mut adj = vec![Vec::new(); 5];
        let e = |adj: &mut Vec<Vec<Edge>>, from: u32, to: u32, millis: u32| {
            adj[from as usize].push(Edge { to, millis });
            adj[to as usize].push(Edge { to: from, millis }); // two-way streets
        };
        e(&mut adj, 0, 1, 60_000);
        e(&mut adj, 1, 2, 60_000);
        e(&mut adj, 2, 4, 60_000);
        e(&mut adj, 0, 3, 30_000);
        e(&mut adj, 3, 4, 120_000);
        RoadGraph {
            adj,
            lat: vec![0.0, 0.0, 0.01, -0.01, 0.0],   // rough grid around origin
            lng: vec![0.0, 0.01, 0.01, 0.01, 0.02],
        }
    }

    #[test]
    fn dijkstra_and_astar_agree() {
        let g = tiny_city();
        assert_eq!(dijkstra(&g, 0, 4).unwrap().0, 150_000);
        let (millis, path) = astar(&g, 0, 4).unwrap();
        assert_eq!(millis, 150_000);
        assert_eq!(path, vec![0, 3, 4]);
    }

    #[test]
    fn traffic_reroutes() {
        let mut g = tiny_city();
        // Congestion report: the 3 -> 4 segment now takes 600 s.
        g.adj[3].iter_mut().find(|e| e.to == 4).unwrap().millis = 600_000;
        g.adj[4].iter_mut().find(|e| e.to == 3).unwrap().millis = 600_000;
        // No code path changes — the "traffic-aware" router is the same
        // algorithm reading different weights.
        assert_eq!(dijkstra(&g, 0, 4).unwrap().1, vec![0, 1, 2, 4]);
    }
}
```

2.14.3 The traffic window: from GPS fixes to edge weights

The aggregator at the heart of Section 2.12: a per-segment sliding window of observed speeds, the live/historical blend, and the weight the graph stores.

```
use std::collections::VecDeque;

/// Per-segment sliding-window speed average (Section 2.12).
pub struct SpeedWindow {
    samples: VecDeque<(u64, f64)>, // (timestamp_secs, observed speed m/s)
    window_secs: u64,              // e.g. 15 * 60
}

impl SpeedWindow {
    pub fn new(window_secs: u64) -> Self {
        Self { samples: VecDeque::new(), window_secs }
    }
```

```
    pub fn add(&mut self, now: u64, speed_mps: f64) {
        self.samples.push_back((now, speed_mps));
        self.evict(now);
    }

    fn evict(&mut self, now: u64) {
        while let Some(&(t, _)) = self.samples.front() {
            if now.saturating_sub(t) > self.window_secs { self.samples.pop_front(); }
            else { break; }
        }
    }

    /// Current average, or None when there is too little live data —
    /// the caller then blends in the historical pattern (Section 2.12).
    pub fn average(&mut self, now: u64, min_samples: usize) -> Option<f64> {
        self.evict(now);
        if self.samples.len() < min_samples { return None; }
        let sum: f64 = self.samples.iter().map(|&(_, s)| s).sum();
        Some(sum / self.samples.len() as f64)
    }
}

/// Effective speed for a segment: live average where we trust it,
/// the historical pattern for this day/time otherwise, and never above
/// the posted limit (we do not route people as if speeding were planned).
pub fn effective_speed(live: Option<f64>, historical: f64, limit: f64) -> f64 {
    live.unwrap_or(historical).min(limit)
}

/// The edge weight the routing graph stores (Section 2.7): expected
/// crossing time. Clamped away from zero so weights stay finite and
/// Dijkstra's non-negativity requirement is never endangered.
pub fn weight_millis(segment_length_m: f64, speed_mps: f64) -> u32 {
    (segment_length_m / speed_mps.max(0.5) * 1000.0) as u32
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn window_evicts_old_samples() {
        let mut w = SpeedWindow::new(900);          // 15-minute window
        w.add(1_000, 30.0);                          // highway at full speed
        w.add(1_700, 8.0);                           // jam begins
        w.add(1_800, 9.0);
        // The t=1000 sample is now 800 s old: inside the window.
        assert_eq!(w.samples.len(), 3);
        w.add(2_000, 10.0);
        // t=1000 is 1000 s old: evicted. Average over the jam only.
        let avg = w.average(2_000, 3).unwrap();
        assert!((avg - 9.0).abs() < 0.01);
    }

    #[test]
    fn sparse_segments_fall_back_to_history() {
        let mut w = SpeedWindow::new(900);
        w.add(100, 22.0);                            // one car at 3 a.m.
        assert_eq!(w.average(200, 10), None);        // not enough samples
        // Historical pattern says 20 m/s here at this day/time; limit 25.
```

```
        assert_eq!(effective_speed(None, 20.0, 25.0), 20.0);
        // Live data says 30 m/s, but we never plan on speeding.
        assert_eq!(effective_speed(Some(30.0), 20.0, 25.0), 25.0);
    }

    #[test]
    fn weight_is_crossing_time() {
        assert_eq!(weight_millis(1_000.0, 10.0), 100_000); // 1 km at 10 m/s
        assert!(weight_millis(1_000.0, 0.0) > 0);          // clamped, finite
    }
}
```

2.14.4 The reroute decision

Section 2.13's state machine, reduced to its durable core: distance from the planned polyline, sustained across fixes. (Distance uses a local flat-earth approximation — within a few hundred meters of the route, curvature is noise; production systems run the full map-matching of Section 2.12 here.)

```
/// Approximate meters-per-degree at a given latitude: 111.32 km per degree
/// of latitude everywhere; longitude degrees shrink with cos(latitude).
fn meters_per_deg(lat: f64) -> (f64, f64) {
    (111_320.0, 111_320.0 * lat.to_radians().cos())
}

/// Distance from point p to segment a-b, in meters (local approximation).
fn point_segment_m(p: (f64, f64), a: (f64, f64), b: (f64, f64)) -> f64 {
    let (m_lat, m_lng) = meters_per_deg(p.0);
    let (px, py) = (p.1 * m_lng, p.0 * m_lat);
    let (ax, ay) = (a.1 * m_lng, a.0 * m_lat);
    let (bx, by) = (b.1 * m_lng, b.0 * m_lat);
    let (dx, dy) = (bx - ax, by - ay);
    let len2 = dx * dx + dy * dy;
    let t = if len2 == 0.0 { 0.0 } else {
        (((px - ax) * dx + (py - ay) * dy) / len2).clamp(0.0, 1.0)
    };
    let (cx, cy) = (ax + t * dx, ay + t * dy);   // closest point on a-b
    ((px - cx).powi(2) + (py - cy).powi(2)).sqrt()
}

/// Distance from a fix to the nearest point anywhere on the route polyline.
pub fn distance_to_route_m(p: (f64, f64), route: &[(f64, f64)]) -> f64 {
    route.windows(2)
        .map(|w| point_segment_m(p, w[0], w[1]))
        .fold(f64::MAX, f64::min)
}

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Guidance { OnRoute, OffRouteSuspected, Reroute }

/// The hysteresis filter: ONE noisy fix must never trigger a reroute
/// (Section 2.13's pitfall). `strikes` counts consecutive off-route fixes.
pub struct RerouteFilter {
    pub distance_threshold_m: f64, // e.g. 40 m
    pub required_consecutive: u8,  // e.g. 3 fixes
    strikes: u8,
}

impl RerouteFilter {
    pub fn new(distance_threshold_m: f64, required_consecutive: u8) -> Self {
        Self { distance_threshold_m, required_consecutive, strikes: 0 }
```

```
    }

    pub fn update(&mut self, fix: (f64, f64), route: &[(f64, f64)]) -> Guidance {
        if distance_to_route_m(fix, route) > self.distance_threshold_m {
            self.strikes += 1;
        } else {
            self.strikes = 0;
        }
        if self.strikes >= self.required_consecutive { Guidance::Reroute }
        else if self.strikes > 0 { Guidance::OffRouteSuspected }
        else { Guidance::OnRoute }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    // A straight eastbound route along lat 37.0.
    fn route() -> Vec<(f64, f64)> { vec![(37.0, -122.0), (37.0, -121.9)] }

    #[test]
    fn on_route_fix_stays_quiet() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0001, -121.95), &route()), Guidance::OnRoute);
    }

    #[test]
    fn sustained_deviation_reroutes() {
        let mut f = RerouteFilter::new(40.0, 3);
        // ~56 m north of the route: beyond threshold.
        let lost = (37.0005, -121.95);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update(lost, &route()), Guidance::Reroute);
    }

    #[test]
    fn one_ghost_fix_recovers() {
        let mut f = RerouteFilter::new(40.0, 3);
        assert_eq!(f.update((37.0005, -121.95), &route()), Guidance::OffRouteSuspected);
        assert_eq!(f.update((37.0, -121.95), &route()), Guidance::OnRoute);
    }
}
```

2.15 Scaling & Geographic Sharding

Chapter 1 defined sharding; here the shard key is **geography itself**.

Tier	Sharding strategy	Why
Tiles	No sharding logic at all — the `(z,x,y)` address space partitions naturally; CDN edges cache by URL	Addressing **is** the partition scheme; popularity skew does the rest
Road graph	Partitioned by region (continent → subregion); each shard is a full in-	A route is overwhelmingly inside one region; replicas add both capacity and failover

Tier	Sharding strategy	Why
	memory copy of its region's CH graph, replicated	
Cross-region routes	Solved hierarchically: mega-segments cross region borders at a handful of highway gateways	The hierarchy of Section 2.8 doubles as the distributed-systems answer
GPS stream	Topic partitioned by geohash prefix; each stream-processor instance owns a disjoint set of cells	Map-matching and aggregation are embarrassingly parallel over geography
Traffic cache	Sharded by `segment_id` hash; total size 500 MB	Tiny, hot, and latency-critical — keep it in memory, close to routing
Place index	Replicated per region; sharded by quadkey prefix within a region	Search is read-heavy and geographically scoped (Section 2.14)

KEY INSIGHT — Geography is a forgiving shard key

Unlike user data (Chapter 1's documents), geography does not move, grow legs, or go viral. A region's road graph changes on the scale of weeks; its traffic weights change on the scale of minutes but are **regenerated**, not migrated. The one genuinely hot spot — a stadium emptying at 11 p.m. — is absorbed inside one partition by the aggregation pipeline, not spread across the system.

2.16 Failure Modes & Recovery

Failure	Handling
WS gateway dies	Clients reconnect to any gateway; the connection manager re-registers them; the navigation service resumes pushes to the new socket. Unacked client updates are retried and deduplicated (Chapter 1's idempotency discipline).
Navigation node dies	Sessions are small (route + progress). The client's reconnect carries its last known position; a fresh node rebuilds the session from the route store and continues. User sees a pause, not a crash.
Stream backpressure / Kafka lag	Freshness degrades first and **visibly**: per-segment entries age past their TTL, and the system slides to historical speeds automatically — the fallback of Section 2.12 is the degradation mode, by design. Recovery replays the log.
Traffic cache loss	Rebuilt from the stream within one window (15 min); historical speeds serve in the meantime.
Routing node dies	Each region shard is replicated; traffic shifts to a replica. A lost node reloads its graph from the segment store plus the traffic cache (Section 2.10).
Tile origin down	The CDN keeps serving — 95%+ of requests never noticed the origin existed. `stale-while-revalidate` semantics make even expired tiles safe, since the base map changes weekly at worst.

Failure	Handling
Bad or spoofed GPS input	Per-segment sample thresholds ignore singletons; per-source scoring quarantines systematically wrong feeds (Section 2.12).
Full region outage	The phone is the last fallback: cached route, tiles, and step list keep guiding offline (Section 2.13) while sessions re-home to another region.

2.17 Trade-offs & Alternatives

Decision	Benefit purchased	Cost accepted
Vector tiles over raster	50% fewer bytes; restyle without re-render; smooth zoom	Client GPU does the work; a raster fallback must exist for old clients
CH / mega-segments over plain A*	1000× query speedup; the only way 20k QPS fits a small fleet	Hours of preprocessing; shortcut weights must be rebuilt or bubbled-up when traffic moves — extra machinery (Section 2.8)
WebSocket push for navigation	Reroutes and ETA updates arrive instantly; one connection also carries GPS upstream	Millions of long-lived connections to hold and to register (connection manager)
Hybrid live + historical traffic	Coverage everywhere, graceful under sparse data and pipeline lag	Two data paths to keep consistent; blending logic to get right
Quadkey/geohash indexing	Proximity becomes a string-prefix range scan in ordinary KV stores	Cell-boundary artifacts — always query the neighbor ring too
Precompute over on-demand render	Tiles and hierarchy make reads O(cache hit)	A planet of batch jobs to run, version, and roll back

2.18 Observability & SLOs

DEFINITION — MAPE (mean absolute percentage error)

The average of `|predicted − actual| / actual` over many predictions — the standard accuracy metric for forecasts, and the natural SLI for ETA quality. Per **completed** trip, we know both the ETA we gave and the realized travel time, so ETA MAPE is measurable exactly, per city, per hour, per route class. The source talk is emphatic on this point: you cannot improve what you do not score, and ETA accuracy is the metric this product lives by.

What we instrument, at minimum: **ETA error distribution** (predicted vs. realized, per region — the chapter's headline SLI, targeting the ±10% of Section 2.4); **route acceptance rate** (how often users actually drive the recommended route — a recommendation nobody takes is a signal something is wrong with the routes, per the talk's analytics discussion); reroute rate per session; tile cache hit ratio

and origin QPS; route latency p50/p99; pipeline lag (fix → weight update); traffic freshness (median age of live segment speeds); navigation session drops and recovery times.

2.19 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *"Offline maps?"* — Ship regional bundles (tiles + contracted graph) to the device; routing runs locally; traffic and reroute quality silently degrade until reconnect. Everything we precomputed server-side is exactly what the bundle contains.
- *"Public transit?"* — A **time-dependent** graph (edges exist only when a vehicle does); Dijkstra generalizes, but production transit uses schedule-based algorithms (RAPTOR and friends). Name it, don't build it.
- *"GPS noise in urban canyons?"* — The HMM map-matching of Section 2.12, plus sensor fusion (accelerometer, gyroscope, wheel ticks where available).
- *"Better ETAs?"* — Features into a learned model: current and historical speeds, weather, events, turn and signal penalties; graph neural networks model congestion **spreading** along the road graph. Google reported up to 50% ETA-error reductions in some cities from exactly this move — say the number as motivation, then defend the simple hybrid as the baseline it must beat.
- *"Walking and cycling?"* — Same graph, same algorithms, different edge eligibility and weights (stairs, bike lanes, no highways). The design's generality is the answer.
- *"Location privacy?"* — Aggregate, don't store: segment statistics instead of traces, retention limits on raw updates, minimum sample counts that double as k-anonymity for sparse segments.

If you remember five things:

1. The map is a precomputed pyramid of addressable squares; the CDN, not your fleet, serves it.
2. Roads are a time-weighted directed graph, and **traffic is just edge weights that move**.
3. Dijkstra is exact and unusable; bidirectional and A* are free wins; hierarchy (mega-segments / contraction hierarchies) is the 1000× that makes it a product.
4. Live traffic is a streaming pipeline: GPS fix → map-match → per-segment windowed average → weights and overlay tiles, with history as the fallback.
5. Navigation correctness lives on the phone: the client caches the route and survives the server.

2.20 Summary & Further Reading

We designed a maps and navigation service for 1B monthly users: a pre-rendered, CDN-served tile pyramid addressed by `(z, x, y)` and quadkeys; a time-weighted road graph held in memory and searched with an algorithm ladder that ends at contraction hierarchies; a live traffic loop that turns 3M GPS updates per second into per-segment speeds, edge weights, and overlay tiles, with historical patterns as the fallback; and a turn-by-turn navigation service that guides, refreshes ETAs, and reroutes with hysteresis — degrading to the phone when the network or we fail.

Primary source for this chapter:

- **"13: Google Maps" — Systems Design Interview Questions With Ex-Google SWE (Jordan has no life)** — the walkthrough this chapter expands.

Foundations worth reading:

- The OSRM and Valhalla open-source routing engines — contraction hierarchies in production code, not just papers.
- Geisberger et al., *Exact Routing in Large Road Networks Using Contraction Hierarchies* (2008) — the CH formulation.
- Newson & Krumm, *Hidden Markov Map Matching Through Noise and Sparseness* (2009) — the map-matching standard.

- Google's S2 geometry library documentation, and Uber's H3 — the real spatial indexes behind quadkey-style addressing.
- The Bing Maps tile system documentation — the canonical quadkey reference.
- Google DeepMind's write-up on learning ETAs with graph neural networks (2020) — the ML direction for Section 2.19's follow-up.

2.21 Chapter 2 Glossary

A one-glance index of every term this chapter defined. Chapter 1's glossary is assumed; later chapters assume both.

Term	Meaning in one line
ETA	Predicted travel time / arrival time; the quality bar of this chapter
Freshness	Age of the data behind an answer; traffic targets minutes
Map tile	Fixed 256×256 square of map; the unit of storage, cache, transfer
Zoom level	Scale integer *z*; each step splits every tile into four
Slippy-map scheme	Deterministic (z, x, y) tile addressing over Web Mercator
Quadkey	Tile address as a digit string; a quadtree path, prefix-searchable
CDN	Planet-wide edge caches that absorb static reads near the user
TTL / hit ratio	How long cache entries may live / fraction of requests they absorb
Content addressing	Identical tiles (oceans) stored once, by content hash
Polyline encoding	Compact ASCII encoding of a route's coordinate sequence
Graph	Nodes + edges; directed and weighted for roads
Segment	One directed road piece between nodes; the unit of traffic
Adjacency list	Per-node outgoing-edge lists; the in-memory graph layout
Shortest path	Minimum-total-weight route; with time weights, the fastest path
Dijkstra's algorithm	Exact breadth-by-distance search with a priority queue
Priority queue	O(log n) "give me the current minimum" structure
Bidirectional search	Two half-searches meeting mid-route; half the work
A* / heuristic	Dijkstra steered by a remaining-distance estimate
Admissible heuristic	Never overestimates; preserves exactness
Haversine distance	Great-circle distance between coordinates
Mega-segment	Precomputed synthetic edge summarizing a segment chain
Contraction hierarchy	Node-importance ordering + shortcuts; µs–ms queries
Kafka-class stream	Durable partitioned event log; decouples and replays
Stream processing	Computing over data as it arrives, not in batch
Map-matching	Snapping noisy GPS fixes to the true road segment
Sliding window	Moving time interval defining "current" speed per segment
Polyglot persistence	Different engines for different access patterns

Term	Meaning in one line
Connection manager	Registry of which gateway holds which user’s socket
MAPE	Mean absolute percentage error; the ETA accuracy SLI

CHAPTER 3

Designing a Distributed Rate Limiter

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"7: Design a Rate Limiter"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs a rate-limiting service for a public API: why limiting is a business feature and not just a shield, the classic limiting algorithms, and how to enforce limits correctly across a fleet of stateless servers. This chapter follows the same arc, deepened with full definitions, capacity mathematics, protocol details, and Rust reference implementations.

3.1 The Problem Statement

The interviewer looks up and says:

> *"Design a rate limiter. Our public API is used by a million developers, and we need to make sure no single caller can overwhelm the service — or use more than their plan allows."*

Chapters 1 and 2 designed user-facing products. This chapter designs a piece of **infrastructure** — a component that sits in front of every request and decides, in well under a millisecond, whether the request may proceed. The difficulty is not the concept ("count requests, reject past a threshold" is a five-line program on one machine). The difficulty is doing it **correctly and cheaply on the hot path of a distributed system**: fifty gateway servers must enforce one shared limit per caller, under concurrency, without meaningful latency, and without the limiter itself becoming the outage it exists to prevent.

DEFINITION — Rate limiting / throttling

Controlling the **rate** at which a caller may invoke a system: at most n requests per time window per identity (API key, user, IP address, tenant). Calls beyond the limit are rejected (or queued, or slowed — hence *throttling*). Rate limiting serves three distinct masters: **protection** (abuse, brute force, accidental stampede), **capacity** (one noisy caller must not starve the rest), and **monetization** (usage tiers are limits with a price tag). Keep all three in mind: they produce different requirements.

DEFINITION — Quota vs. rate limit

A *quota* is a budget over a long period ("100,000 calls per month") used for billing and plan enforcement; it tolerates approximate, batched accounting. A *rate limit* governs short windows ("100 calls per second") to shape live traffic; it must be enforced **now**, on the request's critical path. This chapter is about rate limits; Section 3.18 discusses how quotas differ.

3.2 Scope & Clarifying Questions

The prompt spans everything from login-page brute-force protection to planetary DDoS absorption. Narrow it:

Speaker	Dialogue
Candidate	"What are we limiting — HTTP API calls, or something else like messages or bytes?"
Interviewer	"HTTP requests to our public REST API. Limit per API key."
Candidate	"Are limits the same for everyone, or tiered — free keys vs. paid plans?"
Interviewer	"Tiered. Rules must be configurable per key and per route, and changeable without redeploying anything."
Candidate	"Scale of the API being protected?"
Interviewer	"Peak 200,000 requests per second, spread across a fleet of about 50 stateless gateway servers. One million registered API keys."
Candidate	"How exact must enforcement be? If a key is limited to 100 requests per second, is 101 a scandal?"
Interviewer	"Never reject a caller who is under their limit. Small overshoot during failures is acceptable; systematic overshoot is not."
Candidate	"How much latency may the check add to each request?"
Interviewer	"A millisecond or two at most — this runs on every single request."
Candidate	"And if the limiter's own state store is down — block everything, or let traffic through?"
Interviewer	"Great question. Decide, and defend the decision."

AGREED SCOPE

Per-API-key rate limits on a public REST API; tiered limits, configurable per key and per route, hot-reloadable; enforced **globally** across 50 stateless gateway nodes at **200k RPS** peak; ≤1–2 ms added latency; **no false rejections**, bounded overshoot; and an explicit, defended policy for limiter failure (fail-open vs. fail-closed — Section 3.15).

INTERVIEW TIP — The last question is the interview

"What do we do when the limiter breaks?" separates candidates who build components from candidates who build **systems**. Ask it yourself if the interviewer doesn't — every subsequent design decision (centralized store, local fallbacks, timeouts) is downstream of the answer.

3.3 Functional Requirements

Chapter 1 defined functional requirements; ours:

1. **FR-1 — Enforcement.** A caller exceeding its limit receives HTTP **429 Too Many Requests** with a `Retry-After` hint; everyone else passes through.
2. **FR-2 — Configurable rules.** Limits are defined per API key, optionally per route or method, with named tiers (free, pro, enterprise); an admin API changes them at runtime.

3. **FR-3 — Transparency.** Responses carry the caller's limit state (limit, remaining, reset time) in standard headers, so well-behaved clients can self-throttle **before** being rejected.
4. **FR-4 — Global enforcement.** The limit holds across the entire gateway fleet, not per server — a caller with 100 req/s cannot get 5,000 req/s by being load-balanced across 50 nodes.
5. **FR-5 — Burst policy.** Short bursts above the sustained rate are allowed up to a defined allowance (real traffic is bursty; rejecting every microburst punishes normal clients).
6. **FR-6 — No false rejections.** A caller under its limit is never rejected. (Overshoot is a soft failure; false rejection is a hard one — it breaks innocent customers deterministically.)

3.4 Non-Functional Requirements

Three qualities dominate, and they pull against each other — naming the tension is half the answer:

Quality	Target
Added latency	≤ 1 ms in-region, p99 — the check is on **every** request's critical path
Accuracy	Zero false rejections; steady-state overshoot within 1% of the limit; degraded-mode overshoot explicitly bounded
Availability of the API	The limiter must never be a new single point of failure: if the limiter fails, the API stays up (fail-open), with a local safety cap
Scale	200k RPS enforcement across 50 nodes; 1M keys; 100M+ active keys per day tolerable
Config freshness	Rule changes propagate to all gateways within seconds

DEFINITION — False rejection / overshoot

The two ways a limiter can be wrong. A *false rejection* denies a caller who is under the limit — an availability bug, visible and unforgivable. *Overshoot* allows more than the limit — a protection degradation, tolerable in small doses. Accuracy requirements should always be stated in this asymmetric vocabulary, because the two errors have opposite costs and different causes.

KEY INSIGHT — Latency, accuracy, availability: pick the trade-off consciously

Exact global counting wants a synchronous round trip to a strongly consistent store (latency + availability cost). **Zero added latency** wants purely local counting (accuracy cost: 50 nodes × local limit). **Maximum availability** wants fail-open everywhere (protection cost). Every real design is a point in this triangle; the rest of the chapter locates ours and prices it.

3.5 Back-of-the-Envelope Estimation

Assumptions (stated, per Chapter 1's discipline):

- Peak gateway traffic: **200k requests/sec**; every request needs exactly one limit check.
- 1M registered keys; up to **100M keys active in a day** (many keys are used once and idle for weeks).
- Typical limit: 100 req/s per key; token-bucket state per active key is two numbers plus the key itself — 50–60 bytes.
- A modern in-memory store serves 100k simple ops/sec per shard.

Derived numbers:

Quantity	Estimate	How
Check operations	200k ops/s peak	one per request; reads+writes combined
Store shards needed	4–6 (+ replicas)	200k ops/s ÷ 100k ops/s per shard, with headroom
State memory (counters)	≈ 6 GB worst case	100M active keys × 60 B
State memory (sliding log)	≈ 800 GB worst case	100M keys × up to 1,000 time-stamps × 8 B — **infeasible**
Added latency budget	≤ 1 ms	one in-region round trip, or zero with local state
Config size	a few MB	1M keys × rule record; trivially cacheable everywhere

KEY INSIGHT — The scarce resource is round trips, not hardware

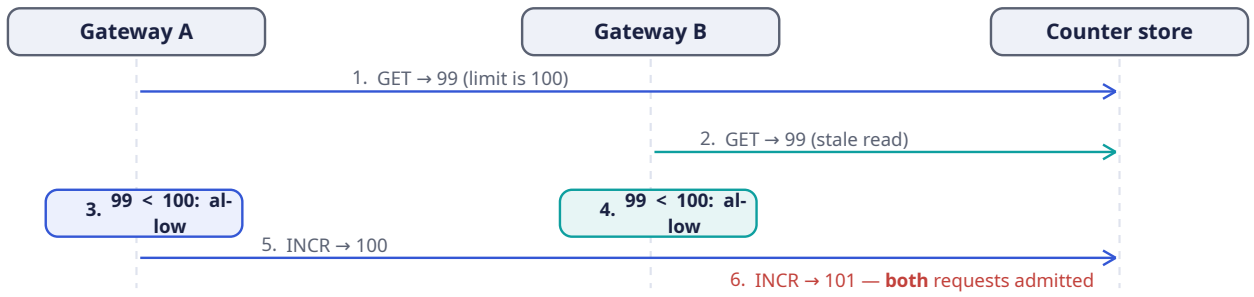
Six gigabytes of state and a few hundred thousand ops per second are, by Chapter 2's standards, nothing. The entire design problem is: **where does the check happen relative to the request?** A centralized exact check costs one network round trip (1 ms in-region — the whole budget). A local check costs zero round trips but is approximate. Estimation tells you this is a **latency topology** problem, not a capacity problem — design accordingly.

3.6 The Core Challenge: Counting Correctly Under Concurrency

The limit check is a read-modify-write: **read** the caller's count, **decide**, **write** the incremented count. Two naive placements fail in instructive ways.

Naive strategy 1 — count locally on each gateway. No shared state, zero added latency. But the limit is **per key**, and a key's requests land on all 50 gateways. If each node enforces 100 req/s locally, the caller effectively has **5,000 req/s** — FR-4 is unmet. Lowering each node's limit to 100/50 = 2 req/s misfires the other way: a caller whose traffic happens to hit one node gets strangled. Per-instance limits cannot express a global limit.

Naive strategy 2 — a shared counter store with plain reads and writes. Put the counters in one shared, in-memory store; each gateway does `GET`, compare, `INCR`. This fails under concurrency:



DEFINITION — Race condition / check-then-act (TOCTOU)

A *race condition* is a bug whose outcome depends on the uncontrolled interleaving of concurrent operations. The specific species above is **time-of-check to time-of-use**: the decision ("count is 99, under the limit") is based on state that another actor changes before our write lands. Both gateways checked

honestly; both were correct **at check time**; the limit was still exceeded. No amount of retries fixes a TOCTOU hole — the check and the update must become **one indivisible operation**.

DEFINITION — Atomicity / compare-and-swap (CAS)

An operation is *atomic* if it executes entirely or not at all, with no observable intermediate state. *Compare-and-swap* is the primitive form: "set this value to V_2 only if it currently equals V_1," performed as one hardware/server-side step. Our options for making the check atomic: a server-side script executed atomically by the store (Section 3.12), a CAS loop, or a single atomic increment whose **returned value** is the decision input. The last two need no scripting at all — `INCR` already returns the post-increment count, which **is** the atomic observation we need.

So the core challenge decomposes into two questions that the rest of the chapter answers: **what exactly is the count** (which algorithm — Section 3.7), and **how is the check made atomic across the fleet** (Section 3.12).

3.7 The Algorithm Zoo

Five algorithms cover the entire design space. Each is a different answer to "what does 'at most *n* per window' mean, exactly?"

DEFINITION — Fixed window counter

Divide time into fixed windows (e.g., each clock minute) keyed `rl:{key}:{window_id}`; each request increments the current window's counter; reject when it exceeds the limit. O(1) memory per key, one atomic increment per request — and a famous flaw: **the boundary burst**. A caller can fire 100 requests at 0:59.9 and 100 more at 1:00.0 — 200 requests in 0.1 seconds, all legal, because the two bursts sit in different windows.

DEFINITION — Sliding window log

Store the **timestamp of every request** in a per-key log; on each request, drop timestamps older than the window and reject if the remaining count reaches the limit. Perfectly accurate, no boundary problem — but memory is O(limit) per key. Section 3.5 priced this at 800 GB at our scale: the log is the algorithm you describe to show you know the trade-off, not the one you ship.

DEFINITION — Sliding window counter

The fixed-window/log hybrid. Keep the current and previous window counters only, and estimate the sliding window as:

```
estimate = current_count + previous_count × (1 − elapsed / window)
```

If 15 seconds of a 60-second window have elapsed, the previous window's 84 requests count as `84 × 0.75 = 63`. O(1) memory, one increment per request, no boundary burst — at the price of being an **estimate** (it assumes the previous window's traffic was spread evenly, which is fair on aggregate; published analyses put the misclassification rate well under 1%).

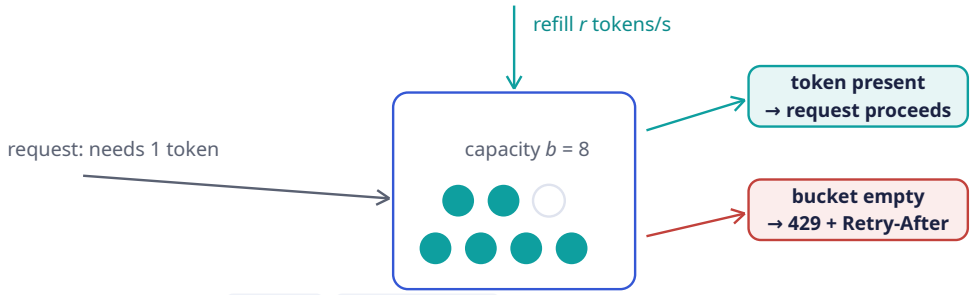
DEFINITION — Token bucket

Each key owns a bucket holding up to *b* tokens, refilled continuously at *r* tokens per second. A request takes one token; an empty bucket rejects. The parameters map directly onto product language: *r* is the sustained rate, *b* is the burst allowance (FR-5). O(1) memory (two numbers: tokens, last-refill

timestamp), and refill is computed **lazily** on each request — no timers, no background jobs. This is the industry's default for API limiting, and our choice; Section 3.11 deep-dives it and Section 3.13 implements it.

DEFINITION — Leaky bucket / GCRA

Requests enter a conceptual queue and "leak" out at a fixed rate; a request is admitted only if the queue has room. Where the token bucket **permits** bursts, the leaky bucket **forbids** them — output is perfectly smooth. Its stateless formulation is the **Generic Cell Rate Algorithm**: store one theoretical-arrival-time per key; admit if now is not earlier than that time minus tolerance. The right tool when you meter **into** a fragile downstream (payment gateways, SMS providers), overkill for request admission.



Two numbers per key — `tokens`, `last_refill` — and no timers: refill is computed when a request arrives.

The comparison that ends the discussion:

Algorithm	Memory/key	Exact?	Bursts?	Notes
Fixed window	O(1)	window-granular	2× at boundary	Simplest; the boundary burst is the interview trap
Sliding log	O(limit)	exact	no	Memory kills it at scale (Section 3.5)
Sliding counter	O(1)	1% error	smoothed	Best accuracy-per-byte when bursts are unwanted
Token bucket	O(1)	exact accounting	**allowed** up to *b*	**Our choice**: burst policy is a product feature here
Leaky / GCRA	O(1)	exact pacing	forbidden	Choose when downstream needs smooth flow

3.8 API & Protocol Design

A rate limiter's "API" is mostly **other people's responses**. The contract that matters:

DEFINITION — HTTP 429 / Retry-After

Status **429 Too Many Requests** (RFC 6585) means "you, specifically, are calling too fast." The `Retry-After` header tells the caller how many seconds to wait before retrying. Together they convert

> a rejection into a negotiation: a well-built client backs off exactly as instructed instead of hammering harder. A limiter that rejects without `Retry-After` trains clients to retry blindly — worse for everyone.

Header	Meaning
`X-RateLimit-Limit`	The caller's limit for this route, e.g. `100`
`X-RateLimit-Remaining`	Requests left in the current window / tokens left
`X-RateLimit-Reset`	When the budget refills (epoch seconds or seconds-from-now)
`Retry-After`	On 429 only: minimum wait before retry, seconds

(An IETF draft standardizes these as `RateLimit-*` fields; the `X-` forms remain the de-facto convention. Either is defensible — knowing both exist is the point.)

A rejection response, end to end:

```
HTTP/1.1 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 45
Retry-After: 1
Content-Type: application/json

{ "error": "rate_limit_exceeded", "limit": 100, "window": "1s", "plan": "free" }
```

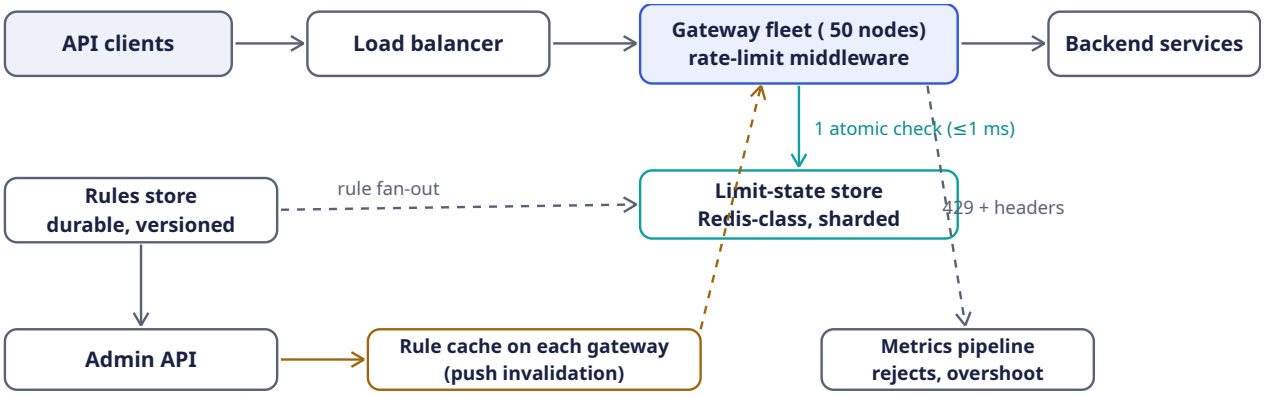
The **admin** surface (FR-2, FR-6) is ordinary REST: rules as records — `{ api_key or tier, route pattern, algorithm, limit/rate, burst, window }` — created and updated through an admin API, versioned, and pushed to every gateway within seconds (Section 3.10).

3.9 Data Model & Storage

Entity	Contents	Store
Limit state	`rl:{api_key}:{route}` → `{tokens, last_refill_ms}` (bucket) or window counters; TTL = window so idle keys self-clean	In-memory store cluster (Redis-class), sharded by key hash
Rule	tier or key → {algorithm, rate, burst, window, route pattern}; versioned	Durable KV/relational store, fanned out to gateway-local caches
Quota ledger	monthly usage per key for billing	Append-only log → batch aggregation; **not** on the request path (Section 3.2)

Two decisions to name. First, state entries carry a **TTL** (Chapter 2) equal to the window, so the 100M-active-keys problem is self-cleaning: idle keys vanish without a garbage collector. Second, rules are cached **on each gateway** with push-based invalidation — a per-request rule lookup must never add a second network hop.

3.10 High-Level Architecture



The middleware is a read-modify-write on the hot path: one atomic store op per request, rules read locally.

Component	Responsibility	Why it is shaped this way
Rate-limit middleware	First code every request touches on the gateway: load rule → atomic check → pass or 429	Middleware placement means **no backend code changes** and one consistent policy for every route
Limit-state store	Atomic counters/buckets for all active keys	One shared store is what makes the limit **global** (FR-4); sharded by key hash, replicated for availability
Rules store + admin API	Versioned rule records; runtime updates	Config is tiny and read-hot: push it to gateway memory, never fetch per request
Gateway rule cache	Rules in process memory, invalidated by push	Zero added hops for config (Section 3.9)
Metrics pipeline	Every decision emitted as an event	Section 3.17: you cannot tune what you cannot see

> **KEY INSIGHT — The one-op rule**
>
> Per request, the design allows itself **exactly one** atomic store operation and zero synchronous config reads. Everything — the algorithm choice (O(1) state), the TTL self-cleaning, the gateway rule caches — exists to preserve that invariant. When an interviewer pushes on any component, return to it.

3.11 Deep Dive: The Token Bucket, Precisely

The token bucket's elegance is that **time is never advanced by a timer**. State is two numbers, updated lazily when a request arrives:

1. `tokens` — current balance, capped at capacity *b*.
2. `last_refill_ms` — when the balance was last recomputed.

On each request at time `now`:

```
elapsed_s   = (now - last_refill_ms) / 1000
tokens      = min(b, tokens + elapsed_s * r)   // lazy refill, capped
```

```
last_refill = now
if tokens >= 1: tokens -= 1; ALLOW (remaining = floor(tokens))
else:           DENY  (retry_after_ms = (1 - tokens) / r * 1000)
```

Reading the parameters as product language: r = sustained rate ("100 req/s"), b = burst allowance ("… with bursts up to 150"). The retry hint is free: `retry_after` is exactly how long until one token exists.

A worked trace (limit 5 req/s, burst 8), to have in hand at the whiteboard:

Time	Event	Balance after	Why
`t = 0.0 s`	8 requests arrive together	8 → 0	Full bucket: all 8 pass — the burst allowance
`t = 0.0 s`	9th request	0	Empty: 429, `Retry-After: 1` (1/5 s until one token)
`t = 1.0 s`	1 request	5 → 4	One second refilled 5 tokens
`t = 3.0 s`	1 request	8 → 7	Two more seconds refill 10, capped at capacity 8

COMMON PITFALL — Refill by background timer

Implementing refill as a cron or timer per key means a million timers, clock drift between timer and request paths, and state that never cleans itself up. Lazy refill has none of these: no timers, no drift (one clock, read once per request), and a TTL on the key reclaims idle state for free. If you find yourself scheduling work per key, you have re-invented the sliding log's costs inside the token bucket.

NOTE — One clock, and make it monotonic

Distributed machines disagree about wall-clock time (clock skew), and even on one machine the wall clock can jump backwards (NTP corrections) — a backward jump would **refund** tokens. Two defenses: (1) perform the bucket update **inside the store** with a server-side script, so one clock governs every gateway (Section 3.12); (2) where local time is unavoidable, read a **monotonic** clock (one that never moves backwards), as Section 3.13's Rust does.

3.12 Deep Dive: Distributed Enforcement & the Race

Section 3.6's race came from splitting **check** and **update** across a network. Three production-grade fixes, in increasing order of sophistication:

Fix 1 — atomic increment-as-decision. For window algorithms, one atomic `INCR` already returns the post-increment value: the store's returned count **is** the check. Reject when the returned value exceeds the limit. One round trip, no client-side race, and the first increment of a window sets the key's TTL in the same atomic step. Fixed window and sliding counter ship this way.

Fix 2 — server-side script for read-modify-write. Token bucket needs read-compute-write (refill, compare, decrement). Executing that sequence **on the store**, as one atomic script, removes the race identically: the store serializes script execution, so no two requests ever observe the same balance. Section 3.13 shows the client side, with the script embedded as data.

Fix 3 — approximate local counting. The radical option: skip the store on the hot path entirely. Each gateway keeps local counters and periodically **publishes** them; a background aggregator broadcasts fleet-wide totals, and each node denies when `local_estimate + fleet_total ≥ limit`. Zero added latency, and the store is off the critical path (fail-open for free) — but between syncs, overshoot is

bounded by roughly `limit × (sync_lag / window)`. Choose it when protection matters more than precision (abuse mitigation); never for billing-adjacent limits.

Approach	Latency	Accuracy	Choose when
Centralized, atomic op	+1 RTT (1 ms)	exact	Default. Paid tiers, contractual limits
Centralized, non-atomic	+1 RTT	races under concurrency	Never — Section 3.6
Local + async sync	0	bounded overshoot	Abuse protection at extreme scale, or store-failure fallback
Sticky routing + local	0	near-exact	Only if your load balancer can pin keys to nodes — fragile on failover

INTERVIEW TIP — Say the quiet part: exactness is a product decision

"How exact does this need to be?" has no technical answer — it depends on whether the limit protects revenue (exact: paid tiers), infrastructure (approximate fine: abuse), or other users' experience (approximate fine: fairness). Volunteering this framing turns an algorithm comparison into a senior-level requirements discussion.

3.13 Deep Dive: Rust Reference Implementations

Three pieces, all executable: the token bucket with deterministic-time tests, the fixed-window/sliding-counter pair showing the boundary burst and its cure, and the atomic store check with the server-side script embedded as data.

3.13.1 Token bucket with a testable clock

Time is injected through a `Clock` trait so tests control it exactly — the difference between a test that **suggests** correctness and one that **proves** it.

```rust
use std::sync::Mutex;

/// Time source, injected so tests can control it (Section 3.11).
pub trait Clock: Send + Sync {
    fn now_ms(&self) -> u64;
}

/// Production clock: monotonic — it can never jump backwards and refund
/// tokens (Section 3.11's note).
pub struct MonotonicClock(std::time::Instant);
impl MonotonicClock {
    pub fn new() -> Self { Self(std::time::Instant::now()) }
}
impl Clock for MonotonicClock {
    fn now_ms(&self) -> u64 { self.0.elapsed().as_millis() as u64 }
}

/// The decision every algorithm returns: pass, or reject with a retry hint
/// that becomes the Retry-After header (Section 3.8).
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub enum Decision {
```

```
    Allow { remaining: u64 },
    Deny { retry_after_ms: u64 },
}

/// Token bucket (Section 3.7): `capacity` tokens, refilled lazily at
/// `refill_per_sec`. The Mutex makes the read-modify-write atomic *within*
/// one process; Section 3.12 handles the fleet-wide version.
pub struct TokenBucket<C: Clock> {
    capacity: f64,
    refill_per_sec: f64,
    state: Mutex<(f64, u64)>, // (tokens, last_refill_ms)
    clock: C,
}

impl<C: Clock> TokenBucket<C> {
    pub fn new(capacity: f64, refill_per_sec: f64, clock: C) -> Self {
        let now = clock.now_ms();
        Self { capacity, refill_per_sec,
               state: Mutex::new((capacity, now)), clock }
    }

    pub fn try_acquire(&self) -> Decision {
        let now = self.clock.now_ms();
        let mut s = self.state.lock().unwrap();
        let elapsed_s = (now - s.1) as f64 / 1000.0;
        let mut tokens = (s.0 + elapsed_s * self.refill_per_sec).min(self.capacity);
        s.1 = now;
        if tokens >= 1.0 {
            tokens -= 1.0;
            s.0 = tokens;
            Decision::Allow { remaining: tokens as u64 }
        } else {
            s.0 = tokens;
            let wait_ms = ((1.0 - tokens) / self.refill_per_sec * 1000.0).ceil() as u64;
            Decision::Deny { retry_after_ms: wait_ms.max(1) }
        }
    }
}
```

Tests, including the concurrency proof that `Mutex` buys us exactly-once accounting across threads:

```
#[cfg(test)]
mod tests {
    use super::*;
    use std::sync::{Arc, atomic::{AtomicU64, Ordering}};

    /// A clock the test controls explicitly.
    struct TestClock(AtomicU64);
    impl Clock for TestClock {
        fn now_ms(&self) -> u64 { self.0.load(Ordering::SeqCst) }
    }

    #[test]
    fn burst_then_reject_then_refill() {
        let clock = Arc::new(TestClock(AtomicU64::new(0)));
        let b = TokenBucket::new(8.0, 5.0, clock.clone()); // 8 burst, 5/s
        for i in 0..8 {
            assert!(matches!(b.try_acquire(), Decision::Allow { .. }), "burst #{i}");
        }
        // Bucket empty: denied, and told to wait 1/5s for one token.
```

```
        assert_eq!(b.try_acquire(), Decision::Deny { retry_after_ms: 200 });
        // One second passes: 5 tokens return (capped at capacity).
        clock.0.store(1000, Ordering::SeqCst);
        assert_eq!(b.try_acquire(), Decision::Allow { remaining: 4 });
    }

    #[test]
    fn exactly_capacity_across_threads() {
        let clock = Arc::new(TestClock(AtomicU64::new(0)));
        let b = Arc::new(TokenBucket::new(100.0, 1.0, clock));
        let allowed = Arc::new(AtomicU64::new(0));
        std::thread::scope(|s| {
            for _ in 0..8 {
                let (b, allowed) = (b.clone(), allowed.clone());
                s.spawn(move || {
                    for _ in 0..25 {   // 8 threads x 25 = 200 attempts
                        if matches!(b.try_acquire(), Decision::Allow { .. }) {
                            allowed.fetch_add(1, Ordering::SeqCst);
                        }
                    }
                });
            }
        });
        assert_eq!(allowed.load(Ordering::SeqCst), 100); // never one more
    }
}
```

3.13.2 Fixed window, its boundary burst, and the sliding counter's cure

```
/// Fixed window counter (Section 3.7): one counter per window per key.
pub struct FixedWindow {
    window_ms: u64,
    count: u64,
    window_start_ms: u64,
}

impl FixedWindow {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, count: 0, window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms;   // window start
        if w != self.window_start_ms {                      // new window
            self.window_start_ms = w;
            self.count = 0;
        }
        self.count += 1;
        self.count <= limit
    }
}

/// Sliding window counter: previous window, weighted by overlap (Section 3.7).
pub struct SlidingWindowCounter {
    window_ms: u64,
    prev_count: u64,
    curr_count: u64,
    curr_window_start_ms: u64,
}
```

```
impl SlidingWindowCounter {
    pub fn new(window_ms: u64) -> Self {
        Self { window_ms, prev_count: 0, curr_count: 0, curr_window_start_ms: 0 }
    }

    pub fn try_acquire(&mut self, now_ms: u64, limit: u64) -> bool {
        let w = now_ms / self.window_ms * self.window_ms;
        if w != self.curr_window_start_ms {
            self.prev_count = if w == self.curr_window_start_ms + self.window_ms {
                self.curr_count                          // adjacent window: carry over
            } else { 0 };                                // gap longer than a window
            self.curr_count = 0;
            self.curr_window_start_ms = w;
        }
        let elapsed = (now_ms - w) as f64 / self.window_ms as f64;
        let estimate = self.curr_count as f64
                     + self.prev_count as f64 * (1.0 - elapsed);
        if estimate + 1.0 <= limit as f64 {
            self.curr_count += 1;
            true
        } else {
            false
        }
    }
}
```

And the test that **demonstrates the flaw** — fixed window admits a 2× burst at the boundary, sliding counter rejects it:

```
#[cfg(test)]
mod window_tests {
    use super::*;

    #[test]
    fn fixed_window_admits_boundary_burst() {
        let mut fw = FixedWindow::new(60_000);        // 100 req/min
        // 100 requests at t = 59.5 s .. 59.9 s: all pass (window 0).
        for i in 0..100 {
            assert!(fw.try_acquire(59_500 + i, 100));
        }
        // 100 MORE at t = 60.0 s .. 60.4 s: all pass (window 1).
        // 200 requests in under a second — legal. That is the bug.
        for i in 0..100 {
            assert!(fw.try_acquire(60_000 + i * 4, 100));
        }
    }

    #[test]
    fn sliding_counter_rejects_the_same_burst() {
        let mut sw = SlidingWindowCounter::new(60_000);
        for i in 0..100 {
            assert!(sw.try_acquire(59_500 + i, 100));
        }
        // Just past the boundary the estimate is ~100 x overlap: the
        // previous window still counts almost fully. Burst denied.
        assert!(!sw.try_acquire(60_200, 100));
        assert!(!sw.try_acquire(61_000, 100));
        // Half a window later, weight has decayed: traffic flows again.
```

```
        let mut ok = 0;
        for i in 0..60 {
            if sw.try_acquire(90_000 + i * 500, 100) { ok += 1; }
        }
        assert!(ok > 40, "expected roughly half the limit to be available");
    }
}
```

3.13.3 The fleet-wide atomic check

Within one process, a `Mutex` makes read-modify-write atomic. Across fifty gateways, the atomicity must live **in the store**: the whole check ships as one server-side script that the store executes without interleaving. In production Rust this is a string constant handed to the client library — the script is data; the engineering is Rust:

```
/// Fleet-wide fixed-window check (Section 3.12, Fix 1). Executed atomically
/// by the store, so the interleaving of Section 3.6 is impossible.
/// KEYS[1] = "rl:{api_key}:{route}:{window_id}", ARGV[1] = window seconds.
const FIXED_WINDOW_LUA: &str = r#"
    local n = redis.call("INCR", KEYS[1])
    if n == 1 then redis.call("EXPIRE", KEYS[1], ARGV[1]) end
    return n
"#;

/// One gateway node's decision: exactly one round trip, and the *returned*
/// count is the check — no client-side read-modify-write at all.
pub async fn allowed(
    con: &mut redis::aio::MultiplexedConnection,
    api_key: &str,
    route: &str,
    limit: u64,
    window_secs: u64,
    now_secs: u64,
) -> redis::RedisResult<Decision> {
    let window_id = now_secs / window_secs;
    let key = format!("rl:{api_key}:{route}:{window_id}");
    let n: u64 = redis::Script::new(FIXED_WINDOW_LUA)
        .key(key)
        .arg(window_secs)
        .invoke_async(con)
        .await?;
    Ok(if n <= limit {
        Decision::Allow { remaining: limit - n }
    } else {
        let reset = (window_id + 1) * window_secs;
        Decision::Deny { retry_after_ms: (reset - now_secs) * 1000 }
    })
}
```

Note what the code makes structural: the key **embeds the window id**, so window rollover is just a new key (old windows expire by TTL — self-cleaning, Section 3.9); and the retry hint falls out of the window arithmetic for free.

3.13.4 Where the check sits

The middleware contract, sketched against a generic handler: rules come from the gateway-local cache (zero hops), the decision from the fleet store (one hop), and `Deny` maps to the 429 contract of Section 3.8.

```
```

```rust
/// Gateway middleware shape (Section 3.10). `handler` is the real API.
pub async fn handle(
    req: Request,
    rules: &RuleCache,            // gateway-local, push-refreshed
    store: &mut Store,            // fleet-wide limit state
) -> Response {
    let rule = rules.for_key_and_route(req.api_key(), req.route());
    match store.check(req.api_key(), req.route(), &rule).await {
        Ok(Decision::Allow { remaining }) => {
            let mut resp = handler(req).await;
            resp.headers_mut().insert("X-RateLimit-Limit", rule.limit.into());
            resp.headers_mut().insert("X-RateLimit-Remaining", remaining.into());
            resp
        }
        Ok(Decision::Deny { retry_after_ms }) => Response::too_many_requests(
            retry_after_ms,          // -> Retry-After + X-RateLimit-Reset
            &rule,
        ),
        Err(_) => fail_open(req).await, // Section 3.15: never take the API down
    }
}
```

3.14 Scaling & Sharding

Sharding (Chapter 1) is by API key — the natural key, since every limit check is scoped to exactly one:

- **Limit-state store**: keys hash-sharded across 4–6 shards with replicas (Section 3.5's arithmetic). Each check touches exactly one key, so there is no cross-shard coordination on the hot path.
- **Gateways**: stateless; any request can be checked anywhere, because the state lives in the shared store.
- **Rules**: fully replicated to every gateway — config is megabytes, and the read pattern (every request) demands local memory.

> **DEFINITION — Hot key**
>
> A single key whose traffic concentrates on one shard hard enough to matter — here, one viral API key doing tens of thousands of checks per second. A single atomic `INCR` or script call is O(1) and microseconds; a hot key is far more likely to be **rejected** fast than to melt a shard. If a key ever outgrows one shard, split its counter across sub-shards (`rl:key#0..15`) and sum at check time — the same trick Chapter 2's stream used per segment.

Multi-region note. A truly global limit across regions forces synchronous cross-region round trips on the hot path — hundreds of milliseconds, violating the latency NFR. Production answer: enforce per-region limits sized to divide the global one, and reconcile asynchronously (Chapter 1's PACELC framing: consistency would cost latency, so availability and latency win, and the **bounded overshoot** vocabulary of Section 3.4 is how you state the price).

3.15 Failure Modes & Recovery

> **DEFINITION — Fail-open / fail-closed**
>
> What a dependent system does when its dependency fails. *Fail-open* serves traffic anyway (availability over enforcement); *fail-closed* refuses (enforcement over availability). The choice is per-route: abuse protection on a public API fails open with a local cap; a login endpoint defending against brute force

fails **closed-ish** — refusing logins during a store outage is cheaper than admitting a credential-stuffing wave.

Failure	Handling
Limit-state store shard down	Fail over to the replica. If all replicas are gone: **fail open** with a per-gateway local token bucket sized `limit / fleet_size` — traffic flows, protection degrades to a bounded multiple of the limit, metrics scream (Section 3.17).
Store unreachable from some gateways (partition)	Those gateways degrade to local caps; the rest enforce exactly. Converges on heal, no manual action.
Clock skew between gateways	Structurally impossible to matter: bucket/window math runs on the **store's** clock via the server-side script (Section 3.12); local code uses monotonic time (Section 3.13).
Bad rule pushed	Rules are versioned; roll back to the previous version. Rate of change is small, so a human-in-the-loop push is fine.
Rule fan-out stalls	Gateways enforce last-known rules and alert; a stale rule for 60 s beats a missing check for 60 s.
Retry storm after a 429 wave	`Retry-After` spreads retries; add jitter client-side guidance in docs. A limiter that herds retries into the same second recreates the burst it just rejected.
Hot key	Sub-shard the counter and sum (Section 3.14). Rare in practice — hot keys are usually already over their limit.

3.16 Trade-offs & Alternatives

Decision	Benefit purchased	Cost accepted
Token bucket	Bursts as a product feature; O(1) state; lazy refill needs no timers	Two-number accounting must be atomic fleet-wide (script in the store)
Sliding window counter	No boundary burst, still O(1)	1% estimate error; slightly more math at check time
Fixed window alone	One atomic INCR; trivially explainable	2× boundary burst (Section 3.13's test proves it)
Centralized exact check	FR-4 truly holds; no false rejections	+1 RTT on every request; store on the critical path

Decision	Benefit purchased	Cost accepted
Local + async sync	Zero added latency; fail-open for free	Bounded overshoot; unusable for billing-adjacent limits
Fail-open default	The API never dies because the limiter did	During store outages, protection is a bounded approximation

3.17 Observability & SLOs

SLIs and SLOs (Chapter 1) for a limiter are unusual in one way: the system **working** looks like rejections. Instrument accordingly:

- **Decision rates** per rule: allows vs. 429s, over time. A 429 spike after a rule change is a config bug until proven otherwise.
- **Overshoot audit**: sampled per-key realized rates vs. limits — the direct measurement of Section 3.4's accuracy NFR.
- **Check latency** p50/p99, split by path (local rules lookup vs. store call); SLO: the ≤1 ms budget of Section 3.4.
- **Store health**: shard ops/sec, script latency, replica lag — the one dependency on the hot path.
- **Degraded-mode counters**: how many gateways are currently failing open, and with what local cap. This number should be zero; when it is not, paging is legitimate.
- **Top rejected keys**: the abuse list and the sales list are the same list — chronically limited keys are upgrade candidates (limits as monetization, Section 3.1).

3.18 Interview Wrap-Up

Likely follow-ups, with one-line answers:

- *"Per-key AND per-IP AND per-route at once?"* — Composite rules: evaluate each applicable limit; deny if any denies. Order checks cheapest-first.
- *"Monthly quotas for billing?"* — Different system: append-only usage log, batch aggregation, delayed enforcement. Rate limits shape traffic; quotas shape invoices (Section 3.1).
- *"Limiter as a shared service vs. a library?"* — A library removes a hop but re-introduces per-language drift and per-instance state; a sidecar/service centralizes policy at the price of the hop. State which you picked and why.
- *"Exactly-distributed without a central store?"* — Gossiped counters or CRDTs (Chapter 1) give you **approximate** global counts with zero coordination; exactness without coordination is not a thing, and saying so is the answer.
- *"L3/L4 DDoS?"* — Out of scope by layer: SYN floods and volumetric attacks are absorbed at the CDN/edge (Chapter 2) with connection-level defenses. This design is L7, per-authenticated-caller.
- *"Adaptive limits under load?"* — Load-shedding's cousin: when the backend browns out, tighten limits dynamically by a global multiplier. Nice extension; say it, don't build it live.

If you remember five things:

1. A rate limiter is a read-modify-write on the hot path; the design is the art of making that operation atomic and ≤ 1 ms.
2. Local counters cannot express a global limit; a non-atomic shared counter races. Atomicity lives **in the store**.
3. Token bucket = sustained rate + burst allowance, with lazy refill and self-cleaning TTL state.
4. Accuracy is asymmetric: never false-reject, bound the overshoot — and know that exactness is a product decision, not a technical one.
5. The limiter must never be the outage: fail open with a local cap, and page on degraded mode.

3.19 Summary & Further Reading

We designed a distributed rate limiter for a 200k-RPS public API: a gateway middleware making exactly one atomic check per request against a sharded in-memory store; the algorithm zoo (fixed window, sliding log, sliding counter, token bucket, leaky/GCRA) with token bucket chosen for its burst semantics; the TOCTOU race and its three fixes (atomic increment-as-decision, server-side scripts, approximate local counting); the 429 + `Retry-After` contract that turns rejections into negotiations; runtime-reloadable tiered rules pushed to gateway memory; and a defended fail-open policy with bounded local caps.

Primary source for this chapter:

- **"7: Design a Rate Limiter" — Systems Design Interview Questions With Ex-Google SWE (Jordan has no life)** — the walkthrough this chapter expands.

Foundations worth reading:

- Stripe's engineering blog on rate limiters — production limiter design at API-company scale.
- Figma's *An alternative approach to rate limiting* — the leaky-bucket-in-Redis variant, honestly costed.
- The IETF draft *RateLimit header fields for HTTP* — the emerging standard response contract.
- Brandur Leach's writing on GCRA and the Redis-cell module — the stateless leaky bucket, implemented.
- NGINX's rate-limiting documentation — the leaky bucket hiding inside a commodity reverse proxy.

3.20 Chapter 3 Glossary

A one-glance index of every term this chapter defined. Chapters 1–2 are assumed; later chapters assume all three.

Term	Meaning in one line
Rate limiting / throttling	Capping requests per identity per window; protection, capacity, monetization
Quota	Long-period budget for billing; batched, approximate accounting
False rejection	Denying an under-limit caller — the unforgivable error
Overshoot	Admitting over-limit traffic — the bounded, tolerable error
Race condition / TOCTOU	Check and use separated in time; state changes in between
Atomicity / CAS	All-or-nothing operation / conditional swap as one step
Fixed window counter	One counter per window; O(1), but 2× boundary bursts
Sliding window log	Every timestamp kept; exact; memory O(limit) — infeasible at scale
Sliding window counter	Two windows weighted by overlap; O(1), 1% error, no burst
Token bucket	Capacity b refilled at r/s; bursts allowed; lazy refill
Leaky bucket / GCRA	Constant-rate drip; bursts forbidden; smooth output
HTTP 429 / Retry-After	"Too fast" status + how long to wait — rejection as negotiation
X-RateLimit-* headers	Limit, remaining, reset — client-side self-throttling data
API gateway middleware	First code every request meets; policy without backend changes
Server-side script (Lua)	Read-modify-write executed atomically inside the store

Term	Meaning in one line
Local + async sync	Zero-latency approximate enforcement with bounded overshoot
Monotonic clock	Time source that never moves backwards; no refunded tokens
Fail-open / fail-closed	Dependency down: serve anyway / refuse — chosen per route
Hot key	One key concentrating load on one shard; sub-shard and sum
Tiered limits	Different limits per plan; limits as pricing
Rule cache	Gateway-local copy of config; zero hops per request
Key TTL self-cleaning	Idle keys expire with their window; no garbage collector
Boundary burst	Two legal window-fulls fired at a window edge — 2× in seconds
Degraded-mode cap	Local emergency limit while the fleet store is down

CHAPTER 4

Designing a Logging & Metrics Platform

PROBLEM SOURCE

This chapter solves the problem posed in the talk **"14: Distributed Logging & Metrics Framework"** from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The talk designs the observability backbone for a microservices fleet: how telemetry leaves thousands of hosts, where it is buffered, how logs and metrics are stored and queried, and how alerts fire. This chapter follows the same arc, deepened with full definitions, capacity mathematics, query and retention design, and Rust reference implementations.

4.1 The Problem Statement

The interviewer looks up and says:

> *"We run thousands of services across tens of thousands of machines. When something breaks at 3 a.m., engineers need to see what happened — search every log, graph every metric, and get paged before users notice. Design the logging and metrics platform that makes that possible."*

The previous chapters designed systems that **serve users**. This one designs the system that watches all of them — including, recursively, the ones from Chapters 1–3. It is a data pipeline problem wearing an operations costume: an enormous, never-ending write stream on one side, and on the other a handful of humans asking very expensive questions at the worst possible moment (during an outage, when both data volume and query volume spike together).

DEFINITION — Observability / telemetry

The ability to answer questions about a system's internal state from its **outputs**: the data it emits as it runs. *Telemetry* is that emitted data. The canonical "three pillars" are **logs** (timestamped event records), **metrics** (numeric measurements over time), and **traces** (the path of one request through many services). This chapter designs the platform for the first two; traces are a follow-up (Section 4.18).

DEFINITION — Log vs. metric

A *log record* is a discrete, semi-structured event: "at 03:14:22.017, service `checkout` on host `i-9f2` logged `ERROR payment timeout user=…`" — high volume, low per-record value, queried by **searching text**. A *metric* is a named numeric measurement repeated over time: `http_requests_total{service="checkout", status="500"} = 87341` — small, structured, queried by **range scans and aggregations**. They look like cousins and demand opposite storage engines. Holding that distinction is the core of this chapter (Section 4.6).

4.2 Scope & Clarifying Questions

Speaker	Dialogue
Candidate	"What's in scope — logs, metrics, distributed tracing, all three?"
Interviewer	"Logs and metrics. Tracing is a future extension; don't design it, but don't preclude it."
Candidate	"Scale of the fleet being observed?"
Interviewer	"About 50,000 service instances across a few thousand hosts, in several regions."
Candidate	"How quickly must emitted data be queryable?"
Interviewer	"Logs within about a minute; metrics within about thirty seconds. Alerts within a minute of the condition."
Candidate	"How long do we keep data?"
Interviewer	"Logs: two weeks fast-searchable, three months retrievable. Metrics: thirteen months, and nobody needs per-15-second resolution on data from last year."
Candidate	"Is some data loss acceptable under extreme overload?"
Interviewer	"Metrics — yes, they're statistical; drop samples rather than fall over. Logs — avoid it; engineers notice gaps. State how you'd degrade."
Candidate	"Who queries, and how often?"
Interviewer	"A few thousand engineers. Dashboards refresh continuously; searches spike during incidents."

AGREED SCOPE

Logs + metrics for a **50k-instance** fleet. Logs: full-text search, queryable ≤ 1 min after emission, 2 weeks hot + 3 months cold. Metrics: label-filtered queries and dashboards, fresh ≤ 30 s, 13 months retained with downsampling. Alerting on both, notification ≤ 1 min from condition. Metrics may drop under extreme load; logs degrade by **sampling**, never silent gaps.

4.3 Functional Requirements

1. **FR-1 — Collection.** Every instance's logs and metrics flow to the platform automatically, within seconds of emission, with no per-service plumbing by engineers.
2. **FR-2 — Log search.** Engineers query logs by full text, structured fields, and time range; results over hot data return in seconds.
3. **FR-3 — Metric queries & dashboards.** Engineers query series by name and labels over time ranges, with aggregations (rate, sum, percentile), rendered as auto-refreshing dashboards.
4. **FR-4 — Alerting.** Rules over logs and metrics evaluate continuously; when a condition holds for its configured duration, notifications fire — once, not repeatedly (Section 4.12).
5. **FR-5 — Retention & tiering.** Data ages automatically through hot → warm → cold → deleted, per configured policy, with no manual intervention.
6. **FR-6 — Self-service config.** Teams register services, set retention, and define alerts through an API/UI — the platform team is not in the loop.

4.4 Non-Functional Requirements

DEFINITION — Ingestion latency (freshness, telemetry sense)

The time from an event being emitted on a host to it being **queryable**. Chapter 2's freshness measured data age; this one measures pipeline speed. Our targets: logs ≤ 60 s, metrics ≤ 30 s — fast enough that an engineer watching a deploy sees its effect while watching.

Quality	Target
Ingestion throughput	10M log events/s average, 3× that in bursts; 0.7M metric samples/s (Section 4.5)
Ingestion latency	Logs queryable ≤ 60 s; metrics ≤ 30 s
Query latency	Hot log search p95 ≤ 5 s; dashboard refresh ≤ 2 s
Durability	Logs: at-least-once, gaps are visible and rare. Metrics: loss-tolerant by nature (sampling tolerates gaps)
Availability	Ingestion path ≥ 99.9% — it is the fleet's black-box recorder; query path may degrade first
Cost discipline	Storage dominates; compression and tiering are first-class requirements, not optimizations

KEY INSIGHT — The system's worst day is everyone else's

When the fleet has an incident, two things happen at once: services emit **more** telemetry (error storms, stack traces) and engineers query **harder** (everyone opens dashboards). The platform must be sized for the correlated spike — the day it is needed most is the day it is loaded most. This observation drives the buffer-first architecture of Section 4.11.

4.5 Back-of-the-Envelope Estimation

Assumptions:

- 50k service instances; each emits 200 log lines/s average (quiet services less, chatty ones more), 200 B per line.
- Each instance exposes 200 metric series (name + label combinations); sampled every 15 s.
- Dashboards/searches: 2k engineers, spiking to 5k queries/s during incidents.

Derived numbers:

Quantity	Estimate	How
Log event rate	≈ 10M events/s avg, 30M burst	50k instances × 200 lines/s, ×3 incident factor
Log bandwidth	≈ 2 GB/s avg	10M × 200 B
Raw logs per day	≈ 170 TB	2 GB/s × 86,400 s
Hot log tier (14 days)	≈ 0.5–1 PB	sampling/filtering trims to 30–50 TB/day indexed; × 14 days, plus index overhead
Metric sample rate	≈ 0.7M samples/s	50k × 200 series ÷ 15 s

Quantity	Estimate	How
Metric storage per day	≈ 120 GB **compressed**	0.7M/s × 86,400 s × 2 B per compressed sample (Section 4.9)
Metrics, 13 months	tens of TB	after rollups; an order of magnitude less than logs' **daily** raw volume
Buffer capacity	≈ 2–5 GB/s sustained	a Kafka-class cluster of 20–30 brokers, 200 partitions

KEY INSIGHT — What the math tells us

Two facts shape everything. First, **logs are the cost center**: 170 TB/day raw versus metrics' 120 GB/day — three orders of magnitude apart. Storage design for logs is really **cost** design (sampling, tiering, compression); storage design for metrics is **query-shape** design (fast range scans over tiny values). Second, the write rate utterly dwarfs the query rate, so the system is organized as a pipeline: absorb first, index second, query third.

4.6 The Core Challenge: One Firehose, Two Shapes of Data

Logs and metrics arrive on the same wire from the same hosts — and must part ways immediately, because they are different kinds of data with different query patterns and different economics:

Property	Logs	Metrics
Record	Semi-structured text event	Name + labels + (timestamp, number)
Volume	170 TB/day raw	120 GB/day compressed
Per-record value	Low individually; priceless during one incident	Low individually; aggregates are the product
Query pattern	Full-text search, field filters, recent time ranges	Range scan by name/labels; aggregations over time
Index needed	Inverted index over tokens (Section 4.8)	Compressed time-ordered blocks (Section 4.9)
Loss tolerance	Low — gaps break investigations	High — statistics tolerate dropped samples

COMMON PITFALL — One store for both

The classic junior move: "just put it all in one database." Indexing metrics as log lines wastes the log pipeline's expensive text machinery on tiny numbers; storing logs in a time-series engine forfeits text search entirely. The chapter's architecture forks **right after the buffer** into two purpose-built stores — Section 4.8 and 4.9 exist because this fork exists.

4.7 Data Ingestion: Agents, Push vs. Pull, and the Buffer

DEFINITION — Telemetry agent

A small daemon running on every host (or beside every workload as a sidecar), owned by the platform team: it tails log files, collects or receives metrics, batches, compresses, and ships both upstream — buffering locally when upstream is unavailable. Agents are how collection (FR-1) happens "automatically": a service writes to stdout and exposes a metrics endpoint; the agent does the rest.

The one genuine design fork in collection is **who initiates the metrics transfer**:

DEFINITION — Push vs. pull (scrape)

Push: the source (or its agent) sends metrics upstream when it has them. *Pull*: the collector periodically **requests** metrics from each target's endpoint ("scrapes" `/metrics` every 15 s). Pull gives the collector control of sample timing, automatic **up/down detection** (a target that doesn't answer is itself a signal), and easy correctness (one place enforces the schedule). Push fits short-lived jobs that vanish before they can be scraped, and crossing trust boundaries outward. Production fleets commonly run both: pull for services, push gateways for batch jobs.

DEFINITION — Backpressure

What a pipeline does when a downstream stage is slower than upstream: the pressure must propagate backward and be **absorbed somewhere** — a queue grows, a producer is slowed, or data is deliberately shed. A pipeline without a backpressure plan converts every slow consumer into a cascading outage. Our answer: a durable buffer between agents and processors, plus explicit drop policies per data class (metrics shed first, logs sampled — Section 4.2's degradation agreement).

KEY INSIGHT — The buffer is the shock absorber

A Kafka-class log (Chapter 2) between agents and processors buys four things at once: spikes are absorbed (the incident-storm insight of Section 4.4); indexing can lag and replay after outages; a bad deploy of the indexing fleet costs **lag**, not data; and new consumers (the future tracing pipeline, a billing tap) attach without touching producers. Decoupling ingestion from indexing is the single highest-leverage decision in the chapter.

And a deliberate callback: the buffer's tenants are protected from each other with per-team **ingestion quotas** — Chapter 3's rate limiter, applied to telemetry. One team's debug-level flood must not evict everyone else's logs.

4.8 Log Storage: The Inverted Index

Log search answers "find the records containing these tokens, in this time range, with these field values, among billions of records" in seconds. A scan is impossible; the index must be inverted.

DEFINITION — Inverted index / posting list

The data structure behind every text search engine: a map from **token** to the sorted list of records containing it (the *posting list*). `"payment"` → [rec 17, rec 203, rec 8811, …]. A query like `payment AND timeout` intersects two posting lists — set intersection over sorted lists, not a scan of the data. Building the index is work done **at write time** so that reads stay fast: logs are a write-once, read-rarely, read-expensively workload, and the inverted index prices exactly that.

DEFINITION — Tokenization / structured logging

Tokenization splits raw text into searchable terms (lower-cased, split on punctuation). *Structured logging* emits logs as JSON records —

`{"level":"ERROR","service":"checkout","msg":"payment timeout",...}` — so fields are indexed **as fields** (`service:checkout`) rather than dug out of text at query time. The platform ingests both; structure makes queries faster and sampling smarter, and FR-6's self-service story nudges teams toward it.

Three storage decisions carry the load:

1. **Immutable segments, merged in the background.** Incoming records are buffered into small index **segments** written to disk immutably; a background process merges small segments into larger ones. Appends and merges are sequential I/O — the cheapest disk work there is (same lesson as Chapter 1's operation log).
2. **Sharding by time.** The index is partitioned into time buckets (e.g., one index per day per tenant class). Queries prune whole days by timestamp before touching a posting list — and **retention becomes deletion of an old index file**, not a billion per-record deletes. Time-sharding turns FR-5 into `rm`.
3. **Hot / warm / cold tiers.** Today's indices live on fast SSD nodes with replicas (hot); last week's on cheaper nodes (warm); months-old data as compressed archives in object storage (cold), rehydrated on the rare historical query. Cost per GB falls by an order of magnitude per tier — Section 4.5 made clear this **is** the design.

4.9 Metrics Storage: The Time-Series Database

DEFINITION — Time series / sample / label set

A *time series* is a named, labeled sequence of **(timestamp, value)** pairs: `http_requests_total{service="checkout", status="500"}` sampled every 15 s. A *sample* is one such pair. The *labels* (tags) are the dimensions queries filter and group by. The number of **distinct series** — the product of all label-value combinations — is the *cardinality*, and it is the metric system's one true nemesis.

COMMON PITFALL — Cardinality explosion

Labels must be **dimensions**, never **identifiers**. `status="500"` is a dimension (a handful of values); `user_id="u_91827"` is an identifier — millions of series, each seen once, index structures bloated beyond RAM, queries scanning garbage. Rule of thumb: a label whose value set grows with **traffic or users** does not belong on a metric; it belongs in a log or a trace. Every metrics interview eventually arrives here — arrive first.

Why a purpose-built store: samples of one series arrive in time order and change slowly, so they compress almost to nothing:

DEFINITION — Delta encoding / delta-of-delta

Store differences, not values. Timestamps sampled every 15 s delta-encode to a constant 15000 ms; the **delta-of-delta** (difference of consecutive deltas) is 0 almost always — compressible to a single bit per timestamp in the production scheme (Facebook's Gorilla, which also XOR-encodes values so only meaningful bits are stored). 16-byte samples become 1–2 bytes. Section 4.13 implements the delta+zigzag+varint core of the idea.

DEFINITION — Downsampling / rollup

> Replacing old high-resolution samples with precomputed aggregates — keep 15-second resolution for a week, 1-minute for a quarter, 1-hour beyond: `avg/min/max/sum/count` per bucket, computable incrementally. Nobody asks for 15-second data from last year (Section 4.2), and rollups turn 13 months of retention from a petabyte problem into a terabyte one.

Queries are then range scans over compressed blocks, sharded by metric-name hash: fetch the blocks for matching series in the range, decompress, aggregate. Dashboard panels are exactly this, cached and refreshed.

4.10 API & Query Design

Three surfaces — ingestion, query, config — each deliberately boring:

Method	Path	Purpose
`POST`	`/v1/logs:batch`	Agent push: compressed batch of log records with source metadata
`GET`	`/metrics` (on targets)	The scrape endpoint pull collectors call every 15 s
`POST`	`/v1/metrics:write`	Push path for short-lived jobs
`GET`	`/v1/logs/search?q=&from=&to=`	Log search: query string, time range, cursor-paginated
`POST`	`/v1/metrics/query`	Instant and range queries over series
`POST`	`/v1/alerts/rules`	Create/update alert rules (FR-4, FR-6); versioned like Chapter 3's rules

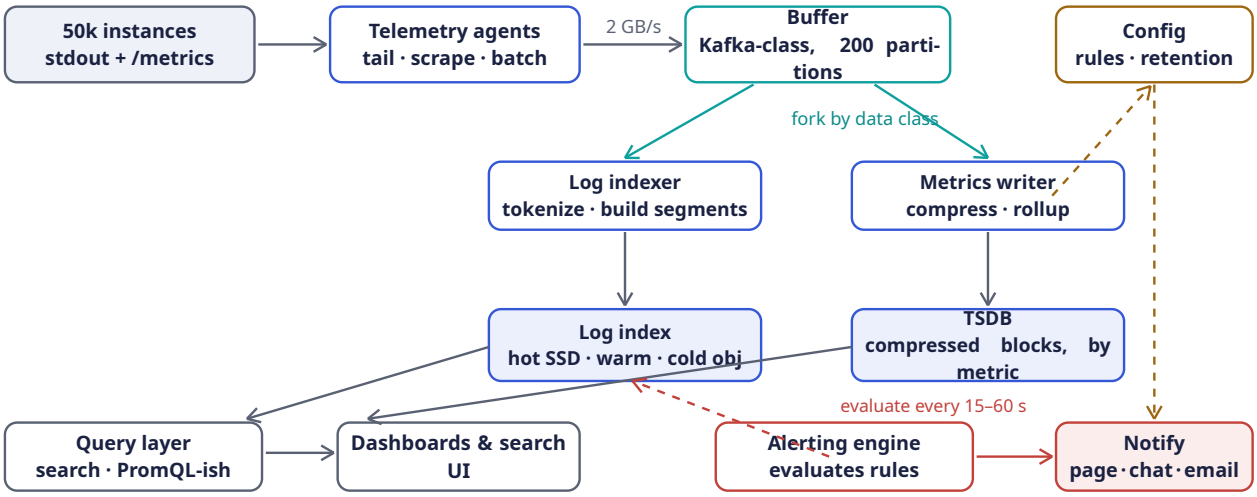
Query languages, sketched to show the shape (not to design in full):

```
-- logs: field filters + free text + time range
service:checkout AND level:ERROR AND "payment timeout"   @ last 1h

-- metrics: series selector + aggregation over a range
rate(http_requests_total{service="checkout", status=~"5.."}[5m])
  |> sum by (region)
```

Alert rules are records: `{ name, query, threshold, comparison, for_duration, severity, notify_channels }` — evaluated by the alerting engine of Section 4.12.

4.11 High-Level Architecture



One write path, two stores. The buffer absorbs incident storms; the query layer federates across both stores.

Component	Responsibility	Why it is shaped this way
Telemetry agents	Tail logs, scrape/collect metrics, batch + compress, ship	Per-host daemon = zero per-service work (FR-1); local buffer rides out upstream blips
Buffer (Kafka-class)	Durable, replayable queue between agents and processors	The shock absorber of Section 4.7: spikes, replays, new consumers
Log indexer	Parse, tokenize, build inverted-index segments	Stateless consumers of the log topic; scale with partition count
Log index cluster	Hot/warm/cold inverted indices, time-sharded	Search needs posting lists; retention needs cheap tiers (Section 4.8)
Metrics writer	Compress samples into blocks; maintain rollups	Stateless consumers of the metrics topic
TSDB cluster	Compressed time-series blocks; label index	Range scans over 2 B samples (Section 4.9)
Query layer	Federate queries across both stores; cache dashboard panels	One door for engineers; caches absorb dashboard refresh storms
Alerting engine	Evaluate rules continuously; notify without flapping	Section 4.12 — its correctness is the product's voice
Config service	Rules, retention, quotas; versioned, pushed	Chapter 3's lesson: config never fetched on the hot path

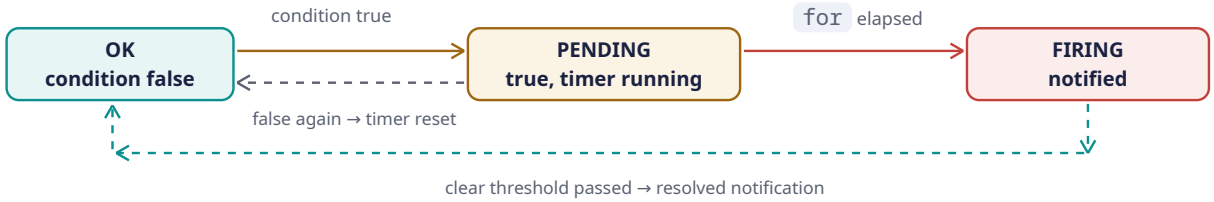
4.12 Deep Dive: The Alerting Engine

Alerting is where the metrics store earns its keep. The engine's loop is trivially stated — **every evaluation interval (15–60 s), run every rule's query, compare against the threshold** — and the difficulty is entirely in not crying wolf.

DEFINITION — Flapping / hysteresis / "for" duration

> *Flapping*: an alert that oscillates OK → FIRING → OK → FIRING as the metric jitters around the threshold, paging a human every oscillation until the rule is muted and the signal lost. The cures: a *"for" duration* — the condition must hold continuously for N minutes before firing (a PENDING state); and *hysteresis* — firing and clearing use **different** thresholds (fire at p99 > 500 ms for 5 min, clear only when p99 < 450 ms for 10 min), so a metric straddling one line cannot cross both. Alert fatigue is a **system failure mode**, and it is engineered against, exactly like throughput.

The per-rule state machine:



Three more engineering points:

- **Deduplication and grouping.** One dying dependency fires forty service alerts; the engine groups by root labels (`region`, `dependency`) into **one** page with the forty attached as context. Notifications are facts about **state transitions**, not about every evaluation.
- **Evaluation must be independent of ingestion health of the thing it watches.** If the alerting engine shares a failure domain with the metrics pipeline, the one outage that must page is the one that can't. It runs on separate infrastructure and — critically — can fire on **absence** of data (Section 4.15).
- **Self-service with guardrails (FR-6).** Teams write rules in config; the platform validates them (query parses, series cardinality bounded, threshold sane) and versions every change, exactly the Chapter 3 rule-lifecycle pattern.

4.13 Rust Reference Implementations

Four distilled pieces, each with a deterministic test. They are teaching skeletons of what runs at platform scale: a mini inverted index for log search, timestamp compression for the TSDB, incremental rollups, and the alert state machine.

4.13.1 A Mini Inverted Index for Log Search

Structured records are tokenized; each token maps to a sorted posting list of record ids. A two-term `AND` query intersects the lists in linear time.

```rust
use std::collections::HashMap;

/// One structured log record, as the indexer sees it after parsing.
#[derive(Debug, Clone)]
pub struct LogRecord {
    pub id: u64,
    pub timestamp_ms: u64,
    pub service: String,
    pub level: String,
    pub message: String,
}

/// Lower-case, split on anything that isn't alphanumeric:
/// "payment TIMEOUT after 3s" -> ["payment", "timeout", "after", "3s"].
fn tokenize(text: &str) -> Vec<String> {
    text.to_lowercase()
```

```
        .split(|c: char| !c.is_alphanumeric())
        .filter(|t| !t.is_empty())
        .map(str::to_string)
        .collect()
}

/// token -> sorted ids of records containing it ("posting list").
/// Also index the searchable fields under `field:value` tokens so that
/// `service:checkout` is just another posting-list lookup.
#[derive(Default)]
pub struct InvertedIndex {
    postings: HashMap<String, Vec<u64>>,
}

impl InvertedIndex {
    pub fn add(&mut self, rec: &LogRecord) {
        let mut tokens = tokenize(&rec.message);
        tokens.push(format!("service:{}", rec.service.to_lowercase()));
        tokens.push(format!("level:{}", rec.level.to_lowercase()));
        tokens.sort();
        tokens.dedup();
        for tok in tokens {
            let list = self.postings.entry(tok).or_default();
            // ids are appended in increasing order, so lists stay sorted;
            // a real engine would also store positions for phrase queries.
            list.push(rec.id);
        }
    }

    pub fn lookup(&self, token: &str) -> &[u64] {
        self.postings.get(token).map(Vec::as_slice).unwrap_or(&[])
    }

    /// Intersection of two sorted lists: the heart of `A AND B`.
    pub fn and(&self, a: &str, b: &str) -> Vec<u64> {
        let (xs, ys) = (self.lookup(a), self.lookup(b));
        let (mut i, mut j, mut out) = (0, 0, Vec::new());
        while i < xs.len() && j < ys.len() {
            match xs[i].cmp(&ys[j]) {
                std::cmp::Ordering::Less => i += 1,
                std::cmp::Ordering::Greater => j += 1,
                std::cmp::Ordering::Equal => {
                    out.push(xs[i]);
                    i += 1;
                    j += 1;
                }
            }
        }
        out
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn rec(id: u64, service: &str, level: &str, msg: &str) -> LogRecord {
        LogRecord { id, timestamp_ms: id, service: service.into(),
                    level: level.into(), message: msg.into() }
    }
```

```
    #[test]
    fn search_intersects_posting_lists() {
        let mut ix = InvertedIndex::default();
        ix.add(&rec(1, "checkout", "ERROR", "payment timeout after 3s"));
        ix.add(&rec(2, "checkout", "INFO",  "payment authorized"));
        ix.add(&rec(3, "search",   "ERROR", "timeout talking to index"));
        ix.add(&rec(4, "checkout", "ERROR", "payment timeout after 5s"));

        // free text
        assert_eq!(ix.lookup("timeout"), &[1, 3, 4]);
        // field query
        assert_eq!(ix.lookup("service:checkout"), &[1, 2, 4]);
        // the incident query: payment AND timeout, checkout only
        assert_eq!(ix.and("payment", "timeout"), &[1, 4]);
        assert_eq!(
            ix.and("service:checkout", "level:error")
                .into_iter()
                .collect::<Vec<_>>(),
            vec![1, 4]
        );
    }
}
```

Production engines add positions (for phrase queries), compression of posting lists, skip pointers for faster intersection, and segment files instead of a `HashMap` — but the **algorithm** the interviewer wants to hear is the sorted intersection above, unchanged since the 1960s.

4.13.2 Delta + Zigzag + Varint: Compressing Timestamps

The honest core of Gorilla-style compression: store the first timestamp, then **delta-of-deltas**; encode each as a zigzag varint so small numbers — including small **negative** corrections — cost one byte.

```
/// Zigzag maps signed -> unsigned so small magnitudes (either sign)
/// become small unsigned ints: 0->0, -1->1, 1->2, -2->3, 2->4, ...
fn zigzag(n: i64) -> u64 {
    ((n << 1) ^ (n >> 63)) as u64
}
fn unzigzag(z: u64) -> i64 {
    ((z >> 1) as i64) ^ -((z & 1) as i64)
}

/// Unsigned varint: 7 bits per byte, high bit = "more bytes follow".
fn write_varint(mut z: u64, out: &mut Vec<u8>) {
    loop {
        let byte = (z & 0x7f) as u8;
        z >>= 7;
        if z == 0 { out.push(byte); break; }
        out.push(byte | 0x80);
    }
}
fn read_varint(bytes: &[u8], pos: &mut usize) -> u64 {
    let (mut z, mut shift) = (0u64, 0);
    loop {
        let byte = bytes[*pos];
        *pos += 1;
        z |= ((byte & 0x7f) as u64) << shift;
        if byte & 0x80 == 0 { return z; }
        shift += 7;
    }
```

```
}

/// Compress a sorted series of millisecond timestamps:
/// first value verbatim, then delta-of-delta as zigzag varints.
pub fn compress_timestamps(ts: &[u64]) -> Vec<u8> {
    assert!(!ts.is_empty());
    let mut out = Vec::new();
    write_varint(ts[0], &mut out);
    let (mut prev_ts, mut prev_delta) = (ts[0] as i64, 0i64);
    for w in ts.windows(2) {
        let delta = w[1] as i64 - prev_ts;
        let dod = delta - prev_delta;                 // 0 for a steady 15s scrape
        write_varint(zigzag(dod), &mut out);
        prev_ts = w[1] as i64;
        prev_delta = delta;
    }
    out
}

pub fn decompress_timestamps(bytes: &[u8], count: usize) -> Vec<u64> {
    let mut pos = 0;
    let first = read_varint(bytes, &mut pos) as i64;
    let mut out = vec![first as u64];
    let (mut prev_ts, mut prev_delta) = (first, 0i64);
    for _ in 1..count {
        let dod = unzigzag(read_varint(bytes, &mut pos));
        let delta = prev_delta + dod;
        prev_ts += delta;
        prev_delta = delta;
        out.push(prev_ts as u64);
    }
    out
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn zigzag_roundtrip_and_small_magnitudes() {
        for n in [-2, -1, 0, 1, 2, 15000, -15000] {
            assert_eq!(unzigzag(zigzag(n)), n);
        }
        assert_eq!(zigzag(-1), 1);
        assert_eq!(zigzag(0), 0);
    }

    #[test]
    fn steady_15s_scrape_compresses_to_about_a_byte_per_point() {
        let base = 1_700_000_000_000u64;           // some epoch-ms
        let ts: Vec<u64> = (0..1000).map(|i| base + i * 15_000).collect();
        let packed = compress_timestamps(&ts);
        assert_eq!(decompress_timestamps(&packed, ts.len()), ts);
        // 1000 x 8-byte timestamps -> ~1007 bytes: ~1 byte per point.
        assert!(packed.len() < 1100, "got {} bytes", packed.len());
    }

    #[test]
    fn jitter_and_gaps_survive_roundtrip() {
        // scrape jitter (+/- 3ms) plus one 30s gap where a sample was lost
```

```
        let mut ts = vec![1_000u64, 16_003, 31_002, 61_001, 75_997];
        let packed = compress_timestamps(&ts);
        assert_eq!(decompress_timestamps(&packed, ts.len()), ts);
        ts.clear(); // prove the bytes alone reconstruct everything
        assert!(ts.is_empty());
    }
}
```

Gorilla additionally XORs consecutive **values** and stores only meaningful bit-runs — the same philosophy: at scale, the difference between 16 bytes and 1.4 bytes per sample is the difference between a petabyte and a rack.

4.13.3 Incremental Rollups for Retention

A rollup bucket carries just enough state to be mergeable: `sum`, `min`, `max`, `count`. Raw samples stream in; the bucket answers the five aggregates without remembering any sample.

```
/// One downsampled bucket: mergeable in O(1) without raw samples.
/// This is why rollups scale: merging two 1-hour buckets is
/// adding sums, taking min/max, adding counts.
#[derive(Debug, Clone, Copy, PartialEq)]
pub struct Bucket {
    pub sum: f64,
    pub min: f64,
    pub max: f64,
    pub count: u64,
}

impl Bucket {
    pub fn empty() -> Self {
        Bucket { sum: 0.0, min: f64::INFINITY, max: f64::NEG_INFINITY, count: 0 }
    }
    pub fn add(&mut self, value: f64) {
        self.sum += value;
        self.min = self.min.min(value);
        self.max = self.max.max(value);
        self.count += 1;
    }
    /// Merge another bucket's aggregates in O(1) — no raw samples needed.
    /// This associativity is exactly why rollups scale: two hour-buckets
    /// combine into a day-bucket with four cheap operations.
    pub fn merge(&mut self, other: &Bucket) {
        if other.count == 0 { return; }
        self.sum += other.sum;
        self.min = self.min.min(other.min);
        self.max = self.max.max(other.max);
        self.count += other.count;
    }
    pub fn avg(&self) -> Option<f64> {
        (self.count > 0).then(|| self.sum / self.count as f64)
    }
}

/// Assign a timestamp (ms) to its bucket of `width_ms`, then fold in.
pub fn rollup(samples: &[(u64, f64)], width_ms: u64)
    -> std::collections::BTreeMap<u64, Bucket>
{
    let mut buckets = std::collections::BTreeMap::new();
    for &(ts, v) in samples {
        buckets.entry(ts / width_ms).or_insert_with(Bucket::empty).add(v);
```

```
    }
    buckets
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn rollup_buckets_match_hand_computation() {
        // 15s samples across two 1-minute buckets; base aligned to a
        // minute boundary so the first four samples share bucket 0.
        let t = 1_700_000_040_000u64; // == 28_333_334 * 60_000
        let samples: Vec<(u64, f64)> = (0..8)
            .map(|i| (t + i * 15_000, (100 + i * 10) as f64))
            .collect();
        let m = rollup(&samples, 60_000);
        let keys: Vec<u64> = m.keys().copied().collect();
        assert_eq!(keys.len(), 2);
        let b0 = m[&keys[0]];
        assert_eq!(b0.count, 4);                        // t, t+15s, t+30s, t+45s
        assert_eq!(b0.sum, 100.0 + 110.0 + 120.0 + 130.0);
        assert_eq!(b0.min, 100.0);
        assert_eq!(b0.max, 130.0);
        assert_eq!(b0.avg(), Some(115.0));
    }

    #[test]
    fn buckets_merge_without_raw_samples() {
        let mut a = Bucket::empty();
        for v in [10.0, 20.0, 30.0] { a.add(v); }
        let mut b = Bucket::empty();
        for v in [5.0, 50.0] { b.add(v); }
        a.merge(&b);
        assert_eq!(a.count, 5);
        assert_eq!(a.sum, 115.0);
        assert_eq!(a.min, 5.0);
        assert_eq!(a.max, 50.0);
        assert_eq!(a.avg(), Some(23.0));
    }
}
```

4.13.4 The Alert Evaluator: OK → PENDING → FIRING

The state machine of Section 4.12, made executable. The test demonstrates the two failure modes it prevents: a single bad spike must **not** page (no `for` violation), and a sustained breach must page **once**, not on every evaluation.

```
/// States of Section 4.12's machine. `pending_since` is Some(ms) while
/// the condition has been continuously true but `for_ms` hasn't elapsed.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum AlertState {
    Ok,
    Pending { since_ms: u64 },
    Firing,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Notification {
    Fired,
```

```
    Resolved,
}

pub struct AlertRule {
    pub threshold: f64,
    pub for_ms: u64,
}

pub struct AlertEvaluator {
    pub rule: AlertRule,
    state: AlertState,
}

impl AlertEvaluator {
    pub fn new(rule: AlertRule) -> Self {
        AlertEvaluator { rule, state: AlertState::Ok }
    }

    /// Feed one evaluation (one query result) at time `now_ms`.
    /// Returns a notification only on *transitions*.
    pub fn evaluate(&mut self, breached: bool, now_ms: u64) -> Option<Notification> {
        use AlertState::*;
        match (self.state, breached) {
            (Ok, true) => {
                self.state = Pending { since_ms: now_ms };
                None
            }
            (Pending { since_ms }, true) => {
                if now_ms - since_ms >= self.rule.for_ms {
                    self.state = Firing;
                    Some(Notification::Fired)          // page once
                } else {
                    None                               // still waiting
                }
            }
            (Firing, true) => None,                    // already paged; stay quiet
            (Firing, false) => {
                self.state = Ok;
                Some(Notification::Resolved)
            }
            (Pending { .. }, false) => {
                self.state = Ok;                       // spike ended: reset timer
                None
            }
            (Ok, false) => None,
        }
    }

    pub fn is_breach(&self, value: f64) -> bool {
        value > self.rule.threshold
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    fn evaluator() -> AlertEvaluator {
        AlertEvaluator::new(AlertRule { threshold: 500.0, for_ms: 300_000 })
    }
```

```
    #[test]
    fn single_spike_never_pages() {
        let mut ev = evaluator();
        // p99 jumps over threshold for one 15s evaluation, then recovers
        assert_eq!(ev.evaluate(true, 0), None);        // -> Pending
        assert_eq!(ev.evaluate(false, 15_000), None);  // recovered: reset
        assert_eq!(ev.state, AlertState::Ok);
    }

    #[test]
    fn sustained_breach_pages_once_then_resolves() {
        let mut ev = evaluator();
        let mut t = 0u64;
        let step = 60_000;                             // evaluate every minute
        let mut notes = Vec::new();
        // breach for 6 consecutive minutes, `for` = 5 minutes
        for _ in 0..6 {
            if let Some(n) = ev.evaluate(true, t) { notes.push((t, n)); }
            t += step;
        }
        // fired exactly once, when the timer elapsed (t = 5 min)
        assert_eq!(notes, vec![(300_000, Notification::Fired)]);
        // keep breaching: silence
        assert_eq!(ev.evaluate(true, t), None);
        // recovery -> one resolved notification
        assert_eq!(ev.evaluate(false, t + step), Some(Notification::Resolved));
        assert_eq!(ev.state, AlertState::Ok);
    }

    #[test]
    fn threshold_boundary_is_not_a_breach() {
        let ev = evaluator();
        assert!(!ev.is_breach(500.0));                 // strictly greater
        assert!(ev.is_breach(500.1));
    }
}
```

4.14 Scaling the Platform

Layer	Scale axis	Mechanism
Agents	Fleet size (50k+)	Embarrassingly parallel: one per host; sizing is a per-host CPU/RAM budget, not a cluster problem
Buffer	Ingestion GB/s	Partitions (200) spread over 20–30 brokers; keyed by tenant+service so one team's flood is contained and quota'd (Chapter 3 callback)
Log indexers	Events/s	Stateless consumers, one per partition lane; autoscale on consumer lag
Log index	Data volume + query QPS	Time-sharded indices spread over nodes; hot tier replicated; incident

Layer	Scale axis	Mechanism
		query storms absorbed by the query-layer cache
Metrics writers	Samples/s	Stateless consumers; 0.7M samples/s is small — headroom is free
TSDB	Series cardinality × retention	Shard by metric-name hash; rollups shrink old data 100× before it becomes a storage problem
Alerting engine	Rule count × eval rate	Partition rules across evaluators by hash; evaluations are independent, so this scales linearly

KEY INSIGHT — The incident storm is the sizing case

Average load (2 GB/s) is not the design point; **everyone's worst day at once** is. An outage multiplies error logs 3–5×, and two thousand engineers open dashboards simultaneously. The buffer absorbs the write spike; the query cache absorbs the read spike; per-tenant quotas keep one service's panic from evicting the logs everyone else needs to debug it. A telemetry platform sized for the average is a platform that dies exactly when it is needed.

4.15 Failure Modes & Degradation

Failure	Symptom	Response
Telemetry agent dies	A host goes silent	Agents buffer locally, so restarts lose nothing; and silence itself is a signal — a heartbeat-absence alert fires (the alerting engine evaluates **absence**, not just thresholds)
Buffer fills / brokers down	Agents' local buffers grow	Local disk spooling buys hours; on recovery, consumers replay the backlog. Durability is the buffer's whole job
Log indexer lags or crashes	Search results fall behind ingest	Lag is visible as a metric on the buffer (meta-observability, Section 4.17); stateless indexers are replaced and replay from last offset
Index node lost	A shard of one day's index unavailable	Replicas on the hot tier serve queries; the shard is rebuilt from the buffer while it is still retained
TSDB node lost	Gap in dashboards	Replicas; and because metrics are statistics, a brief gap de-

Failure	Symptom	Response
		grades a chart without lying to an alert — `for` durations span it
Alerting engine down	The silent killer	It runs in an **independent failure domain** (separate infra, separate credentials), and a meta-monitor outside the platform watches its heartbeat — Section 4.17
Poison record crashes a parser	One consumer stuck in a crash loop	Dead-letter queue: N failed attempts → the record is parked aside, processing continues, an operator is notified
Storage budget blown	Hot tier at capacity	Enforced sampling on the noisiest tenants (never silent gaps — Section 4.2), accelerated warm/cold migration, quota renegotiation

COMMON PITFALL — Monitoring the monitor

The most dangerous failure of an observability platform is **looking healthy while being blind**: dashboards render cached panels, no alerts fire, and meanwhile ingestion silently stopped. Every layer must therefore emit telemetry about itself into an **independent** pipeline — if the platform's own metrics flow through the platform, its failure erases its own death certificate. Section 4.17 formalizes this as meta-observability.

4.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Metrics collection	Pull (scrape) for services + push gateway for batch jobs	Pure push: loses scrape-time up/down detection and centralized scheduling; pure pull: can't catch short-lived jobs
Log write path	Index at write time (inverted index)	Schema-on-read / index-at-query (cheap writes, brutal query scans) — fine for archive-only data; our FR-2 demands interactive search
Log retention	Hot/warm/cold tiers with time-sharded indices	Uniform hot storage: correct but 10× the cost for data that is queried <1% of the time
Metric resolution	Fixed 15 s + rollup tiers	High-res forever: cardinality × retention makes storage explode; variable/adaptive reso-

Decision	Chosen	Alternative & why not (here)
		lution: complex, inconsistent dashboards
Alert condition	Threshold + `for` duration + clear hysteresis	Fire on every breach: flapping burns trust; anomaly detection everywhere: powerful for **exploration**, too opaque as the paging contract
Buffer	Durable log (Kafka-class) between agents and processors	Direct agent→indexer RPC: no spike absorption, no replay, couples every producer to every consumer's outages
Degradation under overload	Shed metrics first, sample logs, never silently	Hard block (429 to agents): telemetry backs up into **application** hosts — the observer becomes the outage

4.17 SLOs & Meta-Observability

The platform that measures everyone else's SLOs needs its own — measured from **outside**, by a small independent "meta-monitor" in a separate failure domain:

Indicator	Definition	Target
Ingestion availability	Share of batches accepted by the buffer	≥ 99.9%
Log searchability delay	Event timestamp → present in index, p95	≤ 60 s
Metric freshness	Scrape timestamp → queryable, p95	≤ 30 s
Query latency (hot)	Log search / dashboard render, p95	≤ 5 s / ≤ 3 s
Alert latency	Condition true → notification sent, p95	≤ 1 min + `for`
Data loss	Acked bytes later found missing, per month	≈ 0 (buffer replay)
Pipeline health coverage	Hosts whose agent heartbeat is watched	100%

> **INTERVIEW TIP — State the SLO of the telemetry system in the interview**
>
> Candidates design observability for everyone else and forget the platform itself. Naming meta-observability — "who watches the watcher, and from which failure domain" — is a senior-level signal: it shows you have been paged by the monitoring system being down, not just by the systems it monitors.

4.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Add distributed tracing — what changes?"** A third pillar with its own shape: traces are **trees** (spans with parent ids), stored per-trace-id with heavy head-based or tail-based sampling (1–5% head, or tail-sampling that keeps errors and slow outliers). Trace ids are **correlation keys**: log records carry `trace_id` so a log search pivots to the trace, and exemplars link metric spikes to example traces. The platform's buffer and tiering lessons transfer directly.

2. **"How do you cut the log bill in half?"** Rank tenants by bytes; enforce structured logging; sample repetitive INFO lines at the agent (log 1-in-N, keep all WARN+); shorten hot retention for the noisiest services; negotiate quotas. The answer is organizational as much as technical.
3. **"PII ends up in logs — now what?"** Scrubbing is cheapest at the agent (pattern rules before anything leaves the host), enforced again at the indexer; access to raw logs is itself logged; cold-tier encryption keys rotate. Treat telemetry as production data with a blast radius.
4. **"A team creates a label with user ids in it — what breaks?"** Cardinality explosion (Section 4.9): the TSDB's series index balloons, queries slow globally, and you are now storing identifiers you may not be allowed to store. Guardrails: reject writes whose label sets exceed per-metric cardinality budgets, and alert **the platform team** on the growth itself.
5. **"Why not just use a vendor solution?"** Buy-versus-build: the architecture above is exactly what the vendors run internally; the interview question is whether you can reason about their trade-offs. At 170 TB/day the build case is real; at 170 GB/day it usually is not.

4.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A logging-and-metrics platform is **one firehose, two shapes of data**. Telemetry agents collect from every host (pull for services, push for batch jobs); a durable buffer absorbs the incident storm and decouples ingestion from processing; then the data forks: logs into a time-sharded **inverted index** (tokenization, posting lists, immutable segments, hot/warm/cold tiers that make retention a file deletion), metrics into a **time-series store** (delta-of-delta compression, label cardinality as the nemesis, mergeable rollups). Alerting is a per-rule state machine — OK → PENDING → FIRING — that pages on transitions, never on repetitions, and runs in its own failure domain. The platform watches itself from outside. The sizing insight: design for everyone else's worst day, because that is precisely when this system becomes the most important one in the building.

Further reading.

- The source video: *"14: Distributed Logging & Metrics Framework — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=p_q-n09B8KA`
- Pelkonen, Franklin, Teller, Cavallaro, Huang, Meza, Veeraraghavan — *"Gorilla: A Fast, Scalable, In-Memory Time Series Database"* (VLDB 2015) — the delta-of-delta / XOR compression scheme Section 4.9 distills.
- The Prometheus documentation — the pull model, the label model, and the `for` / alerting-rule semantics this chapter borrows.
- McCandless, Hatcher, Gospodnetić — *Lucene in Action* — inverted indices, segments, and merges in production depth.
- *Google SRE Book*, chapters "Monitoring Distributed Systems" and "Alerting on SLOs" — symptoms vs. causes, and the philosophy behind Section 4.12.

4.20 Chapter Glossary

Term	Meaning
Alert fatigue	The learned ignoring of alerts caused by excessive false or repeated pages; a system failure mode, engineered against with `for` durations and hysteresis
Backpressure	Propagation of slowness upstream through a pipeline, absorbed by queues, throttling, or deliberate shedding

Term	Meaning
Cardinality	The number of distinct time series a metric's label combinations produce; the metric platform's primary scaling constraint
Cold / warm / hot tier	Storage classes of decreasing cost and speed holding progressively older telemetry; retention is implemented by migrating and deleting time-sharded indices
Dead-letter queue (DLQ)	A side channel where records that repeatedly fail processing are parked so the pipeline proceeds
Delta-of-delta encoding	Compressing a timestamp series by storing the difference of consecutive deltas — zero for a steady scrape, near-zero with jitter
Downsampling / rollup	Replacing old high-resolution samples with precomputed mergeable aggregates (sum/min/max/count) per bucket
Failure domain	The set of components that can fail together; the alerting engine and meta-monitor must live outside the domain they watch
Flapping	An alert oscillating between firing and clearing as a metric jitters around its threshold
Hysteresis	Using different thresholds to enter and leave a state, preventing oscillation at a single boundary
Inverted index	Map from token to the sorted list of records containing it; the data structure behind text search
Label	A key=value dimension on a metric series; dimensions, never identifiers
Meta-observability	Telemetry about the telemetry platform itself, emitted into an independent pipeline so the platform's death cannot erase its own certificate
Observability / telemetry	The practice (and data) of inferring a system's internal state from its outputs; pillars: logs, metrics, traces
Posting list	The sorted list of record ids attached to one token in an inverted index; queries intersect posting lists
Pull (scrape) model	Metrics collection where the collector periodically requests samples from each target's endpoint, gaining up/down detection for free
Push model	Metrics collection where sources send samples upstream; suits short-lived jobs and outbound trust boundaries
Sample	One (timestamp, value) pair of a time series
Sampling (logs)	Keeping a deterministic 1-in-N subset of a repetitive log class; degrades volume without creating silent gaps
Schema-on-read	Indexing little at write time and interpreting data at query time; cheap writes, expensive queries
Segment (index)	An immutable shard of an inverted index; small segments are merged in the background

Term	Meaning
Structured logging	Emitting logs as fielded records (JSON) rather than free text, enabling field queries and smart sampling
Telemetry agent	The per-host daemon that tails logs, collects metrics, batches, buffers, and ships telemetry upstream
Time series	A named, labeled sequence of timestamped values; the atom of the metrics pillar
Tokenization	Splitting text into searchable terms at index time
Zigzag encoding	A signed-to-unsigned integer mapping that makes small magnitudes of either sign varint-cheap

— End of Chapter 4 · Next: Chapter 5—

CHAPTER 5

Designing a Hierarchical Comment System

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"15: Reddit Comments"* from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the commenting system of a Reddit-scale discussion site — nested reply threads, up/down voting, and the several ways users can sort a thread. The interesting parts are the **data shape** (a forest of unbounded trees) and the **ranking mathematics** (votes plus time decay plus statistical confidence), not raw storage volume — a deliberate contrast with Chapter 4's firehose. All terms are defined before use; all reference code is Rust with deterministic tests.

5.1 The Problem Statement

The interviewer draws a post with a familiar indented conversation underneath it and says:

"Users comment on posts and reply to each other, arbitrarily deep. They upvote and downvote comments. A thread can be sorted by 'best', 'top', 'new', or 'controversial'. Popular posts get tens of thousands of comments, so users page through them and expand subtrees on demand. Design the system that stores, ranks, and serves these comment threads."

The previous chapters designed infrastructure; this one designs a **user-facing product surface** whose entire value is ordering: the same fifty thousand comments are useful or useless depending on which fifteen the reader sees first. Two hard problems hide inside — representing arbitrarily deep trees so that subtrees can be fetched and paged cheaply, and ranking votes in a way that is **statistically honest** and **time-aware** at write rates of thousands of votes per second.

> **DEFINITION — Comment thread / tree**
>
> The replies under one post form a *tree*: the post is the root context, each comment has exactly one parent (the post or another comment), and replies fan out to arbitrary depth and breadth. The set of trees under all posts is a *forest*. "The thread" usually means one post's whole tree, or a rendered window into it.

> **DEFINITION — Score / vote tally**
>
> A comment's *score* is `upvotes - downvotes`, the single number users see. Votes serve two masters: they produce the visible score, and they feed the **ranking functions** (Section 5.8) that decide display order — and those two consumers want different things from the same data, which is where the design gets interesting.

5.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
How deep can nesting go?	Unbounded in principle; the UI collapses past 10 levels with a "continue this thread" link
Edit and delete?	Edits allowed briefly (mark "edited"); deletes are soft — the comment becomes a tombstone but its replies survive
Are votes final?	No — users can change or retract votes; one vote per user per comment, enforced
Real-time?	New comments should appear within seconds on refresh; no live push needed. Counts can lag a little
Moderation?	Assume a separate moderation pipeline flags/removes; we must honor removals fast. Don't design moderation itself
Sort orders?	best, top, new, controversial — "best" is the default and the tricky one
Scale?	Reddit-scale: 70M daily users, viral posts with 10^5 comments
Anonymous reading?	Yes — most reads are logged-out; that population is cache-friendly
Media in comments?	Text plus links only; media uploads are the Maps chapter's CDN problem, already solved

NOTE — Agreed scope

1. **Post and reply** to arbitrary depth; soft-delete with surviving replies.
2. **Vote** up/down/retract; exactly one vote per user per comment.
3. **Sort** a thread by best / top / new / controversial, with sound mathematics behind each.
4. **Page** through large threads: windowed top-level listing, expandable subtrees, collapsed deep chains.
5. **Counts**: post-level comment counts and comment scores may be seconds stale, never lost.
6. Out: live push updates, moderation tooling, rich media, recommendations.

5.3 Functional Requirements

DEFINITION — Ranking mode

One of the supported orderings of a thread's comments. *Top*: highest score. *New*: most recent first. *Best*: highest **statistical confidence** of being liked (Section 5.8's Wilson score — not the same as top!). *Controversial*: most balanced **and** voluminous disagreement. Each mode is a different pure function of the same vote data — an important honesty property.

1. **FR-1 — Create.** Post a top-level comment on a post, or a reply to any existing comment, recording parent, author, timestamp, and body.
2. **FR-2 — Vote.** Cast, change, or retract one up/down vote per user per comment; the score reflects the change within seconds.
3. **FR-3 — Read a thread window.** Given a post, a ranking mode, and a cursor, return the next page of the ranked tree — top-level comments each with the first few levels of their best children.
4. **FR-4 — Expand a subtree.** Given any comment, return the next window of its children ("more replies", "continue this thread").

5. **FR-5 — Soft-delete and removal.** Deleted comments render as a tombstone that preserves thread structure; moderator removals disappear entirely, quickly.
6. **FR-6 — Post-level counters.** Comment counts per post for listing pages, eventually consistent within seconds.

5.4 Non-Functional Requirements

DEFINITION — Read-to-write ratio

The proportion of read requests to write requests a system serves. It dictates architecture: high ratios justify aggressive caching and denormalization (precomputing for reads at write time), because every write is amortized across thousands of reads. Comment systems are extreme — Section 5.5 derives roughly **35:1** on requests, and far higher on bytes.

Requirement	Target & reasoning
Thread read latency	p95 ≤ 200 ms for a cached window, ≤ 700 ms uncached; this is the product's main surface
Write latency	p95 ≤ 300 ms for comment and vote writes; users tolerate a beat before their own comment appears
Availability	Reads ≥ 99.99% (stale cache is acceptable degradation); writes ≥ 99.9%
Durability	A confirmed comment or vote is never lost — it is social content, not telemetry (contrast Chapter 4's metrics)
Vote integrity	One-vote-per-user enforced exactly; tallies reconcile to the true vote log eventually
Consistency	Read-your-own-write for the author; seconds of staleness acceptable for everyone else
Ranking freshness	A vote visibly affects ordering within 1 minute on hot threads; scores tick visibly on refresh
Cost	Storage is cheap here (11 TB/year); spend engineering on read shape and ranking, not capacity

KEY INSIGHT — This is a shape problem, not a size problem

Chapter 4 drowned in bytes (170 TB/day); this chapter stores 30 GB/day — three orders of magnitude less. The difficulty is that the data is a forest of unbounded trees that must be **ranked four ways, paged stably, and re-ranked continuously** as votes arrive. Interviewers use this problem to test data modeling and algorithmic reasoning, not capacity arithmetic.

5.5 Back-of-the-Envelope Estimation

Assumptions (stated, then derived from): 70M daily active users; 20% read comment threads, averaging 10 thread views; 1 in 35 thread readers writes a comment; every comment author and 10 lurkers vote on comments per reader; average comment body 500 bytes.

Quantity	Estimate	Derivation
Thread views	700M/day ≈ 8k/s avg, 40k/s peak	14M readers × 10 views; peak ×5 for a viral moment
Comments written	20M/day ≈ 230/s avg, 1.2k/s peak	20M posts' worth of discussion; 1-in-35 readers comment
Votes cast	200M/day ≈ 2.3k/s avg, 12k/s peak	10 votes per active voter; the write-heavy path
Comment storage	30 GB/day → 11 TB/year	20M × 1.5 KB with ids, indexes, overhead
Vote records	40 GB/day raw, compacted small	200M × 50 B (user, comment, value, time); idempotency needs them
Hot cache	50 GB	Top 1M threads' first ranked windows × 50 KB
Read bandwidth	800 MB/s at avg load	8k views/s × 100 KB rendered window

KEY INSIGHT — Votes, not comments, are the write storm

Comments arrive at 230/s; **votes** at 2.3k/s average and 12k/s peak — and each vote is a read-modify-write on **at least four** things: the voter's idempotency record, the comment's up/down tallies, the cached score, and eventually the ranked order. Naively, that is a hot-row update per vote on viral comments. The vote pipeline (Section 5.11) exists to make the busiest path in the system also the cheapest.

The pipeline of the rest of the chapter: Section 5.6 isolates the two hard problems; 5.7 models the trees; 5.8 does the ranking mathematics; 5.9–5.11 design APIs, architecture, and the vote pipeline; 5.12 serves threads; 5.13 implements all of it in Rust; 5.14–5.20 scale, harden, and review.

5.6 The Core Challenge: Trees and Honest Ordering

Two problems carry this design, and neither is capacity.

Problem one — represent a forest of unbounded trees so that windows of it are cheap to read. Every read asks a deceptively simple question: "give me top-level comments 41–60 by best, each with its first two levels of best children." A relational row per comment answers that badly unless the schema is designed for it; Section 5.7 compares the three classical tree encodings and picks a hybrid.

Problem two — rank honestly. "Best" cannot mean "highest score": a comment at +1 (1 up, 0 down) would outrank none and a comment at +190 (200 up, 10 down) deserves to beat one at +2 (2 up, 0 down) — raw differences ignore **sample size**. Ranking must be a statistical statement about votes, plus a time statement (freshness), and it must be recomputed continuously as votes stream in. Section 5.8 derives the three functions used in production practice.

COMMON PITFALL — Score is not rank

Sorting "best" by `up - down` is the most common wrong answer in this interview. It ranks a 1-up-0-down comment above a 200-up-10-down one (+1 vs +190 is fine, but +2 vs +190 is not — and +1, 0 down beats +190 never, yet **percentage** sorting fails oppositely: 1/1 = 100% beats 200/210 = 95%). Both

naive orders are statistically illiterate; the fix is a confidence bound (Section 5.8). Say this early in the interview — it is the question the interviewer is waiting for.

5.7 Data Model: Encoding Comment Trees

DEFINITION — Adjacency list / materialized path / nested sets

The three classical ways to store trees in a relational store. *Adjacency list*: each row stores `parent_id` — trivial writes, but fetching a subtree needs recursive queries. *Materialized path*: each row stores its whole ancestor chain (`/a1/b7/c3`), so a subtree is one `WHERE path LIKE '/a1/b7/%'` prefix scan and depth is the path length — at the cost of longer keys and painful subtree **moves**. *Nested sets*: each row stores `lft` / `rgt` bounds from a depth-first numbering, making subtrees a range scan but every **insert** renumbers half the tree — catastrophic at comment write rates.

Property	Adjacency list	Materialized path	Nested sets
Insert a reply	One row	One row (path = parent's + own id)	One row + mass renumbering
Fetch a subtree	Recursive	One prefix/range scan	One range scan
Fetch top-level page	Indexed `parent_id IS NULL` scan	`depth = 1` or path prefix	Range union
Depth of a comment	Unknown without walking	Path segment count	Stored or derived
Move a subtree	One update	Rewrite all descendant paths	Renumber the tree
Fit for comments	Good bones	**Best fit** — subtrees are read, rarely moved	Unacceptable writes

The chosen schema — adjacency bones, path muscle, denormalized tallies:

```
comments
  comment_id      bigint pk
  post_id         bigint          -- shard key (Section 5.14)
  parent_id       bigint null     -- adjacency: direct parent
  path            text            -- materialized: /c1/c7/c42 (segment per ancestor)
  depth           smallint        -- = segments in path; drives collapse rules
  author_id       bigint
  body            text            -- null when tombstoned
  created_utc     timestamp
  up / down       bigint          -- denormalized tallies (vote pipeline owns them)
  state           enum            -- live | tombstone | removed
  index (post_id, path)           -- serves every window query in Section 5.12
```

`votes` is a separate table keyed `(user_id, comment_id)` — the idempotency backbone of Section 5.11. A post's `comment_count` lives denormalized on the `posts` row, maintained asynchronously (FR-6).

DEFINITION — Tombstone (soft delete)

A deletion that erases content but keeps structure: body and author are blanked, `state` becomes `tombstone`, and the row **stays** so its replies still have a parent. Rendering shows "[deleted]". Hard deletion would orphan entire subtrees; moderation removals (`removed`) are the exception — they may take the subtree with them per policy.

5.8 Deep Dive: The Ranking Mathematics

Three pure functions over the same two integers (`up`, `down`) plus time. Each answers a different question honestly.

5.8.1 "Top" and "New" — trivial orders

`top` sorts by score `up - down` descending (ties by age, newest first); `new` sorts by `created_utc` descending. Both are indexable directly. Their weakness is statistical, which is exactly why "best" exists.

5.8.2 "Best" — the Wilson lower confidence bound

DEFINITION — Wilson score interval (lower bound)

Treat each comment's votes as `n = up + down` Bernoulli trials estimating the **true probability** `p` that a random viewer upvotes it. A confidence interval around the observed ratio `p̂ = up/n` narrows as `n` grows; the *lower bound* of the Wilson interval (at z = 1.96, ≈95%) is a pessimistic estimate of `p` that fairly compares comments with **different sample sizes**. Small-sample comments are penalized toward 0; as `n` → ∞ the bound approaches `p̂`. It needs only `up` and `down`, is monotonic in both, and costs a few flops — computable at vote time and stored.

$$\text{wilson}(u,d) = \frac{\hat{p} + \frac{z^2}{2n} - z\sqrt{\frac{\hat{p}(1-\hat{p}) + \frac{z^2}{4n}}{n}}}{1 + \frac{z^2}{n}}, \quad n = u + d, \hat{p} = u/n$$

The numbers that make the point: a comment with 1 up / 0 down scores ≈ 0.21; with 10 / 0, ≈ 0.72; with 100 / 0, ≈ 0.96. And 100 up / 100 down (p̂ = 50% but n = 200, ≈ 0.42) correctly beats 1 / 0 — volume of evidence matters more than a lone perfect record. Section 5.13 implements and tests exactly these.

5.8.3 "Controversial" — balanced disagreement, weighted by volume

$$\text{controversial}(u,d) = \begin{cases} 0 & \text{if } \min(u,d) = 0 \\ (u+d)^{\min(u,d)/\max(u,d)} & \text{otherwise} \end{cases}$$

A 50/50 split on 100 votes (`balance` = 1, magnitude 100) crushes a 105-vote comment with a 100/5 split (balance = 0.05 → magnitude ≈ 1.26): near-even splits dominate, and among equal splits the **louder** argument wins.

5.8.4 "Hot" — score with time decay (for posts and fast threads)

DEFINITION — Time-decay ranking ("hot")

Reddit's post ranking: `"hot" = log_10 max(|s|, 1) + "sign"(s) · (t - t_0) / 45000`, with `s = up - down`, `t` the comment's epoch seconds, and `t_0` an arbitrary fixed epoch. The log makes the first 10 votes worth as much as the next 100 (diminishing returns); dividing age by 45,000 s (12.5 h) means a post must be **10× more popular** to tie one 12.5 hours newer. Fresh content cycles through the front page; score advantages decay gracefully instead of cliffing.

KEY INSIGHT — Rank is a stored, recomputed column — not a query-time sort

All four functions are pure over `(up, down, created_utc)`. So the vote pipeline recomputes a comment's rank keys **on every vote** and stores them; reads are then index scans, never sorts. "Best" order changes only when votes change — and votes arrive at 2.3k/s, exactly the pipeline built in Section 5.11. Sorting 50k comments per page-view would be malpractice.

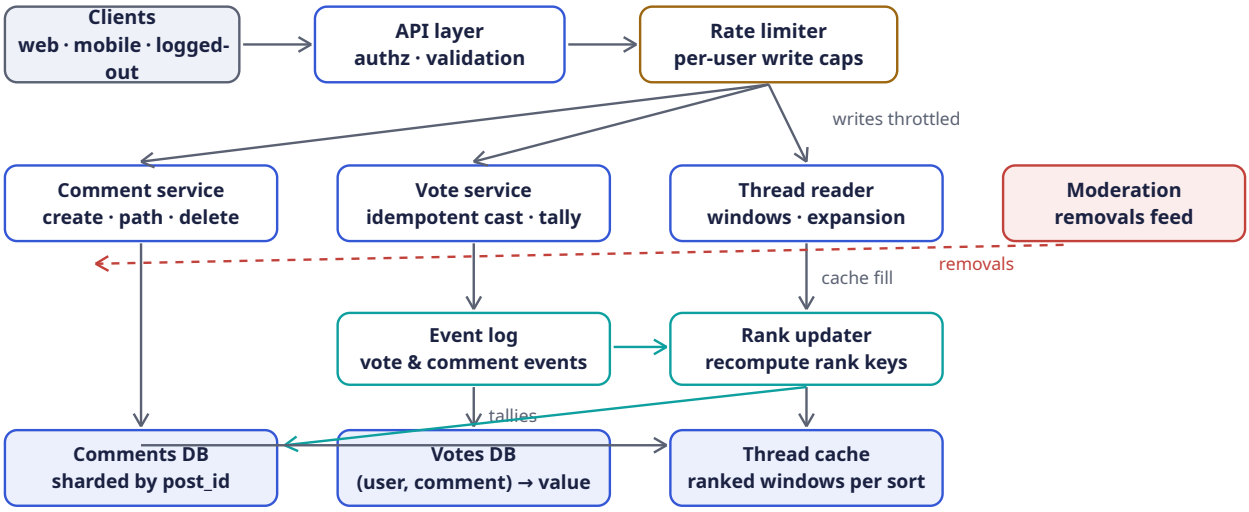
5.9 API Design

Method	Path	Purpose
`POST`	`/v1/posts/{id}/comments`	Top-level comment (FR-1); body, author
`POST`	`/v1/comments/{id}/replies`	Reply to a comment (FR-1); server computes path, depth
`PUT`	`/v1/comments/{id}/vote`	Cast/change/retract vote: `{value: 1|-1|0}` — idempotent (FR-2)
`GET`	`/v1/posts/{id}/comments?sort=&cursor=&limit=`	Ranked thread window with shallow children (FR-3)
`GET`	`/v1/comments/{id}/children?cursor=&limit=`	Expand a subtree window (FR-4)
`DELETE`	`/v1/comments/{id}`	Soft-delete → tombstone (FR-5); author-only
`GET`	`/v1/posts/{id}/comments/count`	Post-level counter (FR-6); cacheable, seconds-stale

Two deliberate choices:

- **Opaque cursors, never offsets.** A cursor encodes the last seen rank key and comment id (`base64(wilson, id)`); offsets break the moment a new comment inserts above the reader's position — Chapter 1's cursor lesson applied to ranked lists. Ties are impossible: `(rank_key, comment_id)` is a total order.
- **Votes are `PUT`, not `POST`.** Repeating or flipping a vote is one idempotent operation on a `(user, comment)` resource, not an event stream the client can double-fire. Retries are free.

5.10 High-Level Architecture



Component	Responsibility	Why it is shaped this way
API layer	Auth, validation, routing	Stateless; scales horizontally behind a load balancer
Rate limiter	Per-user caps on comment & vote writes	Chapter 3 verbatim: comments/hour, votes/minute — the first anti-brigading wall
Comment service	Create replies (computing `path` / `depth`), edits, tombstones	Owns the tree invariants; the only writer of `comments` rows
Vote service	Idempotent cast/change/retract; owns `votes` table	One writer per vote record; the idempotency enforcer (Section 5.11)
Event log	Durable stream of vote & comment events	Decouples tally/rank recomputation from the write path — Chapter 4's buffer lesson
Rank updater	Recompute tallies, Wilson, hot, controversial; write rank keys; bump post counters	Turns 2.3k votes/s into batched, coalesced rank updates
Comments DB	Tree rows, sharded by `post_id`	One post's whole thread lives on one shard — window queries stay single-shard
Votes DB	`(user, comment) → value`	Sharded by `comment_id`; the ground truth for reconciliation
Thread cache	Pre-rendered ranked windows per post × sort	35:1 read ratio + logged-out majority = the tier that actually serves traffic
Moderation feed	Removals pushed to comment service	Honored in seconds (agreed scope); separate pipeline, not designed here

5.11 Deep Dive: The Vote Pipeline

The busiest path in the system deserves the most engineering. Requirements: one vote per user (exactly), changeable, never lost, score visible in seconds, rank keys refreshed continuously.

> **DEFINITION — Idempotent write / natural idempotency key**
>
> A write that can be applied any number of times with the same effect. Retries, double-clicks, and client bugs make **at-least-once** delivery the honest assumption, so the handler must deduplicate. Here the key is free: `(user_id, comment_id)` is the natural primary key of a vote — one row per pair, upserted, is the deduplication mechanism itself. No tokens, no expiry windows (contrast Chapter 3's idempotency keys on payment APIs).

The write path for `PUT /v1/comments/c42/vote {value: -1}`:

1. **Rate-limit** the user (Chapter 3: sliding window, votes/minute).
2. **Upsert** `votes[(u, c42)] = -1` in one statement returning the previous value — the atomic read-modify-write that Chapter 3's TOCTOU section warns about; the unique key makes it safe:

```
-- one atomic statement; all clauses share one snapshot, so `old`
-- reads the pre-upsert value
WITH old AS (
  SELECT value FROM votes WHERE user_id = $1 AND comment_id = $2
)
INSERT INTO votes (user_id, comment_id, value, updated_utc)
VALUES ($1, $2, $3, now())
ON CONFLICT (user_id, comment_id)
DO UPDATE SET value = $3, updated_utc = now()
RETURNING (SELECT value FROM old) AS prev_value;
```

3. **Emit** a `VoteChanged{user, comment, prev, new}` event to the log. The request returns here — the write path is **two** durable operations.
4. **Rank updater** consumes events, **coalesces** bursts per comment (200 votes in a second become one recompute), recomputes tallies `up += (new==1) - (prev==1)`, `down += (new==-1) - (prev==-1)`, re-derives Wilson/hot/controversial, writes them back, and **version-stamps** the thread cache so the next read repopulates.

> **KEY INSIGHT — Coalescing is the whole trick**
>
> A viral comment can take thousands of votes per minute, but its **rank key** only needs recomputing a few times a minute — readers cannot perceive faster change, and sorting is stable between recomputes. Treating votes as a stream to be compacted (latest-wins per user; batch-sum per comment) turns the hottest write path into a trickle of rank updates. The score users see may lag the truth by seconds — the agreed scope allows exactly that, and spends the savings on reads.

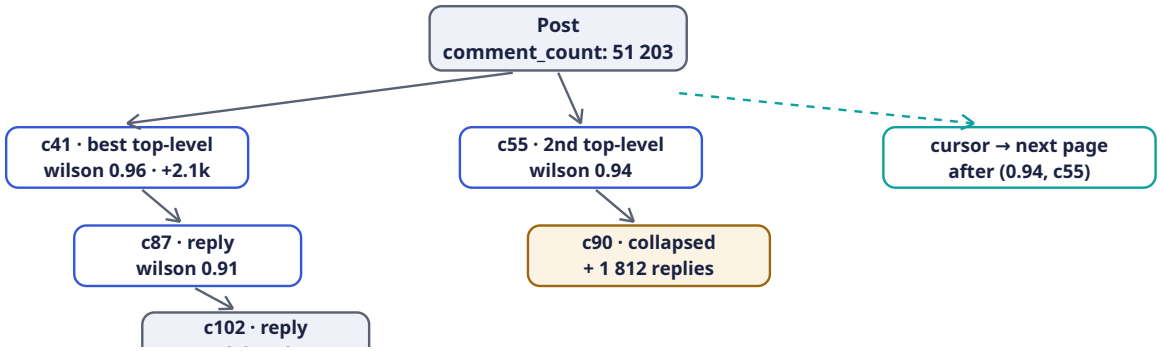
Reconciliation. Tallies are derived state; `votes` is ground truth. A nightly job (and a per-comment repair tool) recomputes tallies from `votes` and fixes drift — the same discipline as Chapter 4's buffer replays: **derived numbers must be rebuildable from the event record.**

5.12 Deep Dive: Reading a Thread Window

A thread read assembles a **window** of the tree: a page of top-level comments, each carrying its first levels of best children, with everything past the depth cap collapsed behind an affordance. With the schema of Section 5.7 the whole window is two queries:

1. **Top-level page**: `WHERE post_id = $1 AND depth = 1 ORDER BY rank_key DESC, comment_id DESC LIMIT $2` with the cursor supplying the rank-key boundary — a single index range scan on `(post_id, rank)` per sort mode.
2. **Children fill**: for the ≤20 parents in the page, one batched prefix scan `WHERE post_id = $1 AND path ~ ANY ('/c41/%', '/c87/%', ...) AND depth <= 4 ORDER BY path` — materialized paths turn "first two levels of every parent on the page" into **one** query instead of N recursive ones (the N+1 that sinks the naive adjacency-list design).

Assembly is in memory: parents in rank order, children nested under them by path prefix, tombstones rendered as "[deleted]" placeholders, subtrees beyond depth 10 replaced by a `{continue_thread: comment_id}` token the client expands through `/children` (FR-4).



Caching. The rendered window JSON is cached under `(post_id, sort, cursor)` for logged-out readers — the majority (Section 5.2) — and the rank updater **version-stamps** the post on every coalesced update, so stale windows age out within seconds without any invalidation fanout. Logged-in readers get the same window plus a per-user vote overlay (`SELECT comment_id, value FROM votes WHERE user_id = $1 AND comment_id = ANY(...)`) merged at read time — personalization over a shared cache, instead of a cache per user.

> **INTERVIEW TIP — Window, don't tree**
>
> The API never returns "the thread" — only windows: ranked top-level pages, shallow children, explicit expansions. A 50k-comment thread is 50k rows the user will never scroll; serving the **shape** of the conversation (best subthreads first, depth capped) is both the product-correct and the systems-correct answer. Candidates who serialize whole trees lose the room.

5.13 Rust Reference Implementations

Four pieces with deterministic tests: the ranking trio, the tree model with windowing, the idempotent vote service, and cursor pagination.

5.13.1 The Ranking Functions

```rust
/// z = 1.96 -> ~95% confidence. The lower Wilson bound of the true
/// upvote probability, from (up, down) alone. Pessimistic for small n.
pub fn wilson_lower(up: u64, down: u64) -> f64 {
    let n = (up + down) as f64;
    if n == 0.0 { return 0.0; }
    let z: f64 = 1.96;
    let phat = up as f64 / n;
    let denom = 1.0 + z * z / n;
```

```
    let centre = phat + z * z / (2.0 * n);
    let margin = z * ((phat * (1.0 - phat) + z * z / (4.0 * n)) / n).sqrt();
    (centre - margin) / denom
}

/// Reddit's "hot": log score + age bonus. 45_000s = 12.5h; a comment must
/// be 10x more popular to tie one 12.5h newer. t0 is a fixed epoch.
pub fn hot(up: u64, down: u64, epoch_secs: i64) -> f64 {
    const T0: i64 = 1_134_028_003; // Reddit's original launch epoch
    let s = up as i64 - down as i64;
    let order = s.unsigned_abs().max(1) as f64;
    let order = order.log10();
    let sign = (s > 0) as i8 as f64 - (s < 0) as i8 as f64;
    order + sign * (epoch_secs - T0) as f64 / 45_000.0
}

/// Balanced disagreement weighted by volume: (u+d)^(min/max).
pub fn controversial(up: u64, down: u64) -> f64 {
    let (lo, hi) = (up.min(down), up.max(down));
    if lo == 0 { return 0.0; }
    ((up + down) as f64).powf(lo as f64 / hi as f64)
}

#[cfg(test)]
mod ranking_tests {
    use super::*;

    #[test]
    fn wilson_grows_with_sample_size_at_same_ratio() {
        // all 100% approval, increasing n: 0.21 -> 0.72 -> 0.96
        let (w1, w10, w100) =
            (wilson_lower(1, 0), wilson_lower(10, 0), wilson_lower(100, 0));
        assert!((w1 - 0.2065).abs() < 1e-3, "w1 = {w1}");
        assert!((w10 - 0.7225).abs() < 1e-3, "w10 = {w10}");
        assert!((w100 - 0.9630).abs() < 1e-3, "w100 = {w100}");
        assert!(w1 < w10 && w10 < w100);
    }

    #[test]
    fn wilson_prefers_proven_mediocre_to_lucky_perfect() {
        // 100/100 (phat 50%, n 200) beats 1/0: evidence over luck.
        let proven = wilson_lower(100, 100);
        assert!((proven - 0.4307).abs() < 1e-3, "{proven}");
        assert!(proven > wilson_lower(1, 0));
        assert!(wilson_lower(0, 0) == 0.0);
        assert!(wilson_lower(0, 5) < wilson_lower(1, 5));
    }

    #[test]
    fn hot_decay_needs_10x_score_per_12h30m() {
        let t = 1_700_000_000;
        let older = hot(100, 0, t);
        let newer = hot(10, 0, t + 45_000);
        assert!((older - newer).abs() < 1e-9);
        assert!(hot(0, 10, t) < hot(0, 1, t)); // downvotes push below zero-side
    }

    #[test]
    fn controversial_needs_both_balance_and_volume() {
        let even_split = controversial(50, 50);      // 100^1.0
```

```
        let loud_blowout = controversial(100, 5);    // 105^0.05
        assert!((even_split - 100.0).abs() < 1e-9);
        assert!((loud_blowout - 1.2619).abs() < 1e-3, "{loud_blowout}");
        assert!(even_split > loud_blowout);
        assert_eq!(controversial(0, 100), 0.0);      // no disagreement
    }
}
```

5.13.2 The Tree Model: Paths, Depth, Tombstones

```
use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum State { Live, Tombstone }

#[derive(Debug, Clone)]
pub struct Comment {
    pub id: u64,
    pub parent_id: Option<u64>,
    pub created_ms: u64,
    pub up: u64,
    pub down: u64,
    pub state: State,
    pub body: String,
}

impl Comment {
    pub fn score(&self) -> i64 { self.up as i64 - self.down as i64 }

    /// Materialized path segment: parent path + own id.
    pub fn path(&self, parent_path: &str) -> String {
        format!("{}/{}", parent_path, self.id) // "" -> "/1" -> "/1/2" -> ...
    }

    pub fn render_body(&self) -> &str {
        match self.state {
            State::Live => &self.body,
            State::Tombstone => "[deleted]",
        }
    }
}

/// Depth-first flatten of a thread for rendering: parents before children,
/// siblings ranked by score (stand-in for "best" at read time). Tombstones
/// keep their position so replies are not orphaned.
pub fn flatten_thread(rows: &[Comment]) -> Vec<(usize, &Comment)> {
    let mut children: HashMap<Option<u64>, Vec<&Comment>> = HashMap::new();
    for c in rows {
        children.entry(c.parent_id).or_default().push(c);
    }
    for kids in children.values_mut() {
        kids.sort_by(|a, b| {
            b.score()
                .cmp(&a.score())
                .then(b.created_ms.cmp(&a.created_ms)) // newer wins ties
                .then(a.id.cmp(&b.id))                 // total order: no ambiguity
        });
    }
    let mut out = Vec::with_capacity(rows.len());
```

```
    let mut stack: Vec<(usize, &Comment)> = Vec::new();
    if let Some(roots) = children.get(&None) {
        for c in roots.iter().rev() {
            stack.push((1, *c)); // top-level comments have depth 1
        }
    }
    while let Some((depth, c)) = stack.pop() {
        out.push((depth, c));
        if let Some(kids) = children.get(&Some(c.id)) {
            for k in kids.iter().rev() {
                stack.push((depth + 1, *k));
            }
        }
    }
    out
}

#[cfg(test)]
mod tree_tests {
    use super::*;

    fn c(id: u64, parent: Option<u64>, up: u64, state: State) -> Comment {
        Comment { id, parent_id: parent, created_ms: id * 1000,
                  up, down: 0, state, body: format!("body {id}") }
    }

    #[test]
    fn window_is_dfs_with_ranked_siblings() {
        let rows = vec![
            c(1, None, 10, State::Live),
            c(2, Some(1), 5, State::Live),
            c(3, Some(1), 8, State::Live),
            c(4, Some(2), 1, State::Live),
            c(5, None, 20, State::Tombstone),
            c(6, Some(5), 3, State::Live),
        ];
        let flat = flatten_thread(&rows);
        let ids: Vec<u64> = flat.iter().map(|(_, c)| c.id).collect();
        // roots by score: 5 (20) then 1 (10); children of 1: 3 (8) then 2 (5)
        assert_eq!(ids, vec![5, 6, 1, 3, 2, 4]);
        // depth tracks nesting
        assert_eq!(flat.iter().find(|(_, c)| c.id == 4).unwrap().0, 3);
        assert_eq!(flat.iter().find(|(_, c)| c.id == 1).unwrap().0, 1);
    }

    #[test]
    fn tombstone_hides_body_but_keeps_subtree() {
        let rows = vec![c(5, None, 20, State::Tombstone),
                        c(6, Some(5), 3, State::Live)];
        let flat = flatten_thread(&rows);
        assert_eq!(flat[0].1.render_body(), "[deleted]");
        assert_eq!(flat[1].1.render_body(), "body 6");
    }

    #[test]
    fn materialized_paths_compose() {
        let (c1, c2, c4) = (c(1, None, 0, State::Live),
                            c(2, Some(1), 0, State::Live),
                            c(4, Some(2), 0, State::Live));
        let p1 = c1.path("");
```

```
        let p2 = c2.path(&p1);
        assert_eq!(c4.path(&p2), "/1/2/4");
        assert_eq!(p2.matches('/').count(), 2); // segment count == depth
    }
}
```

5.13.3 The Idempotent Vote Service

```
use std::collections::HashMap;

/// Natural idempotency key: (user, comment). One row per pair; upserts.
#[derive(Default)]
pub struct VoteService {
    votes: HashMap<(u64, u64), i8>, // value in {-1, 0, +1}; 0 == retracted
}

impl VoteService {
    /// Cast / change / retract a vote. Returns (delta_up, delta_down) for
    /// the rank updater to apply to the comment's tallies — the whole
    /// effect of the vote in two integers.
    pub fn cast(&mut self, user: u64, comment: u64, value: i8) -> (i64, i64) {
        assert!((-1..=1).contains(&value));
        let prev = self.votes.insert((user, comment), value).unwrap_or(0);
        let delta = |old: i8, new: i8, side: i8| {
            ((new == side) as i64) - ((old == side) as i64)
        };
        (delta(prev, value, 1), delta(prev, value, -1))
    }
}

#[cfg(test)]
mod vote_tests {
    use super::*;

    #[test]
    fn repeat_and_flip_and_retract() {
        let mut svc = VoteService::default();
        assert_eq!(svc.cast(7, 42, 1), (1, 0));   // new upvote
        assert_eq!(svc.cast(7, 42, 1), (0, 0));   // retry: no double count
        assert_eq!(svc.cast(7, 42, -1), (-1, 1)); // flip: score swings by 2
        assert_eq!(svc.cast(7, 42, 0), (0, -1));  // retract
        assert_eq!(svc.cast(8, 42, 1), (1, 0));   // another user unaffected
    }

    #[test]
    fn folding_deltas_reproduces_true_tallies() {
        let mut svc = VoteService::default();
        let deltas = [svc.cast(7, 42, 1), svc.cast(7, 42, -1),
                      svc.cast(8, 42, 1)];
        let (up, down) = deltas
            .iter()
            .fold((0i64, 0i64), |(u, d), &(du, dd)| (u + du, d + dd));
        assert_eq!((up, down), (1, 1)); // 7's flip + 8's upvote
    }
}
```

5.13.4 Cursor Pagination over a Ranked List

```rust
/// Map an f64 to bits whose unsigned order matches the float's order —
/// so rank keys can live in opaque cursors and database indexes.
pub fn ordered_bits(f: f64) -> u64 {
    let b = f.to_bits();
    if b & (1 << 63) == 0 { b | (1 << 63) } else { !b }
}

/// Total order per comment: (rank desc, id desc). No ties, ever.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub struct RankKey {
    pub wilson: u64, // ordered_bits(wilson_lower(up, down))
    pub id: u64,
}

pub type Cursor = RankKey; // the client treats it as opaque base64

/// Next page strictly after `after` from a list sorted descending.
pub fn page_after(
    sorted_desc: &[RankKey],
    after: Option<Cursor>,
    limit: usize,
) -> (Vec<RankKey>, Option<Cursor>) {
    let start = match after {
        None => 0,
        Some(c) => sorted_desc
            .iter()
            .position(|&k| k == c)
            .map(|i| i + 1)
            .unwrap_or(sorted_desc.len()),
    };
    let page: Vec<RankKey> =
        sorted_desc.iter().skip(start).take(limit).copied().collect();
    let next = match page.last() {
        Some(&last) if start + page.len() < sorted_desc.len() => Some(last),
        _ => None,
    };
    (page, next)
}

#[cfg(test)]
mod page_tests {
    use super::*;
    use crate::wilson_lower; // from the ranking listing (same crate)

    fn key(up: u64, down: u64, id: u64) -> RankKey {
        RankKey { wilson: ordered_bits(wilson_lower(up, down)), id }
    }

    #[test]
    fn ordered_bits_preserves_float_order() {
        assert!(ordered_bits(0.72) > ordered_bits(0.21));
        assert!(ordered_bits(-1.0) < ordered_bits(0.0));
    }

    #[test]
    fn pages_cover_everything_once_and_are_stable() {
        let mut keys = vec![
            key(100, 0, 1), // 0.963
            key(10, 0, 2),  // 0.722
            key(10, 0, 3),  // 0.722 — tie with id 2, broken by id desc
```

```
            key(1, 0, 4),   // 0.207
            key(0, 1, 5),   // 0.0
        ];
        keys.sort_by(|a, b| b.cmp(a)); // (wilson desc, id desc)

        let (p1, c1) = page_after(&keys, None, 2);
        let (p2, c2) = page_after(&keys, c1, 2);
        let (p3, c3) = page_after(&keys, c2, 2);
        let ids: Vec<u64> =
            [p1.as_slice(), &p2, &p3].concat().iter().map(|k| k.id).collect();
        assert_eq!(ids, vec![1, 3, 2, 4, 5]); // no skips, no duplicates
        assert_eq!(c1.unwrap().id, 3);        // cursor = last item of page
        assert!(c3.is_none());                // exhausted
    }
}
```

5.14 Scaling the Platform

Layer	Scale axis	Mechanism
API / services	Request rate	Stateless; horizontal scaling behind load balancers
Comments DB	Posts × comments	Shard by `post_id` — a whole thread on one shard, so every window query is single-shard; viral posts are one **hot** shard, handled by cache, not resharding
Votes DB	Users × votes	Shard by `comment_id` — tallies per comment stay local; lookup by user is the rare path (vote overlay), served by a secondary index
Event log / rank updater	Vote rate 2.3k/s avg	Partitioned by `comment_id`; coalescing collapses bursts — consumer count tracks partitions, not votes
Thread cache	Concurrent readers	Window JSON is immutable between version stamps → any number of replicas; logged-out traffic is fully cacheable
Read bandwidth	800 MB/s avg	Edge/CDN for logged-out windows; the origin only serves logged-in overlays and cache misses

KEY INSIGHT — The viral-post problem is a cache problem, not a database problem

A post that makes the front page concentrates a meaningful share of the **entire site's** read QPS onto one shard's rows. Sharding cannot help — one post has one home. What helps: the ranked-window cache (one fill serves millions of reads between version stamps), read replicas of the hot shard for

cache misses, and coalesced rank updates so the vote storm does not translate into row-write storms. Measure success as **origin QPS during a viral event** staying flat.

5.15 Failure Modes & Degradation

Failure	Symptom	Response
Tally drift	Displayed score ≠ vote log sum	Expected in small doses (coalescing, replays); nightly reconciliation recomputes tallies from `votes`; a repair endpoint fixes single comments
Rank updater lag	Scores update, order doesn't	Visible as consumer lag (Chapter 4's meta-metrics); degraded mode = windows re-sorted by stored (stale) rank keys — the product blurs, never breaks
Event log outage	Writes accepted, ranks frozen	Vote service buffers events locally (Chapter 4's agent pattern); on recovery the backlog replays and coalesces
Thread cache loss	Cache cold on hot posts	Stampede protection: single-flight fill per window (one miss rebuilds, the rest wait), stale-while-revalidate serving
Replica lag on reads	User posts, refreshes, comment missing	Read-your-own-write for authors: route their reads to the primary, or merge their pending writes client-visible via token
Votes DB shard loss	Votes on one slice of comments fail	Fail **closed** for integrity (a queued vote is better than a lost vote); tallies already computed remain served
Brigading wave	Hundreds of votes/s on one thread from fresh accounts	Chapter 3's rate limiter plus velocity anomaly detection (Chapter 4's metrics!); flagged threads shift to slower rank recompute while investigated

5.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Tree encoding	Adjacency + materialized path hybrid	Pure adjacency: recursive subtree reads; nested sets: write amplification on every insert

Decision	Chosen	Alternative & why not (here)
“Best” ranking	Wilson lower bound on up/(up+down)	Raw score: statistically illiterate at small n; ratio: worse — 1/1 beats 200/210; Bayesian average: defensible, but needs a global prior and is harder to explain to users
Rank computation	Recompute on vote, store rank keys	Sort at query time: 50k-row sorts per page view; scheduled recompute: stale beyond tolerance on fast threads
Vote writes	Synchronous upsert + async event	Fully async queue-first: lowest latency, but integrity (one-vote-per-user) becomes eventual — not acceptable
Tallies	Denormalized, reconciled nightly	Count from `votes` per read: honest but scans the largest table on the hottest path
Thread caching	Version-stamped windows, per-user overlay	Per-user full windows: personalization cost multiplies cache by user count; no cache: origin melts on viral posts
Deep nesting	Depth cap + “continue this thread”	Unbounded inline rendering: pathological subtrees (10^4-deep chains) DoS the renderer and the reader
Comment count on posts	Async maintained counter	Synchronous increment on the post row: every comment write contends on one hot row — the exact anti-pattern Section 5.5 warned about

5.17 Observability & SLOs

Indicator	Definition	Target
Thread read latency	Window fetch, cached / uncached, p95	≤ 200 ms / ≤ 700 ms
Comment write latency	Create reply, p95	≤ 300 ms
Vote latency	PUT vote, p95	≤ 150 ms
Rank freshness	Vote → reflected in stored rank keys, p95	≤ 60 s
Read-your-own-write	Author sees own comment after create, p99	≤ 1 s
Tally accuracy	Comments whose tally ≠ vote-log sum, sampled	≤ 0.1%, self-healing nightly

Indicator	Definition	Target
Removal propagation	Moderation removal → gone from all caches	≤ 60 s
Availability	Thread reads / writes	99.99% / 99.9%

Every one of these is a metric with labels and an alert with a `for` duration — Chapter 4's platform, pointed at this chapter's system. The reconciliation job emits tally-drift as a metric; brigading detection is a velocity alert on votes per thread. The book's chapters compose, and saying so in the interview is a senior signal.

5.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Make it real-time — new comments appear without refresh."** Add a subscription tier: WebSocket/SSE gateways subscribed per post; the event log fans `CommentCreated` events to gateways. Key decision: live comments arrive **appended** (by time), not re-sorted — re-ranking a live view under the reader's eyes is a product bug; re-sort applies on next window load.
2. **"How do you detect vote manipulation?"** Signals: velocity anomalies on single threads, account-age and IP clustering of voters, vote-ring graph analysis (accounts that only vote each other). Enforcement: shadow-exclude flagged votes from **rank keys** while keeping them in the visible score — manipulation loses its feedback signal.
3. **"Users edit comments to bait-and-switch after they rank."** Keep edit history; reset or decay a comment's rank confidence on substantive edits (re-open the Wilson interval); show "edited" markers. Ranking trust is the product — this is an integrity issue, not a UX nicety.
4. **"Sharding: what about a post with 10^6 comments?"** One shard holds it — $10^6 \times 1.5$ KB ≈ 1.5 GB, fine; reads are cached windows; votes partition by comment. The real ceiling is per-shard write QPS, and comment writes even on viral posts stay 10^2/s. If it ever breaks, sub-shard the thread by top-level comment id.
5. **"Design comment search."** Chapter 4's inverted index, fed from the same event log: tokenize bodies, index `(post_id, comment_id, tokens)`, rank results by stored Wilson score. Pipelines compose; nothing new is needed.

5.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A comment system is a **shape** problem wearing a scale costume. The data is a forest of unbounded trees: store it as an adjacency list with materialized paths, so any subtree is one prefix scan and depth is a stored fact; delete with tombstones so replies are never orphaned. Ranking is statistics, not arithmetic: "best" is the Wilson lower confidence bound (evidence beats luck), "controversial" is balanced volume, "hot" is log score plus 12.5-hour decay — all pure functions of `(up, down, time)`, recomputed on every vote and **stored**, so reads are index scans. Votes are the write storm: idempotent upserts keyed by `(user, comment)`, coalesced rank recomputation, tallies reconciled against the vote log nightly. Reads are windows — ranked pages with shallow children — cached per sort mode and version-stamped. The thread is never serialized; the conversation's shape is what ships.

Further reading.

- The source video: *"15: Reddit Comments — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=BO2gRisnBcA`

- Randall Munroe / Reddit — *"How Reddit ranking algorithms work"* (redditblog, 2009) — the original public write-up of hot/best/controversial.
- Evan Miller — *"How Not To Sort By Average Rating"* — the Wilson score interval derivation Section 5.8 distills, with the canonical worked numbers.
- Joe Celko — *Trees and Hierarchies in SQL for Smarties* — the encyclopedic treatment of adjacency lists, materialized paths, and nested sets.
- Amir Salihefendić — *"How Reddit ranking algorithms work"* commentary and Hacker News ranking write-ups — production variations on time decay.

5.20 Chapter Glossary

Term	Meaning
Adjacency list	Tree encoding where each row stores its parent's id; cheap writes, recursive reads
Coalescing	Collapsing many updates to the same entity into one recomputation; how vote bursts become trickles of rank writes
Comment count	Denormalized per-post tally of comments, maintained asynchronously for listing pages
Confidence interval	A range estimating a true probability from samples; wider (more pessimistic) when data is scarce
Controversial ranking	$(u+d)^{\min/\max}$ — balanced disagreement weighted by volume
Cursor (pagination)	Opaque token encoding the last seen sort key; stable under insertions, unlike offsets
Denormalization	Storing derived values (tallies, rank keys) to make reads cheap; the derived value must be rebuildable from truth
Depth cap	Maximum inline nesting rendered before collapsing behind "continue this thread"
Forest	The set of all comment trees, one rooted at each post
Hot ranking	$\log_{10}\max(\lvert s\rvert,1)+\text{sign}(s)\cdot(t-t_0)/45000$ — log-score with 12.5-hour decay
Idempotency key	A deduplication token making retried writes safe; for votes it is the natural key (user, comment)
Materialized path	Tree encoding storing each row's full ancestor chain; subtrees become prefix scans
Nested sets	Tree encoding with depth-first lft/rgt bounds; range-scan reads, catastrophic insert renumbering
N+1 query	One query per parent to fetch children; the adjacency-list trap that batched path scans eliminate
Rank key	A stored, indexed, orderable encoding of a comment's rank per sort mode; recomputed on vote
Read-to-write ratio	Reads per write; extreme ratios justify aggressive caching and denormalization

Term	Meaning
Read-your-own-write	Guarantee that a writer immediately sees their own write; routed to primary or merged client-side
Score	`up - down`, the visible number; deliberately **not** the "best" ordering
Single-flight fill	One cache miss rebuilds a value while concurrent misses wait; stampede protection
Soft delete / tombstone	Content erased, structure kept, so replies stay threaded under a "[deleted]" placeholder
Time decay	Ranking term that ages content out, forcing continuous freshness
Version stamp	A per-post counter bumped by rank updates; cached windows keyed by it age out without invalidation fanout
Vote overlay	Per-user vote values merged over a shared cached window at read time
Wilson score interval	Binomial confidence interval on the true upvote probability; its lower bound ranks "best" honestly across sample sizes

— End of Chapter 5 · Next: Chapter 6—

CHAPTER 6

Designing a Top-K Leaderboard

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"17: Top K Leaderboard"* from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the leaderboard system of a hit online game — millions of players submit scores after every match, and the service answers "who are the top K?" and "what is **my** rank?" in milliseconds, on boards that reset daily, weekly, and run all-time. The challenge is not volume but a very specific primitive — **rank as a first-class query** over an ordered set that mutates 25,000 times a second. All terms are defined before use; all reference code is Rust with deterministic tests.

6.1 The Problem Statement

The interviewer sketches a phone screen — a podium of three avatars, a scrolling list below, and a highlighted row pinned at the bottom: **"You: rank 1 247 003"** — and says:

"Players finish matches and submit scores. Players can view the global top 100, page deeper, and always see their own rank and neighborhood even when they are a million places down. Boards exist per season and all-time. Ties go to whoever got the score first. Design it."

This problem looks small next to Chapters 2–5 — and that is the trap. The arithmetic is gentle (Section 6.5); the difficulty is that the two headline queries, **top-K** and **rank-of-player**, must stay O(log n)-ish over an ordered collection of 10^8 entries while that collection is being updated at storm rates. Choose the wrong structure and either the reads or the writes eat the system. Choose the right one and most of the chapter is careful engineering around it: windows, ties, sharding, and cheating.

> **DEFINITION — Leaderboard / rank / top-K**
>
> A *leaderboard* is an ordering of players by score within some scope (a game, a season, a region, a friend group). A player's *rank* is their 1-based position in that ordering. A *top-K* query returns the first K entries in order. The pair {top-K, rank} is the complete query surface — every UI screen is one or the other.

> **DEFINITION — Best-score vs. every-run boards**
>
> Two admission semantics. *Best-score*: a player occupies exactly one slot, holding their personal best; a worse new score changes nothing. *Every-run* (arcade-style): each match is its own entry and one player may hold many slots. We design best-score — the common default — and note where every-run differs (Section 6.10).

6.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Score semantics?	Personal best per player per board; ties broken by who achieved it first
Which boards?	Global all-time, plus daily and weekly; same game, one platform
Rank granularity?	Exact rank for the querying player; "top X%" displays for everyone are fine approximate
Freshness?	A submitted score should be reflected in rank/top-K within 5 seconds
Live updates?	No push; clients poll or refresh. The list may change between pages — acceptable
Anti-cheat?	Hook it in; don't design it. Suspicious scores can be quarantined
Scale?	20M daily players, 200M matches/day, 200M registered accounts
Friends boards?	Yes — a player vs. their friends ($\leq$ a few hundred)
Score range?	Non-negative integers, 0 to 10^9

NOTE — Agreed scope

1. **Submit score** after each match; best-score semantics per player per board; ties to the earliest achiever.
2. **Top-K** with cursor paging ($K \leq 100$ per page).
3. **My rank** — exact, fast, at any depth, plus a neighborhood view (the $\pm r$ players around me).
4. **Time-windowed boards** — daily/weekly boards rotate; all-time persists.
5. **Friends leaderboard** — my standing among my friend list.
6. **Anti-cheat quarantine** hook on the ingestion path.
7. Out: live push, team boards, tournaments, cross-game boards.

6.3 Functional Requirements

DEFINITION — Neighborhood query

The $\pm r$ entries around a given player's rank — the "you are here" view. It is the query that makes the problem famous: it requires **rank** (to locate the player) and **ordered iteration** (to walk r places in both directions), at any depth in the ordering, in milliseconds.

1. **FR-1 — Submit score.** `(player_id, score, achieved_at, match_id)` per board scope; stored if it beats the player's current best; idempotent under retries and duplicate match reports.
2. **FR-2 — Top-K page.** Ordered entries `(rank, player, score)` from a cursor, per board.
3. **FR-3 — Player rank.** Exact rank and score for one player on one board.
4. **FR-4 — Neighborhood.** The $\pm r$ window around a player, ranks included.
5. **FR-5 — Windowed boards.** Daily/weekly/all-time boards of the same game, rotating automatically, old windows archived read-only.
6. **FR-6 — Friends board.** Top-K/rank restricted to a player's friend set.

6.4 Non-Functional Requirements

DEFINITION — Order-statistics query

A query about **position** in an ordering — "rank of X", "the k-th element", "count of entries above Y" — as opposed to a lookup by key. Supporting order statistics efficiently under inserts/deletes requires an ordered structure augmented with subtree/span counts (Section 6.7); plain hash indexes cannot answer them at all, and plain sorted scans answer them at O(n).

Requirement	Target & reasoning
Score ingest throughput	25k updates/s peak sustained; a launch event may 5× that briefly
Top-K read latency	p95 ≤ 50 ms (cached pages: ≤ 10 ms); it is a hot UI surface
Rank / neighborhood latency	p95 ≤ 100 ms at 10^8-entry boards — this kills naive designs
Freshness	Score → visible in rank/top-K ≤ 5 s; rank need not be transactional with the match
Durability	A confirmed personal best must survive node loss (replication + journaling)
Availability	Reads ≥ 99.99% (stale board pages acceptable); writes ≥ 99.9%
Integrity	One slot per player per board; ties deterministic; quarantined scores never render

KEY INSIGHT — Reads and writes are both modest — the operation mix is the design

Nothing here rivals Chapter 4's firehose or Chapter 5's vote storm. What is unusual: the system must maintain a **total order** over 10^8 entries, mutate it 25k times a second, and answer positional queries — top-K, rank, neighborhood — in milliseconds. Data structure choice **is** the architecture; everything else is distribution plumbing around one ordered set.

6.5 Back-of-the-Envelope Estimation

Assumptions: 20M daily players; 10 matches per player per day; every match submits one score; each player opens leaderboard surfaces 5×/day; a board entry is 100 B with index overhead.

Quantity	Estimate	Derivation
Score submissions	200M/day ≈ 2.3k/s avg, 25k/s peak	20M players × 10 matches; evening × event multiplier ≈ ×10
Leaderboard reads	100M/day ≈ 1.2k/s avg, 12k/s peak	20M × 5 views; top-K pages + rank lookups
Global all-time board	20 GB in RAM	200M players × 100 B (id, score, ts, structure overhead)
Daily board	2 GB	20M active × 100 B; born and dies daily
Top-100 page	5 KB JSON	100 × (id, name ref, score, rank)
Update cost	27 pointer hops	$\log_2(2\times10^8) \approx 27$ — the ordered set makes writes cheap
Snapshots	20 GB/board, minutes to write	Sequential dump; recovery = snapshot + journal replay

KEY INSIGHT — The whole working set fits in RAM — spend it

20 GB for the all-time board is one machine's RAM, or a few shards' worth. That licenses the chapter's central decision: keep boards **in memory** in a purpose-built ordered structure with O(log n) everything, replicate for reads and failover, journal for durability. A disk-first database would answer the same queries 10–100× slower for zero capacity benefit. Capacity is solved; latency and structure are the game.

The flow of the chapter: Section 6.6 isolates why rank is the crux; 6.7 picks the data structure (with the skip-list mechanics drawn out); 6.8–6.9 design APIs and architecture; 6.10–6.12 go deep on ingestion semantics, sharded top-K, and windowed/friends boards; 6.13 implements it all in Rust; 6.14–6.20 scale, harden, and review.

6.6 The Core Challenge: Rank Is Not a Lookup

Strip the product away and the service is one abstract data type with four operations: `upsert(player, score)`, `top(k, cursor)`, `rank(player)`, `neighborhood(player, r)` — under 25k mutations/s and 12k queries/s, on a collection of 10^8. Walk the candidate structures:

Structure	upsert	top-K	rank(player)	Verdict
Hash map	O(1)	O(n log n)	O(n log n)	Rank does not exist; every query re-sorts 10^8 entries
Sorted array	O(n)	O(K)	O(log n) by score	Reads fine; O(n) write shifts ruin 25k updates/s
B-tree index (DB)	O(log n)	O(log n + K)	O(n) count scan	The classic wrong answer — see below
Heap	O(log n)	O(K log n)	unsupported	No rank, no paging; wrong shape entirely
Skip list + hash map, spans	**O(log n)**	**O(log n + K)**	**O(log n)**	Ordered, rank-augmented, in-memory — Section 6.7

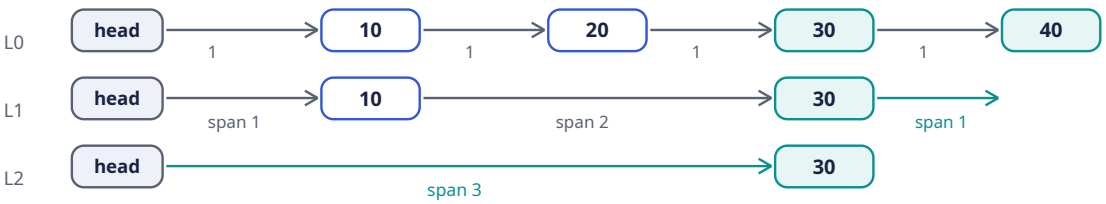
COMMON PITFALL — The `COUNT(*)` rank trap

The most common wrong answer: store scores in a relational table, index the score column, and compute rank as `SELECT COUNT(*) WHERE score > :mine`. The index makes top-K fine, but rank-by-count touches **every entry above the player** — for rank 1.2M of 200M, that is a 198.8M-row index scan per query, thousands of times a second. Rank must be **maintained by the structure**, not computed by counting. This is the sentence the interviewer is listening for.

6.7 Deep Dive: The Data Structure — Skip List with Spans

DEFINITION — Skip list / span

A *skip list* is a probabilistically balanced ordered structure: a sorted linked list (level 0) with express lanes above it — each node is promoted to the next level with probability p (e.g. 1/4), so level i holds p^i of the nodes. Search starts on the top express lane and drops down, skipping 1/p nodes per hop: O(log n) expected search, insert, delete. A *span* is an integer on every forward pointer counting how many level-0 steps it covers; spans turn the skip list into an **order-statistics** structure: summing spans along a search path yields the rank directly, and rank + forward walks yield neighborhoods — still O(log n).



rank(40) = spans along the search path: 3 (L2, head→30) + 1 (L1, 30→40) = 4 — position without counting

Why this over a balanced tree: same asymptotics, but level-promotion makes inserts/deletes local (no rotations), concurrent implementations are far simpler (only level 0 needs careful linking), and span bookkeeping is an 10-line addition. It is exactly the structure at the core of the industry's canonical sorted-set implementations — `hash map player→node` for O(1) lookup, skip list for order and rank.

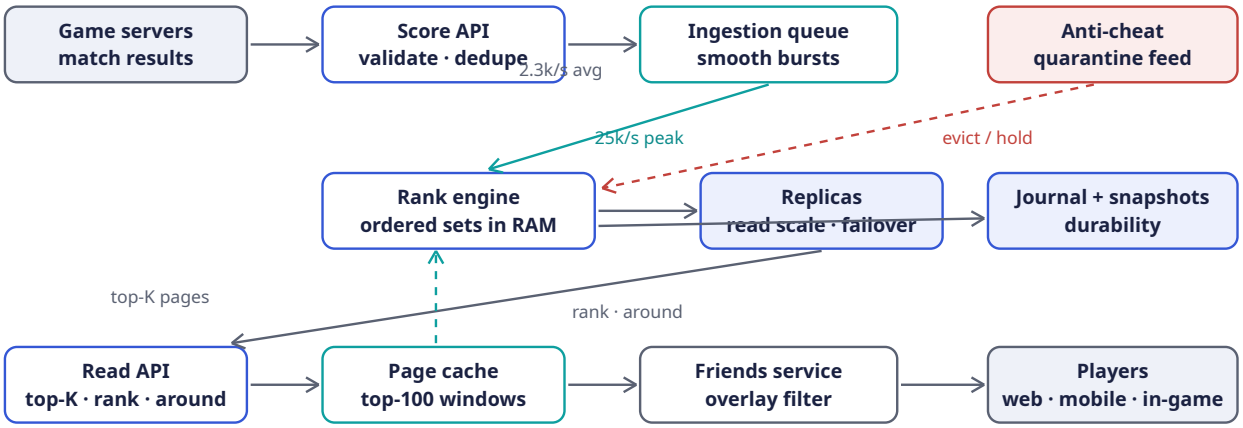
Ties slot in cleanly: the ordering key is not `score` but the triple `(score desc, achieved_at asc, player_id)` — deterministic total order, no equal keys, "first to achieve" falls out of the comparison itself. Section 6.13 implements it on an ordered map (same complexity story; Rust's std ordered map lacks spans, and the listing says so honestly).

6.8 API Design

Method	Path	Purpose
POST	/v1/scores	Submit match score {player_id, board, score, achieved_at, match_id} — best-score + idempotent (FR-1)
GET	/v1/leaderboards/{board}/top?cursor=&limit=	Top-K page (FR-2); cursor = last (score, ts, id) triple
GET	/v1/leaderboards/{board}/players/{id}	Exact rank + score (FR-3)
GET	/v1/leaderboards/{board}/players/{id}/around?r=	Neighborhood window (FR-4)
GET	/v1/leaderboards/{board}/players/{id}/friends	Friends board (FR-6)
POST	/v1/admin/quarantine	Anti-cheat hook: remove/hold entries (scope)

Notes: `board` encodes window and scope (`global:alltime`, `global:2026-W36`, `region:eu:daily:2026-09-05`). Cursors are the rank triple, opaque and base64'd — Chapter 5's pagination lesson, and here the cursor **is** the ordering key, so paging is a pure `range()` from the cursor position. Submission responses return `{accepted, improved, current_best}` so clients can celebrate personal bests without a second call.

6.9 High-Level Architecture



Writes: validate → queue → in-memory ordered sets (replicated, journaled). Reads: cached pages for the top, live rank queries for everything else

Component	Responsibility	Why it is shaped this way
Score API	Authenticate game servers, validate, dedupe by `match_id`	Writes enter through one narrow, auditable door
Ingestion queue	Buffer score events; absorb event-day bursts	Chapter 4's shock absorber; ingestion lag becomes a metric, not an outage
Rank engine	Holds all boards as in-memory ordered sets; applies best-score updates; serves rank/top-K/around	The structure of Section 6.7 **is** the system; 20 GB fits in RAM (Section 6.5)
Replicas	Read scale-out + hot standby	Rank reads ×12k/s spread; promotion covers node loss
Journal + snapshots	Append-only update log; periodic board dumps	Recovery = latest snapshot + replay; the board is rebuildable, never lost
Anti-cheat	Consumes the same stream; quarantines suspects	A consumer like any other — removal is just a delete from the set
Page cache	Top-100 pages per board, few-second TTL	The podium is read 10^3× more often than it changes (top-100 churn is slow; Section 6.14)
Friends service	Intersection of friend list with board entries	Small overlays computed on read; Section 6.12

6.10 Deep Dive: Score Ingestion Semantics

Best-score admission makes the write path almost suspiciously simple:

> **DEFINITION — Monotonic (idempotent) update**
>
> An update whose repeated application changes nothing after the first time. Best-score submission is monotonic: `board[player] = max(old, new)` absorbs retries, duplicates, and out-of-order redeliveries for free — no dedup tokens, no exactly-once machinery on the hot path. (Chapter 5's votes needed

an upsert table; here the **semantics** do the work.) The one non-monotonic case — same player, same score, **earlier** timestamp stealing the tie-break — is excluded by rule: a player's slot keeps its first achieving timestamp.

The ingestion contract, end to end:

1. Game server submits `(player, board, score, achieved_at, match_id)`; the Score API dedupes by `match_id` briefly (same match reported twice) and validates shape.
2. The queue buffers; the rank engine applies `max` updates in stream order. Bursts only stretch the freshness budget (≤5 s) — never correctness.
3. Anti-cheat consumes the same stream independently; a quarantine verdict deletes the player's entries from every board (a delete is just another update) and parks their submissions until cleared. The leaderboard never renders a quarantined score — integrity requirement, Section 6.4.
4. **Every-run variant** (arcade boards): entries keyed `(match_id)` instead of player; dedupe by match id is still free; top-K is unchanged; "my rank" becomes "my best run's rank". One sentence in the interview shows the abstraction holds.

6.11 Deep Dive: Sharding and the Global Top-K

One node holds 20 GB and applies 25k skip-list updates/s — a single beefy rank engine covers our numbers with replicas. But boards grow (new regions, games, seasons), so the sharding answer must be ready:

Shard by `hash(player_id)`. Each shard owns a complete ordered set of its players. Then:

- **Top-K, exact**: fetch the top K from **every** shard and k-way merge. Correctness: any member of the global top-K has at most K−1 players outranking it globally, hence at most K−1 on its own shard — so it must appear in its shard's top-K. Fetching K per shard is sufficient and necessary. Cost: `shards × K` entries per query — 8 shards × 100 = 800 entries per page, trivial.
- **My rank, exact**: `1 + Σ_shards count(better than me)` — a scatter-gather count across all shards, O(log n) per shard with spans, but fan-out per query. Fine at 1.2k rank-reads/s; painful at 10^5/s.
- **"Top X%" displays, approximate**: each shard also maintains a fixed-boundary **histogram** of its scores. Histograms are **mergeable by addition** — sum the bucket counts and interpolate (Section 6.13) — so a global percentile is a cheap aggregate with no global ordering at all. This is how "top 0.4%" renders without ever computing 200M ranks.

> **KEY INSIGHT — Exact where it is read, approximate where it is glanced**
>
> The podium (top-100) must be exact — it is the product. A player's own rank must be exact — they will screenshot it. The "top 2.3%" badge on a profile is **glanced at**, not audited — approximate it with mergeable histograms and save the global scatter-gather entirely. Precision is a budget; spend it only on surfaces where users check the math.

6.12 Deep Dive: Windowed and Friends Boards

Time windows are just keys. A board is identified by `(scope, period)` — `global:alltime`, `global:weekly:2026-W36`, `global:daily:2026-09-05`. Every score submission writes to **all live windows** for its scope (daily + weekly + all-time = three ordered-set updates, still cheap). Rotation is a scheduler flipping the live key: yesterday's daily board stops accepting writes, gets snapshotted to cold storage, and stays readable as a frozen archive. No migration, no reindexing — the empty new window is created lazily by the first write.

Friends boards are overlays, not boards. A friend list is ≤ a few hundred players. On read: fetch each friend's current `(score, ts)` by direct player lookup (the hash-map side of the ordered set, O(1) each), sort in memory, rank locally. Milliseconds, zero storage. The alternative — materializing a per-user friends board at write time — multiplies every score update by the player's appearance in friend lists (a Chapter 2-style fanout write amplification) to answer a rare query. Compute on read wins decisively here.

6.13 Rust Reference Implementations

Three pieces with deterministic tests: the leaderboard itself, the sharded top-K merge, and the mergeable percentile histogram.

6.13.1 The Leaderboard: Ordered Set + Player Index

```rust
use std::cmp::Reverse;
use std::collections::{BTreeMap, HashMap};

/// Ordering key: score desc, then earliest achieved_at, then player id —
/// a total order, so entries never tie and "first to achieve" is free.
type Key = (Reverse<u64>, u64, u64);

/// In-memory board. `entries` is the ordered set (a stand-in for the
/// span-augmented skip list of Section 6.7 — same interface and big-O,
/// except rank, noted below); `by_player` is the O(1) side index.
#[derive(Default)]
pub struct Leaderboard {
    entries: BTreeMap<Key, ()>,
    by_player: HashMap<u64, (u64 /*score*/, u64 /*achieved_at*/)>,
}

impl Leaderboard {
    /// Best-score admission: strictly better scores only. Monotonic, so
    /// retries and duplicate submissions are safe by construction.
    pub fn submit(&mut self, player: u64, score: u64, achieved_at: u64) -> bool {
        if let Some(&(old, _)) = self.by_player.get(&player) {
            if score <= old { return false; }
        }
        if let Some(&(old, old_ts)) = self.by_player.get(&player) {
            self.entries.remove(&(Reverse(old), old_ts, player));
        }
        self.entries.insert((Reverse(score), achieved_at, player), ());
        self.by_player.insert(player, (score, achieved_at));
        true
    }

    pub fn top_k(&self, k: usize) -> Vec<(u64, u64)> {
        self.entries
            .keys()
            .take(k)
            .map(|&(Reverse(s), _, id)| (id, s))
            .collect()
    }

    /// 1-based rank. NOTE: Rust's ordered map keeps no order statistics, so
    /// this counts the better half — O(rank). The span skip list answers the
    /// same call in O(log n); only the implementation changes.
    pub fn rank_of(&self, player: u64) -> Option<u64> {
        let &(s, ts) = self.by_player.get(&player)?;
        let better = self.entries.range(..(Reverse(s), ts, player)).count();
```

```
        Some(better as u64 + 1)
    }

    /// ±r window around a player: (rank, player, score), edges clamped.
    pub fn neighborhood(&self, player: u64, r: usize) -> Vec<(u64, u64, u64)> {
        let rank = match self.rank_of(player) {
            Some(r) => r as usize,
            None => return vec![],
        };
        let lo = rank.saturating_sub(r + 1); // 0-based start
        let hi = (rank + r).min(self.entries.len()); // exclusive end
        self.entries
            .keys()
            .enumerate()
            .skip(lo)
            .take(hi - lo)
            .map(|(i, &(Reverse(s), _, id))| (i as u64 + 1, id, s))
            .collect()
    }
}

#[cfg(test)]
mod board_tests {
    use super::*;

    #[test]
    fn best_score_ties_and_rank() {
        let mut lb = Leaderboard::default();
        assert!(lb.submit(1, 500, 100)); // alice
        assert!(lb.submit(2, 900, 100)); // carol
        assert!(lb.submit(3, 500, 200)); // bob: same score, later ts
        assert!(lb.submit(4, 100, 100)); // dave
        // board order: 2 (900), 1 (500@100), 3 (500@200), 4 (100)
        assert_eq!(lb.top_k(2), vec![(2, 900), (1, 500)]);
        assert_eq!(lb.rank_of(1), Some(2)); // tie broken by earlier ts
        assert_eq!(lb.rank_of(3), Some(3));
        assert_eq!(lb.rank_of(4), Some(4));
        assert_eq!(lb.rank_of(99), None);
        // monotonic admission: worse or equal scores change nothing
        assert!(!lb.submit(1, 400, 300));
        assert!(!lb.submit(1, 500, 50));
        assert_eq!(lb.rank_of(1), Some(2));
        // improvement repositions
        assert!(lb.submit(1, 950, 400));
        assert_eq!(lb.top_k(2), vec![(1, 950), (2, 900)]);
    }

    #[test]
    fn neighborhood_windows_clamp_at_edges() {
        let mut lb = Leaderboard::default();
        for i in 0..10u64 {
            lb.submit(i, 1000 - i * 10, 1); // id i -> rank i+1
        }
        let w = lb.neighborhood(4, 2); // rank 5 -> ranks 3..=7
        let ranks: Vec<u64> = w.iter().map(|e| e.0).collect();
        let ids: Vec<u64> = w.iter().map(|e| e.1).collect();
        assert_eq!(ranks, vec![3, 4, 5, 6, 7]);
        assert_eq!(ids, vec![2, 3, 4, 5, 6]);
        let w0 = lb.neighborhood(0, 2); // top edge
        assert_eq!(w0.iter().map(|e| e.0).collect::<Vec<_>>(), vec![1, 2, 3]);
```

```
        assert!(lb.neighborhood(99, 2).is_empty());
    }
}
```

6.13.2 Sharded Top-K: The K-Way Merge

```rust
use std::cmp::Reverse;
use std::collections::BinaryHeap;

/// One shard's top entries, already in board order (score desc, ts asc).
pub type ShardTop = Vec<(u64 /*player*/, u64 /*score*/, u64 /*ts*/)>;

/// Global top-k by merging each shard's top-k. Sound because any global
/// top-k member has < k entries outranking it globally, hence < k on its own
/// shard — so it is always inside its shard's top-k contribution.
pub fn merge_top_k(shards: &[ShardTop], k: usize) -> Vec<(u64, u64, u64)> {
    // max-heap on (score, Reverse(ts)): highest score, earliest ts pops first
    let mut heap = BinaryHeap::new();
    for (s, shard) in shards.iter().enumerate() {
        if let Some(&(p, sc, ts)) = shard.first() {
            heap.push((sc, Reverse(ts), p, s, 0usize));
        }
    }
    let mut out = Vec::with_capacity(k);
    while let Some((sc, Reverse(ts), p, s, i)) = heap.pop() {
        out.push((p, sc, ts));
        if out.len() == k { break; }
        if let Some(&(p2, sc2, ts2)) = shards[s].get(i + 1) {
            heap.push((sc2, Reverse(ts2), p2, s, i + 1));
        }
    }
    out
}

#[cfg(test)]
mod merge_tests {
    use super::*;

    #[test]
    fn merge_matches_brute_force_global_sort() {
        let shards: Vec<ShardTop> = vec![
            vec![(1, 900, 5), (4, 300, 5)],
            vec![(2, 950, 5), (5, 250, 5)],
            vec![(6, 500, 1), (3, 500, 5)], // tie at 500: earlier ts first
        ];
        let merged = merge_top_k(&shards, 4);
        let ids: Vec<u64> = merged.iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![2, 1, 6, 3]);

        let mut all: Vec<(u64, u64, u64)> = shards.concat();
        all.sort_by(|a, b| b.1.cmp(&a.1).then(a.2.cmp(&b.2)));
        let oracle: Vec<u64> = all.into_iter().take(4).map(|e| e.0).collect();
        assert_eq!(ids, oracle);
    }

    #[test]
    fn k_per_shard_is_sufficient() {
        // all three global leaders live on one shard; asking only k=3
        // per shard still recovers the exact global top-3
```

```
        let shards: Vec<ShardTop> = vec![
            vec![(1, 100, 1), (2, 90, 1), (3, 80, 1), (4, 70, 1)],
            vec![(5, 60, 1)],
        ];
        let per_shard: Vec<ShardTop> =
            shards.iter().map(|s| s.iter().take(3).cloned().collect()).collect();
        let ids: Vec<u64> =
            merge_top_k(&per_shard, 3).iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![1, 2, 3]);
    }
}
```

6.13.3 Mergeable Percentile Histograms

```
/// Fixed-boundary histogram for approximate "top X%" displays.
/// counts[i] = scores in [bounds[i-1], bounds[i]); the last bucket catches
/// everything >= the final bound. Histograms merge by adding counts —
/// that is what makes global percentiles cheap across shards.
pub struct ScoreHistogram {
    bounds: Vec<u64>,
    counts: Vec<u64>,
    total: u64,
}

impl ScoreHistogram {
    pub fn new(bounds: Vec<u64>) -> Self {
        assert!(!bounds.is_empty());
        let m = bounds.len();
        ScoreHistogram { bounds, counts: vec![0; m + 1], total: 0 }
    }

    pub fn add(&mut self, score: u64) {
        let i = self.bounds.partition_point(|&b| score >= b);
        self.counts[i] += 1;
        self.total += 1;
    }

    /// Merge another shard's histogram (identical bounds) by addition.
    pub fn merge(&mut self, other: &ScoreHistogram) {
        assert_eq!(self.bounds, other.bounds);
        for (a, b) in self.counts.iter_mut().zip(&other.counts) {
            *a += b;
        }
        self.total += other.total;
    }

    /// Estimated fraction of players scoring below `score`, with linear
    /// interpolation inside the straddling bucket.
    pub fn percentile_of(&self, score: u64) -> f64 {
        if self.total == 0 { return 0.0; }
        let (mut below, mut lower) = (0.0f64, 0u64);
        for (i, &upper) in self.bounds.iter().enumerate() {
            if score >= upper {
                below += self.counts[i] as f64;
                lower = upper;
            } else {
                if score > lower {
                    let frac = (score - lower) as f64 / (upper - lower) as f64;
                    below += frac * self.counts[i] as f64;
```

```
                }
                return below / self.total as f64;
            }
        }
        // score at/past the final bound: overflow bucket counts only when
        // strictly below `score` (bucket contents are >= the last bound)
        if score > *self.bounds.last().unwrap() {
            below += self.counts[self.bounds.len()] as f64;
        }
        below / self.total as f64
    }
}

#[cfg(test)]
mod hist_tests {
    use super::*;

    #[test]
    fn percentile_tracks_uniform_distribution() {
        let mut h =
            ScoreHistogram::new(vec![100, 1_000, 10_000, 100_000, 1_000_000]);
        for i in 0..1000u64 { h.add(i * 1_000); } // 0, 1000, ..., 999_000
        assert_eq!(h.total, 1000);
        assert_eq!(h.percentile_of(0), 0.0);
        let p50 = h.percentile_of(500_000); // interpolates in [100k, 1M)
        assert!((p50 - 0.5).abs() < 0.05, "p50 = {p50}");
        assert_eq!(h.percentile_of(2_000_000), 1.0);
        // "top X%" badge: the 999_500 score is elite
        assert!((1.0 - h.percentile_of(999_500)) * 100.0 < 1.0);
    }

    #[test]
    fn histograms_merge_by_addition() {
        let bounds = vec![100, 1_000];
        let (mut a, mut b) =
            (ScoreHistogram::new(bounds.clone()), ScoreHistogram::new(bounds));
        for s in [0, 50, 500] { a.add(s); }
        for s in [5_000, 9_999] { b.add(s); }
        a.merge(&b);
        assert_eq!(a.total, 5);
        assert!((a.percentile_of(1_000) - 0.6).abs() < 1e-9); // 3 of 5 below
    }
}
```

6.14 Scaling the Platform

Layer	Scale axis	Mechanism
Score API	Submission rate	Stateless; horizontal
Ingestion queue	25k/s avg→peak bursts	Partitioned by board+shard key; lag is a metric, not an outage (Chapter 4 pattern)
Rank engine	Board entries × update rate	One node per board-group until RAM or write QPS demands sharding by `hash(player)`; O(log n) updates

Layer	Scale axis	Mechanism
		mean a single core does 10^6 ops/s — 40× headroom over peak
Read path	Rank/top-K reads	Replicas + the page cache; the podium page is shared by everyone and changes slowly — a few-second TTL absorbs the viral majority
Storage	200M-entry all-time board	20 GB RAM per replica; snapshots to object storage; journal retention days
Windows	Board count = scopes × live windows	Lazy creation; frozen archives are read-only snapshots — zero live cost

KEY INSIGHT — Top-100 churn is glacial compared to mid-table churn

The millionth-best player changes rank with every match; the podium changes a few times an hour. So cache **top pages** aggressively (they barely move), serve **rank/neighborhood** live from the ordered set (they move constantly and are queried rarely per position), and never cache mid-table pages — they are wrong the moment they are written. The cache policy mirrors the physics of the data.

6.15 Failure Modes & Degradation

Failure	Symptom	Response
Rank engine node loss	Board reads fail over, writes pause seconds	Promote a replica; it is already warm. Recovery to a fresh node = snapshot + journal replay
Journal gap / corruption	Replay inconsistency on recovery	Checksummed journal frames; on gap, rebuild the board from match-history re-ingest (slow but total) — the board is derived state over score facts
Queue backlog on event day	Freshness degrades past 5 s	Degrade **freshness**, never integrity: clients show "updating…" affordance; Chapter 3's limiter sheds anonymous poll traffic first
Replica lag	Stale ranks on some reads	Version the board (update count); reads report the version so clients never go backwards within a session
Hot shard	One shard's players dominate writes	Reshard by splitting the hash range; top-K merge is shard-count-agnostic, so reads need no coordination
Anti-cheat pipeline down	Suspect scores render	Fail toward quarantine for **new** suspicious velocity; previously

Failure	Symptom	Response
		cleared scores unaffected. Integrity beats availability here — Section 6.4
Clock skew in `achieved_at`	Wrong tie-breaks	Ties are rare and low-stakes; still, trust server receipt time over client clocks, accept `achieved_at` only within a tolerance window

6.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Storage of boards	In-memory ordered sets + journal	Disk-first DB: 10–100× slower rank ops for zero capacity gain — the working set fits in RAM
Rank computation	Structure-maintained (spans)	Rank-by-count SQL: O(better-entries) per query — melts (Section 6.6)
Score semantics	Best-score, monotonic	Every-run: right for arcade boards; same machinery, different key (match_id)
Global top-K	Per-shard top-K + k-way merge	One global sorted set across shards: distributed ordering is a consensus problem nobody needs for a leaderboard
Global percentile	Mergeable histograms	Exact scatter-gather counts: fine at 10^3/s, wrong tool for every profile badge
Friends board	Computed on read from the friend list	Materialized per-user boards: write fanout per score — Chapters 2/5's fanout lesson
Freshness	≤5 s via async apply	Synchronous end-to-end: turns every match-end into a distributed transaction
Live push	Out of scope; polling + short TTL	WebSocket fanout of rank changes: a fine Chapter 1-style add-on, but rank updates at 25k/s make push storms the default state

6.17 Observability & SLOs

Indicator	Definition	Target
Ingest freshness	Score accepted → reflected in rank engine, p95	≤ 5 s

Indicator	Definition	Target
Top-K latency	Page read, cached / live, p95	≤ 10 ms / ≤ 50 ms
Rank latency	rank + neighborhood, p95	≤ 100 ms
Board integrity	Sampled players whose stored slot ≠ submitted best	≈ 0 (journal-rebuildable)
Quarantine latency	Verdict → entries removed from all boards	≤ 60 s
Availability	Reads / writes	99.99% / 99.9%

Chapter 4's platform carries all of it: ingestion lag as a consumer-lag metric, freshness as a histogram, an alert with a `for` duration on replica version drift. The reconciliation sampler — comparing stored slots against recomputed bests from the journal — is this chapter's version of Chapter 5's tally audit.

6.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Push live rank changes to clients."** Fan-out problem: 25k updates/s each reorder thousands of ranks — nobody's visible rank actually moved. Push only **material** changes: podium swaps, personal bests, crossing friends. Everything else is noise dressed as real-time.
2. **"Team leaderboards."** Team score = aggregate of members (sum, or ELO-style rating); the ordered set is identical, keyed by team. The interesting part is member-change handling and anti-stack incentives, not the structure.
3. **"Regional boards at 50 regions × 3 windows?"** Boards are cheap keys — 150 extra ordered sets, each small. Writes fan out per applicable board (a player's score lands on their region's set + global); reads unchanged.
4. **"How do you catch cheaters?"** Server-side authority (the game server signs match results, clients never self-report), statistical anomaly detection on score distributions per level (a Chapter 4 alert on percentile outliers), velocity checks (Chapter 3's limiter: matches/hour per player), and quarantine-not-delete so false positives recover.
5. **"10^9 players — what breaks first?"** Single-node RAM (100 GB is still one node, but uncomfortable) and the top-K merge fan-out staying flat — the design already shards; the rank scatter-gather for exact global rank is what you would approximate next (histograms), keeping exact rank only within a player's region shard.

6.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A leaderboard is one primitive — **rank over a mutating total order** — and the chapter is the engineering that protects it. The structure is an ordered set: hash map for player lookup, skip list with spans for O(log n) rank, top-K, and neighborhoods; ties are a total ordering key `(score, achieved_at, id)`. Writes are monotonic best-score updates — idempotent by construction, journaled for durability, replicated for reads. Reads split by physics: the podium is cached (it barely moves), personal rank is live (it always moves), percentiles are approximated with mergeable histograms (they are glanced at, not audited). Sharding by player hash keeps top-K exact through a k-way merge — any global top-K member is provably inside its shard's top-K. Windows are keys; friends boards are overlays; cheating is a stream consumer with the power to delete. The lesson that transfers: **when the query is positional, the data structure is the architecture.**

Further reading.

- The source video: *"17: Top K Leaderboard — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=nQpkRONzEQI`
- William Pugh — *"Skip Lists: A Probabilistic Alternative to Balanced Trees"* (1990) — the structure of Section 6.7, including span-based ranks.
- Redis sorted sets documentation (`ZADD` / `ZRANK` / `ZRANGE`) — the canonical production embodiment of this chapter's ordered set.
- Cormen et al., *Introduction to Algorithms* — the order-statistics tree chapter (subtree sizes) for the balanced-tree equivalent of spans.
- *"Elo rating system"* and TrueSkill papers — for the follow-up where the board ranks **skill** rather than scores.

6.20 Chapter Glossary

Term	Meaning
Best-score semantics	One slot per player per board holding their personal best; worse scores are no-ops
Cursor	Opaque paging token; here literally the ordering key triple `(score, achieved_at, id)` of the last entry
Every-run semantics	Arcade-style admission where each match is a separate entry and a player may hold many slots
Exact rank	A player's precise 1-based position; required for self-queries, wasteful for global percentiles
Friends board	A leaderboard restricted to a player's friend set; computed as a read-time overlay, not stored
Histogram (mergeable)	Fixed-bucket score distribution; bucket counts add across shards, giving cheap approximate percentiles
Idempotent / monotonic update	An update whose repetition changes nothing; best-score `max` admission has this property for free
Journal (write-ahead)	Append-only record of updates enabling replay after snapshot restore
K-way merge	Merging k-sorted shard outputs with a heap; sound for top-K because K entries per shard always suffice
Leaderboard	An ordering of players by score within a scope (window, region, friends)
Neighborhood query	The ±r window around a player's rank — needs both rank and ordered iteration
Order-statistics query	A positional query (rank, k-th, count-above); needs an augmented ordered structure, not a plain index
Ordered set	The abstract data type: map from member to score plus rank-ordered iteration; hash map + skip list in practice
Percentile / "top X%"	Approximate positional display computed from merged histograms instead of exact ranks

Term	Meaning
Quarantine	Anti-cheat state: scores held out of all boards pending verdict; deletable, recoverable
Rank	1-based position in a leaderboard's ordering
Shard sufficiency (top-K)	The theorem that global top-K needs only K entries per shard, since a global top-K member is outranked by < K entries anywhere
Skip list	Probabilistically balanced ordered list with express lanes; O(log n) search/insert/delete without rotations
Span	Count on a skip-list forward pointer of level-0 steps covered; summing spans along a search path yields rank
Tie-break	Deterministic total order extension: `(score desc, achieved_at asc, id)` — first to achieve wins
Top-K	The first K entries of a board in order; the podium query
Windowed board	A board scoped to a time period (daily/weekly); rotation is a key change, archives are frozen snapshots

— End of Chapter 6 · Next: Chapter 7 —

CHAPTER 7

Designing a Recommendation Engine

NOTE — Problem source

This chapter solves the problem posed in the talk *"22: Recommendation Engine (YouTube, TikTok)"* from the series *Systems Design Interview Questions With Ex-Google SWE* (channel: *Jordan has no life*). The task: design the personalized home feed of a video platform — hundreds of millions of users, a billion-item catalog, and 300 milliseconds to decide which twenty items a given user sees next. It is the chapter where the book's infrastructure (event pipelines, ranking, caching) meets machine learning as a **systems** concern: feature stores, model freshness, and feedback loops. All terms are defined before use; all reference code is Rust with deterministic tests.

7.1 The Problem Statement

The interviewer opens a video app's home screen — an endless, uncannily well-chosen scroll — and says:

"When a user opens the app, we show them a personalized feed of videos. We know everything they've watched, liked, skipped, and how long they lingered. A million new videos arrive every day. Design the system that decides what goes on this screen."

Every prior chapter had a crisp correct answer to compute; this one computes a **guess**. That changes the engineering: the system's job is to take a billion-item catalog and a user's entire history, and under a hard latency budget produce an ordering that maximizes a business metric nobody can measure directly. The design that emerged across the industry — a **funnel** of candidate generation followed by ranking — is the spine of this chapter, and understanding **why** it has that shape is most of the interview.

DEFINITION — Recommendation / the feed

A *recommendation* is a predicted-relevance ordering of catalog items for a specific user at a specific moment. The *feed* is its product form: the ranked, paginated, infinite scroll. Unlike search (Chapter 4's inverted index answers "what matches this query"), the feed answers "what does this user want, having asked nothing" — the query is the user's history.

DEFINITION — Implicit vs. explicit feedback

Explicit feedback is deliberate: likes, ratings, follows, "not interested". *Implicit* feedback is behavioral exhaust: impressions (the item was shown), plays, watch time, skips, rewatches. Implicit feedback is 1000× more abundant and is what actually trains the system — watch time and skips are votes every user casts on every item, whether they mean to or not (Chapter 5's votes, but continuous and uninvited).

7.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
What surfaces?	Just the home feed. Search, subscriptions page, and ads are separate systems
What do we optimize?	Watch time / session satisfaction — assume a business metric exists; your job is the machinery
Freshness requirements?	A great new video should be discoverable within minutes; user actions should affect their feed within the session
How personalized?	Fully — no two users' feeds need overlap. But logged-out users get a fallback (trending)
Model training?	Not the focus — data scientists own model internals. You own data flow, features, serving, latency
Content safety?	A policy filter gate exists; respect it, don't design it
Scale?	500M daily users, 1B public videos, 2B feed loads/day
Cold start?	Must have an answer for new users and new videos

NOTE — Agreed scope

1. **Feed generation**: personalized, ranked, paginated home feed per user.
2. **Feedback loop**: impressions/plays/watch-time/likes recorded and reflected — in-session effects within seconds, model retraining daily.
3. **Candidate diversity**: multiple retrieval channels (follows, similarity, embeddings, trending), blended.
4. **Freshness & cold start**: new items get an exploration quota; new users get trending + rapid personalization from first signals.
5. **Guardrails**: dedup, policy filter, diversity caps in the final mix.
6. Out: search, ads ranking, model architecture design, content moderation itself.

7.3 Functional Requirements

DEFINITION — Candidate generation (retrieval) vs. ranking (scoring)

The two mandatory stages of any industrial recommender. *Candidate generation* cheaply selects 10^3–10^4 plausibly-relevant items from the billion (per-channel: what your follows posted, items similar to what you loved, embedding neighbors, trending). *Ranking* expensively scores those few thousand with the full feature set and orders them. Retrieval is recall-shaped ("don't miss anything they'd love"); ranking is precision-shaped ("order these twenty slots perfectly"). Neither can do the other's job at scale — Section 7.6 derives this.

1. **FR-1 — Feed.** `GET /feed` returns the next page of ranked items for the user, with an opaque cursor; the first page is the home screen.
2. **FR-2 — Event intake.** Impressions, plays, watch time, likes, skips, "not interested" recorded durably and at scale.
3. **FR-3 — Session adaptation.** Signals from **this** session (last N watches) influence the next page within seconds.
4. **FR-4 — Channel blend.** Every page mixes sources: followed creators, similar-to-history, embedding neighbors, trending, and an exploration quota for new items.

5. **FR-5 — Negative feedback.** "Not interested" removes the item and down-weights its ilk immediately.
6. **FR-6 — Cold start.** New users receive trending + category picks, personalizing from the very first events; new videos receive a small, guaranteed stream of test impressions.

7.4 Non-Functional Requirements

DEFINITION — Latency budget

The total time a request may consume, allocated across stages. Ours: 300 ms p95 end-to-end for a feed page — 50 ms retrieval fan-out, 150 ms ranking 5k candidates, 50 ms re-rank + assemble, 50 ms network and slack. The budget **dictates the architecture**: anything that cannot fit its allocation must be precomputed offline.

Requirement	Target & reasoning
Feed latency	p95 ≤ 300 ms per page; it is the app's front door
Event throughput	580k events/s avg, 3M/s peak — Chapter 4's firehose, with a job to do
Feed freshness	A just-watched video influences the next page ≤ 10 s (session features); new **model** daily
New-item discoverability	Every eligible new video gets its exploration impressions ≤ 1 h from upload
Availability	Feed ≥ 99.95%; graceful degradation = trending + follows (never an empty screen)
Diversity	No category/creator may dominate a page beyond caps; the feed must not collapse to one topic
Model staleness	Serving model ≤ 36 h old; features ≤ 10 s (session) / ≤ 1 h (rest)

KEY INSIGHT — Relevance is a budget problem

Scoring one (user, item) pair well is a solved ML problem. The system's entire shape comes from arithmetic: 2B requests/day × a 1B-item catalog means you may score, at most, a few thousand items per request. Everything — retrieval channels, approximate nearest neighbors, precomputed similarities, the funnel itself — is a device for spending that tiny scoring allowance on the right few thousand items.

7.5 Back-of-the-Envelope Estimation

Assumptions: 500M daily users, 4 feed sessions each, 20 items viewed per first page; 100 events per user per day; catalog 1B videos, growing 20M/day.

Quantity	Estimate	Derivation
Feed requests	2B/day ≈ 23k/s avg, 100k/s peak	500M users × 4 sessions; peaks at regional evenings
Feedback events	50B/day ≈ 580k/s avg, 3M/s peak	100 events/user/day: impressions dominate
Candidates per request	5k retrieved → 20 shown	Retrieval channels × quotas; ranking budget

Quantity	Estimate	Derivation
Scoring rate	115M inferences/s avg	23k req/s × 5k candidates — why ranking runs on inference-optimized hardware
User feature store	500 GB	500M users × 1 KB dense features
Item feature/embedding store	1.25 TB	1B items × (1 KB features + 128-dim fp16 embedding)
Training data	10 TB/day	50B events × 200 B per featurized example
Feed page size	50 KB JSON	20 items × metadata + preview refs

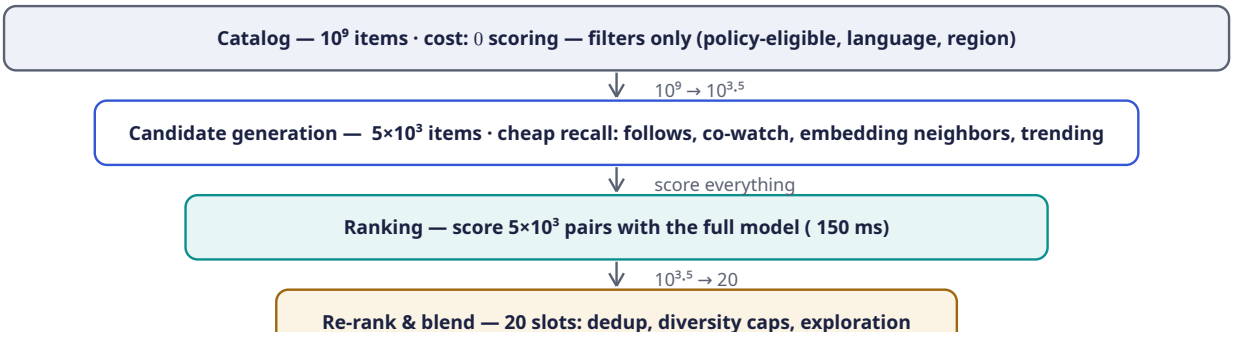
KEY INSIGHT — Three systems wearing one trench coat

The estimation reveals the real architecture: a **telemetry firehose** (580k events/s — Chapter 4 verbatim), a **serving system** (23k requests/s with a 300 ms budget — Chapters 5 and 6 territory), and an **offline factory** (10 TB/day of training data in, one fresh model out daily). The recommendation interview is three familiar interviews stitched by two seams: the feature store and the model store.

The pipeline of the chapter: Section 7.6 derives the funnel; 7.7–7.9 walk its stages (retrieve, rank, re-rank); 7.10–7.12 design APIs, architecture, and the training/serving split; 7.13 implements the core pieces in Rust; 7.14–7.20 scale, harden, and review.

7.6 The Core Challenge: The Funnel

Why must the system be two-staged? Cost arithmetic. A good ranking model costs 10^6–10^7 flops per (user, item) pair. Scoring the whole catalog per request is 10^{15} flops per page view — a data center per user. But **not** scoring is worse: a feed of random items is a dead product. The resolution is to apply cheap relevance filters first and expensive ones last:



COMMON PITFALL — Ranking the catalog

The naive answer — "score every video for every user" — fails by six orders of magnitude of compute (Section 7.5: it would need 10^{11} inferences/s average). Equally wrong in the other direction: **only** candidate channels with no learned ranking, which can't order by the actual objective. The funnel is not an optimization; it is the only shape that satisfies both relevance and the latency budget. State this trade explicitly — it is the pivot of the whole interview.

7.7 Deep Dive: Candidate Generation — Recall at 50 ms

Retrieval runs several **channels** in parallel, each a cheap, independent source of a few hundred to few thousand candidates with a quota in the final mix:

Channel	What it retrieves	How it is served cheaply
Follows	Recent uploads from creators the user follows	Reverse-chronological pull from the subscription index (Chapter 2's pull fanout — feeds are **pulled** here, not pushed)
Co-watch similarity	Items watched by people who watched what you watched	Precomputed item→item similarity table (Section 7.13 computes it); lookup by recent history — no runtime model
Embedding neighbors	Items nearest the user's taste vector	User embedding → approximate nearest-neighbor index; a 10 ms query over 1B vectors
Trending	What's hot globally/regionally right now	Chapter 6's leaderboard: time-decayed watch counts, top-K cached per region
Exploration pool	New/underexposed items earning their first data	A reserved quota (Section 7.9); without it new items starve — the rich-get-richer loop

DEFINITION — Collaborative filtering / co-watch similarity

Collaborative filtering: recommending items via the behavior of **other users** ("users like you liked…"), needing no understanding of content. Its cheapest industrial form is *item-item co-watch similarity*: count how often pairs of items are watched by the same users, normalize into a cosine similarity, and precompute each item's most-similar list offline. Serving a user is then `union(similar-to(x) for x in recent_history)` — table lookups, no math at request time.

DEFINITION — Embedding / ANN index

An *embedding* is a learned dense vector (e.g. 128 floats) representing a user or item such that **distance encodes taste**: items near a user's vector are items they'd probably like (matrix factorization / two-tower models produce them). Retrieval asks for the nearest 500 item vectors to the user vector — over 1B items, done with an *approximate nearest neighbor* (ANN) index (graph- or quantization-based), trading 1% recall for 10 ms latency. Exact search would eat the whole budget.

7.8 Deep Dive: Ranking — Precision at 150 ms

The ranker scores each surviving candidate with the expensive model. As systems engineers we care about **what the model needs at request time**:

- **Item features**: category, duration, language, age since upload, popularity counters, embedding.
- **User features**: historical category affinities, average watch time, creator affinities, embedding.
- **Cross features** (the gold): does this item's category match this user's affinity; similarity between the two embeddings.

- **Context/session features**: time of day, device, and the last N in-session watches — the reason the feed reacts "within the session" (FR-3): these features are updated by the event stream in seconds, **without retraining the model**.

> **DEFINITION — Feature store**
>
> The serving-time database of precomputed ML inputs: user features, item features, and embeddings, keyed for point lookup at ms, refreshed by the event stream (fast path) and batch jobs (slow path). It is the seam between the offline factory and online serving — the ranker never computes features from raw events at request time; it **reads** them.

The model outputs a score per candidate — typically a blend like predicted watch time, or `P(click) × E[watch | click]`. The training label comes from yesterday's implicit feedback: an impression with 80% watch-through is a strong positive; a 2-second skip is a strong negative. **The product's metric choice is the model's soul** — optimize raw clicks and you get clickbait; optimize watch time and you get binge; Section 7.17's guardrail metrics exist because the objective is a policy decision with an engineering delivery mechanism.

7.9 Deep Dive: Re-Ranking — The Last 50 ms

Raw score order is not the feed. The final stage applies product rules to the ranked few hundred:

1. **Dedup** — the same video arriving from three channels appears once.
2. **Diversity caps** — e.g., ≤2 per category/creator per page (Section 7.13 implements this exactly). Uncapped, the ranker collapses the feed to whatever topic the user binged last night — the **filter bubble** failure mode.
3. **Freshness boost** — new uploads from followed creators get a bounded bonus; subscriptions must mean something.
4. **Exploration slot** — reserve 1 of 20 slots for the exploration pool.

> **DEFINITION — Exploration vs. exploitation / multi-armed bandit**
>
> *Exploitation*: show what the model already believes is best. *Exploration*: spend a small quota on items whose value is uncertain, to **learn** — new videos literally cannot be ranked honestly until someone watches them. The *multi-armed bandit* framing formalizes the trade: an ε-greedy policy exploits with probability $1-\varepsilon$ and explores uniformly with probability ε. Without a guaranteed exploration quota, the feedback loop is a closed circle: only shown items get data, only items with data get shown — new creators starve and the catalog fossilizes. Exploration is not charity; it is the system's only source of new information.

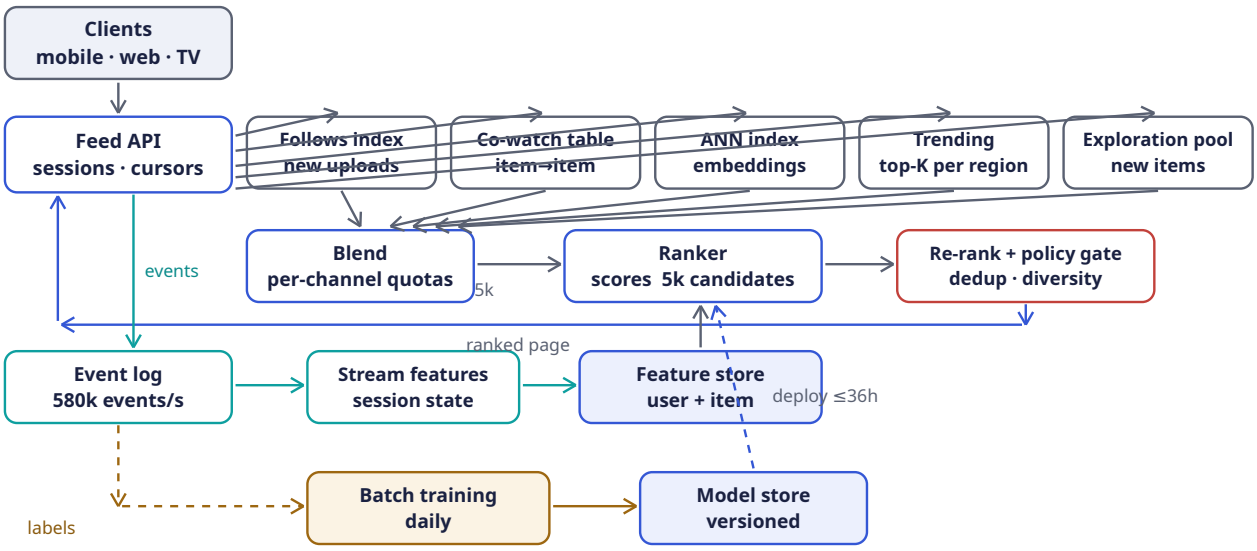
> **KEY INSIGHT — The feed is controlled by whoever sets the caps**
>
> Notice what happened across three sections: the **model** orders candidates, but the **product** owns the final screen — dedup rules, diversity caps, freshness boosts, exploration quota. This is deliberate and healthy: ML optimizes the measurable; humans govern the mixture. In the interview, calling out this separation (and where each lever lives) reads as having operated such a system, not just read about one.

7.10 API Design

Method	Path	Purpose
GET	/v1/feed?cursor=&limit=	Next ranked page (FR-1); cursor encodes session + page state, opaque
POST	/v1/events:batch	Impressions, plays, watch-time, likes, skips — compressed batches (FR-2)
POST	/v1/feedback/not-interested	Negative feedback, effective immediately (FR-5)
GET	/v1/trending?region=	Logged-out / cold-start fallback (FR-6); Chapter 6's board, cached
GET	/v1/feed/explain/{item}	"Why am I seeing this?" — the channel that surfaced it (transparency, cheap to add)

Feed responses carry `{items: [{id, rank, channel}], cursor}`. The `channel` tag is returned on purpose: it powers explainability, per-channel metrics (Section 7.17), and client-side diversity hints. Event ingestion is fire-and-forget for the client (202 Accepted), exactly the Chapter 4 contract — the feed must never block on its own telemetry.

7.11 High-Level Architecture



Component	Responsibility	Why it is shaped this way
Feed API	Session context, cursor state, final assembly	Stateless; the only client-facing door
Candidate channels	Each retrieves 10^2–10^3 plausible items its own way	Independent, parallel, replaceable; quotas blend them (Section 7.7)
Ranker	Scores all candidates with the serving model	The expensive stage: reads the feature store, never computes from raw events
Re-rank + policy	Dedup, diversity caps, freshness boost, exploration slot, safety filter	Product rules the model cannot be trusted with (Section 7.9)

Component	Responsibility	Why it is shaped this way
Event log	Durable buffer for all feedback	Chapter 4's shock absorber, third time in this book
Stream features	Session state and counters updated in seconds	In-session adaptation (FR-3) without model retraining
Feature store	Point-lookup user/item features + embeddings at ms	The seam between offline factory and online serving
Batch training	Daily: labels → features → model → evaluation	Section 7.12; model **internals** are out of scope, its **supply chain** is not
Model store	Versioned models, canary deploys, rollback	A bad model is a deploy incident, treated exactly like bad code

7.12 Training vs. Serving: The Two-Factory Split

The **offline factory** runs daily: join yesterday's 50B events into labeled examples (impression ↔ watch outcome within an attribution window), compute features, train, evaluate against holdout, and publish a versioned model. The **online factory** serves 23k requests/s with that frozen model plus fresh features. Between them sits the discipline this split exists to protect:

> **COMMON PITFALL — Training/serving skew**
>
> The subtlest recommender bug: the model trains on features computed one way (batch SQL over the warehouse) and serves on features computed another (stream jobs with different semantics) — e.g., "watches in last 7 days" counted by calendar week offline and rolling 168 h online. The model silently degrades and no dashboard alarms, because **both** computations are internally correct. Defense: one feature **definition** compiled to both paths (a feature store with shared definitions), plus canary evaluation of every new model on live traffic before promotion. If you name one ML systems pitfall in the interview, name this one.

Freshness policy: the model is ≤36 h old (daily train + evaluation + canary); user/item features ≤1 h; session features ≤10 s. Each layer of staleness is a deliberate price paid for stability, and each is an SLO in Section 7.17.

7.13 Rust Reference Implementations

Four pieces with deterministic tests: item-item collaborative filtering, the diversity re-ranker, ε-greedy exploration, and embedding similarity.

7.13.1 Item-Item Collaborative Filtering

```rust
use std::collections::{HashMap, HashSet};

/// The co-watch relation: user -> items watched, item -> watchers.
/// Implicit positives only — a watch is a vote (Chapter 7.1).
#[derive(Default)]
pub struct WatchMatrix {
    pub by_user: HashMap<u64, HashSet<u64>>,
    pub watchers: HashMap<u64, HashSet<u64>>,
}

impl WatchMatrix {
```

```
    pub fn from_pairs(pairs: &[(u64, u64)]) -> Self {
        let mut m = WatchMatrix::default();
        for &(u, item) in pairs {
            m.by_user.entry(u).or_default().insert(item);
            m.watchers.entry(item).or_default().insert(u);
        }
        m
    }

    /// Cosine similarity over the co-watch relation:
    /// |watchers(a) ∩ watchers(b)| / sqrt(|watchers(a)| * |watchers(b)|).
    /// Precomputed offline per item in production; computed inline here.
    pub fn similarity(&self, a: u64, b: u64) -> f64 {
        let (wa, wb) = match (self.watchers.get(&a), self.watchers.get(&b)) {
            (Some(x), Some(y)) => (x, y),
            _ => return 0.0,
        };
        let (small, big) = if wa.len() <= wb.len() { (wa, wb) } else { (wb, wa) };
        let co = small.iter().filter(|u| big.contains(u)).count() as f64;
        if co == 0.0 { return 0.0; }
        co / (wa.len() as f64 * wb.len() as f64).sqrt()
    }

    /// Recommend unseen items, scored by summed similarity to the user's
    /// whole watch history — the recall channel of Section 7.7.
    pub fn recommend(&self, user: u64, n: usize) -> Vec<(u64, f64)> {
        let seen = self.by_user.get(&user).cloned().unwrap_or_default();
        let mut score: HashMap<u64, f64> = HashMap::new();
        for &item in &seen {
            for &other in self.watchers.keys() {
                if seen.contains(&other) { continue; }
                let s = self.similarity(item, other);
                if s > 0.0 { *score.entry(other).or_default() += s; }
            }
        }
        let mut v: Vec<(u64, f64)> = score.into_iter().collect();
        v.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap().then(a.0.cmp(&b.0)));
        v.truncate(n);
        v
    }
}

#[cfg(test)]
mod cf_tests {
    use super::*;

    fn matrix() -> WatchMatrix {
        // users 1,2,3 ; items 10,20,30,40
        WatchMatrix::from_pairs(&[
            (1, 10), (1, 20),
            (2, 10), (2, 20), (2, 30),
            (3, 30), (3, 40),
        ])
    }

    #[test]
    fn similarity_counts_co_watchers() {
        let m = matrix();
        assert_eq!(m.similarity(10, 20), 1.0);       // identical audiences
        assert_eq!(m.similarity(10, 30), 0.5);       // 1 shared of 2 & 2
```

```
        assert!((m.similarity(30, 40) - 0.7071).abs() < 1e-3); // 1/sqrt(2)
        assert_eq!(m.similarity(10, 40), 0.0);       // no shared watchers
    }

    #[test]
    fn recommendations_exclude_seen_and_rank_by_evidence() {
        let m = matrix();
        let recs = m.recommend(1, 5);
        // user 1 watched {10, 20}; item 30 scores sim(10,30)+sim(20,30) = 1.0,
        // item 40 scores 0 and is not surfaced
        assert_eq!(recs, vec![(30, 1.0)]);
        assert!(!recs.iter().any(|&(id, _)| id == 10 || id == 20));
    }
}
```

7.13.2 The Re-Ranker: Dedup + Diversity Quotas

```
use std::collections::{HashMap, HashSet};

#[derive(Debug, Clone)]
pub struct Candidate {
    pub id: u64,
    pub category: &'static str,
    pub score: f64,
}

/// Same item arriving from several channels: keep its best score, once.
pub fn dedup(ranked: Vec<Candidate>) -> Vec<Candidate> {
    let mut seen = HashSet::new();
    ranked.into_iter().filter(|c| seen.insert(c.id)).collect()
}

/// Take a page of n from score order, allowing at most `per_category`
/// items per category — pass 1 respects the cap, pass 2 fills any
/// remaining slots in score order so the page is never short.
pub fn rerank_with_quota(
    mut ranked: Vec<Candidate>,
    n: usize,
    per_category: usize,
) -> Vec<u64> {
    ranked.sort_by(|a, b| b.score.partial_cmp(&a.score).unwrap().then(a.id.cmp(&b.id)));
    let mut out = Vec::with_capacity(n);
    let mut counts: HashMap<&str, usize> = HashMap::new();
    let mut leftover = Vec::new();
    for c in ranked {
        let cnt = counts.entry(c.category).or_insert(0);
        if *cnt < per_category && out.len() < n {
            out.push(c.id);
            *cnt += 1;
        } else {
            leftover.push(c.id);
        }
    }
    for id in leftover {
        if out.len() < n { out.push(id); }
    }
    out
}
```

```rust
#[cfg(test)]
mod rerank_tests {
    use super::*;

    fn c(id: u64, cat: &'static str, score: f64) -> Candidate {
        Candidate { id, category: cat, score }
    }

    #[test]
    fn quota_caps_a_dominant_category_but_fills_the_page() {
        let ranked = vec![
            c(1, "sport", 9.0), c(2, "sport", 8.5), c(3, "sport", 8.0),
            c(4, "music", 7.5), c(5, "news", 7.0),
        ];
        // cap 2 per category, page of 5
        assert_eq!(rerank_with_quota(ranked, 5, 2), vec![1, 2, 4, 5, 3]);
    }

    #[test]
    fn dedup_keeps_first_best_occurrence() {
        let dup = vec![c(1, "sport", 9.0), c(1, "sport", 7.0), c(2, "news", 8.0)];
        let unique = dedup(dup);
        assert_eq!(unique.len(), 2);
        assert_eq!(unique[0].score, 9.0); // first occurrence wins
    }
}
```

7.13.3 ε-Greedy Exploration (Multi-Armed Bandit)

```rust
/// Tiny deterministic RNG (xorshift64) so tests are reproducible.
pub struct XorShift(pub u64);
impl XorShift {
    pub fn next(&mut self) -> u64 {
        let mut x = self.0;
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        self.0 = x;
        x
    }
    pub fn f64(&mut self) -> f64 { (self.next() >> 11) as f64 / (1u64 << 53) as f64 }
    pub fn below(&mut self, n: u64) -> u64 { self.next() % n }
}

/// ε-greedy: exploit the best-estimate arm, explore uniformly with
/// probability ε. Arms = channels or new-item pools earning data.
pub struct EpsilonGreedy {
    pub means: Vec<f64>,
    pub counts: Vec<u64>,
    pub epsilon: f64,
    rng: XorShift,
}

impl EpsilonGreedy {
    pub fn new(arms: usize, epsilon: f64, seed: u64) -> Self {
        EpsilonGreedy {
            means: vec![0.0; arms],
            counts: vec![0; arms],
            epsilon,
            rng: XorShift(seed.max(1)),
        }
```

```
    }

    pub fn pull(&mut self) -> usize {
        // initialize: every arm once, in order
        if let Some(i) = self.counts.iter().position(|&c| c == 0) { return i; }
        if self.rng.f64() < self.epsilon {
            self.rng.below(self.means.len() as u64) as usize
        } else {
            self.means
                .iter()
                .enumerate()
                .max_by(|a, b| a.1.partial_cmp(b.1).unwrap())
                .map(|(i, _)| i)
                .unwrap()
        }
    }

    /// Incremental mean update — the bandit learns from implicit feedback.
    pub fn record(&mut self, arm: usize, reward: f64) {
        self.counts[arm] += 1;
        let n = self.counts[arm] as f64;
        self.means[arm] += (reward - self.means[arm]) / n;
    }
}

#[cfg(test)]
mod bandit_tests {
    use super::*;

    #[test]
    fn zero_epsilon_exploits_after_initialization() {
        let true_means = [0.1, 0.5, 0.9];
        let mut b = EpsilonGreedy::new(3, 0.0, 42);
        let mut counts = [0u64; 3];
        for _ in 0..103 {
            let a = b.pull();
            counts[a] += 1;
            b.record(a, true_means[a]);
        }
        assert_eq!(counts, [1, 1, 101]); // 3 init pulls, then all arm 2
    }

    #[test]
    fn pure_exploration_spreads_uniformly() {
        let true_means = [0.1, 0.5, 0.9];
        let mut b = EpsilonGreedy::new(3, 1.0, 88_172_645_463_325_252);
        let mut counts = [0u64; 3];
        for _ in 0..303 {
            let a = b.pull();
            counts[a] += 1;
            b.record(a, true_means[a]);
        }
        assert_eq!(counts.iter().sum::<u64>(), 303);
        for &c in &counts {
            assert!((60..=140).contains(&c), "counts = {counts:?}");
        }
    }

    #[test]
    fn running_mean_converges() {
```

```
        let mut b = EpsilonGreedy::new(1, 0.0, 1);
        b.record(0, 1.0);
        b.record(0, 0.0);
        assert_eq!(b.means[0], 0.5);
    }
}
```

7.13.4 Embedding Similarity: Cosine Top-K

```
/// Cosine similarity between taste vectors; distance encodes preference.
pub fn cosine(a: &[f64], b: &[f64]) -> f64 {
    let dot: f64 = a.iter().zip(b).map(|(x, y)| x * y).sum();
    let na = a.iter().map(|x| x * x).sum::<f64>().sqrt();
    let nb = b.iter().map(|x| x * x).sum::<f64>().sqrt();
    if na == 0.0 || nb == 0.0 { 0.0 } else { dot / (na * nb) }
}

/// Exact nearest neighbors for a user vector. Honest scope note: this is
/// O(n·d) — the teaching core. Production swaps in an ANN index
/// (Section 7.7) with identical interface and ~99% of the recall.
pub fn top_k_similar(
    query: &[f64],
    items: &[(u64, Vec<f64>)],
    k: usize,
) -> Vec<(u64, f64)> {
    let mut scored: Vec<(u64, f64)> =
        items.iter().map(|(id, v)| (*id, cosine(query, v))).collect();
    scored.sort_by(|a, b| b.1.partial_cmp(&a.1).unwrap().then(a.0.cmp(&b.0)));
    scored.truncate(k);
    scored
}

#[cfg(test)]
mod ann_tests {
    use super::*;

    #[test]
    fn nearest_neighbors_by_direction_not_magnitude() {
        let q = vec![1.0, 0.0, 0.0];
        let items = vec![
            (1, vec![0.9, 0.1, 0.0]),  // nearly aligned
            (2, vec![0.0, 1.0, 0.0]),  // orthogonal
            (3, vec![5.0, 0.0, 0.0]),  // identical direction, 5x magnitude
        ];
        let top2 = top_k_similar(&q, &items, 2);
        let ids: Vec<u64> = top2.iter().map(|e| e.0).collect();
        assert_eq!(ids, vec![3, 1]);          // direction beats magnitude
        assert_eq!(top2[0].1, 1.0);           // perfectly aligned
        assert!((top2[1].1 - 0.9939).abs() < 1e-3);
        assert_eq!(cosine(&q, &items[1].1), 0.0);
    }
}
```

7.14 Scaling the Platform

Layer	Scale axis	Mechanism
Feed API / ranker	23k req/s avg	Stateless services, horizontal; ranking replicas scale with scoring rate (115M inferences/s avg) on inference-optimized hardware
Candidate channels	Catalog size	Each channel scales independently: follows index (Chapter 2 pull), precomputed co-watch table (offline), ANN shards (embedding space partitioned), trending cache (Chapter 6)
Event pipeline	580k events/s avg, 3M/s peak	Chapter 4 verbatim: partitioned durable log, stream processors for session features, batch for training
Feature store	500 GB user + 1.25 TB item	Sharded key-value with point lookups; read-heavy, so replicas carry the load
Training	10 TB/day examples	Data-parallel GPU/TPU fleet; the daily cadence bounds both cost and staleness
Feed cache	Logged-out / cold-start	Trending pages are shared and cached hard; personalized pages are never shared — cache the **candidates**, not the feed

KEY INSIGHT — Personalization is cache-hostile by definition

Chapter 5 cached windows shared by millions; here **every response is unique**. The caching strategy inverts: cache the shared ingredients (trending boards, co-watch lists, item features, embeddings) and assemble the personalized result fresh. A recommender that caches feeds has accidentally built trending-with-extra-latency.

7.15 Failure Modes & Degradation

Failure	Symptom	Response
Event pipeline lag	Session features stale; feed feels unreactive	Degrade gracefully to non-session features; lag alerts (Chapter 4); catch-up replay
One candidate channel down	Feed still full, less varied	Quotas redistribute to surviving channels; the blend is designed for exactly this — no single point of taste
Ranker overload	Latency budget breached	Score fewer candidates (5k → 1k): relevance drops imper-

Failure	Symptom	Response
		ceptibly before latency does; then trending fallback
Bad model deploy	Feed quality craters at scale	Canary + business-metric guardrails on the deploy pipeline; one-click rollback to previous model version — model incidents are deploy incidents
Feature store stale	Model serves on old affinities	Staleness SLO with alerts; beyond threshold, ranker down-weights stale features
Training pipeline fails	Model ages past 36 h	Yesterday's model keeps serving; freshness alert pages; investigate like any failed batch job
Feedback loop spiral	One topic floods a user's feed	Diversity caps bound the damage structurally; guardrail metrics (Section 7.17) detect drift
Trending fallback poisoned	Manipulated trending board	Chapter 6's integrity tools apply: velocity anomalies, quarantine feed

7.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Architecture	Funnel: retrieve → rank → re-rank	Single-stage scoring of the catalog: 6 orders of magnitude over compute budget (Section 7.6); retrieval-only: no objective-driven order
Retrieval	Multi-channel blend with quotas	One giant embedding index alone: best average relevance, worst failure modes — no guarantee follows or fresh uploads ever surface
Model freshness	Daily retrain + real-time session features	Online learning (update weights per event): fresher, but a feedback-loop amplifier and an operational nightmare to roll back
Objective	Watch time with guardrail metrics	Pure CTR: optimizes clickbait; pure satisfaction surveys: too sparse to train on
Exploration	Guaranteed quota (ε-greedy slot)	Pure exploitation: the rich-get-richer loop fossilizes the catalog; Thompson

Decision	Chosen	Alternative & why not (here)
		sampling: better theory, same quota slot in practice
Feed consistency	Cursor paginates over a snapshot	Re-rank every page live: infinite scroll jumps and repeats; the session cursor pins the candidate set
Cold start	Trending + category picks, then rapid session adaptation	Signup interest quizzes: fine complement, cannot be the only path (friction, stale answers)

7.17 Observability & SLOs

Two metric families — system health (Chapter 4's bread and butter) and **quality** health, which only this kind of system has:

Indicator	Definition	Target
Feed latency	Page generation p95	≤ 300 ms
Event lag	Event → session feature visible	≤ 10 s
Model freshness	Age of serving model	≤ 36 h, alert past
Candidate coverage	Pages served with all channels contributing	≥ 99%
CTR / watch time	Clicks and minutes per impression, per model version	Guardrail: never ship a regression
Diversity index	Distinct categories/creators per page	≥ policy floor
Exploration health	Share of new items reaching N impressions in 1 h	≥ 95%
Negative-feedback rate	"Not interested" per impression	Watch trend, alert on spikes

KEY INSIGHT — Guardrail metrics are the product's conscience

Every optimization target can be gamed by the optimizer itself: CTR invites clickbait, watch time invites doomscrolling. Production recommenders therefore ship **pairs** of metrics — the objective and the guardrails (diversity, negative feedback, long-term retention) — and a model that improves the objective while moving a guardrail does not ship. Encoding this in the deploy pipeline is systems work, and it is the difference between a recommender and a slot machine.

7.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Make it real-time: the feed reacts to every watch instantly."** Already half-built: session features update in seconds. The next step — updating the **user embedding** mid-session — is streaming inference, not training: recompute the user vector from the new history with the frozen item tower. True online weight updates trade the rollback story for freshness; say what you'd be giving up.

2. **"A new creator with zero viewers — how do they ever surface?"** The exploration quota (Section 7.9) plus **content-based** features: with no watch history, the video's own metadata (category, title embeddings, thumbnail features) place it near known items. First impressions go to users whose taste vector is near the **content** vector — cold start is why content features exist at all.
3. **"TikTok vs. YouTube — what differs in the system?"** The graph: YouTube leans on subscriptions (the follows channel is strong); TikTok is graph-free — nearly pure engagement ranking, which makes its exploration quota and per-session adaptation much larger shares of the mix. Same funnel, different quotas. Architecture is the constants.
4. **"Where do ads fit?"** A second ranking system with its own objective (expected revenue × relevance), blended into slots by an auction — and the reason feed slots are strictly partitioned between organic and sponsored before re-ranking.
5. **"How would you detect the model amplifying harmful content?"** Guardrail metrics per content class, policy-gate recalls, and audit sampling of what the exploration pool promotes — plus the honest answer: ranking amplifies whatever the objective correlates with, so this is fought at the objective-and-guardrail layer, not with filters alone.

7.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> A recommender is three systems in a trench coat: a telemetry firehose (Chapter 4), a latency-bound serving funnel, and a daily model factory. The funnel exists because of arithmetic: you may score 5k of 10^9 items per request, so cheap recall channels (follows, co-watch, embedding ANN, trending, exploration) feed one expensive ranker, and a re-ranker applies the product's conscience — dedup, diversity caps, freshness, policy. Implicit feedback is the fuel; the feature store is the seam that keeps training and serving honest (skew is the killer bug); exploration is the quota that keeps the feedback loop open; guardrail metrics decide what ships. The lesson that transfers: **in ML systems, the model is a component — the system around it is the design.**

Further reading.

- The source video: *"22: Recommendation Engine (YouTube, TikTok) — Systems Design Interview Questions With Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=QrZTmiZSRcw`
- Covington, Adams, Sargin — *"Deep Neural Networks for YouTube Recommendations"* (2016) — the canonical two-stage funnel paper this chapter's architecture follows.
- Koren, Bell, Volinsky — *"Matrix Factorization Techniques for Recommender Systems"* (2009) — embeddings before they were called embeddings.
- Sutton & Barto, *Reinforcement Learning* — the multi-armed bandit chapters behind Section 7.9's exploration quota.
- Chapter 4's monitoring reading applies twice here — guardrail metrics are SLOs with a conscience.

7.20 Chapter Glossary

Term	Meaning
A/B test	Controlled experiment over model/parameter versions; how guardrail and objective metrics earn their keep
ANN index	Approximate nearest-neighbor structure over embeddings; 1% recall traded for 10 ms latency over 1B vectors

Term	Meaning
Candidate generation	The recall stage: cheap channels selecting 10^3 plausible items from the catalog
Clickbait / CTR trap	The failure of optimizing clicks alone: the model learns to promise, not to satisfy
Cold start	Serving users or items with no history: trending/category fallbacks for users, content features + exploration for items
Collaborative filtering	Recommendation from other users' behavior, no content understanding required
Co-watch similarity	Item-item cosine over shared watchers; precomputed offline, looked up at serve time
Cross feature	A feature combining user and item (e.g., category × affinity); where ranking accuracy concentrates
Diversity cap	Re-rank rule bounding any category/creator per page; structural filter-bubble defense
Embedding	Learned dense vector for a user or item; distance encodes taste
ε-greedy	Bandit policy exploiting the best arm with probability $1-\varepsilon$, exploring uniformly otherwise
Exploration quota	Reserved feed slots for uncertain items; the only source of new information in the loop
Feature store	Serving-time database of precomputed user/item features with shared offline/online definitions
Feedback loop (rich-get-richer)	Shown items get data, data earns impressions; without exploration the catalog fossilizes
Filter bubble	Feed collapse to one topic under pure score order; fought with diversity caps
Funnel	The retrieve → rank → re-rank shape; a compute-budget necessity, not an optimization
Guardrail metric	A metric that may not regress even when the objective improves; ships alongside the objective
Implicit feedback	Behavioral signals (watch time, skips, impressions) used as training labels
Latency budget	300 ms p95 allocated across retrieval, ranking, re-ranking; dictates what must be precomputed
Matrix factorization / two-tower	Model families producing user and item embeddings from interaction history
Model store	Versioned model registry with canary deploys and rollback; models are deploys
Multi-armed bandit	The exploration/exploitation formalism: arms are channels or items, rewards are engagement

Term	Meaning
Ranking	The precision stage: scoring retrieval's candidates with the full model under 150 ms
Re-ranking	The product-rules stage: dedup, diversity, freshness, policy, exploration slot
Session features	In-session signals (last N watches) updated in seconds; feed reactivity without retraining
Training/serving skew	Feature definitions diverging between offline training and online serving; the silent killer of recommenders
Trending	Time-decayed popularity board per region — Chapter 6's leaderboard reused; the cold-start and outage fallback

— End of Chapter 7 · Next: Chapter 8 —

CHAPTER 8

Designing a Database Index: How B-Trees Work

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"How do B-Tree Indexes work?"* from the series *Systems Design Interview: 0 to 1 with Google Software Engineer* (channel: *Jordan has no life*). Unlike the product chapters, this is a **fundamentals** deep dive — the interviewer asks you to design the index structure of a relational database's storage engine: something that answers point lookups and range scans in milliseconds over billions of rows, survives a write-heavy workload, and lives on hardware whose physics you must respect. It is also the keystone chapter: Chapters 5 and 6 stored data in "the database"; this chapter designs what they were standing on. All terms are defined before use; all reference code is Rust with deterministic tests.

8.1 The Problem Statement

The interviewer draws a table with a billion rows and says:

"Queries filter on equality (`email = ?`) and on ranges with ordering (`created BETWEEN ? AND ?`, `ORDER BY created`). The table lives on disk. Design the index structure that makes those fast — and explain exactly why yours beats the alternatives."

This is a design problem with an unusually well-defined physics: the bottleneck is not CPU or capacity but **disk I/O shape** — every random read costs 10^5 times a memory reference, so the entire game is minimizing the number of disk pages touched per operation. The B+ tree won this game fifty years ago; the interview tests whether you can re-derive **why** from first principles, not whether you can recite it.

> **DEFINITION — Index**
>
> A redundant, derived data structure that maps a search key to the location of full rows, maintained by the database on every write. An index trades write cost and space for read speed: every `INSERT` pays to keep it current, and every matching query repays that cost a thousandfold. "Adding an index" is never free — it is a bet that reads outnumber writes.

> **DEFINITION — Point lookup / range scan / sorted iteration**
>
> The three read shapes an index can serve: *point lookup* (`key = ?`), *range scan* (`lo ≤ key ≤ hi`), and *sorted iteration* (`ORDER BY key` without a sort step). A structure that serves all three is dramatically more valuable than one serving only the first — this asymmetry eliminates the hash index from the running almost immediately (Section 8.7).

8.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Workload shape?	OLTP: many small point reads/writes, plus frequent range scans and ORDER BY pagination
Data resident where?	On disk (or SSD); tables far exceed RAM. Indexes may be partially cached
Read/write mix?	Mixed; neither extreme. Reads dominate slightly, writes must stay cheap
Key types?	Integers, strings, composite (multi-column) keys
Concurrency?	Mention latching strategy; don't design a full lock manager
Durability?	Committed writes must survive crashes — bring in a WAL when you get to the write path
Distribution?	Single node. Sharding is Chapter 5's topic; indexes replicate/shard with their table

NOTE — Agreed scope

1. Design the on-disk index structure for a single-node relational storage engine: **point lookup**, **range scan**, **sorted iteration**, **insert**, **delete**.
2. Respect disk physics: I/O in fixed-size pages; random I/O is the scarce resource.
3. Cover the write path (buffer pool, WAL, splits) and the B+ tree's classic variants (clustered, secondary, covering, composite).
4. Position the LSM tree as the write-optimized alternative and know when it wins.
5. Out: concurrency control protocols, distributed indexes, query optimization beyond index selection basics.

8.3 Functional Requirements

1. **FR-1 — Point lookup.** `key → row location` in a small, bounded number of page reads regardless of table size.
2. **FR-2 — Range scan.** All keys in `[lo, hi]` in sorted order, touching only pages that contain them.
3. **FR-3 — Sorted iteration.** Full or partial ordered traversal (for `ORDER BY ... LIMIT`) without an external sort.
4. **FR-4 — Insert.** Add a key, maintaining the structure's invariants, with bounded page writes.
5. **FR-5 — Delete.** Remove a key, keeping pages reasonably full (no slow decay into a sparse, space-wasting structure).
6. **FR-6 — Crash safety.** No committed state is lost and the structure is never corrupted by a crash mid-write (with the WAL, Section 8.10).

8.4 Non-Functional Requirements

DEFINITION — Page / block I/O

Disks do not read bytes; they read **pages** — fixed-size blocks (4–16 KB) that are the atomic unit of I/O and of the buffer pool. Every structure in this chapter is designed so that **one node** = **one page**: reading a node is one I/O, and the cost model of every algorithm is counted in pages touched, not comparisons made.

DEFINITION — Fanout

The number of children per internal node — for a page-sized node holding keys and child pointers, fanout ≈ `page_size / (key + pointer)` ≈ hundreds to a thousand. Fanout is the whole game: tree height is `log_fanout(rows)`, and height is disk reads per lookup.

Requirement	Target & reasoning
Lookup depth	≤ 4 page reads for 10^9 rows — the height math of Section 8.5 makes this the headline NFR
Write cost	O(height) page writes per insert, amortized; splits rare at high fanout
Space overhead	Index ≈ 10–15% of table size; pages 70–90% full on average
Range locality	Range scans read mostly **sequential** pages — leaves are linked and physically clustered
Cache friendliness	Top levels of the tree permanently resident in the buffer pool; most lookups hit disk once
Predictable tail latency	No unbounded rebalancing cascades; split cost is bounded and rare

KEY INSIGHT — You are designing for the disk, not the data

Every prior chapter counted requests and bytes; this one counts **page touches**. A binary search tree does 30 comparisons for a billion rows — and if each node is a random page, 30 disk reads. A B+ tree does more comparisons per node but touches **four** pages. On disk, the "slower" CPU structure is 10× faster. The interview is won by whoever counts the right resource.

8.5 Back-of-the-Envelope: The Physics and the Height Math

Latency pyramid (orders of magnitude — memorize these):

Operation	Latency	Consequence
RAM reference	100 ns	The free resource; comparisons are noise
SSD random page read	100 µs	1000× RAM — the unit this chapter minimizes
SSD sequential read	10× cheaper per page	Why leaf links and clustered layout matter
HDD random I/O (seek)	10 ms	100,000× RAM; mechanical arm physics — the original design constraint
Network round trip	0.5–1 ms	For context: why a database page miss still beats a remote call

The height derivation — the most important arithmetic in the chapter:

Quantity	Value	Derivation
Page size	16 KB	Typical database page
Index entry	16 B	8 B key + 8 B child pointer (interior)
Fanout per page	1000	16 KB / 16 B

Quantity	Value	Derivation
Rows reachable, height 2	10^6	1000 leaf pages × 1000 entries… precisely: root → 1000 internal pages → leaves; leaves hold 16KB/ (16B+ptr to row) ≈ 500–1000 entries
Rows reachable, height 3	10^9	A billion rows in **three** levels of internal nodes + leaves
Page reads per lookup	≤ 4, usually 1–2	Root and level-1 live in the buffer pool — only the leaf read hits disk
Index size for 10^9 rows	16 GB leaves + 0.02 GB interior	Interior levels are negligible — that is why they are always cached

KEY INSIGHT — High fanout collapses height

Binary trees give height $\log_2$ — 30 levels for a billion rows. A fanout of 1000 gives $\log_{1000}$ — **three**. Since the top two levels are tiny (16 MB) they live in RAM permanently, and a billion-row lookup costs about **one disk read**. That is the entire magic trick: not cleverer comparisons, just a tree so wide it is nearly flat. Every B-tree property — page-sized nodes, separator keys, splits that propagate up rarely — exists to protect this fanout.

8.6 The Core Challenge: One Structure for Reads and Writes

The tension: reads want sorted, dense, immutable layouts; writes want cheap, localized mutation. Structures that ace one side fail the other:

COMMON PITFALL — The two seductive wrong answers

Hash index: O(1) point lookups — and **nothing else**: no ranges, no ordering, and rehashing a billion-key table is an outage. It answers the question you weren't asked. *Sorted file*: binary search gives O(log n) reads and ranges are sequential — but one insert into the middle rewrites half the file. Append and it degrades into a log you must compact (that idea, pursued seriously, becomes the LSM tree of Section 8.11 — the legitimate rival). The B+ tree threads the needle: sorted **and** cheaply mutable, because mutation is localized to one page and its ancestors.

8.7 Candidate Structures

Structure	Point lookup	Range scan	Insert	Verdict
Hash table	O(1)	unsupported	O(1)	Point lookups only; no order, no ranges
Sorted array / file	O(log n)	O(log n + k)	O(n)	Reads fine; a single insert rewrites the file
Binary search tree	O(n) worst	O(n)	O(n) worst	Unbalanced under real key distributions (sorted inserts!)

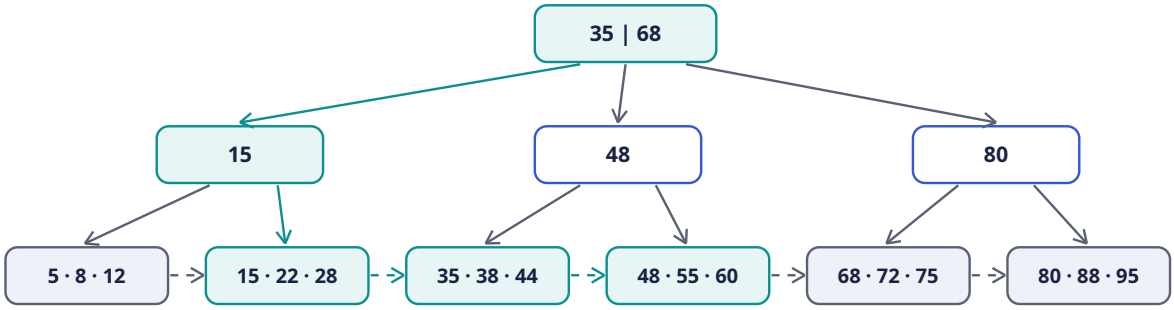
Structure	Point lookup	Range scan	Insert	Verdict
Balanced BST (AVL/ red-black)	O(log n)	O(log n + k)	O(log n)	Right complexity, wrong physics: 30 random pages per lookup
B+ tree	≤4 pages	≤4 + sequential leaves	O(height) pages	Page-sized nodes, fanout 1000, linked leaves — Section 8.9

8.8 Deep Dive: B-Tree Mechanics

DEFINITION — B-tree / node split

A *B-tree* is a self-balancing search tree where every node is one disk page holding up to **m** sorted keys and m+1 child pointers. Invariants: all leaves at the same depth; every non-root node at least half full; keys sorted within and across nodes. Insertion descends to the target leaf; if the leaf overflows, it *splits* in two and pushes a separator key up into the parent — which may itself split, in a bounded cascade that, at the root, grows the tree **taller by one level**. Growth by root splits is why the tree stays perfectly balanced under any key order — including the sorted inserts that destroy naive BSTs.

Search is one comparison loop per level: at each internal page, binary search the separator keys, follow the child pointer, read one more page. Height 3–4 (Section 8.5) ⇒ ≤4 page reads, and the upper pages are cached.



Range scan [22, 55]: descend root → internal(15) → leaf 2 (22, 28), then follow leaf links through 55 — never returning upward.

Deletion is the mirror: remove from the leaf; if a node falls below half-full, borrow from a sibling or merge siblings and pull the separator down. In practice engines under-fill rather than merge eagerly (merges are latch-heavy), and a background process rebalances — Section 8.15's "index bloat" is what happens when even that is neglected.

8.9 Deep Dive: The B+ Tree Refinements

The production variant differs from the textbook B-tree in three deliberate ways:

1. **Values live only in leaves.** Internal pages hold pure separator keys — no row data — which **increases fanout** (more separators per page) and protects the height math.
2. **Leaves are linked** in key order (the dashed arrows above). A range scan descends once, then walks sideways — sequentially, the disk's favorite pattern. This is what makes `ORDER BY ... LIMIT 20` nearly free.
3. **Interior keys are copies.** The separator `35` also exists as a real key in a leaf; internal copies are pure routing.

Three index **usages** ride on the same structure:

> **DEFINITION — Clustered / secondary / covering / composite index**
>
> A *clustered* index **is** the table: leaf pages hold full rows, physically ordered by the key (one per table; range scans read rows directly). A *secondary* index's leaves hold `(key → primary key)`: a lookup then re-descends the clustered index (a "bookmark lookup") — extra I/O per row that makes non-covering secondary scans expensive. A *covering* index stores every column the query needs, so the second lookup never happens. A *composite* index keys on `(a, b, c)` as a tuple — usable by queries constrained on a **leftmost prefix** (`a`, or `a,b`), useless for `b` alone, because the order is lexicographic (Section 8.13 implements the rule).

8.10 The Write Path: Buffer Pool and WAL

Writes never touch disk directly:

> **DEFINITION — Buffer pool**
>
> The database's page cache: a fixed RAM arena of page frames with an LRU-ish eviction policy. Reads check it first (the tree's top levels are effectively pinned by popularity); writes **modify pages in memory** and mark them dirty, flushing lazily. Most "database performance" is buffer pool hit ratio — Section 8.13 implements the LRU core.

> **DEFINITION — Write-ahead log (WAL)**
>
> The durability trick: before any dirty page may flush, the **intention** — an append-only log record of the change — must be durable on disk. Appends are sequential and cheap; a crash replays the log to redo committed changes. The WAL is what lets the buffer pool defer random page writes without losing committed data — the same append-log bet Chapter 1 made for operations and Chapter 6 made for score journals.

The split dance on insert: descend with latches, split the leaf if full, push the separator up — each split is 2–3 page writes plus a WAL record, rare at fanout 1000 (a leaf absorbs 500 inserts between splits), and amortized into the background flush stream.

8.11 The Rival: LSM Trees

> **DEFINITION — LSM tree / memtable / SSTable / compaction**
>
> The *log-structured merge tree* is the write-optimized alternative: writes land in an in-memory sorted buffer (the *memtable*), which flushes periodically to immutable sorted files (the *SSTables* — Chapter 4's log index segments, exactly), and a background process *compacts* levels by merge-sorting them into fewer, bigger files. Reads check the memtable, then the newest SSTables outward, with Bloom filters skipping files that can't contain the key.

Property	B+ tree	LSM tree
Write path	Random page updates + splits	Append + flush: almost purely sequential
Write amplification	Low-moderate	Higher: compaction rewrites data repeatedly

Property	B+ tree	LSM tree
Read amplification	Lowest: ≤4 pages, one structure	Higher: check memtable + several SSTable levels (Bloom filters help)
Range scans	Excellent: linked leaves	Good, but must merge across levels
Space	Pages 70–90% full	Compaction leaves transient duplication
Best fit	Read-heavy OLTP (this chapter's scope)	Write-heavy logs/time-series (Chapter 4's workload)

KEY INSIGHT — B+ vs LSM is a read/write ratio decision

The two structures dominate modern storage engines, and the choice is not fashion: B+ trees minimize read I/O, LSM trees minimize write I/O. State the workload's read/write ratio and choose — that sentence, with the amplification vocabulary to defend it, is the entire "which storage engine" follow-up.

8.12 The Storage Engine's Interface

The structure above serves a deliberately small API — this is what the rest of the database (and Chapter 5's and 6's tables) stand on:

Operation	Semantics
`get(key)`	Point lookup → row or location; ≤4 page reads
`scan(lo, hi)` / cursor	Ordered range iteration; the cursor holds a page + position — Chapter 5's cursor at the storage layer
`insert(key, row)`	Descend, insert in leaf, split as needed, WAL first
`delete(key)`	Remove from leaf; underflow handled lazily
`create index (cols)`	Build a secondary B+ tree by scanning the table once, sorting, and bulk-loading leaves left-to-right (far cheaper than n inserts)

8.13 Rust Reference Implementations

Four pieces with deterministic tests: the B+ tree itself, a sparse-index page lookup, the leftmost-prefix rule, and the LRU buffer pool.

8.13.1 A B+ Tree with Splits

```
const ORDER: usize = 4; // max children per node — tiny so tests exercise splits
const MAX_KEYS: usize = ORDER - 1;

#[derive(Debug, Clone)]
enum Node {
    Leaf { keys: Vec<u64>, vals: Vec<u64> },
    Internal { keys: Vec<u64>, children: Vec<Node> }, // keys[i] = min key of children[i+1]
}

pub struct BPlusTree {
    root: Node,
```

```
}

enum Split {
    None,
    Split { sep: u64, right: Node },
}

impl BPlusTree {
    pub fn new() -> Self {
        BPlusTree { root: Node::Leaf { keys: vec![], vals: vec![] } }
    }

    pub fn get(&self, key: u64) -> Option<u64> {
        let mut node = &self.root;
        loop {
            match node {
                Node::Leaf { keys, vals } => {
                    return keys.binary_search(&key).ok().map(|i| vals[i]);
                }
                Node::Internal { keys, children } => {
                    // first separator > key bounds the descent from the left
                    let i = keys.partition_point(|&k| key >= k);
                    node = &children[i];
                }
            }
        }
    }

    pub fn insert(&mut self, key: u64, val: u64) {
        if let Split::Split { sep, right } = insert_rec(&mut self.root, key, val) {
            let old = std::mem::replace(
                &mut self.root,
                Node::Leaf { keys: vec![], vals: vec![] },
            );
            self.root = Node::Internal { keys: vec![sep], children: vec![old, right] };
        }
    }

    /// Ordered [lo, hi] scan — in production this descends once and walks
    /// linked leaves; here we prune recursively, same result set.
    pub fn range(&self, lo: u64, hi: u64) -> Vec<(u64, u64)> {
        let mut out = Vec::new();
        collect_range(&self.root, lo, hi, &mut out);
        out
    }

    pub fn height(&self) -> usize {
        let (mut h, mut node) = (1, &self.root);
        while let Node::Internal { children, .. } = node {
            node = &children[0];
            h += 1;
        }
        h
    }
}

fn insert_rec(node: &mut Node, key: u64, val: u64) -> Split {
    match node {
        Node::Leaf { keys, vals } => {
            let pos = keys.partition_point(|&k| k < key);
```

```
            if pos < keys.len() && keys[pos] == key {
                vals[pos] = val; // upsert: no duplicate keys
                return Split::None;
            }
            keys.insert(pos, key);
            vals.insert(pos, val);
            if keys.len() <= MAX_KEYS { return Split::None; }
            let mid = keys.len() / 2;
            let rkeys = keys.split_off(mid);
            let rvals = vals.split_off(mid);
            let sep = rkeys[0]; // B+ copy-up: the key stays in the leaf too
            Split::Split { sep, right: Node::Leaf { keys: rkeys, vals: rvals } }
        }
        Node::Internal { keys, children } => {
            let i = keys.partition_point(|&k| key >= k);
            match insert_rec(&mut children[i], key, val) {
                Split::None => Split::None,
                Split::Split { sep, right } => {
                    keys.insert(i, sep);
                    children.insert(i + 1, right);
                    if keys.len() <= MAX_KEYS { return Split::None; }
                    let mid = keys.len() / 2;
                    let up = keys[mid];
                    let rkeys = keys.split_off(mid + 1);
                    keys.pop(); // internal split pushes the middle UP, no copy
                    let rchildren = children.split_off(mid + 1);
                    Split::Split {
                        sep: up,
                        right: Node::Internal { keys: rkeys, children: rchildren },
                    }
                }
            }
        }
    }
}

fn collect_range(node: &Node, lo: u64, hi: u64, out: &mut Vec<(u64, u64)>) {
    match node {
        Node::Leaf { keys, vals } => {
            for (i, &k) in keys.iter().enumerate() {
                if k >= lo && k <= hi {
                    out.push((k, vals[i]));
                }
            }
        }
        Node::Internal { keys, children } => {
            for (i, child) in children.iter().enumerate() {
                let lower = if i == 0 { 0 } else { keys[i - 1] };
                let upper = if i == keys.len() { u64::MAX } else { keys[i] };
                if hi >= lower && lo <= upper {
                    collect_range(child, lo, hi, out);
                }
            }
        }
    }
}

#[cfg(test)]
mod btree_tests {
    use super::*;
```

```
    fn check(node: &Node, depth: usize, leaf_depths: &mut Vec<usize>) {
        match node {
            Node::Leaf { keys, .. } => {
                assert!(keys.windows(2).all(|w| w[0] < w[1]));
                leaf_depths.push(depth);
            }
            Node::Internal { keys, children } => {
                assert_eq!(keys.len() + 1, children.len());
                assert!(keys.windows(2).all(|w| w[0] < w[1]));
                assert!(keys.len() <= MAX_KEYS);
                for c in children {
                    check(c, depth + 1, leaf_depths);
                }
            }
        }
    }

    #[test]
    fn insert_search_range_stay_consistent() {
        let mut t = BPlusTree::new();
        let mut keys: Vec<u64> = (0..100).collect();
        keys.sort_by_key(|&k| (k * 37) % 101); // deterministic shuffle
        for &k in &keys {
            t.insert(k, k * 10);
        }
        for k in 0..100u64 {
            assert_eq!(t.get(k), Some(k * 10));
        }
        assert_eq!(t.get(100), None);
        let r = t.range(25, 42);
        assert_eq!(r.len(), 18);
        assert_eq!(r.first().unwrap().0, 25);
        assert_eq!(r.last().unwrap().0, 42);
        assert!(r.windows(2).all(|w| w[0].0 < w[1].0));
        let mut depths = Vec::new();
        check(&t.root, 1, &mut depths);
        assert!(depths.iter().all(|&d| d == depths[0])); // perfectly balanced
        assert!(t.height() <= 6, "height {}", t.height());
    }

    #[test]
    fn upsert_overwrites_without_duplication() {
        let mut t = BPlusTree::new();
        t.insert(5, 50);
        t.insert(5, 500);
        assert_eq!(t.get(5), Some(500));
        assert_eq!(t.range(0, 1000).len(), 1);
    }
}
```

8.13.2 Sparse-Index Page Lookup

```
/// A sorted run of keys with an in-memory sparse index: one sample per
/// SPAN keys (the LSM/SSTable trick — binary search the samples, then
/// search one tiny block; a full index would cost O(n) memory).
pub struct SparseIndex {
    keys: Vec<u64>,
    sample: Vec<(u64, usize)>, // (first key of block, block start index)
```

```
}

impl SparseIndex {
    const SPAN: usize = 4;

    pub fn new(mut keys: Vec<u64>) -> Self {
        keys.sort();
        let sample = keys
            .iter()
            .enumerate()
            .step_by(Self::SPAN)
            .map(|(i, &k)| (k, i))
            .collect();
        SparseIndex { keys, sample }
    }

    pub fn find(&self, key: u64) -> Option<usize> {
        if self.keys.is_empty() { return None; }
        // last sample whose key <= key
        let idx = self.sample.partition_point(|&(k, _)| k <= key);
        if idx == 0 { return None; } // key precedes the first sample
        let start = self.sample[idx - 1].1;
        let end = (start + Self::SPAN).min(self.keys.len());
        self.keys[start..end]
            .binary_search(&key)
            .ok()
            .map(|off| start + off)
    }
}

#[cfg(test)]
mod sparse_tests {
    use super::*;

    #[test]
    fn finds_via_sample_then_block() {
        let idx = SparseIndex::new((0..40).map(|i| i * 3).collect());
        assert_eq!(idx.find(27), Some(9));   // 27 = 9*3
        assert_eq!(idx.find(0), Some(0));    // first sample boundary
        assert_eq!(idx.find(117), Some(39)); // last block
        assert_eq!(idx.find(118), None);     // past the end
        assert_eq!(idx.find(1), None);       // inside a block, absent
    }
}
```

8.13.3 The Leftmost-Prefix Rule

```
/// How many leading columns of a composite index `(a, b, c)` a query can
/// use for equality lookups: the longest run of index columns that all
/// have equality constraints. After the prefix, at most one range column
/// remains usable — columns past it are unreachable by the index order.
pub fn usable_prefix(index_cols: &[&str], equality_cols: &[&str]) -> usize {
    index_cols
        .iter()
        .take_while(|c| equality_cols.contains(c))
        .count()
}

#[cfg(test)]
```

```
mod prefix_tests {
    use super::*;

    #[test]
    fn lexicographic_order_decides_usability() {
        let idx = ["a", "b", "c"];
        assert_eq!(usable_prefix(&idx, &["a", "b"]), 2); // a, b usable
        assert_eq!(usable_prefix(&idx, &["b"]), 0);      // b alone: useless
        assert_eq!(usable_prefix(&idx, &["a", "c"]), 1); // a only; c skipped over b
        assert_eq!(usable_prefix(&idx, &["a", "b", "c"]), 3);
        assert_eq!(usable_prefix(&idx, &[]), 0);
    }
}
```

8.13.4 The Buffer Pool (LRU)

```
use std::collections::{HashMap, VecDeque};

/// Page cache with LRU eviction. O(n) recency refresh for clarity;
/// production uses an intrusive linked list for O(1) touches (and often
/// LRU-K or clock variants to resist scan pollution).
pub struct BufferPool {
    cap: usize,
    frames: HashMap<u64, Vec<u8>>, // page id -> contents
    recency: VecDeque<u64>,        // front = most recently used
}

impl BufferPool {
    pub fn new(cap: usize) -> Self {
        BufferPool { cap, frames: HashMap::new(), recency: VecDeque::new() }
    }

    pub fn get(&mut self, page: u64) -> Option<&[u8]> {
        if self.frames.contains_key(&page) {
            self.touch(page);
            return self.frames.get(&page).map(Vec::as_slice);
        }
        None
    }

    pub fn put(&mut self, page: u64, data: Vec<u8>) {
        if self.frames.contains_key(&page) {
            self.frames.insert(page, data);
            self.touch(page);
            return;
        }
        if self.frames.len() == self.cap {
            let victim = self.recency.pop_back().expect("pool nonempty");
            self.frames.remove(&victim);
        }
        self.frames.insert(page, data);
        self.touch(page);
    }

    fn touch(&mut self, page: u64) {
        self.recency.retain(|&p| p != page);
        self.recency.push_front(page);
    }
}
```

```
#[cfg(test)]
mod pool_tests {
    use super::*;

    #[test]
    fn lru_evicts_least_recently_used() {
        let mut bp = BufferPool::new(2);
        bp.put(1, b"a".to_vec());
        bp.put(2, b"b".to_vec());
        assert!(bp.get(1).is_some()); // refresh page 1 -> page 2 is now LRU
        bp.put(3, b"c".to_vec());     // evicts page 2
        assert!(bp.get(2).is_none());
        assert_eq!(bp.get(1).unwrap(), b"a");
        assert_eq!(bp.get(3).unwrap(), b"c");
    }

    #[test]
    fn overwrite_refreshes_recency() {
        let mut bp = BufferPool::new(2);
        bp.put(1, b"a".to_vec());
        bp.put(2, b"b".to_vec());
        bp.put(1, b"a2".to_vec()); // refresh via overwrite
        bp.put(3, b"c".to_vec());  // evicts page 2, not page 1
        assert_eq!(bp.get(1).unwrap(), b"a2");
        assert!(bp.get(2).is_none());
    }
}
```

8.14 Scaling the Structure

Axis	How the B+ tree scales	What breaks first
Rows	Height grows as $\log_{1000}$ — billions stay at depth 3–4	Nothing, structurally; table **breadth** (columns × row size) pressures page economics first
Read concurrency	Buffer pool serves cached levels; leaf reads parallelize across I/O	Buffer pool hit ratio — watch it fall as the working set outgrows RAM
Write rate	Splits amortized at fanout 1000; WAL absorbs burstiness	Random-page flush bandwidth; past it, the workload is telling you LSM
Key size	Fanout = page/entry — fat keys flatten it	Oversized keys (long strings) cut fanout and deepen the tree; hash or prefix-compress them
Distribution	The index shards with its table (Chapter 5)	Hot partitions get hot index pages — the tree scales, the shard doesn't

8.15 Failure Modes & Degradation

Failure	Symptom	Response
Crash mid-split	Torn or half-written pages	WAL replay redoes committed work; page checksums detect torn writes; never corrupt **silently**
Buffer pool thrashing	Latency spike; hit ratio collapses	A scan read the whole table through the pool — LRU variants (LRU-K, clock) resist scan pollution; pin critical levels
Index bloat	Pages half-empty after mass deletes; scans slow	Lazy underfill + background rebalancing/rebuild; monitor fill factor
Write burst	Split cascades on a hot range	Sequential keys (auto-increment, timestamps) concentrate inserts on the rightmost leaf — the **rightmost hotspot**; hash-prefix or reverse the key
Corrupted page	Checksum mismatch on read	Fail the query loudly, rebuild from replica/WAL; an index is derived state — it can always be rebuilt from the table
Runaway query	Optimizer ignores the index, full-scans	Section 8.18: selectivity, stale statistics; `ANALYZE`, or force the index — then fix the statistics

8.16 Trade-offs & Alternatives

Decision	Chosen	Alternative & why not (here)
Core structure	B+ tree, page-sized nodes	Hash index: no ranges; sorted file: no cheap writes; balanced BST: 30 random I/Os vs 4 (Section 8.7)
Values in leaves only	B+ variant	Classic B-tree stores values in internal nodes too — fatter interior, lower fanout, and it breaks the leaf-link range trick
Write-heavy alternative	LSM tree when writes dominate	Chosen blindly: read amplification and compaction stalls bite read-heavy workloads
Durability	WAL + lazy page flush	Flush pages synchronously per commit: correct and unusably slow
Underflow handling	Lazy (underfill, rebalance later)	Eager merge on every delete: latch contention and churn for negligible space

Decision	Chosen	Alternative & why not (here)
Sequential keys	Accept rightmost hotspot or hash-prefix	Ignore it: one leaf page serializes the entire insert rate
Index everything	No — indexes are write tax	Each index is paid on every insert; cover the queries you have, not the queries you fear

8.17 Observability & SLOs

Indicator	Definition	Target
Buffer pool hit ratio	Reads served from RAM / all page reads	≥ 99% for hot indexes
Pages per query	Page touches per lookup/scan	≤4 point; ≈ k/page-fill for ranges
Write amplification	Bytes written / bytes inserted	Low single digits for B+ trees
Fill factor	Average page occupancy	70–90%; alert on sustained decline
Slow query count	Queries past the latency SLO, from the slow log	Flat; investigate every step up
Split rate	Page splits per second	Proportional to inserts; spikes signal hotspots

8.18 Interview Wrap-Up

Likely follow-ups and the shape of strong answers:

1. **"Why didn't the database use my index?"** Selectivity: if the predicate matches 30% of the table, a scan is **cheaper** than 300M random bookmark lookups — the optimizer is right. Secondary causes: stale statistics, a function wrapped around the column (`WHERE LOWER(email) = ?` defeats the index on `email`), or a composite index whose leftmost prefix the query doesn't constrain (Section 8.13's rule).
2. **"Design the index for this query"** — `WHERE tenant_id = ? AND created > ? ORDER BY created LIMIT 20`: composite `(tenant_id, created)` — equality on the leftmost column, range + order on the second; the sort is free and the LIMIT reads one page. This one composite index is the single most common production index shape; know it cold.
3. **"B-tree vs hash for a key-value store?"** Hash for pure point gets at extreme write volume (and the LSM if writes dominate); B+ the moment ranges, ordering, or prefix scans appear. Memcached vs the world.
4. **"How do transactions fit in?"** Isolation layers above the structure: latches protect pages physically; locks/MVCC protect transactions logically. The B+ tree doesn't change; version chains or lock tables hang off it. Mentioning the latch/lock distinction is a depth signal.
5. **"Why do UUID primary keys hurt?"** Random keys destroy insert locality — every insert lands on a random leaf, defeating the buffer pool — and bloat every secondary index that references them. Time-ordered UUIDs (ULID/UUIDv7) exist precisely to give keys back their locality.

8.19 Summary & Further Reading

> **NOTE — Chapter summary**
>
> The B+ tree is not a data structure choice; it is **disk physics made flesh**. Random I/O is $10^5\times$ slower than RAM, so the structure minimizes page touches: page-sized nodes give fanout 1000, which collapses a billion rows into height 3–4, of which the top levels live in the buffer pool — one disk read per lookup. Splits keep it balanced under any key order; values-in-leaves and linked leaves make range scans sequential; the WAL buys durability with appends instead of random flushes. The LSM tree is the mirror-image choice when writes dominate. Indexes are a bet paid on every write and collected on every read — composite, covering, and leftmost-prefix design decide whether the bet pays. Chapters 5 and 6 stood on this chapter; this chapter is why they could.

Further reading.

- The source video: *"How do B-Tree Indexes work? — Systems Design Interview: 0 to 1 with Google Software Engineer"* (Jordan has no life): `https://www.youtube.com/watch?v=Z2OaqmxiH20`
- CMU 15-445 (Andy Pavlo's database course, lectures + notes) — buffer pools, B+ trees, and latching in production depth.
- Hellerstein, Stonebraker, Hamilton — *Architecture of a Database System* (2007) — the map of everything around the index.
- O'Neil et al. — *"The Log-Structured Merge-Tree"* (1996) — the rival, in its own words.
- SQLite's file-format documentation — a complete, readable B-tree implementation spec you can finish in an evening.

8.20 Chapter Glossary

Term	Meaning
B-tree	Self-balancing multiway search tree; node = page, fanout high, all leaves level
B+ tree	B-tree with values only in leaves, linked leaves, separator-only interiors; the production variant
Bookmark lookup	The second descent from a secondary index's leaf into the clustered index to fetch the row
Buffer pool	The page cache between the tree and disk; hit ratio is the database's vital sign
Clustered index	An index whose leaves hold full rows — the table itself, ordered by the key
Compaction	LSM background merge of SSTables into fewer levels; reclaims space, restores read shape
Composite index	Index on a tuple of columns; usable by leftmost prefix only
Covering index	An index storing every column a query needs — no bookmark lookup
Fanout	Children per node; page size / entry size; the number that collapses tree height
Fill factor	Average page occupancy; declines with deletes into index bloat
Latch vs. lock	Latch: physical, short-term page protection; lock: logical, transaction-scope protection

Term	Meaning
Leftmost-prefix rule	A composite index serves only queries constraining a leading run of its columns
LSM tree	Log-structured merge tree: memtable + immutable SSTables + compaction; write-optimized
Memtable	The LSM's in-memory sorted write buffer, flushed to SSTables
Order / MAX_KEYS	Max children per node (order m); max keys m−1; overflow triggers splits
Page	The fixed-size I/O and buffer-pool unit (4–16 KB); one node per page
Range scan	Ordered retrieval of [lo, hi]; one descent plus linked-leaf walk
Rightmost hotspot	Sequential keys concentrating all inserts on one leaf page, serializing writes
Secondary index	Index whose leaves map key → primary key, not full rows
Selectivity	Fraction of rows a predicate matches; low selectivity makes the optimizer choose a scan — correctly
Separator key	Interior routing key = minimum key of the right child; a copy in B+ trees
Sparse index	One sample per block over a sorted run; binary search samples, then the block — SSTable lookup
Split	Node overflow → two nodes + separator pushed up; root split grows the tree a level
SSTable	Sorted String Table: immutable sorted file flushed from a memtable; Chapter 4's segments
WAL	Write-ahead log: durable append of intentions before any page flush; crash = replay

— End of Chapter 8 · Next: Chapter 9 —

CHAPTER 9

9

Designing Conflict-Free Replication: How CRDTs Work

NOTE — Problem source

This chapter solves the problem posed in the talk *"CRDTs — Stop Worrying About Write Conflicts"* from the series *Systems Design 0 to 1 with Ex-Google SWE* (channel: *Jordan has no life*). Like Chapter 8, it is a **fundamentals** deep dive rather than a product design: the interviewer asks you to design the data layer of a distributed system in which every replica accepts writes at all times — no leader, no locking, no global ordering — and yet every replica provably converges to the same state. It is also the promised second half of Chapter 1: there we chose Operational Transformation behind a per-document sequencer for the collaborative editor; this chapter walks the road we did not take, and discovers it is paved with some of the most elegant mathematics in distributed systems. All terms are defined before use; all reference code is Rust with deterministic tests.

9.1 The Problem Statement

The interviewer sketches three data centers on three continents and says:

"Every region accepts reads and writes at all times — even when the network link between regions is down for an hour. No region ever waits for another. When the link heals, all regions converge to the same data automatically: no manual conflict resolution, no lost updates, no duplicates. Design the data layer that makes this true."

This prompt outlaws every familiar trick, one by one. A single **leader** that serializes all writes violates "no region waits for another" — and dies as a single point of failure. Distributed **locking** violates it harder: a lock held across a partition is either unavailable or unsafe. "Just take the latest write" loses data silently. The interviewer is not asking for a weaker version of a normal database; they are asking for a **different mathematics of agreement**. That mathematics is the CRDT.

DEFINITION — Replica

An independent copy of some shared state, living on a different machine (here: in a different region), that accepts reads and writes on its own. A system is *replicated* when several replicas of the same logical data exist at once. Replication buys fault tolerance and read locality — and immediately raises the question this chapter answers: how do the copies stay in agreement?

DEFINITION — Network partition

A failure in which two groups of replicas cannot communicate for some period, while each group keeps running. Partitions are not exotic: any severed fiber, misconfigured router, or overloaded link between

regions is one. A design that is correct only when the network is perfect is correct zero percent of the time at planet scale.

DEFINITION — Coordination

Any protocol in which a replica must communicate with others **before** it may complete a write: leader election, two-phase commit, distributed locks, consensus rounds. Coordination is the price of imposing a global order — and under a partition, the side that cannot reach the others must stop accepting writes. Coordination and availability are the two ends of a seesaw.

DEFINITION — Write conflict

The situation in which two replicas accept writes to the same logical datum while unable to see each other — *concurrent* writes. Neither write is "wrong"; the system simply owes the user a single, deterministic merged outcome. A conflict is not an error to be prevented but a fact of physics to be absorbed.

DEFINITION — Convergence

The property that replicas which have received the same set of updates are in the same observable state, no matter the order, duplication, or timing with which those updates arrived. Convergence is the entire contract of this chapter: not "never diverge" (impossible without coordination) but "divergence is temporary and self-healing".

9.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Data model?	A key-space of replicated values: counters, registers, sets, maps, and one sequence (text). Small, independently mutable values — not bulk blobs
Topology?	Three regions, active-active (every region serves real writes). The design must not preclude offline-capable edge clients later
Consistency?	Strong eventual consistency within the store (defined below). No cross-key invariants required — that door stays shut on purpose
Partition behavior?	Writes must never block on a partition: full AP. Reconciliation happens after healing, automatically
Network assumptions?	Messages may be delayed, reordered, duplicated. At-least-once delivery at best
Durability?	A write must survive the loss of any single region once it has been gossiped; locally it is durable on ack
The hard part?	*"Stop worrying about write conflicts"* — the merge must be automatic, deterministic, and provably correct

NOTE — Agreed scope

1. Design an **active-active replicated data layer**: every replica accepts reads and writes with zero cross-replica coordination on the write path.

2. Guarantee **strong eventual consistency**: replicas that have seen the same updates are provably in the same state.
3. Cover **both CRDT families** — state-based and operation-based — and the canonical catalog: counters, registers, sets, and the sequence type that would power Chapter 1's editor.
4. Design the **synchronization layer** (gossip / anti-entropy, delta propagation) and the **metadata lifecycle** (tags, tombstones, garbage collection) that make the theory shippable.
5. Quantify everything: metadata per element, tombstone growth, sync bandwidth, and the bound on convergence time.
6. Out of scope: Byzantine (malicious) replicas, and cross-key global invariants — Section 9.17 explains why CRDTs cannot enforce those and what to reach for instead.

9.3 Functional Requirements

1. **Independent writes.** Every replica accepts reads and writes at all times, with no leader, lock, lease, or quorum on the write path.
2. **Automatic merge.** Concurrent writes to the same datum are merged by the data structure itself — deterministically, identically on every replica, with no application-level conflict handler required.
3. **Convergence.** Any two replicas that have exchanged their updates are in equivalent observable states, regardless of network order, duplication, or delay.
4. **Partition healing.** When a partition heals, the two sides reconcile by exchanging state (or deltas) — never by rolling back acknowledged writes.
5. **A usable type catalog.** The layer must provide the types real applications need: a counter that can go up **and** down, a value register, a set with add **and** remove, maps, and an ordered sequence for text.
6. **Bounded metadata.** Deletes must be real (elements disappear), and the metadata that makes convergence possible must be garbage-collectible without ever blocking writes.

9.4 Non-Functional Requirements

- **Availability first (AP).** Every request — read or write — receives a response from the local replica even during a partition. In CAP terms we choose availability and partition tolerance, and recover consistency continuously rather than atomically.
- **Bounded convergence.** After the network heals, replicas converge within a small multiple of the gossip interval — seconds, not minutes.
- **Bounded overhead.** CRDT metadata per element must stay a small constant multiple of the payload after compaction; unbounded growth is a defect.
- **Protocol robustness.** The synchronization layer must tolerate duplicated, reordered, and delayed messages as a matter of routine.

DEFINITION — Eventual consistency

The guarantee that, if updates stop arriving, all replicas **eventually** reach the same state. By itself this is weak — it says nothing about what happens **while** updates flow, and it allows replicas to disagree arbitrarily in the meantime. It is a liveness property ("the system gets there"), not a safety property ("the states are right").

DEFINITION — Strong eventual consistency (SEC)

Eventual consistency plus a **safety** guarantee: any two replicas that have applied the same set of updates are in **equivalent states right now**, with no further communication needed. SEC is what turns

"eventually the same" from a hope into a theorem, and it is exactly what CRDTs provide: convergence is a property of the data structure, not of the schedule.

DEFINITION — Linearizability

The strongest common consistency model: every operation appears to take effect atomically at some instant between its start and its completion, as if there were exactly one copy of the data. Linearizability requires coordination, so it is exactly what an AP system gives up. This chapter's honesty rule: CRDTs provide SEC per object, **never** linearizability — and Section 9.17 shows which application needs genuinely require the stronger model.

9.5 Back-of-the-Envelope: Metadata, Tombstones, and Gossip

The design stands or falls on three numbers: how much metadata convergence costs per element, how fast tombstones accumulate, and how much bandwidth synchronization burns. Fix a concrete scenario — the canonical CRDT showcase: a **shopping-cart service** in three regions.

Assumptions. Three regions in full mesh; 10^8 active carts; an average of 20 items per cart; item payload 24 B (SKU + quantity); one add-tag of 16 B (replica id 8 B + counter 8 B) per add; churn such that each live item has, on average, two historically removed siblings.

Quantity	Estimate (with assumptions)
Regions / replicas	3, pairwise full mesh (one gossip hop between any two)
Live cart state	10^8 carts × 20 items × (24 B + 16 B) = **80 GB per replica**
Tombstones	10^8 × 40 removed items × 16 B = **64 GB per replica** — nearly half the state is ghosts; GC is not optional
Peak cart writes	10^5 ops/s fleet-wide (add, remove, change-quantity)
Delta sync traffic	10^5 ops/s × 100 B/op (payload + tag + context) ≈ **10 MB/s fleet-wide**, 3.3 MB/s per region — trivial for inter-region links
Full-state sync	144 GB per exchange — **impossible as a routine mechanism**; this number is why delta-state CRDTs exist (Section 9.10)
Convergence bound	1 s gossip interval × 2 rounds ≈ **2 s worst case** on a full mesh; a ring of N replicas would cost ⌈N/2⌉ rounds — topology is a convergence budget
Text metadata	40 B per character (sequence id + parent reference + flags) before compression — Section 9.11's tax on Chapter 1

KEY INSIGHT — The number that shapes the architecture

Nothing in the table is frightening except the last three rows taken together: state is cheap, writes are cheap, but **moving whole states is not**. A naive "send your replica to your peer" anti-entropy design would need to stream 144 GB per exchange; the same convergence achieved by shipping only what changed costs single-digit MB/s. Every production CRDT system — Riak, Redis CR, Akka Distributed Data — converged on delta-state replication for exactly this arithmetic reason.

9.6 The Core Challenge: Agreement Without a Coordinator

Every conflict-resolution strategy you already know is secretly a way of **imposing an order** on writes and then obeying it. A leader imposes order by fiat. A lock imposes order by exclusion. A consensus protocol imposes order by majority vote. Last-writer-wins imposes order by wall clock. The first three require coordination, which a partition forbids; the fourth is coordination-free but **wrong often enough to matter**.

COMMON PITFALL — Last-writer-wins as the default

LWW resolves a conflict by keeping the write with the largest timestamp and discarding the other. Two defects make it a footgun, not a strategy. First, it **throws data away by design**: two users updating the same profile field concurrently means one of them silently never happened. Second, the "winner" is chosen by clocks you do not control: with 100 ms of clock skew between regions — unremarkable in practice — the write that actually happened **later** can lose. LWW is a fine register policy for overwrite-able caches (Section 9.9); it is a terrible **default** for data you care about, and "the database will merge it" is not a design.

So the challenge is precise: **find data structures whose merge does not need an order at all.** The breakthrough idea is to stop asking "which write came first?" — a question that has no answer across a partition — and ask instead "what is the **join** of these two states?": a combination that absorbs both writes symmetrically, the way set union absorbs both contributors without caring who contributed first.

KEY INSIGHT — Order is the enemy; merge is the friend

A total order over all writes is unaffordable (coordination) and, under a partition, **undefinable** — concurrent writes genuinely have no fact-of-the-matter order. The escape is to demand only three properties of the merge function: **commutativity** (order of arrival irrelevant), **associativity** (grouping of arrivals irrelevant), and **idempotence** (duplication irrelevant). Any structure whose merge has these three properties converges under **any** network behavior — delays, reordering, duplication, pairwise gossip in any pattern — because every remaining degree of freedom in the delivery schedule has been declared irrelevant. The rest of the chapter is the craft of building useful data types whose merges obey these three laws.

9.7 Deep Dive: The Mathematics of Convergence

This section gives the three laws above a body: the small amount of order theory that turns "we merge somehow" into "convergence is a theorem".

DEFINITION — Partial order

A relation $\leq$ on a set that is reflexive ($x \leq x$), antisymmetric ($x \leq y$ and $y \leq x$ imply $x = y$), and transitive ($x \leq y$ and $y \leq z$ imply $x \leq z$) — but **not** total: some pairs are simply incomparable, written $x \parallel y$. Replica states form a partial order by "knowledge": state B is $\geq$ state A if B has seen every update A has seen. Two replicas that have seen **different** concurrent updates are incomparable — and that incomparability is the faithful mathematical image of a partition.

DEFINITION — Join (least upper bound) and join-semilattice

The *join* of two states x and y, written $x \vee y$, is the **smallest** state that is $\geq$ both: the cheapest way to know everything either of them knows. A set in which every pair has a join is a *join-semilattice*. Joins are automatically commutative ($x \vee y = y \vee x$), associative ($(x \vee y) \vee z = x \vee (y \vee z)$), and idempotent ($x$

∨ x = x) — the three laws of the insight box, now with names and a proof obligation: to build a state-based CRDT, define a state space, define ∨, and **prove** it is the least upper bound.

DEFINITION — Monotonicity / inflation

An update is *inflationary* (monotonic) if it only ever moves the state **up** the partial order: after any update, new-state ≥ old-state. No rollback, no overwrite, no removal from the **state's knowledge** — a remove operation, as we will see, is re-expressed as the **addition of a tombstone**, which is growth. Monotonicity is what makes convergence safe: since replicas only ever climb the lattice, every merge is progress and no progress is ever undone.

With the vocabulary in place, the central theorem of the field fits in one sentence and one paragraph of proof sketch:

Convergence theorem (state-based CRDTs). *If the state space is a join-semilattice, every update is inflationary, and merge computes the join, then replicas that have exchanged the same set of updates are in equal states — regardless of order, duplication, or gossip topology.*

Why it holds. Each update moves its origin replica to a state ≥ the old one (inflation). Gossip and merge replace a state by a join of states, and joins only ever add knowledge, so every replica's state climbs the lattice and never descends. When two replicas have both absorbed the same set of updates, each one's state is the join of exactly that set — and the join of a set does not depend on the order (commutativity), grouping (associativity), or repetition (idempotence) of its members. Hence the two states are equal. ∎

NOTE — The two CRDT families

The theorem above describes **state-based** CRDTs (*CvRDTs*, "convergent"): replicas ship their states (or state deltas) and merge by join. There is a dual family, **operation-based** CRDTs (*CmRDTs*, "commutative"): replicas broadcast the **operations themselves** (e.g. "add tag t to element e"), and the requirement is that concurrent operations **commute** — applying them in either order yields the same state — and that the broadcast layer delivers each operation **exactly once, in causal order**. Op-based CRDTs send small messages but demand more of the transport; state-based CRDTs tolerate any transport but send larger messages. The two families are expressively equivalent — every design in one has a counterpart in the other — so the choice is pure engineering: message size versus delivery guarantees. Delta-state (Section 9.10) erases most of the size advantage, which is why state-based dominates production systems.

9.8 Deep Dive: The CRDT Catalog

Every CRDT in the catalog is an answer to the same exam question: "define a state and a join for this familiar data type." Seeing five answers in a row is the fastest way to internalize the method.

9.8.1 G-Counter: the hello-world of convergence

A grow-only counter stores not one integer but a **vector** — one slot per replica. Replica *i* increments only slot *i*. The value is the sum of all slots. Merge takes the **component-wise maximum**: for each slot, keep the larger of the two values.

- *Commutative?* max(a, b) = max(b, a). ✓
- *Associative?* max(max(a, b), c) = max(a, max(b, c)). ✓
- *Idempotent?* max(a, a) = a. ✓ — duplicates are absorbed for free.
- *Monotonic?* Slots only grow; max only climbs. ✓

Each slot climbs because a replica's own count only grows, and merge propagates the climb. A replica can never "un-see" an increment. That is the whole design pattern: **partition the state so that each writer owns a piece only it can change, then take per-piece maxima.**

9.8.2 PN-Counter: decrements without going backwards

A counter that can decrement cannot be one G-Counter (a decrement would move a slot **down** — non-monotonic, theorem void). The trick: **two** G-Counters, P for increments and N for decrements; value = sum(P) − sum(N). Decrement increments N. Every physical slot still only grows; the **derived** value may fall. This "two monotonic structures, one derived reading" move recurs constantly in CRDT design.

9.8.3 G-Set and 2P-Set: union, then union twice

A grow-only set merges by union — the most obvious semilattice in the chapter. To support removal, add a second grow-only set of **tombstones**; an element is a member iff it is in the add-set and not in the remove-set. That is the **2P-Set** (two-phase set), and its glaring limitation teaches the next idea: once removed, an element can **never** be re-added, because its tombstone outlives every future add. The remove is blind — it kills adds it never even saw.

> **DEFINITION — Tombstone**
>
> A marker recording that a specific element (or add) was removed, kept in the state so that the removal can be **merged** like any other fact. A tombstone turns deletion — locally a shrinkage — into knowledge growth, preserving monotonicity. Its price is space: tombstones accumulate until garbage collection (Section 9.10), and deleting one naively is the classic way to resurrect deleted data.

9.8.4 OR-Set: letting adds and removes coexist

The **observed-remove set** fixes the 2P-Set's blindness with one word: *observed*. Every `add(e)` stamps `e` with a **globally unique tag** (a replica id plus a counter). `remove(e)` tombstones exactly the tags the removing replica **has seen**. An element is live iff it has a tag with no tombstone. Now watch a concurrent add and remove collide: the remove tombstones the tags it observed; the concurrent add carries a **new** tag the remover could not have seen, so that tag survives and the element lives. This is **add-wins** semantics — the re-add beats the concurrent remove — and it is almost always what users mean: the shopper who re-adds milk while another device removes the stale entry expects milk.

Merge is union of add-tags and union of tombstones — both grow-only, both semilattices, so the composite is a semilattice. The OR-Set is the workhorse of the catalog: shopping carts, friend lists, and collaborative outlines are all OR-Sets in production clothing.

9.8.5 Registers: LWW and multi-value

A **register** holds one value. The LWW register pairs the value with a timestamp and merges by keeping the larger timestamp — the cautionary tale of Section 9.6 in data-type form. The **multi-value register** (MV-register) is the honest version: merge keeps **all** concurrently written values (returning them as a set of "siblings") and collapses to one only when one write causally supersedes the others. LWW chooses for you and sometimes chooses wrong; MV refuses to choose and hands the choice to the application — with a causal context (Section 9.9) so the app's "last word" can be recorded as superseding rather than concurrent.

Type	State	Merge rule	Cost / gotcha
G-Counter	One slot per replica	Per-slot max	Cannot decrement; O(R) space, R = replicas
PN-Counter	Two G-Counters (P, N)	Per-slot max in both	Value is eventual; no invariant like ≥ 0

Type	State	Merge rule	Cost / gotcha
G-Set	One set	Union	No removal — ever
2P-Set	Add-set + remove-set	Union both	No re-add after remove
OR-Set	Set of (element, tag) + tag tombstones	Union both	Tag + tombstone metadata per element
LWW-Register	(value, timestamp)	Keep max timestamp	Drops concurrent writes; clock skew picks the loser
MV-Register	Set of concurrent (value, dot)	Keep causally-maximal values	App must resolve siblings
Sequence (RGA)	Atoms with unique ids + parent links + tombstones	Union by id, order by (parent, id)	Metadata per character; interleaving (§9.11)

INTERVIEW TIP — The catalog is a party trick — use it

Interviewers love watching a candidate **derive** a CRDT on the spot. The method is always the same four steps: (1) partition the state so each replica owns a piece; (2) express every mutation as growth in some component; (3) define merge as a per-component union or max; (4) prove commutativity, associativity, idempotence in one line each. Asked for a replicated "max temperature" reading? Slots per sensor, merge by max — ten seconds. Asked for a leaderboard score per player? That is a per-player max-register — and Chapter 6's best-score monotonicity was a CRDT in spirit before this chapter named it.

9.9 Deep Dive: Causality — Vector Clocks and Dots

"Observed", "concurrent", "supersedes" — the catalog leans on causal vocabulary, so we now give it machinery. The goal: a compact summary of **which updates a replica has seen**, comparable in O(number of replicas).

DEFINITION — Happens-before (→)

Lamport's relation over events in a distributed system: event $a \to b$ if a can **influence** b — same-replica program order, or a is the sending of a message and b its receipt, extended transitively. If neither $a \to b$ nor $b \to a$, the events are **concurrent** ($a \parallel b$): no message path connects them, so there is no fact of the matter about which "really" happened first. Concurrency is not a tie to break; it is a structure to record.

DEFINITION — Vector clock

A map from replica id to a logical counter, one entry per replica that has ever written. A replica increments its own entry on each local event; on receiving a message it merges by component-wise maximum and then ticks. Clock A **dominates** clock B ($A \geq B$) iff every component of A is $\geq$ the matching component of B; $A \to B$ causally iff $A \leq B$ and $A \neq B$; two clocks with neither $A \leq B$ nor $B \leq A$ are **concurrent**. The vector clock is the happens-before relation, compressed into one comparable value.

DEFINITION — Dot (event id) and dotted version vectors

A *dot* is a single (replica, counter) pair identifying one specific event — exactly the OR-Set's add-tag. Production systems (Riak KV being the canonical example) carry state as **dots plus a version**

vector: the vector summarizes the causal history compactly, while the dots identify the freshest events precisely. The combination answers both questions an OR-Set merge asks — "what have you seen in general?" and "which exact adds do you mean?" — without storing a full event log.

Worked example. Replicas EU and US both start with vector clock { }. EU increments a shared counter twice: EU's clock is {EU: 2}. It gossips to US; US merges — US is also {EU: 2} — and performs its own increment: {EU: 2, US: 1}. Meanwhile EU, having seen nothing from US, increments again: {EU: 3}. Compare the two: EU has 3 > 2 in its own slot, US has 1 > 0 in its slot — neither dominates: **concurrent**. After one gossip in each direction, both hold {EU: 3, US: 1}, the component-wise max — equal, and causally after both divergent states. The merge rule for clocks is the G-Counter's rule, because a vector clock **is** a G-Counter whose value we read component-wise instead of summed. The catalog nests inside itself.

9.10 Deep Dive: Tombstones, Deltas, and Anti-Entropy

Three engineering mechanisms carry the mathematics to production. Each neutralizes one row of the estimation table.

9.10.1 Why tombstones cannot just be deleted

Suppose region EU removes "milk" from a cart, tombstoning its tag, and — eager to reclaim space — **deletes the tombstone** immediately. Region APAC was partitioned the whole time; it still holds the live add-tag. When the link heals, APAC gossips its state. EU's merge unions in the add-tag, and there is no tombstone to cover it: **milk rises from the dead**, on every replica, forever. The tombstone was not garbage; it was the **only** record that the remove ever happened.

The safe rule: a tombstone may be collected only when **every** replica has acknowledged a state that includes it — established with a per-replica ack vector exchanged during gossip. Collection is coordination, but it is **off the write path**: asynchronous, batched, retried lazily, and free to be slow, so it costs the system none of its AP guarantees. Note the pattern, because it recurs in LSM compaction (Chapter 8) and log retention (Chapter 4): **deletion is only ever garbage collection of facts everyone already knows.**

9.10.2 Delta-state: shipping the difference

Full-state gossip of 144 GB per exchange is a non-starter; op-based broadcast is small but demands exactly-once causal delivery. **Delta-state CRDTs** take the middle path: after each update, the replica forms a **delta** — the smallest join-semilattice element that, when merged into the old state, yields the new one (for an OR-Set add: just the new tag; for a G-Counter: the one bumped slot) — ships deltas to peers, and merges received deltas by the same join as full states. Deltas are themselves states, so merging them is commutative, associative, and idempotent **automatically**; a delta-buffer per peer, re-sent until acknowledged, gives at-least-once delivery with exactly-once effect via idempotence. Deltas can also be **grouped** — merged into bigger deltas — when a peer falls behind, degrading gracefully toward a full-state transfer. This is the mechanism that turns the estimation table's 10 MB/s from theory into routine.

9.10.3 Anti-entropy and gossip: the sync fabric

DEFINITION — Anti-entropy / gossip protocol

A background process in which each replica periodically picks a peer (randomly, or by topology) and exchanges state — pushes its deltas, pulls the peer's, or both (push-pull) — until the pair agrees. Repetition makes the epidemic complete: with random pairing every **interval**, an update infects the full mesh of R replicas in O(log R) rounds, with no coordinator, no membership ceremony, and graceful

degradation to any topology that stays connected. “Gossip” names the peer selection; “anti-entropy” names the goal: driving the divergence between replicas toward zero.

For **repair** of long-diverged replicas (a region down for a day), pairwise delta exchange is joined by **Merkle-tree** comparison — a hash tree over the key space lets two replicas find the keys that differ in O(log n) hashes instead of shipping everything — the same trick Chapter 4’s segment compaction uses to prove two segments identical. Production CRDT stores layer all three: deltas for the hot path, gossip for dissemination, Merkle repair for the cold path.

9.11 Deep Dive: Sequence CRDTs — Back to Chapter 1

Chapter 1’s collaborative editor faced the same enemy in miniature: two users typing into the “same” position at once. There we solved it with Operational Transformation behind a per-document sequencer — a single point of ordering, chosen deliberately, with CRDTs named as the road not taken. This section walks that road: how do you make **text** — where order of characters is the entire content — converge without any sequencer?

The obstacle is addressing. Integer positions are **unstable under concurrency**: “insert X at position 5” means different things on two replicas whose texts have already drifted — that instability is precisely what OT’s transform matrix compensates for. The CRDT answer is to stop using positions as identities:

DEFINITION — Stable identity addressing (RGA-style)

Give every inserted atom (character) a **globally unique, immutable id** — a (counter, replica) pair — and record each insertion as “insert atom c **after** atom p”, where p is the id of its left neighbor **at the inserting replica**. Position is now expressed relative to a landmark that never moves (atoms are never renumbered), so a concurrent insert elsewhere cannot shift the reference. The document is a linked structure; the visible text is a deterministic walk of it.

Two concurrent inserts after the **same** parent still need a deterministic order among themselves — “Alice typed `!` after `hi`” and “Bob typed `?` after `hi`” must interleave identically everywhere. The rule: among atoms sharing a parent, sort by **descending id**; a newly arriving atom slides in after every same-parent atom whose id is higher, before all whose ids are lower. Both replicas then place Alice’s (3, EU) and Bob’s (1, US) in the same order regardless of which insert arrives first — the Rust listing in Section 9.14 demonstrates exactly this collision, and its test asserts the converged string.

Deletion is the by-now-familiar move: a tombstone on the atom. The atom stays in the structure — concurrent inserts referencing it as a parent must still find their landmark — but the text walk skips it.

DEFINITION — Interleaving anomaly

A known cosmetic defect of first-generation sequence CRDTs: if Alice types “Hello” while Bob types “World” at the same spot, some schemes may merge into “HWeolrllod” — character-level interleaving of the two runs — because the sibling ordering rule compares ids atom by atom. Newer schemes (LSEQ/Logoot’s dense identifiers with tie-breaking bit strategies, and Fugue’s maximal-non-interleaving rule) keep concurrent runs contiguous. Every scheme still converges; the anomaly is about **readability** of merged concurrent runs, not correctness — and in practice two humans rarely type sustained runs into the same word at once.

DEFINITION — Dense / fractional identifiers (LSEQ, Logoot)

The alternative addressing scheme: instead of linking to a parent, assign each atom an identifier from a **dense order** — one that always has another value between any two (like fractions: between 1/2 and

1/3 there is 5/12). Inserting between atoms p and q allocates an identifier strictly between theirs, so the document order is simply identifier order and no parent link is needed. The cost is identifier growth: every split lengthens the identifier, so documents edited mostly at the end accumulate long ids — the mirror image of RGA's tombstone problem, and the reason production libraries (Yjs, Automerge) run length-aware allocation strategies and periodic compaction.

NOTE — OT vs CRDT — the callback settled

Chapter 1's comparison table now has its missing column filled in. OT keeps text lean (no per-character metadata) and history clean, at the price of a sequencer per document and a transform matrix that must be exactly right. Sequence CRDTs need no sequencer — offline editing, peer-to-peer sync, and end-to-end encryption all fall out for free, because no replica is special and no ciphertext needs transforming — at the price of 40 B of metadata per character before compression and tombstone GC forever after. Figma and the Google Docs lineage chose sequencer + transforms; Automerge, Yjs, and most local-first software chose CRDTs. Neither is "the answer"; both are the trade-off, made with open eyes.

9.12 API Design

The store presents a small key-value surface, one family per CRDT type. Two conventions carry the causal machinery: every read returns an opaque **context** (the version vector the replica used to answer), and every conditional write passes the context back — that is how "I read it, now I'm overwriting what I read" is distinguished from "I'm writing blind", which is the difference between superseding and merely concurrent.

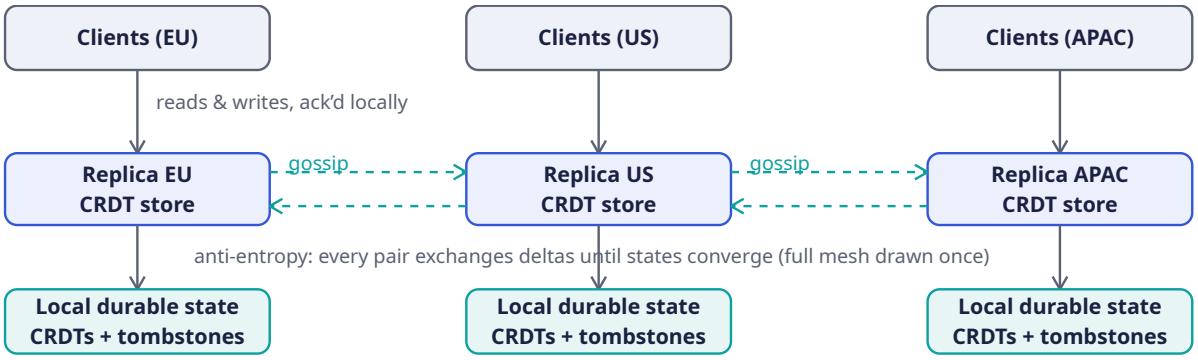
Endpoint	Type	Semantics
`POST /counter/{k}/incr {n}`	PN-Counter	Increment by n (negative n decrements); local, immediate ack; merges by per-slot max
`GET /counter/{k}`	PN-Counter	Returns sum(P) − sum(N) as of this replica's knowledge
`PUT /register/{k} {v, ctx}`	MV / LWW	Write v; with ctx, supersedes exactly the values ctx saw; without ctx, writes blind
`GET /register/{k}`	MV / LWW	Returns value + ctx; MV returns **siblings** (all concurrent values) for the app to resolve
`POST /set/{k}/add {e}`	OR-Set	Add element with a fresh unique tag (replica id + counter)
`POST /set/{k}/remove {e}`	OR-Set	Tombstones every tag for e this replica has observed — add-wins against concurrent adds
`GET /set/{k}`	OR-Set	Returns elements with ≥ 1 live tag
`POST /doc/{k}/edit`	Sequence (RGA)	Batch of (after-id, char) inserts and tombstones; positions are ids, never integers
`GET /doc/{k}?since={ctx}`	Sequence	Returns the delta since ctx — the same mechanism gossip uses, exposed to clients
`POST /sync/{peer}`	internal	Anti-entropy: push-pull delta exchange with one peer; the write path never calls this

KEY INSIGHT — The context parameter is the whole safety story

Every dangerous write in an AP system is a **blind** write — one made without knowing what it overwrites. The opaque context closes that hole without coordination: read-then-write with ctx means "replace exactly what I saw", which the MV-register can honor causally, keeping siblings only for writes that were **genuinely** concurrent. Riak KV built its entire client protocol on this pattern. If the interviewer asks "how do you stop CRDTs from losing user intent?", the answer is this parameter.

9.13 High-Level Architecture

The architecture is almost offensively simple, and that is the point: there is **no box whose failure stops writes**. Each region is a complete, self-sufficient stack; the only cross-region component is the gossip fabric, which is allowed to be late.



NOTE — Reading the diagram

The write path is the **short** path: client → local replica → local disk, acknowledged. Nothing crosses an ocean synchronously; the gossip arrows carry deltas **after** the ack, every 1 s, peer to peer, in both directions. Lose a region and the other two keep serving; lose two links and the isolated region keeps serving; heal the link and anti-entropy merges the diverged states with zero operator action. The architecture diagram of a CRDT system is a list of things that are **absent**: no leader, no lock service, no consensus ring, no conflict-resolution queue.

9.14 Rust Reference Implementations

Four pieces with deterministic tests: the two counters, the vector clock, the OR-Set, and a sequence CRDT for text. Each implements exactly the merge rule its section derived, and each test demonstrates convergence under concurrency, duplication, or both. In a real deployment these states ride inside the sync fabric of Section 9.13; here they run in memory so every property is directly assertable.

9.14.1 Counters: G-Counter and PN-Counter

```rust
use std::collections::BTreeMap;

/// Grow-only counter (state-based). Each replica owns one slot and only
/// ever increments its own slot; merge takes the per-slot maximum.
/// max is commutative, associative, idempotent — the semilattice join.
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct GCounter {
    /// replica id -> count observed at that replica
    counts: BTreeMap<u64, u64>,
}
```

```rust
impl GCounter {
    pub fn increment(&mut self, replica: u64) {
        *self.counts.entry(replica).or_insert(0) += 1;
    }

    /// The counter's value: the sum over all per-replica slots.
    pub fn value(&self) -> u64 {
        self.counts.values().sum()
    }

    /// Join: component-wise maximum. A slot can only climb.
    pub fn merge(&mut self, other: &GCounter) {
        for (&replica, &count) in &other.counts {
            let slot = self.counts.entry(replica).or_insert(0);
            *slot = (*slot).max(count);
        }
    }
}

/// PN-Counter = two G-Counters: P for increments, N for decrements.
/// Every physical slot still only grows; the *derived* value may fall.
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct PNCounter {
    inc: GCounter,
    dec: GCounter,
}

impl PNCounter {
    pub fn increment(&mut self, replica: u64) { self.inc.increment(replica); }
    pub fn decrement(&mut self, replica: u64) { self.dec.increment(replica); }
    pub fn value(&self) -> i64 { self.inc.value() as i64 - self.dec.value() as i64 }
    pub fn merge(&mut self, other: &PNCounter) {
        self.inc.merge(&other.inc);
        self.dec.merge(&other.dec);
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn concurrent_increments_converge() {
        // Two replicas start identical, drift apart, then sync both ways.
        let mut a = GCounter::default();
        let mut b = a.clone();

        a.increment(1);
        a.increment(1);
        b.increment(2);

        let mut a2 = a.clone();
        a2.merge(&b);
        let mut b2 = b.clone();
        b2.merge(&a);
        assert_eq!(a2, b2);            // same updates seen => same state
        assert_eq!(a2.value(), 3);     // no increment lost, none counted twice

        // Idempotence: re-merging an old state changes nothing.
        let snapshot = a2.clone();
```

```
        a2.merge(&b);
        assert_eq!(a2, snapshot);
    }

    #[test]
    fn pn_counter_supports_decrements() {
        let mut a = PNCounter::default();
        let mut b = a.clone();
        a.increment(1);
        a.increment(1);
        a.decrement(1);
        b.decrement(2);
        b.decrement(2);

        a.merge(&b);
        b.merge(&a);
        assert_eq!(a, b);
        assert_eq!(a.value(), 2 - 3);  // two increments minus three decrements
    }
}
```

Note what the tests **cannot** express: a PN-Counter never enforces `value() >= 0`. Two replicas may both decrement the last unit of stock and converge happily to −1. Per-object convergence is free; **invariants** are not — Section 9.17 prices that boundary honestly.

9.14.2 Vector Clocks: Tracking Causality

```
use std::collections::BTreeMap;

/// Vector clock: replica id -> logical counter. The happens-before
/// relation, compressed into one comparable value. (Mechanically this is
/// a G-Counter read component-wise instead of summed.)
#[derive(Debug, Clone, Default, PartialEq, Eq)]
pub struct VectorClock {
    clocks: BTreeMap<u64, u64>,
}

/// The three-way comparison of two causal positions.
#[derive(Debug, PartialEq, Eq)]
pub enum CausalOrder { Equal, Before, After, Concurrent }

impl VectorClock {
    /// Record a local event at `replica`.
    pub fn tick(&mut self, replica: u64) {
        *self.clocks.entry(replica).or_insert(0) += 1;
    }

    pub fn get(&self, replica: u64) -> u64 {
        *self.clocks.get(&replica).unwrap_or(&0)
    }

    /// Component-wise maximum — the join of both causal histories.
    pub fn merge(&mut self, other: &VectorClock) {
        for (&replica, &count) in &other.clocks {
            let slot = self.clocks.entry(replica).or_insert(0);
            *slot = (*slot).max(count);
        }
    }
```

```
    pub fn compare(&self, other: &VectorClock) -> CausalOrder {
        let mut le = true; // self <= other on every component?
        let mut ge = true; // self >= other on every component?
        // union of both key sets (a replica may appear on only one side)
        for (&r, _) in self.clocks.iter().chain(other.clocks.iter()) {
            let a = self.get(r);
            let b = other.get(r);
            if a < b { ge = false; }
            if a > b { le = false; }
        }
        match (le, ge) {
            (true, true)   => CausalOrder::Equal,
            (true, false)  => CausalOrder::Before, // self happened-before other
            (false, true)  => CausalOrder::After,  // self happened-after other
            (false, false) => CausalOrder::Concurrent, // neither saw the other
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn detects_happens_before_and_concurrency() {
        let mut a = VectorClock::default();
        a.tick(1);
        a.tick(1);              // a  = {1: 2}

        let mut b = a.clone();
        b.tick(2);              // b  = {1: 2, 2: 1} — causally after a

        assert_eq!(a.compare(&b), CausalOrder::Before);
        assert_eq!(b.compare(&a), CausalOrder::After);

        // A concurrent branch: a2 moves on without having seen b.
        let mut a2 = a.clone();
        a2.tick(1);             // a2 = {1: 3}
        assert_eq!(a2.compare(&b), CausalOrder::Concurrent);

        // The join dominates both; a clock equals itself.
        let mut m = a2.clone();
        m.merge(&b);            // m  = {1: 3, 2: 1}
        assert_eq!(m.compare(&a2), CausalOrder::After);
        assert_eq!(m.compare(&b), CausalOrder::After);
        assert_eq!(m.compare(&m.clone()), CausalOrder::Equal);
    }
}
```

`compare` is why Rust's `PartialOrd` trait is **not** implemented here: the fourth outcome, `Concurrent`, is the entire point of the data structure, and squeezing it into `None` would invite callers to ignore exactly the case that matters. Make concurrency an explicit variant and the compiler forces every caller to decide what it means.

9.14.3 The OR-Set: Add-Wins with Tombstones

```
use std::collections::{BTreeMap, BTreeSet};

/// Unique stamp for one add operation: (replica id, per-replica counter).
```

```
type Tag = (u64, u64);

/// Observed-Remove Set, add-wins flavour (state-based).
///
/// Per element we keep the tags of every add ever seen and the tags of
/// every remove ever seen (tombstones). An element is live iff it has an
/// add-tag with no matching tombstone. `remove` tombstones exactly the
/// tags *this replica has observed* — so a concurrent add elsewhere,
/// carrying an unseen tag, survives: add wins.
#[derive(Debug, Clone, Default)]
pub struct ORSet {
    adds: BTreeMap<String, BTreeSet<Tag>>,
    removals: BTreeMap<String, BTreeSet<Tag>>,
    replica: u64,
    next: u64,
}

impl ORSet {
    pub fn new(replica: u64) -> Self {
        ORSet { adds: BTreeMap::new(), removals: BTreeMap::new(), replica, next: 0 }
    }

    pub fn add(&mut self, elem: &str) {
        self.next += 1;
        self.adds
            .entry(elem.to_string())
            .or_default()
            .insert((self.replica, self.next));
    }

    pub fn remove(&mut self, elem: &str) {
        // Tombstone every tag we have observed for this element.
        if let Some(tags) = self.adds.get(elem) {
            let observed: Vec<Tag> = tags.iter().copied().collect();
            let gone = self.removals.entry(elem.to_string()).or_default();
            for tag in observed {
                gone.insert(tag);
            }
        }
    }

    fn live_tags(&self, elem: &str) -> usize {
        let empty = BTreeSet::new();
        let added = self.adds.get(elem).unwrap_or(&empty);
        let removed = self.removals.get(elem).unwrap_or(&empty);
        added.difference(removed).count()
    }

    pub fn contains(&self, elem: &str) -> bool {
        self.live_tags(elem) > 0
    }

    pub fn elements(&self) -> Vec<&str> {
        self.adds
            .keys()
            .filter(|e| self.contains(e))
            .map(String::as_str)
            .collect()
    }
```

```
    /// Join: component-wise union of both maps. Union is commutative,
    /// associative, idempotent — both components only ever grow.
    pub fn merge(&mut self, other: &ORSet) {
        for (elem, tags) in &other.adds {
            self.adds
                .entry(elem.clone())
                .or_default()
                .extend(tags.iter().copied());
        }
        for (elem, tags) in &other.removals {
            self.removals
                .entry(elem.clone())
                .or_default()
                .extend(tags.iter().copied());
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    /// Full bidirectional sync: both replicas end in the join state.
    fn sync(a: &mut ORSet, b: &mut ORSet) {
        let (sa, sb) = (a.clone(), b.clone());
        a.merge(&sb);
        b.merge(&sa);
    }

    #[test]
    fn concurrent_add_and_remove_add_wins() {
        let mut a = ORSet::new(1);
        let mut b = ORSet::new(2);
        a.add("milk");                  // tag (1,1)
        sync(&mut a, &mut b);           // both replicas have seen the add

        a.remove("milk");               // tombstones the tag it observed: (1,1)
        b.add("milk");                  // concurrently: a fresh, unseen tag (2,1)

        sync(&mut a, &mut b);
        assert!(a.contains("milk"));    // the concurrent add survives on both
        assert!(b.contains("milk"));
        assert_eq!(a.elements(), vec!["milk"]);
    }

    #[test]
    fn remove_of_observed_add_sticks() {
        let mut a = ORSet::new(1);
        let mut b = ORSet::new(2);
        a.add("milk");
        sync(&mut a, &mut b);           // b has observed the add

        b.remove("milk");               // tombstones the observed tag
        sync(&mut a, &mut b);
        assert!(!a.contains("milk"));   // a non-concurrent remove wins, as it must
        assert!(!b.contains("milk"));
    }

    #[test]
    fn merge_is_commutative_and_idempotent() {
```

```
        let mut a = ORSet::new(1);
        let mut b = ORSet::new(2);
        a.add("eggs");
        a.add("sugar");
        b.add("flour");
        b.remove("flour");              // b tombstones its own observed add

        let mut ab = a.clone();
        ab.merge(&b);
        let mut ba = b.clone();
        ba.merge(&a);
        assert_eq!(ab.elements(), ba.elements());       // commutative
        assert_eq!(ab.elements(), vec!["eggs", "sugar"]);

        let snapshot = ab.elements();
        let mut again = ab.clone();
        again.merge(&b);
        assert_eq!(again.elements(), snapshot);         // idempotent
    }
}
```

The first test is the entire chapter in miniature: two replicas, one partition, conflicting intentions, deterministic reunion — and the user's re-added milk survives because the remove was only allowed to kill what it had **seen**.

9.14.4 RGA: A Sequence CRDT for Text

```
/// Globally unique character id: (per-replica counter, replica id).
/// Tuple order makes "later" ids sort higher — the sibling tie-break.
type Id = (u64, u64);

#[derive(Debug, Clone)]
struct Atom {
    id: Id,
    after: Option<Id>, // id of the atom this one was inserted after (None = start)
    ch: char,
    tombstone: bool,
}

/// Replicated Growable Array: characters are addressed by stable ids,
/// never by integer positions, so concurrent inserts cannot shift each
/// other's meaning. Deletes are tombstones on the atom.
#[derive(Clone)]
pub struct Rga {
    atoms: Vec<Atom>,
    replica: u64,
    next: u64,
}

impl Rga {
    pub fn new(replica: u64) -> Self {
        Rga { atoms: vec![], replica, next: 0 }
    }

    fn alloc(&mut self) -> Id {
        self.next += 1;
        (self.next, self.replica)
    }
```

```
    /// Id of the visible character immediately left of visible position
    /// `pos` (0 = before the first character). Tombstones are invisible.
    fn visible_before(&self, pos: usize) -> Option<Id> {
        let mut seen = 0;
        let mut last = None;
        for atom in &self.atoms {
            if atom.tombstone { continue; }
            if seen == pos { break; }
            seen += 1;
            last = Some(atom.id);
        }
        last
    }

    /// Insert `ch` at visible position `pos`.
    pub fn insert(&mut self, pos: usize, ch: char) {
        let after = self.visible_before(pos);
        let id = self.alloc();
        self.integrate(Atom { id, after, ch, tombstone: false });
    }

    /// Place an atom directly after its parent, but *before* any existing
    /// same-parent atom with a higher id. That single tie-break makes the
    /// placement of concurrent inserts deterministic on every replica.
    fn integrate(&mut self, atom: Atom) {
        let mut i = match atom.after {
            None => 0,
            // Atoms arrive in causal order (a child always sits behind its
            // parent in a replica's array), so the parent is present.
            Some(parent) => self.atoms.iter().position(|a| a.id == parent).unwrap() + 1,
        };
        while i < self.atoms.len()
            && self.atoms[i].after == atom.after
            && self.atoms[i].id > atom.id
        {
            i += 1;
        }
        self.atoms.insert(i, atom);
    }

    /// Delete the visible character at position `pos` (a tombstone, so the
    /// atom survives as a landmark for concurrent inserts).
    pub fn delete(&mut self, pos: usize) {
        if let Some(atom) = self.atoms.iter_mut().filter(|a| !a.tombstone).nth(pos) {
            atom.tombstone = true;
        }
    }

    /// The visible document: the atom walk, skipping tombstones.
    pub fn text(&self) -> String {
        self.atoms.iter().filter(|a| !a.tombstone).map(|a| a.ch).collect()
    }

    /// Join: union by atom id, plus tombstone propagation. Integrating in
    /// the other replica's array order respects causality; re-merging the
    /// same state is a no-op (idempotence).
    pub fn merge(&mut self, other: &Rga) {
        for atom in &other.atoms {
            if !self.atoms.iter().any(|a| a.id == atom.id) {
                self.integrate(atom.clone());
```

```
            }
        }
        for atom in &other.atoms {
            if atom.tombstone {
                if let Some(mine) = self.atoms.iter_mut().find(|a| a.id == atom.id) {
                    mine.tombstone = true;
                }
            }
        }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn concurrent_inserts_converge() {
        let mut a = Rga::new(1);
        let mut b = Rga::new(2);
        a.insert(0, 'h');
        a.insert(1, 'i');
        b.merge(&a);                        // both replicas show "hi"
        assert_eq!(b.text(), "hi");

        // Partition: both type a different character at the end.
        a.insert(2, '!');                   // id (3,1), after "i"
        b.insert(2, '?');                   // id (1,2), after "i"

        // Heal: sync both ways.
        let (sa, sb) = (a.clone(), b.clone());
        a.merge(&sb);
        b.merge(&sa);
        assert_eq!(a.text(), b.text());     // identical documents
        assert_eq!(a.text(), "hi!?");       // higher id first: (3,1) > (1,2)
    }

    #[test]
    fn delete_is_a_tombstone_and_propagates() {
        let mut a = Rga::new(1);
        a.insert(0, 'h');
        a.insert(1, 'i');
        let mut b = Rga::new(2);
        b.merge(&a);

        a.delete(0);                        // tombstone 'h'
        b.merge(&a);                        // the delete travels
        assert_eq!(a.text(), "i");
        assert_eq!(b.text(), "i");

        // The tombstoned atom is still a valid landmark for new inserts.
        b.insert(0, 'H');                   // before 'i', after the ghost 'h'
        a.merge(&b);
        assert_eq!(a.text(), "Hi");
        assert_eq!(b.text(), "Hi");
    }

    #[test]
    fn merge_is_idempotent() {
        let mut a = Rga::new(1);
```

```
        a.insert(0, 'x');
        a.insert(1, 'y');
        let mut b = Rga::new(2);
        b.merge(&a);
        let (text_before, len_before) = (b.text(), b.atoms.len());
        b.merge(&a);                        // same state again
        assert_eq!(b.text(), text_before);
        assert_eq!(b.atoms.len(), len_before);
    }
}
```

The second test quietly demonstrates why tombstones in sequence CRDTs are non-negotiable: Bob inserts `H` **after the deleted** `h` — a landmark only the tombstone still provides — and Alice's replica, which deleted `h` before ever seeing Bob's `H`, still integrates it in the right place. Delete the atom outright and that insert has no anchor; Chapter 1's tombstone line item ("heavier storage and GC") is now fully priced.

> **INTERVIEW TIP — What to say when asked "so which is better, OT or CRDTs?"**
>
> Refuse the framing, then answer precisely: OT centralizes order (a sequencer), pays with a single point of ordering, and gets lean text and clean history; sequence CRDTs decentralize order (stable ids), pay with per-character metadata and GC, and get offline editing, peer-to-peer sync, and end-to-end encryption for free. Then name the boundary condition that actually decides: if the product needs a server that can **read** the document (search, suggestions, AI features), a sequencer and plaintext-friendly transforms fit; if the product promises that no server ever can, CRDTs are close to forced. Figma and Google Docs on one side, Automerge and Yjs on the other — same problem, different promise.

9.15 Scaling the Design

CRDT systems scale along three independent axes, and it is worth naming each because they fail differently.

Sharding by key. Independent keys converge independently, so the key-space shards exactly like Chapter 5's comment store: consistent-hash keys across node groups within a region, and let each shard gossip its own deltas. A hot **key** is only read-hot — writes land in the local replica's slot or tags, which is the entire point — but a hot **structure** is real: one OR-Set with 10^9 members is a 40 GB state whose every delta exchange is painful. The fix is to shard the structure itself: hash elements into 1 024 sub-sets, each an independent OR-Set with independent deltas, unioned at read time. (A counter, pleasantly, is pre-sharded by construction: each replica already owns a private slot.)

Sync economics. Delta groups amortize headers; ack matrices (a per-peer vector clock of delivered deltas) bound retransmission; and when a peer's lag exceeds a threshold, the system falls back to a full-state or Merkle-repair exchange rather than replaying a million deltas one by one. With more than a handful of regions, full-mesh gossip wastes bandwidth, so deployments switch to hierarchical gossip: dense mesh inside a region, designated spokes between regions, convergence bound now O(depth × interval) — the estimation table's "topology is a convergence budget" made operational.

Metadata lifecycle. Tombstone GC runs continuously: a low-priority sweeper that collects any tombstone whose ack vector shows every live replica has merged it. Compaction rewrites sequence-CRDT runs (Yjs and Automerge both collapse adjacent same-replica atoms into run-length records, recovering most of the 40 B/char overhead). Both are background work; neither ever blocks a write, because blocking a write would break the one promise this chapter exists to keep.

9.16 Failure Modes & Degradation

Failure	Symptom	Handling
Duplicated delivery	Same delta arrives twice	Idempotent join absorbs it — the three laws are the retry policy
Reordered deltas	Child atom arrives before parent	State-based: harmless (joins commute). Op-based: causal broadcast buffers until predecessors arrive
Replica loss	Un-gossiped local writes vanish	Local ack = local durability. Critical keys ack only after k-of-n gossip — a durability dial, not a consistency one
Long partition	Deep divergence on heal	Anti-entropy still converges; Merkle repair finds differing keys in O(log n) hashes instead of full transfer
Premature tombstone GC	Deleted elements resurrect when a lagging replica rejoins	Ack-gated GC only: never collect on a timer (Section 9.10)
Clock skew (LWW)	Truly-later write loses silently	Prefer MV-register + context, or hybrid logical clocks; monitor skew as a first-class metric
Split-brain clients	User edits on two devices concurrently	Not a failure — the design's home turf; add-wins / sibling semantics apply

COMMON PITFALL — The resurrection bug is a rite of passage

Nearly every team that ships a CRDT store re-discovers the deleted-data resurrection of Section 9.10 in production: someone "optimizes" tombstone retention from "until all replicas ack" to "keep for 30 days", a replica comes back after 31, and customer data un-deletes itself. Timer-based GC of convergence metadata is never safe; only knowledge-based GC is. If the interviewer asks for war stories, this is the canonical one — Redis CR, Riak, and Cassandra's `gc_grace_seconds` all have scars here, and Cassandra documents the foot-gun by name.

9.17 Trade-offs & Alternatives

Approach	Writes under partition	Metadata	Invariants	Complexity lives in
Single-leader replication	Minority side stalls	None	Full (leader orders all)	Failover & split-brain fences
Consensus (Raft/Paxos)	Minority side stalls	None	Full	Protocol correctness; WAN RTT per write
OT + sequencer (Ch. 1)	Sequencer failover stall	Tiny	Per-document order	Transform-matrix correctness
CRDTs (this chapter)	Never blocked	Tags, tombstones, clocks; GC forever	Per-object only	Merge proofs; metadata lifecycle

Approach	Writes under partition	Metadata	Invariants	Complexity lives in
App-level siblings	Never blocked	None in the store	None	Every client's merge code

Three finer-grained choices recur inside the CRDT camp itself.

- **Add-wins vs remove-wins.** Our OR-Set lets a concurrent add survive a remove; the remove-wins variant tombstones the element so aggressively that even concurrent adds die. Add-wins matches user intent for carts and editors ("I put it back!"); remove-wins matches moderation and revocation ("this must be gone, whoever touched it"). Pick per field, not per system.
- **LWW vs MV registers.** LWW is one line of merge logic and silently discards concurrent writes; MV preserves them as siblings and bills the application. Rule of thumb: LWW for overwrite-able derived state (caches, presence, "last seen"), MV for anything a user would be angry to lose.
- **State-based vs op-based.** State tolerates any transport but moves more bytes; op-based moves few bytes but needs exactly-once causal broadcast. Delta-state closes most of the byte gap, which is why the production mainstream is state-based with deltas.

KEY INSIGHT — The boundary CRDTs cannot cross

CRDTs make **convergence** free; they make **invariants** impossible without coordination. "Balance must never go negative", "a username is taken exactly once", "one winner per auction" — each requires the replicas to agree on a fact **before** acting, which is coordination, which is what we gave up. This is not a limitation of cleverness but CAP itself, wearing a name tag. The production answer is hybrid: CRDTs for the 95% of state that is mergeable, consensus or escrow (pre-reserving a quota of the invariant, e.g. each region may sell 100 of the 300 seats) for the rest. Saying this sentence in the interview is worth more than deriving any merge rule.

9.18 Observability & SLOs

An AP system's vital signs are different from a CP system's: nothing ever blocks, so "is it up?" is uninteresting, and the interesting question is "how far apart are the replicas right now?".

Signal	What it measures	Alert when
Convergence lag	Age of the oldest local write not yet ack'd by all peers	p99 > 10 s: partition or slow peer
Delta backlog	Undelivered deltas per peer	Growing for > 60 s: link down or peer wedged
Tombstone ratio	Tombstones ÷ live entries	> 3× after GC window: sweeper broken or acks missing
Sibling rate	MV-register reads returning multiple values	Sustained rise: clients writing blind (missing contexts)
Merge throughput	Deltas merged per second	Sudden drop: gossip stalled; sudden spike: repair storm
Metadata overhead	CRDT bytes ÷ payload bytes	> 2× post-compaction: allocation strategy regressing

SLOs. Write availability 99.99% per region (writes never coordinate, so this is mostly "is the local process alive"). Convergence: p99 ≤ 2 s within the mesh, ≤ 30 s during a degraded topology. Durability: default keys ack locally; billing-class keys ack after 2-of-3 regions have merged the delta — still coordination-free, just patient. Metadata: ≤ 2× payload after each compaction window.

9.19 Interview Wrap-Up

Likely follow-ups, with the shape of a strong answer.

- *"Enforce 'balance ≥ 0' across regions."* You cannot, not with CRDTs alone — say so instantly, it is the highest-signal sentence available. Then offer escrow (pre-partition the spendable budget per region) or a CP island (consensus on that one key) and note that the rest of the system stays AP.
- *"Unique username registration?"* Same boundary: consensus for the registry key, CRDTs for the profile data. Hybrid designs are the senior answer.
- *"Why not just Cassandra's LWW everywhere?"* Clock skew chooses your losers, and concurrent writes are silently destroyed. Fine for caches, negligent for user data.
- *"How do Redis / Riak / DynamoDB-style stores do multi-region?"* Redis CR is delta-state CRDTs; Riak is OR-Sets and MV-registers with dotted version vectors; Dynamo-style stores push sibling resolution to the client — the "app-level siblings" row of the trade-off table.
- *"Two users type into the same word — what do they see?"* RGA: both strings converge; possible interleaving of the two runs, prevented by LSEQ/Fugue-style allocation; in practice, cursors and presence (Chapter 1's awareness channel) keep humans out of each other's words anyway.
- *"When would you refuse CRDTs?"* Strong global invariants, very large per-object state (a CRDT wrapping a 1 GB blob merges terribly), and teams without the appetite to own merge semantics. A boring CP database is a fine choice when the product can afford it.

Checklist for the whiteboard. (1) Define replica, partition, convergence. (2) State CAP and pick AP explicitly. (3) Derive one CRDT from the three laws — a G-Counter takes sixty seconds. (4) Show one concurrent-conflict merge end to end. (5) Name tombstones and the ack-gated GC. (6) Draw the sync fabric: deltas, gossip, repair. (7) Volunteer the invariant boundary before the interviewer finds it.

9.20 Summary & Further Reading

CRDTs answer the chapter's prompt by changing the question. Instead of ordering concurrent writes — impossible without coordination, undefined across a partition — they build data types whose states form a join-semilattice and whose merge is the join, so that commutativity, associativity, and idempotence make order, grouping, and duplication irrelevant. On that foundation: counters as per-replica slots merged by max; sets as union, then union-twice with tombstones, then observed-remove with unique tags for add-wins; registers as timestamps (dangerous) or sibling sets (honest); text as stable-identity atoms ordered by (parent, id). Causality is tracked with vector clocks and dots; sync is deltas over gossip with Merkle repair; tombstones are collected only when every replica has ack'd them. The price is metadata and the loss of global invariants; the reward is that the write path never, ever waits — Chapter 1's road not taken, now fully mapped.

Further reading.

- The source video: *"CRDTs — Stop Worrying About Write Conflicts — Systems Design 0 to 1 with Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=FG5Varj1Ows`
- Shapiro, Preguiça, Baquero, Zawirski — *"Conflict-free Replicated Data Types"* (2011) and *"A Comprehensive Study of Convergent and Commutative Replicated Data Types"* (2011) — the founding papers; the second is the catalog this chapter compressed.
- Almeida, Shoker, Baquero — *"Delta State Replicated Data Types"* (2018) — the sync-economics fix, formally.

- Lamport — *"Time, Clocks, and the Ordering of Events in a Distributed System"* (1978) — happens-before, in its original eight pages.
- Riak KV's documentation on dotted version vectors, and Redis CR's active-active whitepapers — production CRDT stores, warts included.
- The Ink & Switch *local-first software* essay, and the Yjs and Automerge codebases — sequence CRDTs doing Chapter 1's job in the wild.

9.21 Chapter Glossary

Term	Meaning
Add-wins semantics	A concurrent add survives a concurrent remove; the OR-Set default
Anti-entropy	Background replica-pair reconciliation that drives divergence toward zero
Associativity	Grouping of merges is irrelevant: (a ∨ b) ∨ c = a ∨ (b ∨ c)
CAP	Under partition, choose consistency (CP: stall writes) or availability (AP: merge later); you cannot have both
Causal context	Opaque version vector handed to a reader and echoed on overwrite, separating "supersedes" from "concurrent"
Commutativity	Order of merges is irrelevant: a ∨ b = b ∨ a
Concurrent events	Events with no happens-before path between them; no fact-of-the-matter order exists
Convergence	Replicas that saw the same updates are in the same observable state
Coordination	Pre-write communication (leader, lock, quorum) — the thing partitions disable
CvRDT / CmRDT	State-based (merge states by join) vs op-based (broadcast commuting operations) CRDT families
Delta-state	Shipping the minimal state change instead of full state; deltas merge by the same join
Dot	A (replica, counter) pair naming one event; the OR-Set tag and the version vector's precise companion
Escrow	Pre-allocating shares of an invariant (e.g. quota of seats) so regions can act without coordinating
Eventual consistency	"If updates stop, replicas end up equal" — liveness only, no safety during updates
G-Counter / PN-Counter	Per-replica slots merged by max; PN adds a second counter so the derived value can fall
Gossip	Randomized peer-to-peer exchange; O(log R) rounds to infect R replicas
Happens-before (→)	Lamport causality: program order plus message passing, transitively closed

Term	Meaning
Idempotence	Duplication is irrelevant: a ∨ a = a — the property that makes retries free
Inflation / monotonicity	Updates only move state up the lattice; nothing is ever un-known
Interleaving anomaly	Concurrent text runs merged character-by-character; cosmetic, fixed by newer allocators
Join-semilattice	A partial order where every pair has a least upper bound; merge = that bound
LWW register	Keep the value with the max timestamp; discards concurrent writes, trusts clocks
Merkle repair	Hash-tree comparison of key-spaces to find divergence in O(log n) hashes
MV register	Keeps all concurrent values as siblings; the application resolves with context
OR-Set	Observed-remove set: tagged adds, tombstone-the-observed removes; add-wins
RGA	Replicated Growable Array: sequence CRDT with stable atom ids and parent links
Strong eventual consistency	Eventual consistency plus same-updates ⇒ same-state safety; what CRDTs prove
Tombstone	A mergeable record of deletion; collectible only when all replicas ack it
Vector clock	Replica→counter map encoding causal history; dominates iff ≥ in every component

— End of Chapter 9 · Next: Chapter 10 —

CHAPTER 10

10

Designing a Distributed Job Scheduler

NOTE — Problem source

This chapter solves the problem posed in the talk *"20: Distributed Job Scheduler"* from the series *Systems Design Interview Questions with Ex-Google SWE* (channel: *Jordan has no life*). Unlike Chapters 8 and 9 (fundamentals deep dives), this is a full **product design** in the shape of Chapters 1–7: cron-as-a-service, the machinery behind Airflow, Kubernetes CronJobs, AWS EventBridge Scheduler, and every "remind me in 20 minutes" button on Earth. It quietly reuses almost every earlier chapter: the due-job index is Chapter 8's B+ tree, the dispatch queue is Chapter 4's append-only log, per-tenant fairness is Chapter 3's limiter, and shard ownership borrows Chapter 9's vocabulary of leases and clocks. All terms are defined before use; all reference code is Rust with deterministic tests.

10.1 The Problem Statement

The interviewer draws a clock and says:

"Design a distributed job scheduler. Users register jobs — one-shot ('fire at 3:00 AM'), recurring ('every weekday at 07:30'), or whole workflows of dependent jobs — and the system fires them reliably, at the right time, at scale: tens of millions of registered jobs, tens of thousands of firings per second at the peak. Jobs must not be lost, must not silently double-execute, and the scheduler itself must survive machine and region failures."

Every large company builds this system eventually, because the naive versions are all traps: a single machine running `cron` is a single point of failure; a `sleep()` in application code dies with the process; a database table polled by every worker is a lock convoy. The interview tests whether you can separate the system's three concerns — **knowing when jobs are due**, **deciding who fires them**, and **executing them without duplicates** — and scale each independently.

DEFINITION — Job / run / trigger

A *job* is the registered specification: what to execute (a queue message, an HTTP call, a worker class plus payload), **when** (its *trigger*: a fire timestamp or a schedule), and **how to behave on failure** (retry policy, dead-letter). A *run* (or execution) is one actual firing of a job — jobs are few and durable, runs are many and ephemeral. Confusing the two tables in the data model is the first design error this chapter avoids.

DEFINITION — Schedule / cron expression

A compact description of recurring fire times. The classic *cron expression* has five fields — minute, hour, day-of-month, month, day-of-week — so `30 7 * * 1-5` reads "07:30 on weekdays". Two traps lurk here and a senior candidate names both unprompted: **time zones** (07:30 in whose morning?) and **DST** (on spring-forward day, 02:30 never happens; on fall-back day, it happens twice). The store keeps UTC plus the tenant's zone; the misfire policy (Section 10.9) decides what "the 02:30 that never came" means.

DEFINITION — Misfire and catch-up policy

A *misfire* is a scheduled fire time that passed without firing — because the scheduler was down, the shard was being re-leased, or DST erased the minute. The *catch-up policy* is a per-job choice: **fire once now** (coalesce all misses into one), **replay every miss**, or **skip to the next scheduled time**. A billing job wants replay-every-miss; a cache warmer wants skip; an hourly report wants coalesce. There is no correct default — only a default you chose on purpose.

10.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Job kinds?	One-shot delayed jobs, recurring cron jobs, and DAG workflows of dependent jobs
Execution model?	Scheduler fires by enqueuing to a dispatch queue; worker pools pull and execute. We design both sides
Scale?	10^7 registered jobs, 3×10^3 firings/s average, 30× bursts at top-of-hour
Fire latency?	Seconds matter, microseconds do not: p99 within 1 s of the scheduled time
Delivery semantics?	Push for effectively-once execution; expect to explain why raw exactly-once is impossible
Multi-tenant?	Yes — one tenant's burst must not starve others (Chapter 3 territory)
Durability?	Registered jobs survive region loss; run history retained 90 days

NOTE — Agreed scope

1. Design the **control plane**: job registration (one-shot, cron, DAG), update, cancel, inspect — backed by a durable job store.
2. Design the **trigger side**: how due jobs are found across 10^7 timers without a thread or a full-table scan per timer.
3. Design the **firing side**: sharded scheduler replicas with leases and fencing, a durable dispatch queue, and workers that execute **effectively-once** via idempotency keys.
4. Design **failure behavior** as a first-class feature: retries with backoff and jitter, dead-letter queues, misfire policies.
5. Quantify scale: registered jobs, fire rate, burst shape, worker fleet size, run-history storage.
6. Out of scope: the workers' actual business logic; sub-second precision timers (that's a kernel problem, not an architecture).

10.3 Functional Requirements

1. **Register and manage jobs.** Create, update, cancel, and inspect one-shot jobs (`fire_at = T`), recurring jobs (`cron` expression + time zone), each with payload, target, and retry policy.

2. **Fire on time.** Every due job produces a fire command within the latency SLO of its scheduled time, in all but catastrophe scenarios.
3. **Effectively-once execution.** Clients that follow the idempotency protocol see each run execute at most once, even though the transport underneath is at-least-once.
4. **Bounded retries.** Failed runs retry with exponential backoff and jitter up to a per-job cap, then land in a dead-letter queue with the full error context.
5. **Workflows.** Users submit DAGs of dependent jobs; the system runs each job only after its dependencies succeed, parallelizes independent branches, and applies a per-workflow failure policy (fail-fast vs. complete-what-you-can).
6. **Misfire handling.** After downtime or DST gaps, each job follows its declared catch-up policy: coalesce, replay, or skip.
7. **Tenant fairness.** Per-tenant fire-rate quotas; bursts are smoothed or queued, never allowed to starve other tenants.

10.4 Non-Functional Requirements

- **Durability of the registry.** A registered job must survive the loss of any machine and any region: the job store is replicated (Chapter 8's machinery underneath), and a fire command, once accepted, is durably queued before it is acknowledged.
- **Availability of firing.** The scheduler must keep firing with any one replica — or one region — down: shard leases fail over in ≤ 15 s.
- **No lost runs.** Between scheduler crash, queue outage, and worker death, a due job may be **delayed** or **duplicated**, but never silently dropped. Duplicates are absorbed by the idempotency layer.
- **Bounded staleness.** p99 fire latency ≤ 1 s past due for one-shot jobs; ≤ 5 s for cron jobs (their consumers are batch-shaped).
- **Elastic scale-out.** Adding scheduler shards or worker pools raises capacity linearly; no component is a hard singleton (leases, not process identity, own the shards).

DEFINITION — At-least-once / at-most-once / exactly-once

The three delivery guarantees a system can offer. *At-most-once*: fire and forget — no retries, so failures lose messages. *At-least-once*: retry until acknowledged — failures duplicate messages. *Exactly-once* end-to-end delivery is **provably impossible** over an unreliable network (the receiver cannot distinguish "sender crashed before sending" from "sender crashed after sending but before the ack arrived" — the classical Two Generals impossibility). What **is** achievable — and what this design delivers — is *effectively-once*: at-least-once transport plus **idempotent** processing, so duplicates exist on the wire but are invisible to the application.

DEFINITION — Idempotency / idempotency key

An operation is *idempotent* if performing it twice has the same effect as performing it once. A scheduler achieves this by stamping every fire command with a unique *idempotency key* — `(job_id, scheduled_time)` is the natural choice — and having the executor check a *dedup store* before its first side effect: first arrival claims the key and runs; any duplicate sees the claimed key and exits as a no-op. Chapter 5's vote endpoint used the same trick; here it is the backbone of the whole system.

10.5 Back-of-the-Envelope: Timers, Bursts, and Workers

Three numbers size the design: how many timers exist, how unevenly they fire, and how much execution capacity the fire rate implies.

Assumptions. 10^7 registered jobs; recurring jobs fire hourly on average; job spec ≈ 1 KB; average run duration 30 s; 90-day run history at 1 KB per run record.

Quantity	Estimate (with assumptions)
Registered jobs	10^7 × 1 KB ≈ **10 GB** of job specs — comfortably one replicated store, Chapter 8 style
Average fire rate	10^7 jobs × 1 fire/hour ≈ **2.8×10^3 fires/s**
Top-of-hour burst	50% of cron jobs pinned to minute 0 → 5 × 10^6 fires in one minute ≈ **8 × 10^4 fires/s — a 30× burst**. Splaying (§10.6) is mandatory
Due-job discovery	64 shards × one indexed range scan per second — the B+ tree leaf walk of Chapter 8; trivial against 10^7 rows
Concurrent executions	2.8×10^3 fires/s × 30 s avg ≈ **8.4×10^4 in flight** → 1 000 workers × 100 slots each
Dispatch traffic	≤ 8.3×10^4 msg/s × 1 KB ≈ **83 MB/s peak** — well inside Chapter 4's log throughput
Run history	2.8×10^3 runs/s × 86 400 s × 90 days × 1 KB ≈ **22 TB** → retention windows and rollups, Chapter 4 style
Lease failover	TTL 10 s + probe 5 s → **≤ 15 s** worst-case pause on a shard when its scheduler dies

KEY INSIGHT — The burst, not the average, designs the system

The average fire rate needs one modest scheduler; the top-of-hour spike — half the world's cron jobs pinned to minute 0 — needs thirty. A scheduler designed for the average melts at 00:00 UTC daily; one designed for the peak idles 97% of the day. The resolution is to **reshape the demand**: splay recurring jobs across their period (`0 * * * *` becomes "minute `hash(job_id) mod 60`, every hour"), which flattens a 30× burst to 1.5× — and then size for that. Every scheduler interview should contain the sentence "and then we desynchronize the timers"; it is the same thundering-herd lesson as Chapter 3's jittered retries and Chapter 8's rightmost hotspot, wearing a clock face.

10.6 The Core Challenge: "Exactly Once" Is a Lie We Tell Well

Ask ten engineers what a job scheduler must guarantee and nine say "exactly-once execution". The tenth — the one who has run one — says "effectively-once, and here is why." The reason is not engineering laziness; it is impossibility. Between "scheduler enqueues fire command" and "worker performs the side effect", a network and two processes can fail in combinations no protocol can distinguish: the worker that executed the job and **then** crashed before acking is indistinguishable from the worker that crashed before executing. The scheduler must retry, so duplicates on the wire are unavoidable. The only question is whether anyone can see them.

So the challenge splits in two, and the design addresses each half with a different weapon:

1. **The trigger side is a data-structures problem.** Ten million timers, thirty thousand firing per second at the peak, each fire time computable but volatile (jobs are cancelled and rescheduled constantly). Neither a thread per timer nor a full-table poll survives contact with these numbers; Section 10.7 picks the structure that does.
2. **The firing side is a distributed-systems problem.** Several scheduler replicas must divide the work without double-firing, workers must absorb duplicates they cannot prevent, and a crashed

component's in-flight work must be reclaimed — all without a global lock. Section 10.8 assembles leases, fencing tokens, and idempotency keys into exactly that.

KEY INSIGHT — Effectively-once = at-least-once + idempotency + a dedup store

The industry's entire answer, in one line: **deliver at least once, process idempotently, remember what you processed.** The scheduler guarantees every fire command lands on the queue at least once (durable enqueue before ack); the worker, before any side effect, claims the run's idempotency key in a dedup store (`INSERT ... IF NOT EXISTS` — the same compare-and-set discipline as Chapter 5's votes); the key's TTL outlives any plausible duplicate window. Duplicates still happen on the wire — they just never happen twice to the application. This is the exact pattern behind Kafka's idempotent producers, Stripe's idempotency-keyed API, and every payment retrier you have ever trusted with your card.

10.7 Deep Dive: The Trigger Side — Finding What's Due

The trigger side answers one question, 64 times a second per shard: **which jobs are due right now?** Three designs, in increasing sophistication — name all three in the interview, then defend your pick.

Design A: the indexed scan. Store every job with a materialized `next_fire_at` column and a B+ tree index on `(shard, next_fire_at)` — Chapter 8's structure, doing exactly the job it was designed for. A scheduler tick is one range scan (`next_fire_at ≤ now` within its shard slice) plus a linked-leaf walk; due jobs stream out in fire-time order at index speed. After firing (or enqueueing), the job's `next_fire_at` is recomputed from its cron expression and the row updated — the index maintains itself. This is the design this chapter ships: durable by construction (the timers **are** the database), crash-transparent (a dead scheduler's successor rescans and finds everything), and simple enough to reason about at 2 AM.

Design B: the in-memory min-heap. Keep a priority queue of `(fire_at, job)` keyed by soonest-first (the Rust listing in Section 10.13). O(log n) per schedule or cancel-via-lazy-tombstone, O(1) peek at the next due job, zero index maintenance. The catch: the heap is **volatile** — it must be rebuilt from the store on every failover, and at 10^7 timers that is a 10–30 s cold start during which the shard is blind. Production systems use the heap as a **hot cache in front of Design A**: the tick loop keeps the next few seconds of timers in memory so the scan runs once per window, not once per timer.

Design C: the hashed timing wheel. The classic kernel answer (Kafka, Netty, and the Linux kernel itself): an array of buckets representing time slots — slot `i` holds every timer due at `now + i·tick` mod the wheel's circumference — so insert and expire are O(1) pointer pushes. A **hierarchical** wheel (a seconds-wheel feeding a minutes-wheel feeding an hours-wheel, like a clock's gears) covers far-future timers in O(wheels) per operation. Beautiful, optimal, and entirely in-memory: the same durability problem as the heap, plus resize pain, plus a poor fit for cancel-heavy workloads. Worth describing to show range; wrong as the system of record.

DEFINITION — Timing wheel (hashed / hierarchical)

A ring buffer of buckets indexed by time slot: a timer due in k ticks lands in bucket `(current + k) mod circumference`; each tick, the current bucket's timers fire or cascade. The *hierarchical* variant stacks wheels of coarser granularity (seconds, minutes, hours) so a timer due next year parks cheaply in the year-wheel until it degrades down through the gears. O(1) amortized insert and expiry — the price is volatility and the complexity of cascading on the wheel boundary.

COMMON PITFALL — The poll-everything anti-pattern

The first draft everyone sketches — `SELECT * FROM jobs` every second, filter in application code — turns 10^7 rows into 10^7 row-reads per second per scheduler replica: the database melts, and the interview with it. The index on `next_fire_at` is not an optimization; it **is** the trigger design. If you take one sentence from this section: the due set is a range query, and range queries are a solved problem (Chapter 8) — spend your design budget on the firing side instead.

10.8 Deep Dive: The Firing Side — Leases, Fencing, Idempotency

The firing side answers a harder question: **who** may fire shard 37's due jobs — and how do we stop yesterday's answer from firing them again?

Shard ownership by lease. The 10^7 jobs are hash-partitioned by `job_id` into (say) 64 shards. A scheduler replica acquires a shard by taking its **lease** from a small highly-available coordination store; it renews the lease every few seconds and loses it silently if it stalls (GC pause, network hiccup, death). A standby replica takes over an expired lease — failover in ≤ 15 s, per the SLO.

DEFINITION — Lease

A time-bounded grant of exclusive ownership: "replica R owns shard S until time T, and may renew." Leases convert **membership** ("who owns this shard?") from a consensus-per-operation cost into a heartbeat-per-few-seconds cost. Their danger is the gap between "my lease silently expired" and "I find out": during that gap the old owner still believes itself owner, and two owners double-fire.

DEFINITION — Fencing token

A monotonically increasing number issued by the lease store on every acquisition, which the holder must present with **every** action it performs on behalf of the lease — and which downstream systems check: "accept this dispatch only if its token exceeds every token you have seen for this shard." Fencing closes the lease's blind gap: the stale owner's late dispatches carry an old token and are rejected even if it never learns it was deposed. A lease without fencing is optimism; a lease with fencing is a protocol. (The Rust listing in Section 10.13 implements both in forty lines.)

Why duplicates survive all of this anyway. Suppose leases and fencing work perfectly: single owner per shard, always. A fire command is still delivered at-least-once, because the worker may crash **after** executing but **before** acking, and the queue (correctly) redelivers. Fencing cannot help here — the **same** token legitimately appears twice. This is why the last line of defense lives at the point of side effect: the worker claims the run's idempotency key `(job_id, scheduled_time)` in the dedup store before touching the world, and marks it complete after. A redelivered command finds the key claimed and exits 0. The system is a pipeline of decreasing error rates: leases make double-ownership rare, fencing makes it harmless, queues make delivery at-least-once, and idempotency makes that invisible.

DEFINITION — Heartbeat

A periodic "I am alive and working on X" signal from a component — workers to the scheduler, lease-holders to the lease store. Missed heartbeats past a timeout flip the component to **suspect**, then **dead**, and its work is reclaimed: its lease expires, its in-flight runs return to the queue. The heartbeat's only subtlety is the timeout: too short and GC pauses cause flapping takeovers; too long and failover violates the 15 s SLO. Ten seconds with three-miss tolerance is the field's default for a reason.

10.9 Deep Dive: Retries, Backoff, Jitter, and the Dead-Letter Queue

A run fails. What happens next is policy, and policy is design.

Retry with exponential backoff and full jitter. Attempt k waits `random(0, min(cap, base · 2ᵏ))`: the mean doubles each attempt (giving a struggling dependency room to recover), the cap keeps the tail sane, and the randomization — **full jitter**, drawn fresh per (job, attempt) — spreads a fleet of simultaneous failures across the whole window instead of re-stampeding the dependency in lockstep. This is Chapter 3's token bucket seen from the other side: there we **rate-limited clients**; here we rate-limit **ourselves**, because a retry storm is a self-inflicted denial-of-service.

> **DEFINITION — Dead-letter queue (DLQ)**
>
> The terminal queue for runs that exhaust their retry budget. A DLQ is not a trash can; it is a **quarantine with a promise**: every entry keeps its full context (payload, error, attempt history, stack), an alarm fires on arrival rate, and an operator tool can inspect, patch, and **re-drive** entries back into the dispatch queue once the underlying fault is fixed. The design rule: nothing may fail silently, and no human should ever reconstruct state from logs alone.

Which failures retry and which don't. Distinguish **retryable** errors (timeouts, 502s, connection resets — the dependency might recover) from **permanent** ones (400, schema rejection, "no such user" — retrying is just slow failure). Only retryable errors consume the budget; permanent errors dead-letter immediately. One more taxonomy line separates senior answers: a **poison message** — a run that crashes every worker that touches it — is detected by "attempts exhausted with crash-looping workers" and dead-lettered before it can saw through the fleet.

> **INTERVIEW TIP — Say the quiet part about 2:30 AM**
>
> Retry policy, misfire policy, and DST policy are where schedulers hurt people in production, and interviewers know it. Volunteer the stories: the billing job that double-charged because a retry lacked an idempotency key; the report that silently skipped a day because catch-up defaulted to skip; the 02:30 job that fired twice on fall-back day. You do not need to have lived them — you need to show you know they are **policy choices**, not accidents.

10.10 Deep Dive: Workflows — DAGs of Jobs

Real pipelines are not independent jobs: *extract* must finish before *transform*, which feeds three parallel *load* jobs, which gate *notify*. The scheduler must understand dependencies.

> **DEFINITION — DAG (directed acyclic graph) / workflow**
>
> A *directed* graph: edges mean "depends on" (B's incoming edge from A means A must succeed before B may fire). *Acyclic*: no dependency loops — a cycle has no valid execution order, so it is rejected at **submission time** (the Rust listing's cycle detection), never discovered at 3 AM at runtime. The *layers* of the DAG — Kahn's algorithm peels zero-dependency nodes level by level — are the free parallelism: every job in a layer is independent and may fire concurrently; the workflow's critical path is its longest layer chain.

Mechanics. A workflow is stored as a job graph plus one **run record** per workflow instance. When a run completes, the scheduler decrements the **pending-dependency count** of each child in the store; a child whose count hits zero becomes a normal due job and enters the trigger pipeline — no special machinery downstream. Failure policy is per-workflow: **fail-fast** (first failure cancels pending descendants, workflow run fails) or **complete-branches** (unaffected branches run to completion — right for fan-out reports). Compensation — "payment captured, refund if shipping fails" — is the workflow-level echo of Chapter 9's invariant boundary: it is **application** logic the scheduler must make expressible, not a guarantee it can manufacture.

KEY INSIGHT — The scheduler's data model in one breath

`jobs` (durable specs, indexed by `(shard, next_fire_at)`) · `runs` (one per firing: status, attempt count, idempotency key) · `workflow_edges` (job → depends-on) · `dedup` (claimed idempotency keys with TTL) · `leases` (shard → holder, fencing token, expiry). Five tables. Everything else in this chapter — wheels, retries, takeovers, DLQs — is a process reading and writing those five tables under a discipline. If your whiteboard has more boxes than that, you are designing the org chart, not the system.

10.11 API Design

The surface is deliberately small: jobs and workflows in, runs and dead-letters out, and a handful of internal endpoints for the lease and dispatch machinery. Every mutating client call accepts an idempotency key (Chapter 5's vote endpoint, generalized); every internal dispatch carries a fencing token (Section 10.8).

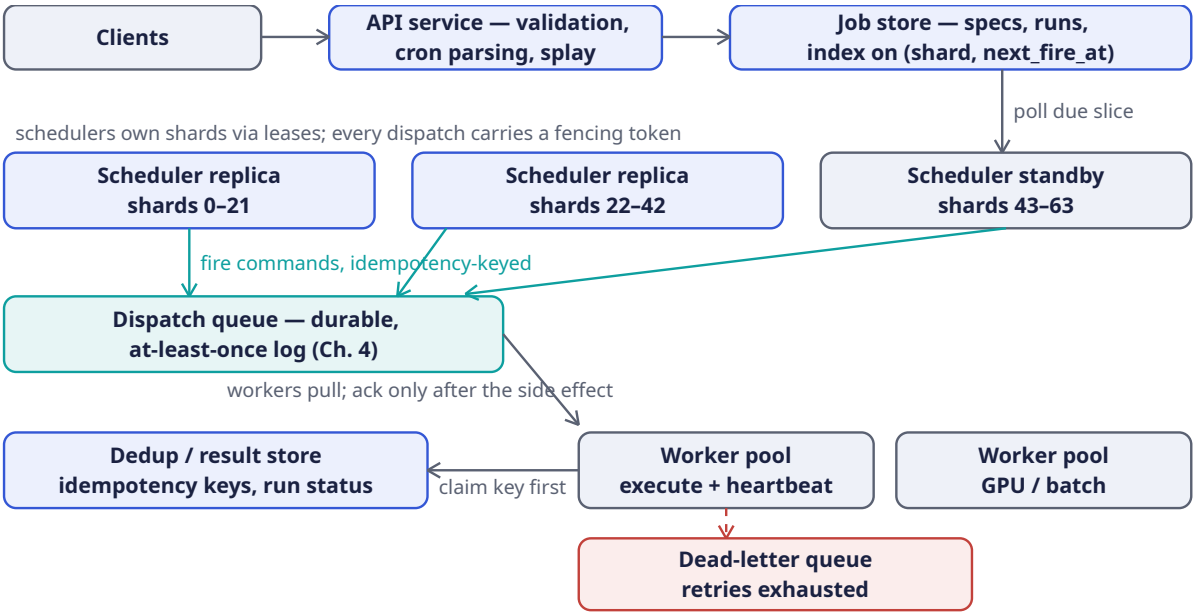
Endpoint	Scope	Semantics
`POST /jobs`	client	Register one-shot (`fire_at`) or recurring (`cron` + `tz`) job; validates the schedule, computes first `next_fire_at` , applies splay offset
`PATCH /jobs/{id}`	client	Update spec; schedule changes re-materialize `next_fire_at` — the old timer dies as a lazy tombstone (§10.13)
`DELETE /jobs/{id}`	client	Cancel: no future fire times; the in-flight run is left to finish or killed per the job's `cancel_policy`
`GET /jobs/{id}`	client	Spec + status + `next_fire_at` + summary of the last run
`POST /workflows`	client	Submit a DAG {jobs, edges}; **cycle-checked at submission** (§10.10); returns a workflow id
`GET /runs?job_id=…&cursor=`	client	Run history, cursor-paginated — Chapter 5's cursor pattern, unchanged
`POST /dlq/{run}/redrive`	operator	Re-enqueue a dead-lettered run once the underlying fault is fixed
`POST /leases/{shard}`	internal	Acquire or renew a shard lease; response carries the new fencing token
`POST /dispatch/pull`	internal	Worker pulls a batch of fire commands; the visibility timeout starts
`POST /dispatch/ack`	internal	Report outcome; un-acked commands redeliver when the timeout lapses

DEFINITION — Visibility timeout

The queue's redelivery timer: a pulled message becomes invisible to other consumers for T seconds; if the puller neither acks nor abandons it within T, the message reappears and another worker takes it. It is the queue-level twin of the shard lease — same trick, same blind gap ("am I still the owner?"), same final answer: idempotency at the point of side effect, because neither lease nor timeout can ever be airtight.

10.12 High-Level Architecture

Read the diagram top to bottom as **registration**, then top to bottom again as **firing**: two flows sharing five tables. The write path a user sees ends at the job store; everything below it is the system talking to itself.



> **NOTE — Reading the diagram**
>
> **Registration** flows right along the top: the API validates the cron expression, stamps the job with its shard (`hash(job_id) mod 64`), its splay offset, and its first `next_fire_at` , and writes it to the job store — done; the user gets an ack without any timer existing yet. **Firing** flows downward: each scheduler replica range-scans the due slice of **its** shards (the Chapter 8 leaf walk), enqueues idempotency-keyed fire commands, workers pull and execute — checking the dedup store before any side effect — and exhausted runs drain to the DLQ with their full context. No box is a global singleton: the API is stateless, the store is replicated, schedulers are fungible behind leases, the queue is a partitioned log, workers are a pool.

10.13 Rust Reference Implementations

Four pieces with deterministic tests: the due-job heap, the lease table with fencing, the backoff schedule, and the DAG layerer. Together they are the scheduler's entire inner loop, small enough to hold in your head and sharp enough to defend line by line.

10.13.1 The Due-Job Heap (Lazy Cancellation)

```rust
use std::collections::{BinaryHeap, HashMap};

/// One scheduled firing. `fire_at` is epoch millis; `seq` breaks ties so
/// equal fire times fire in submission order.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Scheduled {
    pub fire_at: u64,
    pub seq: u64,
    pub job: u64,
}

// BinaryHeap is a MAX-heap; invert the comparison so the SOONEST
```

```rust
// fire_at is "greatest" and pops first.
impl Ord for Scheduled {
    fn cmp(&self, other: &Self) -> std::cmp::Ordering {
        other.fire_at
            .cmp(&self.fire_at)
            .then(other.seq.cmp(&self.seq))
            .then(other.job.cmp(&self.job))
    }
}
impl PartialOrd for Scheduled {
    fn partial_cmp(&self, other: &Self) -> Option<std::cmp::Ordering> {
        Some(self.cmp(other))
    }
}

/// In-memory timer structure sitting in front of the durable store.
/// Cancel/reschedule are lazy: the stale heap entry is left in place and
/// skipped when popped — an entry is valid iff `live[job] == fire_at`.
pub struct Scheduler {
    heap: BinaryHeap<Scheduled>,
    live: HashMap<u64, u64>, // job id -> currently valid fire_at
    seq: u64,
}

impl Scheduler {
    pub fn new() -> Self {
        Scheduler { heap: BinaryHeap::new(), live: HashMap::new(), seq: 0 }
    }

    /// Schedule (or reschedule: a second call for the same job supersedes
    /// the first — the old heap entry becomes a dud on its own).
    pub fn schedule(&mut self, job: u64, fire_at: u64) {
        self.seq += 1;
        self.heap.push(Scheduled { fire_at, seq: self.seq, job });
        self.live.insert(job, fire_at);
    }

    pub fn cancel(&mut self, job: u64) {
        self.live.remove(&job); // heap entry stays; popped-and-skipped later
    }

    /// Drain every job due at or before `now`, soonest first.
    pub fn pop_due(&mut self, now: u64) -> Vec<u64> {
        let mut due = Vec::new();
        while let Some(&top) = self.heap.peek() {
            if top.fire_at > now { break; }
            let entry = self.heap.pop().unwrap();
            if self.live.get(&entry.job) != Some(&entry.fire_at) {
                continue; // cancelled or superseded: a dud
            }
            self.live.remove(&entry.job);
            due.push(entry.job);
        }
        due
    }
}

#[cfg(test)]
mod tests {
    use super::*;
```

```
    #[test]
    fn fires_in_fire_time_order() {
        let mut s = Scheduler::new();
        s.schedule(1, 300);
        s.schedule(2, 100);
        s.schedule(3, 200);
        assert!(s.pop_due(50).is_empty());      // nothing due yet
        assert_eq!(s.pop_due(200), vec![2, 3]); // soonest first
        assert_eq!(s.pop_due(1000), vec![1]);
    }

    #[test]
    fn cancel_prevents_firing() {
        let mut s = Scheduler::new();
        s.schedule(1, 100);
        s.schedule(2, 100);
        s.cancel(1);
        assert_eq!(s.pop_due(100), vec![2]);   // job 1's dud is skipped
        assert!(s.pop_due(1000).is_empty());
    }

    #[test]
    fn reschedule_fires_once_at_new_time() {
        let mut s = Scheduler::new();
        s.schedule(7, 100);
        s.schedule(7, 400);                    // reschedule = supersede
        assert!(s.pop_due(150).is_empty());    // the old time must NOT fire
        assert_eq!(s.pop_due(400), vec![7]);   // the new time fires, once
        assert!(s.pop_due(1000).is_empty());   // and no ghost fires after
    }
}
```

The `live` map is the whole trick: cancellation in a binary heap is O(n) if done eagerly (find the entry, remove it, re-heapify) and O(1) if done lazily (record the truth, let the heap find out when it matters). With cancel-heavy workloads this is the difference between a timer structure and a performance incident — and the same lazy-tombstone pattern as Chapter 9's OR-Set removals.

10.13.2 Leases with Fencing Tokens

```
use std::collections::HashMap;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Lease {
    pub holder: u64,
    pub fence: u64,      // monotonically increasing across ALL acquisitions
    pub expires_at: u64,
}

/// Shard ownership registry. A scheduler replica may dispatch a shard's
/// due jobs only while it holds the lease; the fencing token lets every
/// downstream store reject a stale holder whose lease silently lapsed.
pub struct LeaseTable {
    leases: HashMap<u64, Lease>,
    next_fence: u64,
}

impl LeaseTable {
    pub fn new() -> Self {
```

```
        LeaseTable { leases: HashMap::new(), next_fence: 0 }
    }

    /// Acquire or renew the lease on `shard`. Fails only if another
    /// holder's lease is still live; an expired lease can be taken over.
    pub fn acquire(&mut self, shard: u64, holder: u64, now: u64, ttl: u64) -> Option<Lease>
{
        match self.leases.get(&shard) {
            Some(l) if l.holder != holder && l.expires_at > now => None, // held by someone
else
            _ => {
                self.next_fence += 1;
                let lease = Lease { holder, fence: self.next_fence, expires_at: now + ttl };
                self.leases.insert(shard, lease);
                Some(lease)
            }
        }
    }

    /// The downstream check, kept as per-shard "highest token seen":
    /// accept a dispatch only from a strictly newer holder. A deposed
    /// scheduler waking from a GC pause presents an old token and is
    /// turned away — that rejection is fencing doing its one job.
    pub fn check_fence(highest_seen: &mut u64, fence: u64) -> bool {
        if fence > *highest_seen { *highest_seen = fence; true } else { false }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn lease_excludes_second_holder_until_expiry() {
        let mut t = LeaseTable::new();
        let a = t.acquire(0, 1, 1000, 500).unwrap();    // holder 1 until t=1500
        assert!(t.acquire(0, 2, 1200, 500).is_none());  // contender rejected
        let b = t.acquire(0, 2, 1600, 500).unwrap();    // after expiry: takeover
        assert!(b.fence > a.fence);                     // token strictly rises
    }

    #[test]
    fn renewal_keeps_ownership() {
        let mut t = LeaseTable::new();
        let a = t.acquire(0, 1, 1000, 500).unwrap();
        let r = t.acquire(0, 1, 1400, 500).unwrap();    // same holder renews
        assert_eq!(r.holder, 1);
        assert_eq!(r.expires_at, 1900);
        assert!(r.fence > a.fence);                     // every grant gets a new token
    }

    #[test]
    fn stale_fence_is_rejected() {
        let mut highest = 0;
        assert!(LeaseTable::check_fence(&mut highest, 5));  // fresh holder accepted
        assert!(!LeaseTable::check_fence(&mut highest, 4)); // stale holder deposed
        assert!(!LeaseTable::check_fence(&mut highest, 5)); // replay also rejected
        assert!(LeaseTable::check_fence(&mut highest, 6));  // next holder proceeds
    }
}
```

10.13.3 Exponential Backoff with Full Jitter

```rust
/// Retry schedule: attempt k waits random(0, min(cap, base * 2^k)).
/// The exponential mean gives a struggling dependency room to recover;
/// full jitter spreads a fleet of simultaneous failures across the whole
/// window so they don't re-stampede in lockstep (Section 10.9).
pub struct Backoff {
    pub base_ms: u64,
    pub cap_ms: u64,
    pub max_attempts: u32,
}

impl Backoff {
    /// splitmix64 over (job, attempt): a deterministic, decorrelated
    /// jitter source — reproducible per job, no shared RNG state.
    fn jitter(&self, job: u64, attempt: u32) -> u64 {
        let mut x = job
            .wrapping_mul(0x9E3779B97F4A7C15)
            .wrapping_add(attempt as u64);
        x = (x ^ (x >> 30)).wrapping_mul(0xBF58476D1CE4E5B9);
        x = (x ^ (x >> 27)).wrapping_mul(0x94D049BB133111EB);
        x ^ (x >> 31)
    }

    /// Delay before retry `attempt` (0-based). None = budget exhausted,
    /// send the run to the dead-letter queue.
    pub fn delay(&self, job: u64, attempt: u32) -> Option<u64> {
        if attempt >= self.max_attempts { return None; }
        let exp = self.base_ms.saturating_mul(1u64 << attempt.min(20));
        let ceiling = exp.min(self.cap_ms);
        Some(self.jitter(job, attempt) % (ceiling + 1))
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn backoff_grows_then_caps() {
        let b = Backoff { base_ms: 100, cap_ms: 10_000, max_attempts: 8 };
        for attempt in 0..8u32 {
            let d = b.delay(42, attempt).unwrap();
            let ceiling = (100u64 << attempt).min(10_000);
            assert!(d <= ceiling); // within this attempt's window
        }
        assert_eq!(b.delay(42, 8), None); // exhausted -> dead-letter
    }

    #[test]
    fn jitter_stays_in_bounds_and_varies() {
        let b = Backoff { base_ms: 1000, cap_ms: 60_000, max_attempts: 10 };
        assert_eq!(b.delay(7, 2), b.delay(7, 2)); // deterministic per (job, attempt)
        let first = b.delay(0, 3).unwrap();
        let mut varies = false;
        for job in 0..1000u64 {
            let d = b.delay(job, 3).unwrap();
            assert!(d <= 8000);          // hard bound: base * 2^3
            if d != first { varies = true; }
        }
```

```
        assert!(varies);                // the fleet actually spreads
    }
}
```

10.13.4 DAG Layers (Kahn's Algorithm)

```rust
use std::collections::{BTreeMap, BTreeSet, VecDeque};

/// Execution layers of a job DAG: layer 0 = roots (no dependencies),
/// layer k = jobs whose dependencies all sit in earlier layers. Jobs
/// within a layer are independent and run in parallel; layers run in
/// order. `deps` are (job, depends_on) pairs.
///
/// Returns None on a cycle or an unknown job: a malformed workflow must
/// fail loudly at submission time, never hang silently at runtime.
pub fn layers(jobs: &[u64], deps: &[(u64, u64)]) -> Option<Vec<Vec<u64>>> {
    let mut indegree: BTreeMap<u64, usize> = jobs.iter().map(|&j| (j, 0)).collect();
    let mut children: BTreeMap<u64, Vec<u64>> = BTreeMap::new();
    for &(job, dep) in deps {
        *indegree.get_mut(&job)? += 1;        // unknown job -> reject
        children.entry(dep).or_default().push(job);
    }

    // Kahn's algorithm: repeatedly peel the zero-indegree frontier.
    let mut frontier: VecDeque<u64> = indegree
        .iter()
        .filter(|(_, &d)| d == 0)
        .map(|(&j, _)| j)
        .collect();
    let mut out: Vec<Vec<u64>> = Vec::new();
    let mut placed = 0usize;

    while !frontier.is_empty() {
        let layer: BTreeSet<u64> = frontier.drain(..).collect();
        placed += layer.len();
        let mut next = Vec::new();
        for &j in &layer {
            for &c in children.get(&j).into_iter().flatten() {
                let d = indegree.get_mut(&c).unwrap();
                *d -= 1;
                if *d == 0 { next.push(c); }  // all dependencies satisfied
            }
        }
        out.push(layer.into_iter().collect());
        next.sort_unstable();
        next.dedup();
        frontier = next.into_iter().collect();
    }

    if placed == jobs.len() { Some(out) } else { None } // unplaced => cycle
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn diamond_runs_in_three_layers() {
        //     1
```

```
        //    / \
        //   2   3      layer 2 = {2, 3} runs in parallel
        //    \ /
        //     4
        let jobs = vec![1, 2, 3, 4];
        let deps = vec![(2, 1), (3, 1), (4, 2), (4, 3)];
        let l = layers(&jobs, &deps).unwrap();
        assert_eq!(l, vec![vec![1], vec![2, 3], vec![4]]);
    }

    #[test]
    fn cycle_is_rejected_at_submission() {
        let jobs = vec![1, 2, 3];
        let deps = vec![(2, 1), (3, 2), (1, 3)]; // 1 <- 2 <- 3 <- 1
        assert_eq!(layers(&jobs, &deps), None);
    }

    #[test]
    fn unknown_dependency_is_rejected() {
        assert_eq!(layers(&[1, 2], &[(2, 99)]), None);
    }

    #[test]
    fn independent_jobs_share_layer_zero() {
        let l = layers(&[10, 20, 30], &[]).unwrap();
        assert_eq!(l, vec![vec![10, 20, 30]]);
    }
}
```

The layer view is not just validation — it **is** the workflow execution plan. The scheduler fires layer 0, and each run completion decrements its children's pending counts in the store; a child reaching zero is promoted to a normal due job. The trigger pipeline from Section 10.7 never learns that workflows exist — separation of concerns, enforced by the data model.

> **INTERVIEW TIP — Name the classics when you borrow them**
>
> Kahn's algorithm (1962) for topological layering, the hashed timing wheel (Varghese & Lauck, 1987), the Two Generals problem for exactly-once, splitmix64 for jitter — dropping the name plus the year signals that your design is assembled from proven parts, not invented under pressure. One sentence each is enough; the interviewer will either nod or lean in, and both are wins.

10.14 Scaling the Design

More timers. Sharding by `hash(job_id) mod N` scales the trigger side horizontally: 64 shards today, 1 024 tomorrow — a lease-table rebalance, not a redesign. The due-scan per shard stays a bounded range query no matter how large the job table grows, because the index, not the table size, sets the cost (Chapter 8's height-3 lesson). The in-memory heap of Section 10.13 rides in front of each shard's scan as a hot cache: the replica refills it with the next few seconds of timers per tick, so the store sees one scan per shard-second, not one per timer.

More firing rate. The dispatch queue partitions by shard and scales like Chapter 4's log; worker pools scale independently per execution class — the GPU pool and the HTTP pool share nothing but the queue. Per-tenant fairness is enforced at **enqueue** time with Chapter 3's token bucket: a tenant past its quota has its fire commands delayed, not dropped, and the splay from Section 10.5 keeps the aggregate curve flat enough that quotas rarely bite.

More regions. The job store replicates cross-region (Chapter 9's machinery would make specs available everywhere); schedulers run active-active with region-local shard ownership, so a region loss moves leases, not data. Clock discipline is the quiet dependency: schedulers reason about "now" via NTP-disciplined UTC, and fire times are always stored in UTC with the tenant's zone applied at materialization — the DST trap of Section 10.1 is a data-model decision, not a runtime surprise.

10.15 Failure Modes & Degradation

Failure	Symptom	Handling
Scheduler crash	Shard goes quiet	Lease expires ≤ 10 s; standby takes over and **rescans** — the store is the truth, the queue an accelerator, so nothing due is lost
Scheduler GC pause	Stale holder keeps dispatching after lease loss	Fencing tokens: every downstream write from the old holder is rejected (§10.8)
Queue partition	Fire commands not flowing	Schedulers keep scanning; enqueue retries with backoff; on heal, misfire policy (coalesce/replay/skip) applies per job
Worker death mid-run	Side effect maybe done, ack never sent	Visibility timeout redelivers; dedup store makes the second attempt a no-op — at-least-once + idempotency = effectively-once
Poison message	Run crash-loops workers	Attempt counter exhausts → DLQ; per-target circuit breaker pauses dispatch to the failing dependency
Clock skew	Two schedulers disagree on "now" by seconds	Fire times in UTC, NTP alerting; a job firing 2 s late is a blip, not a bug — the SLO already budgets it
DLQ flood	Operator pager fires	Alarm on inflow rate; redrive tool after the fix; the DLQ is a quarantine, not a graveyard

COMMON PITFALL — The double-fire you designed yourself

The most common self-inflicted duplicate: a scheduler that enqueues the fire command and **then** updates `next_fire_at` — crashing in between means the successor rescans, finds the job still due, and fires again. Two honest answers: (1) claim-then-enqueue — atomically advance `next_fire_at` in the same transaction as recording the fire, so a rescan sees the job already handled; (2) accept the duplicate and let the idempotency key absorb it. Production systems do both: belt at the scheduler, suspenders at the worker. Skipping both is how billing systems end up in the news.

10.16 Trade-offs & Alternatives

Choice	We picked	Alternative	Why the alternative loses (here)
Timer structure	Indexed store scan + heap cache	Hierarchical timing wheel	Wheel is O(1) but volatile; the store scan is durable truth and fast enough — rebuild-on-failover is the wheel's hidden tax
Dispatch	Pull (workers fetch)	Push (scheduler assigns)	Pull is self-balancing backpressure: a slow worker simply stops pulling; push needs a feedback loop to not drown stragglers
Ownership	Sharded leases + fencing	Single leader scheduler	A leader is a throughput ceiling and a failover event; leases make failover routine and per-shard
Execution guarantee	At-least-once + dedup	Distributed transactions	XA/2PC across queue + store + worker is operationally brittle and slow; idempotency keys are cheap and honest
Queue substrate	Durable log (Ch. 4)	Postgres `SKIP LOCKED` as queue	Fine at 10^3 msg/s, painful at 10^5; the log replays and partitions better. A fair second choice at small scale — say so
Scheduling in-app	Central scheduler service	Per-app cron / k8s CronJob	Per-app timers scatter retry, misfire, and audit policy across a thousand repos; the service exists to own the policy

10.17 Observability & SLOs

The scheduler's vital sign is **fire lag**: `now - scheduled_time` at the moment of enqueue. Everything else is a leading indicator of lag.

Signal	What it measures	Alert when
Fire lag	Enqueue time minus scheduled time, per shard	p99 > 1 s sustained: shard under-provisioned or store slow
Due-scan duration	Time per shard range scan	Rising trend: index bloat or hot shard (Ch. 8)
Lease flapping	Takeovers per hour	> a few/day: GC pauses, network instability, or too-short TTL
Queue depth	Unconsumed fire commands	Growing: workers down, or a tenant burst past quota
Dedup hit rate	Duplicates absorbed ÷ runs	Sudden rise = redelivery storm; sustained ≈ 0 is normal
Retry / DLQ inflow	Failures per minute by target	Spike by one target: dependency outage — page its owner, not the scheduler's
Worker slot utilization	Busy slots ÷ total, per pool	Sustained ≈ 100%: capacity is the lag's cause

SLOs. Fire latency: p99 ≤ 1 s for one-shot jobs, ≤ 5 s for cron. Durability: registered jobs and accepted fire commands survive any single region loss. Failover: shard ownership moves in ≤ 15 s. Visible duplicates: ≈ 0 — dedup absorbs ≥ 99.99% of at-least-once redeliveries, and the remainder are reported, not hidden.

10.18 Interview Wrap-Up

Likely follow-ups, with the shape of a strong answer.

- *"But can you give me exactly-once?"* No — and say why in one breath (the Two Generals: the worker that crashes after its side effect is indistinguishable from one that crashed before). Then give the effectively-once recipe and name the dedup store as the price.
- *"A job must run at 09:00 in the user's local zone."* Store UTC + zone; materialize `next_fire_at` against the zone's rules; declare the DST policy (skip the impossible 02:30, coalesce the doubled one). Volunteering DST before being asked is the seniority signal.
- *"Pause a whole tenant right now."* A kill-switch flag checked at **dispatch pull** and at **scan** — two enforcement points, because the queue may already hold their commands.
- *"Charge a credit card on a schedule."* The side effect leaves your blast radius: idempotency key at the payment gateway (Stripe-style), retry only on retryable errors, dead-letter fast on card-declined — retrying a decline is harassment, not reliability.
- *"Design Airflow."* This chapter plus the DAG section is Airflow's skeleton: its scheduler loop, executor pools, and dead-letter cousin (the "failed task" state with manual retry) map one-to-one onto the five tables.
- *"What breaks first at 100×?"* The single-region job store's write throughput on `next_fire_at` updates — every fire rewrites the row. Answer: shard the store along the same hash, batch the re-materialization, and consider LSM-shaped storage (Chapter 8's rival) because this workload is write-dominated.

Checklist for the whiteboard. (1) Split trigger vs firing vs execution in the first five minutes. (2) Give the exactly-once speech unprompted. (3) Put the index on `next_fire_at` and say "range scan, Chapter 8 style". (4) Draw the lease and its fencing token as separate things. (5) Retry with jitter, dead-letter with redrive. (6) Splay the top-of-hour herd. (7) Five tables, no more.

10.19 Summary & Further Reading

A distributed job scheduler is three systems wearing one trenchcoat. The **trigger side** turns 10^7 timers into a per-second range scan over a B+ tree index — durable, resumable, boring in the best way — with an in-memory heap as a hot cache and lazy tombstones for cancels. The **firing side** divides shards among fungible scheduler replicas with leases whose blind gap is closed by fencing tokens, and hands fire commands to a durable at-least-once log. The **execution side** makes at-least-once invisible: workers claim idempotency keys before any side effect, retries decay with full-jitter exponential backoff, exhausted runs quarantine in a dead-letter queue with their context intact, and workflows reduce to per-child dependency counters fed back into the same trigger pipeline. The uncomfortable truths are the design's load-bearing walls: exactly-once is impossible, so effectively-once is engineered; the top-of-hour burst, not the average, sizes the fleet, so demand is splayed flat; and every policy — misfire, retry, cancel, catch-up — is a choice the system makes explicit because the alternative is making it by accident at 3 AM.

Further reading.

- The source video: *"20: Distributed Job Scheduler — Systems Design Interview Questions with Ex-Google SWE"* (Jordan has no life): `https://www.youtube.com/watch?v=pzDwYHRzEnk`
- Varghese & Lauck — *"Hashed and Hierarchical Timing Wheels"* (1987) — the timer structure, from the source.
- Fischer, Lynch, Paterson — *"Impossibility of Distributed Consensus with One Faulty Process"* (1985), and the Two Generals parable — why exactly-once was never on the table.
- Kleppmann — *Designing Data-Intensive Applications*, the chapters on stream processing and fencing — the effectively-once argument in full.
- Airflow, Temporal, and Kubernetes CronJob documentation — three production points in this chapter's design space, with openly documented failure semantics.
- AWS Builder's Library — *"Timeouts, retries, and backoff with jitter"* — the jitter analysis this chapter's backoff listing implements.

10.20 Chapter Glossary

Term	Meaning
At-least-once / at-most-once	Delivery guarantees: retry-until-ack (duplicates) vs fire-and-forget (losses)
Backoff (exponential, full jitter)	Retry delay random(0, min(cap, base·2^k)); the randomization prevents retry stampedes
Catch-up policy	What a misfire costs: coalesce to one firing, replay every miss, or skip to next
Cron expression	Five-field recurring schedule: minute hour day-of-month month day-of-week
DAG / workflow	Directed acyclic graph of dependent jobs; cycles rejected at submission

Term	Meaning
Dead-letter queue	Quarantine for exhausted runs, with full context and a redrive path
Dedup store	Claimed idempotency keys with TTL; the effectively-once memory
Effectively-once	At-least-once transport + idempotent processing; duplicates on the wire, none visible
Fencing token	Monotonic lease stamp checked downstream; deposes stale holders
Fire lag	Enqueue time minus scheduled time — the scheduler's vital sign
Heartbeat	Periodic liveness signal; three misses ≈ dead; work reclaimed
Idempotency key	Unique operation stamp — `(job_id, scheduled_time)` — claimed before side effects
Job vs run	Durable specification vs one ephemeral execution of it
Kahn's algorithm	Peels zero-dependency nodes in layers; both the DAG validator and its execution plan
Lazy tombstone	Cancel by marking, not removing; the heap entry is skipped when popped
Lease	Time-bounded exclusive shard ownership, renewable, expiring silently
Misfire	A scheduled fire time that passed unfired — downtime, failover, or DST
Poison message	A run that crashes every worker it touches; detected, quarantined
Shard	`hash(job_id) mod N` slice of the job space; the unit of scheduler ownership
Splay / desynchronization	Spreading recurring fire times by hash to flatten the top-of-hour herd
Thundering herd	N timers or retries firing in lockstep, DDoS-ing the next layer down
Timing wheel	Ring buffer of time-slot buckets; O(1) timers, volatile memory
Two Generals problem	The impossibility of certain agreement over an unreliable channel
Visibility timeout	Queue redelivery timer for pulled-but-unacked messages; the consumer-side lease

— End of Chapter 10 · Next: Chapter 11 —

CHAPTER 11

Designing for Correctness: How ACID Transactions Work

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"Intro to ACID Database Transactions"* from the series *Systems Design Interview: 0 to 1 with Google Software Engineer* (channel: *Jordan has no life*). It is the third fundamentals deep dive, and it completes a trilogy: Chapter 8 built the storage engine's pages and indexes, this chapter builds the **correctness layer** that lets many statements act as one on top of those pages, and Chapter 9 showed what you must surrender — this chapter's guarantees — when you refuse coordination across regions. Read in that order, the three chapters are one argument: pages, then correctness, then the price of correctness. All terms are defined before use; all reference code is Rust with deterministic tests.

11.1 The Problem Statement

The interviewer draws two bank accounts and says:

"A transfer debits account A by $100 and credits account B by $100. Design the transaction layer of a relational database: let users bundle many reads and writes into one logical unit, keep that unit correct while thousands of other units run concurrently, and keep it correct when the machine crashes in the middle. Define 'correct' — precisely — and then build the machinery that enforces it."

The prompt's sting is in the last sentence. Every engineer can say "ACID"; the interview is won by the candidate who can **define** each letter as an enforceable contract, name the anomaly each isolation level admits, and point at the exact mechanism — log record, lock, version — that prevents it. This chapter builds that machinery from the crash upward.

> **DEFINITION — Transaction**
>
> A bundle of reads and writes treated as one logical unit of work: it either happens completely or not at all, it never exposes half-applied state to others, and once confirmed it is permanent. The transfer above is the canonical example — debit and credit must live or die together, because money is not allowed to be created or destroyed by a power outage.

> **DEFINITION — A — Atomicity**
>
> *All or nothing.* A transaction's writes are applied in full or not at all, even if the machine dies mid-transaction. Enforced by the log: on recovery, the engine **redoes** committed transactions and **undoes** (or discards) uncommitted ones (Section 11.7). Atomicity is a property of the write path, not of careful coding — "be careful not to crash" is not a mechanism.

DEFINITION — C — Consistency

Invariants hold. Every transaction moves the database from one valid state to another: foreign keys resolve, balances stay non-negative, uniqueness holds. The honest detail most answers miss: consistency is largely the **application's** property — the database enforces declared constraints and provides A, I, and D, but "a transfer preserves total money" is logic the transaction author must write correctly. C is the letter the machine assists and the human owns.

DEFINITION — I — Isolation

Concurrent transactions do not see each other's intermediate state. Perfect isolation (serializability) means the outcome equals **some** serial execution of the transactions — nobody can prove they ran in parallel. Perfect isolation is expensive, so the standard sells it in graded levels, each defined by which **anomalies** it forbids (Section 11.8): dirty reads, non-repeatable reads, phantoms, lost updates, write skew. Choosing a level is choosing which anomalies your application can survive.

DEFINITION — D — Durability

Once committed, never lost. When the database acknowledges a commit, the transaction's effects survive process crashes, power loss, and disk failure up to the replication factor. The mechanism is the write-ahead log flushed to stable storage **before** the ack — Chapter 8's WAL, promoted from page insurance to contractual guarantee. Durability is a statement about the log, not about the data pages — those may still be dirty in the buffer pool when we ack, and that is fine.

11.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Single node or distributed?	Single node first (Chapter 8's engine); sketch 2PC and sagas as the distributed answer
Workload?	OLTP: 10^4 txn/s, small read-modify-write transactions, a few hot rows
Which isolation default?	Justify a default; expect to defend READ COMMITTED vs snapshot isolation
Concurrency control?	Show both families: locking (2PL) and versioning (MVCC); know when OCC wins
Crash model?	Process kill and power loss; storage survives. Byzantine corruption is out of scope
Replication?	Mention streaming the WAL to replicas; deep replication design is Chapter 9's territory

NOTE — Agreed scope

1. Define the four guarantees as enforceable contracts and identify the mechanism behind each: log (A, D), locks/versions (I), constraints + application logic (C).
2. Design the **write path**: WAL with commit records, group commit, checkpoints, recovery (redo/undo) in the spirit of ARIES.
3. Design the **concurrency path**: two-phase locking with deadlock handling, MVCC snapshots, and when optimistic control beats both.

4. Enumerate the **anomaly zoo** and map isolation levels onto it — including where snapshot isolation famously stops (write skew).
5. Close with the distributed boundary: 2PC, its blocking failure mode, and sagas — and the bridge to Chapter 9's invariant boundary.
6. All reference code in Rust with deterministic tests.

11.3 Functional Requirements

1. **Transactional demarcation.** `BEGIN`, `COMMIT`, `ABORT`, with savepoints for partial rollback; every statement between them is one atomic unit.
2. **Atomic commit.** A crash at any instant leaves either all of the transaction's effects or none; recovery decides from the log alone.
3. **Durable acknowledgment.** `COMMIT` returns only after the commit record is on stable storage (and, if configured, on a replica).
4. **Selectable isolation.** Per-transaction isolation levels from READ COMMITTED to SERIALIZABLE, with documented anomaly guarantees.
5. **Concurrent correctness.** Readers never block writers and vice versa under MVCC; writers never corrupt each other under any level; deadlocks are detected and broken by victim abort.
6. **Point-in-time consistency.** A transaction reads a stable snapshot for its lifetime (under snapshot isolation), including its own writes.

11.4 Non-Functional Requirements

- **Throughput.** $\geq 10^4$ txn/s single-node for small transactions; the log append and lock table must not become the ceiling.
- **Commit latency.** p99 $\leq$ 2 ms via group commit — one flush amortized over a batch, never one flush per transaction.
- **Recovery time.** After a crash, the database accepts work again in seconds: bounded redo since the last checkpoint, no full-data scans.
- **Version hygiene.** MVCC garbage (dead versions) is collected continuously; a long-lived snapshot may pin versions, and that hazard is surfaced as a metric, not discovered as a full disk.
- **Honest degradation.** Under lock contention the system degrades into waits and rare victim aborts — never into wrong answers.

11.5 Back-of-the-Envelope: The Cost of the Four Letters

Assumptions: 10^4 txn/s; average transaction = 20 row reads + 10 row writes; a redo record $\approx$ 100 B per written row; a row $\approx$ 200 B; SSD flush $\approx$ 0.5–1 ms.

Quantity	Estimate (with assumptions)
WAL bandwidth	10^4 txn/s × 10 writes × 100 B $\approx$ **10 MB/s** of redo — append-friendly, trivial next to an SSD's hundreds of MB/s
Flush discipline	One flush per commit $\Rightarrow \leq$ 1–2k txn/s, not 10^4. **Group commit** batches 100 txns per flush: 10^4 txn/s at 100 flushes/s, + 1 ms commit latency
Write amplification	1 logical write $\Rightarrow$ WAL record + eventual data-page write + replica stream $\approx$ **3**×; checkpoints smooth the page-write side (§11.7)

Quantity	Estimate (with assumptions)
MVCC version churn	10^5 row-writes/s × 200 B ≈ **20 MB/s of new versions** ≈ 1.7 TB/day → vacuum cadence is an operational SLO, and a day-long snapshot can pin all of it
Lock manager load	10^4 txn/s × 30 lock acquisitions ≈ **3 × 10^5 ops/s** on an in-memory table — the table is never the bottleneck; **hot rows** are (Chapter 6's leaderboard keys, again)
Recovery window	Checkpoint every 5 min ⇒ redo replays ≤ 5 min of WAL ≈ 3 GB at 100+ MB/s ⇒ **back up in well under a minute**

KEY INSIGHT — The four letters are bought with four currencies

Atomicity costs log space (undo/redo images). Durability costs flush latency (amortized by group commit). Isolation costs either **waiting** (locks), **aborting** (optimistic validation), or **space** (MVCC versions) — pick your tax. Consistency costs schema constraints and careful transaction authorship. A transaction system is a price list; the interview is reading it aloud without flinching.

11.6 The Core Challenge: Two Enemies, One Log, One Schedule

A transaction layer fights two enemies that never coordinate with each other, which is exactly what makes the problem hard.

Enemy one: the crash. A machine can stop between any two instructions. The transfer transaction writes two pages — debit A, credit B — and the buffer pool (Chapter 8) may flush either, both, or neither, in any order, at any time. If the debit's page reaches disk and the credit's does not, the restart finds money destroyed. The crash does not have to be likely; at 10^4 transactions per second, "unlikely per transaction" becomes "certain this quarter".

Enemy two: concurrency. Thousands of transactions interleave on the same rows. Without control, their reads and writes interleave too — and every anomaly in Section 11.8's zoo is a specific interleaving with a name and a victim. The transfer read by a concurrent reporter mid-flight shows the bank $100 short; two concurrent increments to the same counter produce one increment.

The two enemies share one battlefield — the **order of writes** — and the design answers each with one discipline:

KEY INSIGHT — Log first, then order the interleavings

Against crashes: **never let data reach disk before its description does** — the write-ahead log. One append-only stream, ordered by arrival, holding enough to redo what committed and undo what did not; recovery is replay, and replay is idempotent. Against concurrency: **decide which interleavings are legal** — with locks that forbid them (pessimistic) or versions that route around them (optimistic/MVCC). Every transaction system in production is a combination of one answer from each list. The rest of this chapter is the menu, the prices, and four Rust skeletons that make the menu concrete.

11.7 Deep Dive: Atomicity & Durability — The Write-Ahead Log

The WAL of Chapter 8 insured individual **pages**; here it is promoted to insure **transactions**. The discipline has three rules, and everything else is optimization.

1. **Log before data.** Before any dirty data page may reach disk, every log record describing its changes must already be on stable storage. Then a crash can never leave a change on disk that the log cannot explain.
2. **Commit means flushed.** `COMMIT` appends a **commit record** and flushes the log tail to stable storage (and returns only after the flush acknowledges). The data pages may still be dirty — they can be rebuilt from the log forever. The commit record is the exact instant a transaction becomes real.
3. **Recovery is two passes.** On restart: **redo** the effects of every committed transaction (durability), **undo or discard** the effects of every uncommitted one (atomicity). The Rust listing in Section 11.13 implements the conceptual core in thirty lines.

DEFINITION — Steal / no-force buffer management

The two freedoms that make the WAL necessary and sufficient. *Steal*: the buffer pool may write a page to disk **while a transaction that dirtied it is still uncommitted** (stealing its frame) — legal because the undo information is already in the log. *No-force*: a committing transaction is **not** required to flush its data pages — legal because the redo information is in the log. Steal buys memory flexibility; no-force buys commit speed; the WAL is the price of both. The industrial-strength formulation of all of this is the ARIES protocol (log sequence numbers on every page and record, physiological logging, repeating history before undo) — same three rules, hardened.

DEFINITION — Group commit / checkpoint

Group commit amortizes the one expensive operation — the flush — over a batch: transactions queue at commit, one flush confirms them all (10^4 txn/s at 100 flushes/s from the estimation table). A *checkpoint* periodically forces all dirty pages to disk and records the point in the log up to which recovery never needs to look; it caps the redo window so crash recovery is seconds, not a full replay of history.

11.8 Deep Dive: Isolation Levels and the Anomaly Zoo

"Isolation" becomes precise only when stated negatively: which **anomalies** — named, reproducible bad interleavings — a level forbids. Learn the zoo first; the levels are just fences around subsets of it.

DEFINITION — Isolation anomaly

A specific, nameable interleaving of two or more transactions that produces a result no serial execution could produce (or that leaks uncommitted state). Anomalies are the vocabulary of correctness: "our system is buggy" is a complaint; "our system allows write skew" is a diagnosis with a known cure.

Anomaly	The interleaving	The damage
Dirty read	T1 reads a row T2 has written but not committed; T2 aborts	T1 acted on data that never officially existed
Non-repeatable read	T1 reads a row; T2 updates and commits; T1 re-reads and gets a different value	One transaction, two realities
Phantom	T1 runs a predicate query ("accounts over 1 000"); T2 inserts a matching row and commits; T1 re-runs and sees a new row	Sets change under a reader's feet
Lost update	T1 and T2 both read counter = 10, both write 11	An increment silently vanishes — Chapter 5's vote counters, unprotected

Anomaly	The interleaving	The damage
Read skew	T1 reads A, T2's transfer A→B commits, T1 reads B	T1 saw A before and B after: the $100, mid-flight, visible to no one else
Write skew	T1 and T2 each read a **set** ("doctors on call = 2") and each writes a **different** row (their own status)	Both commit; the invariant dies — snapshot isolation's famous blind spot (§11.13)

The SQL standard sells isolation as four fences, each excluding a subset of the zoo:

Level	Dirty	Non-rep.	Phantom	Lost upd.	Write skew
READ UNCOMMITTED	possible	possible	possible	possible	possible
READ COMMITTED	prevented	possible	possible	possible	possible
REPEATABLE READ (SI)	prevented	prevented	prevented	prevented	**possible**
SERIALIZABLE	prevented	prevented	prevented	prevented	prevented

> **NOTE — The names lie; the mechanisms don't**
>
> The standard's table is a floor, not a description of any real engine. PostgreSQL's REPEATABLE READ is **snapshot isolation** — phantoms prevented, lost updates prevented by first-committer-wins, write skew admitted. MySQL/InnoDB's REPEATABLE READ is **next-key-locked two-phase locking** — phantoms prevented by locking index gaps, write skew admitted unless you `SELECT ... FOR UPDATE`. PostgreSQL's SERIALIZABLE is SSI (snapshot isolation plus lightweight dangerous-structure detection), not a lockout. Asking "which engine, which mechanism, which anomalies **actually** excluded?" is the difference between reciting the standard and knowing a database.

> **DEFINITION — Snapshot isolation (SI)**
>
> The MVCC isolation level: each transaction reads from a **snapshot** — the committed state as of its start, plus its own writes — so its view never changes under it, and writes use **first-committer-wins** (two transactions writing the same row: second to commit aborts). SI forbids every anomaly in the table except **write skew**, because write skew's transactions write **different** rows — no first-committer-wins conflict exists to catch it. The cure is promotion (serialize the dangerous read-write dependency, as SSI does) or materialization (`SELECT ... FOR UPDATE` turns the read rows into write-locked ones), both demonstrated in the Rust section.

11.9 Deep Dive: Pessimistic Concurrency Control — Two-Phase Locking

The oldest working answer: before touching a row, take a lock on it, and let the lock table — not luck — decide which interleavings are legal.

> **DEFINITION — Two-phase locking (2PL); strict 2PL**

Transactions acquire **shared** locks to read and **exclusive** locks to write; shared locks coexist, exclusive locks exclude everything. The two phases: a *growing* phase in which locks are only acquired, and a *shrinking* phase in which they are only released — once a transaction releases any lock, it may take no new ones. That discipline alone guarantees conflict-serializable schedules. Production databases use *strict* 2PL, which holds all exclusive locks until commit/abort — slightly less parallelism, but no transaction ever reads uncommitted data (so no cascading aborts), and recovery stays simple.

2PL's famous failure is not incorrectness but **deadlock**: T1 holds A and waits for B; T2 holds B and waits for A; neither can move. The answers:

- **Detection**: maintain a **wait-for graph** — an edge T→U when T waits on a lock U holds — and find cycles (the Rust listing in Section 11.13 detects them by iteratively trimming transactions nobody waits on). Break a cycle by aborting the **victim**: youngest transaction, least work done, or fewest locks — anything cheap and deterministic.
- **Avoidance**: timeouts (crude, universal) or lock ordering (take locks in key order everywhere — eliminates cycles by construction, at the price of application discipline).

INTERVIEW TIP — Deadlock hygiene is an interview freebie

Three habits kill most production deadlocks, and naming them signals experience: (1) touch rows in a consistent order (primary-key order is fine); (2) keep transactions short — no network calls, no user think time inside a lock scope; (3) take the coarsest necessary lock latest in the transaction. Deadlocks are not a mystery; they are a scheduling choice you can usually design away before detection ever fires.

11.10 Deep Dive: Optimistic Control and MVCC

Locking pays for correctness with **waiting**. Two alternatives pay with other currencies.

Optimistic concurrency control (OCC). Assume conflicts are rare: run the whole transaction against private state, keeping a read set and a write set; at commit, **validate** — did any row I read change underneath me? does my write set collide with a concurrent committer? — then commit or abort. No locks held during execution, so no waits and no deadlocks; the cost is abort storms under contention (on a hot row, OCC aborts exactly the transactions 2PL would merely delay). OCC wins for read-heavy, conflict-sparse workloads; it is the wrong tool for Chapter 6's hot leaderboard keys.

DEFINITION — MVCC — multiversion concurrency control

Never overwrite a row in place; every write creates a new **version** stamped with its creator's transaction id, and readers follow **version chains** to the newest version their snapshot may see. The result is the property application developers actually want: **readers never block writers and writers never block readers** — reads are snapshot reads, writes only contend with concurrent writes to the same row (first-committer-wins). The price is space (Section 11.5's version churn) and a garbage collector (*vacuum* in PostgreSQL vocabulary) that may reclaim a version only when no live snapshot can still see it — which is why a day-old transaction pinning an ancient snapshot is an operational emergency, not a curiosity.

Serializable snapshot isolation (SSI) deserves one paragraph because it is the modern summit: run snapshot isolation as usual, but track **dangerous structures** — the read-write antidependency pairs that underlie write skew — and abort one participant when the pattern completes. Serializable outcomes at near-SI cost: PostgreSQL's SERIALIZABLE since 9.1, and the reason "serializable is too slow" is a decade out of date.

11.11 Deep Dive: The Distributed Boundary — 2PC and Sagas

Everything so far is one machine's machinery. The moment a transaction spans nodes — the transfer's two accounts live in different regions — no local log can commit it atomically, and Chapter 9's physics returns.

DEFINITION — Two-phase commit (2PC)

The distributed atomic-commit protocol. *Prepare*: the coordinator asks every participant to durably promise "I can commit" (each writes a prepared record to its own log and locks its rows). *Commit*: if all vote yes, the coordinator logs the decision and tells everyone to commit; any no (or timeout) means abort. 2PC delivers atomicity across nodes, and its flaw is structural: after voting yes, a participant is **blocked** — locked and unable to finish — until it learns the decision, so a coordinator crash mid-protocol holds locks hostage until recovery. 2PC is correct, blocking, and increasingly confined to single-cluster use; across organizations and regions it is effectively extinct.

DEFINITION — Saga

The microservice-era replacement for cross-service transactions: split the global transaction into a chain of local ACID transactions, each with a **compensating action** that undoes it ("capture payment" ↔ "refund payment"). Steps run without locks held across services; a failure triggers compensations in reverse order. Sagas trade I for availability — intermediate states are visible — which is exactly Chapter 10's workflow-compensation pattern and Chapter 9's invariant boundary restated: when coordination is unaffordable, replace atomicity with apology.

KEY INSIGHT — The trilogy's moral

Chapter 8 built pages; this chapter makes multi-statement changes correct on one node — at the price of coordination (locks, validation, a commit flush). Chapter 9 showed systems that refuse coordination and therefore **cannot** offer this chapter's guarantees across regions. CAP is the seam between them: ACID is what you can promise inside the coordination boundary; CRDTs are what you can promise outside it; and senior system design is choosing — per datum, per operation — which side of the seam each piece of state lives on.

11.12 API Design

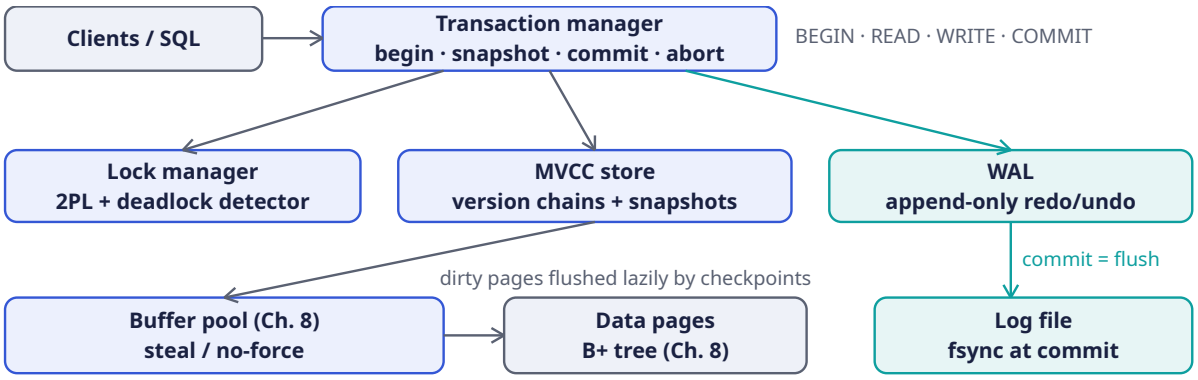
The interface is small because the semantics are heavy. Every verb below maps to exactly one mechanism from the deep dives.

Statement	Mechanism	Semantics
`BEGIN [ISOLATION LEVEL …]`	txn manager	Opens a transaction; under MVCC, takes the snapshot now; under 2PL, opens the lock scope
`SELECT …`	MVCC / S-locks	Snapshot read (no locks) or shared-lock read (strict 2PL), per engine and level
`SELECT … FOR UPDATE`	X-locks	Materializes reads as write locks — the portable write-skew vaccine
`INSERT / UPDATE / DELETE`	versions + X-locks	Writes new versions (MVCC) under exclusive locks; redo records stream to the WAL

Statement	Mechanism	Semantics
`SAVEPOINT name`	partial undo	Marks a point inside the transaction; `ROLLBACK TO` undoes back to it without aborting the whole unit
`COMMIT`	WAL + locks	Validate (OCC/SI) → append commit record → **flush** → release locks → ack
`ABORT`	undo	Discard versions / apply undo records → release locks; idempotent and always safe
`CHECKPOINT` / `VACUUM`	internal	Cap the redo window; reclaim versions no snapshot can see

11.13 High-Level Architecture

The transaction manager is the conductor; the three instruments are the lock table, the version store, and the log. Note how little of this diagram is new: the buffer pool and data pages are Chapter 8, unchanged — transactions are a **layer**, not a rebuild.



> **NOTE — Reading the diagram**
>
> A `COMMIT` traces the teal spine: transaction manager → WAL append → flush to the log file → ack. The data pages are deliberately **not** on that path — no-force means they reach disk later, at checkpoint time, via the buffer pool. That indirection is the whole performance story: the commit path is one sequential append (fast, batchable), while the random page writes it logically implies are deferred and coalesced. Recovery walks the log file, replays committed transactions into the buffer pool, and discards the rest — the diagram's two disks answer the two enemies of Section 11.6.

11.14 Rust Reference Implementations

Four pieces with deterministic tests: an MVCC store with snapshot transactions, a 2PL lock table with deadlock detection, a WAL with recovery, and — using the first — a live demonstration of write skew and its cure. Together they are Section 11.6's "one answer from each list", made executable.

11.14.1 MVCC: Snapshot Transactions

```
use std::collections::{BTreeMap, BTreeSet, HashMap};

/// One version of a key: a value plus the id of the transaction that
/// created it. Old versions are never overwritten in place — readers on
```

```rust
/// old snapshots keep walking the chain. (A real engine also tracks an
/// `end` stamp; vacuum reclaims versions no live snapshot can see.)
#[derive(Debug, Clone)]
struct Version {
    value: u64,
    begin: u64,
}

/// A minimal MVCC store providing snapshot isolation:
///  - each transaction reads the committed state as of its begin, plus
///    its own buffered writes;
///  - writers conflict via FIRST-COMMITTER-WINS on the same key.
pub struct MvccStore {
    data: BTreeMap<String, Vec<Version>>,         // versions, oldest -> newest
    committed: BTreeSet<u64>,
    active: BTreeSet<u64>,
    snapshots: BTreeMap<u64, BTreeSet<u64>>,      // txn -> active set at its begin
    pending: BTreeMap<u64, HashMap<String, u64>>, // txn -> buffered writes
    next_txn: u64,
}

impl MvccStore {
    pub fn new() -> Self {
        MvccStore {
            data: BTreeMap::new(),
            committed: BTreeSet::new(),
            active: BTreeSet::new(),
            snapshots: BTreeMap::new(),
            pending: BTreeMap::new(),
            next_txn: 0,
        }
    }

    pub fn begin(&mut self) -> u64 {
        self.next_txn += 1;
        let id = self.next_txn;
        self.snapshots.insert(id, self.active.clone()); // the snapshot
        self.active.insert(id);
        id
    }

    /// Visible to `txn` iff the version's creator committed BEFORE `txn`
    /// began: lower id, not in the snapshot (i.e. not still active at
    /// begin), and committed. Own writes bypass this via `pending`.
    fn visible(&self, v: &Version, txn: u64) -> bool {
        v.begin < txn
            && self.committed.contains(&v.begin)
            && !self.snapshots[&txn].contains(&v.begin)
    }

    pub fn read(&self, txn: u64, key: &str) -> Option<u64> {
        if let Some(v) = self.pending.get(&txn).and_then(|w| w.get(key)) {
            return Some(*v); // read your own writes
        }
        self.data.get(key)?
            .iter()
            .rev()                            // newest first
            .find(|v| self.visible(v, txn))
            .map(|v| v.value)
    }
```

```
    pub fn write(&mut self, txn: u64, key: &str, value: u64) {
        self.pending.entry(txn).or_default().insert(key.to_string(), value);
    }

    /// Commit, or abort with a write-write conflict: some *concurrent*
    /// transaction (active at my begin, or begun after me) has already
    /// committed a key I also wrote. First committer wins.
    pub fn commit(&mut self, txn: u64) -> Result<(), ()> {
        let conflict = self.pending.get(&txn).map(|writes| {
            writes.keys().any(|key| {
                self.data.get(key).and_then(|vs| vs.last()).map_or(false, |latest| {
                    let c = latest.begin;
                    c != txn
                        && (self.snapshots[&txn].contains(&c) || c > txn)
                        && self.committed.contains(&c)
                })
            })
        }).unwrap_or(false);
        if conflict {
            self.abort(txn);
            return Err(());
        }
        let writes = self.pending.remove(&txn).unwrap_or_default();
        for (k, v) in writes {
            self.data.entry(k).or_default().push(Version { value: v, begin: txn });
        }
        self.committed.insert(txn);
        self.active.remove(&txn);
        self.snapshots.remove(&txn);
        Ok(())
    }

    pub fn abort(&mut self, txn: u64) {
        self.pending.remove(&txn);   // buffered writes simply vanish
        self.active.remove(&txn);
        self.snapshots.remove(&txn);
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn snapshot_isolation_gives_repeatable_reads() {
        let mut s = MvccStore::new();
        let t0 = s.begin();
        s.write(t0, "balance", 100);
        assert!(s.commit(t0).is_ok());

        let reader = s.begin();          // snapshot taken here
        let writer = s.begin();
        s.write(writer, "balance", 200);
        assert!(s.commit(writer).is_ok()); // commits AFTER reader began

        // The reader's snapshot is frozen: same key, same answer, forever.
        assert_eq!(s.read(reader, "balance"), Some(100));
        s.abort(reader);
```

```
        let fresh = s.begin();           // a new snapshot sees the commit
        assert_eq!(s.read(fresh, "balance"), Some(200));
    }

    #[test]
    fn first_committer_wins_on_write_conflict() {
        let mut s = MvccStore::new();
        let t1 = s.begin();
        let t2 = s.begin();
        s.write(t1, "k", 1);
        s.write(t2, "k", 2);
        assert!(s.commit(t1).is_ok());   // first committer...
        assert!(s.commit(t2).is_err());  // ...wins; t2 aborts
        let t3 = s.begin();
        assert_eq!(s.read(t3, "k"), Some(1)); // no lost update, no torn value
    }
}
```

11.14.2 Two-Phase Locking with Deadlock Detection

```
use std::collections::{BTreeMap, BTreeSet};

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum Mode { Shared, Exclusive }

#[derive(Debug, Default)]
struct LockState {
    holders: BTreeMap<u64, Mode>, // txn -> mode held on this key
}

/// A non-blocking strict-2PL lock table: acquisition succeeds or reports
/// the conflict (the caller waits and retries); every conflict is also an
/// edge in the wait-for graph, where cycles are deadlocks. Real engines
/// park waiters on queues; the graph math is identical.
pub struct LockManager {
    locks: BTreeMap<String, LockState>,
    waits: BTreeMap<u64, BTreeSet<u64>>, // waiter -> holders it waits on
}

impl LockManager {
    pub fn new() -> Self {
        LockManager { locks: BTreeMap::new(), waits: BTreeMap::new() }
    }

    /// Shared locks coexist; exclusive locks exclude all but the holder's
    /// own. (Lock UPGRADE is just re-acquisition at a stronger mode.)
    fn compatible(state: &LockState, txn: u64, mode: Mode) -> bool {
        state.holders.iter().all(|(&h, &m)| {
            h == txn || (m == Mode::Shared && mode == Mode::Shared)
        })
    }

    pub fn acquire(&mut self, txn: u64, key: &str, mode: Mode) -> Result<(), ()> {
        let state = self.locks.entry(key.to_string()).or_default();
        if Self::compatible(state, txn, mode) {
            state.holders.insert(txn, mode);
            Ok(())
        } else {
            for &h in state.holders.keys() {
```

```
                if h != txn { self.waits.entry(txn).or_default().insert(h); }
            }
            Err(())
        }
    }

    /// Commit/abort releases everything (strict 2PL) and forgets the
    /// transaction's waits.
    pub fn release_all(&mut self, txn: u64) {
        for state in self.locks.values_mut() {
            state.holders.remove(&txn);
        }
        self.waits.remove(&txn);
        for edges in self.waits.values_mut() {
            edges.remove(&txn);
        }
    }

    /// Deadlock = a cycle in the wait-for graph. Iteratively remove txns
    /// nobody waits on (they can finish, so they can't be stuck); what
    /// remains is deadlocked or doomed behind the deadlocked.
    pub fn deadlock(&self) -> Option<Vec<u64>> {
        let mut g: BTreeMap<u64, BTreeSet<u64>> = BTreeMap::new();
        for (w, holders) in &self.waits {
            for h in holders {
                g.entry(*w).or_default().insert(*h);
                g.entry(*h).or_default();
            }
        }
        loop {
            let mut indeg: BTreeMap<u64, usize> = g.keys().map(|&k| (k, 0)).collect();
            for outs in g.values() {
                for o in outs {
                    if let Some(d) = indeg.get_mut(o) { *d += 1; }
                }
            }
            let free: Vec<u64> = indeg.iter()
                .filter(|(_, &d)| d == 0)
                .map(|(&n, _)| n)
                .collect();
            if free.is_empty() { break; }
            for f in free { g.remove(&f); }
        }
        if g.is_empty() { None } else { Some(g.keys().copied().collect()) }
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn shared_locks_coexist_and_exclusive_waits() {
        let mut lm = LockManager::new();
        assert!(lm.acquire(1, "row", Mode::Shared).is_ok());
        assert!(lm.acquire(2, "row", Mode::Shared).is_ok());
        assert!(lm.acquire(3, "row", Mode::Exclusive).is_err()); // must wait
        assert_eq!(lm.deadlock(), None); // a wait chain is not a cycle
    }
```

```
    #[test]
    fn deadlock_is_detected_and_broken_by_victim_abort() {
        let mut lm = LockManager::new();
        assert!(lm.acquire(1, "a", Mode::Exclusive).is_ok());
        assert!(lm.acquire(2, "b", Mode::Exclusive).is_ok());
        assert!(lm.acquire(1, "b", Mode::Exclusive).is_err()); // 1 waits on 2
        assert!(lm.acquire(2, "a", Mode::Exclusive).is_err()); // 2 waits on 1
        let stuck = lm.deadlock().unwrap();
        assert!(stuck.contains(&1) && stuck.contains(&2));

        lm.release_all(2);                 // victim abort
        assert_eq!(lm.deadlock(), None);
        assert!(lm.acquire(1, "b", Mode::Exclusive).is_ok()); // survivor proceeds
    }
}
```

11.14.3 The WAL and Recovery

```
use std::collections::{BTreeMap, BTreeSet};

/// A write-ahead log record. The discipline is the contract: a record
/// reaches stable storage BEFORE the data change it describes, and
/// COMMIT returns only after this log is flushed.
#[derive(Debug, Clone, PartialEq, Eq)]
enum Record {
    Begin(u64),
    Set(u64, String, u64), // txn, key, value — the redo image
    Commit(u64),
    Abort(u64),
}

pub struct Wal {
    records: Vec<Record>, // production: an append-only file, fsync'd per group commit
}

impl Wal {
    pub fn new() -> Self { Wal { records: vec![] } }

    pub fn append(&mut self, r: Record) { self.records.push(r); }

    /// Replay after a crash. Two passes over one stream: find every
    /// committed transaction, then redo exactly their writes. Committed
    /// work survives (durability); uncommitted work vanishes (atomicity).
    /// ARIES adds page-level LSNs and an undo pass; this is the moral core.
    pub fn recover(&self) -> BTreeMap<String, u64> {
        let mut committed = BTreeSet::new();
        for r in &self.records {
            if let Record::Commit(t) = r { committed.insert(*t); }
        }
        let mut state = BTreeMap::new();
        for r in &self.records {
            if let Record::Set(t, k, v) = r {
                if committed.contains(t) {
                    state.insert(k.clone(), *v);
                }
            }
        }
        state
    }
```

```
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn committed_work_survives_the_crash() {
        let mut wal = Wal::new();
        wal.append(Record::Begin(1));
        wal.append(Record::Set(1, "alice".into(), 100));
        wal.append(Record::Set(1, "bob".into(), 50));
        wal.append(Record::Commit(1));
        let state = wal.recover();
        assert_eq!(state.get("alice"), Some(&100));
        assert_eq!(state.get("bob"), Some(&50));
    }

    #[test]
    fn uncommitted_work_vanishes() {
        let mut wal = Wal::new();
        wal.append(Record::Begin(1));
        wal.append(Record::Set(1, "alice".into(), 100));
        wal.append(Record::Commit(1));
        wal.append(Record::Begin(2));
        wal.append(Record::Set(2, "alice".into(), 999)); // crash: no Commit(2)
        let state = wal.recover();
        assert_eq!(state.get("alice"), Some(&100)); // txn 2 never happened
    }

    #[test]
    fn aborted_work_is_skipped_even_though_logged() {
        let mut wal = Wal::new();
        wal.append(Record::Begin(1));
        wal.append(Record::Set(1, "k".into(), 1));
        wal.append(Record::Abort(1));
        assert_eq!(wal.recover().get("k"), None);
    }
}
```

11.14.4 Write Skew, Demonstrated and Cured

Snapshot isolation's blind spot, reproduced against the MVCC store above (same crate). The setup is the classic on-call rota: the invariant is "at least one doctor is on call"; each doctor's transaction reads **both** rows and writes **only their own** — no write-write conflict exists for first-committer-wins to catch.

```
// uses MvccStore from the MVCC listing (same crate)

#[test]
fn write_skew_survives_snapshot_isolation() {
    let mut s = MvccStore::new();
    let t0 = s.begin();
    s.write(t0, "alice", 1); // 1 = on call
    s.write(t0, "bob", 1);
    s.commit(t0).unwrap();

    let ta = s.begin();
    let tb = s.begin();
```

```
    // Both doctors check the rota: two on call, safe to step away.
    assert_eq!(s.read(ta, "alice"), Some(1));
    assert_eq!(s.read(ta, "bob"), Some(1));
    assert_eq!(s.read(tb, "alice"), Some(1));
    assert_eq!(s.read(tb, "bob"), Some(1));
    // Each takes THEMSELVES off call — different keys, no conflict.
    s.write(ta, "alice", 0);
    s.write(tb, "bob", 0);
    assert!(s.commit(ta).is_ok());
    assert!(s.commit(tb).is_ok()); // SI allows it...

    let check = s.begin();
    assert_eq!(s.read(check, "alice"), Some(0));
    assert_eq!(s.read(check, "bob"), Some(0)); // ...and nobody is on call.
}

#[test]
fn materializing_the_read_catches_the_skew() {
    // The `SELECT ... FOR UPDATE` cure: each transaction also WRITES the
    // row it depends on, turning the read-write dependency into a
    // write-write conflict that first-committer-wins can see.
    let mut s = MvccStore::new();
    let t0 = s.begin();
    s.write(t0, "alice", 1);
    s.write(t0, "bob", 1);
    s.commit(t0).unwrap();

    let ta = s.begin();
    let tb = s.begin();
    s.write(ta, "alice", 0);
    s.write(ta, "bob", 1);   // "I depend on bob being on call"
    s.write(tb, "alice", 1); // "I depend on alice"
    s.write(tb, "bob", 0);
    assert!(s.commit(ta).is_ok());
    assert!(s.commit(tb).is_err()); // conflict on both keys -> abort

    let check = s.begin();
    assert_eq!(s.read(check, "bob"), Some(1)); // invariant saved
}
```

COMMON PITFALL — Testing the happy path only

Notice what the first test proves: **the invariant died with every operation returning success.** No error, no conflict, no log line — SI did exactly what it promises, and the rota still emptied. Correctness bugs at the isolation layer are silent; the only defenses are choosing the level with the anomaly table in hand, materializing reads that carry invariants, and writing tests (like these) that execute the dangerous interleaving on purpose. If your test suite only ever runs one transaction at a time, it has tested none of this.

11.15 Scaling the Design

Single-node transaction throughput scales up (bigger buffer pools, faster flushes, more parallel group-commit batches) until the write wall arrives; then the levers are structural.

Read scale is easy, write scale is earned. Replicas stream the WAL and serve snapshot reads — Chapter 9's machinery carrying this chapter's log — but every write still funnels through one log tail. Sharding writes by key range (Chapter 5's approach) splits the log per shard-group; the price is

that cross-shard transactions now need 2PC or must be designed away by **co-locating** what transacts together (account pairs on the same shard — the transfer example is deliberately co-locatable).

Contention, not capacity, is usually the ceiling. A hot row — the merchant's settlement account in the morning, Chapter 6's top leaderboard entry — serializes every transaction that touches it, whatever the isolation level. The playbook: batch counter increments through intermediate rows and sum at read; shorten transactions so locks are held for microseconds; move truly hot aggregates out of the transactional path entirely (Chapter 6's leaderboard does not need serializability per score).

Version GC is a throughput feature. Vacuum that falls behind turns every table into a history of itself: indexes bloat (Chapter 8's fill factor), snapshots walk longer chains, and the buffer pool fills with garbage. Autovacuum tuning is unglamorous and load-bearing.

11.16 Failure Modes & Degradation

Failure	Symptom	Handling
Crash mid-transaction	Partial effects possibly on disk	Recovery: redo committed, discard uncommitted — the WAL decides, pages never lie (§11.7)
Crash after flush, before ack	Client unsure whether commit landed	Client retries with its idempotency key (Chapter 10's rule); the transaction either is or isn't in the log — never half
Deadlock	Two txns frozen	Wait-for cycle detected; victim aborted with a retryable error; app retries
Long-lived snapshot	Version bloat, vacuum blocked	Metric + alert on snapshot age; statement timeouts; kill the session, not the server
Lock convoy on a hot row	Throughput collapses to serial	Shorten txns; batch via intermediate rows; move the aggregate out of the transactional path
Log disk full	All commits block — total write outage	The WAL is the single point of **write** failure: alert early, checkpoint diligently, never let it fill
Replica lag	Stale reads on replicas	Monotonic-read tokens (Ch. 9's contexts) or route reads-after-writes to the leader

COMMON PITFALL — Retrying without idempotency

The commit-ack ambiguity row deserves emphasis because it bites weekly in production: the log flushed, the network dropped the ack, and the client genuinely cannot know whether its transfer exists. Retrying blindly risks double-charging; not retrying risks losing a payment. The only correct shape is Chapter 10's: the client owns an idempotency key, the retry is safe, and the ambiguity is absorbed by the dedup table. Transactions make **the database** correct; end-to-end correctness is a protocol between client and server.

11.17 Trade-offs & Alternatives

Choice	We picked	Alternative	When the alternative wins
Concurrency control	MVCC (snapshot isolation)	Strict 2PL / OCC	2PL: strong consistency semantics with simpler reasoning; OCC: conflict-sparse, read-mostly workloads where aborts stay rare
Default isolation	READ COMMITTED or SI	SERIALIZABLE everywhere	Hot, invariant-carrying paths justify SSI's abort cost; most web workloads never see an anomaly at RC
Durability	Local flush + replica ack	Local flush only / fully synchronous multi-region	Latency budget vs. RPO=0 across regions; Chapter 9 prices the far end of this dial
Distributed atomicity	Avoid; co-locate + sagas	2PC across services	Single-cluster, short-lived, coordinator-protected transactions — 2PC's safe habitat
Write path	Steal/no-force + WAL	Force-at-commit (no redo)	Tiny embedded engines where simplicity beats commit latency

KEY INSIGHT — Isolation is a product decision, not a database default

Every isolation level is a price-performance point on the correctness curve, and the right answer depends on what an anomaly **costs the business**. A social feed tolerates phantoms; a ledger does not tolerate anything. The senior move is per-workload selection: SI for the OLTP core, SERIALIZABLE (or materialized reads) exactly where invariants carry money, READ COMMITTED where speed matters and anomalies don't — and a written note, in the design doc, saying which anomalies each service has agreed to live with.

11.18 Observability & SLOs

Signal	What it measures	Alert when
Commit latency	Time from COMMIT to ack (group-commit batching)	p99 > 2 ms: flush path slow or batches starving
WAL flush rate / backlog	Log throughput vs. generation	Backlog growing: disk saturated — the write ceiling
Lock waits	Time blocked per txn; wait-for edge count	P95 wait climbing: contention shift or a new hot row
Deadlocks / aborts	Victim aborts, SI conflicts per minute	Spike = a deploy changed access order; SSI abort storms on hot paths

Signal	What it measures	Alert when
Snapshot age	Oldest active snapshot	> minutes: vacuum is pinned, bloat incoming — page before the disk fills
Recovery drill	Time-to-open after kill −9	Practiced, not theoretical: measured in game days

SLOs. Commit latency p99 ≤ 2 ms (group commit); recovery ≤ 60 s to accepting writes; zero committed-data loss at single-node fault (RPO = 0 with replica ack); deadlock resolution ≤ 100 ms from formation; snapshot age ≤ 5 min at p99. Every one of these is a mechanism from this chapter plus a number.

11.19 Interview Wrap-Up

Likely follow-ups, with the shape of a strong answer.

- *"Which ACID letter is not the database's job?"* C. The database enforces constraints and provides A, I, D; the invariant "money is conserved" is authored by the transaction writer. Candidates who say this unprompted stand out immediately.
- *"Default isolation for a new service?"* READ COMMITTED or SI, chosen by naming the anomalies you are accepting — the table in Section 11.8 is the answer, not a footnote.
- *"PostgreSQL vs MySQL REPEATABLE READ?"* SI vs next-key-locked 2PL: same name, different anomalies excluded. Know one engine's mechanism deeply rather than both engines' marketing.
- *"Why is 2PC dying?"* The blocking window: a coordinator crash holds participants' locks hostage until recovery. Inside one cluster with a protected coordinator it is fine; across services it is a reliability anti-pattern — sagas won.
- *"Speed up commits?"* Group commit; then steal/no-force (already assumed); then async commit with an explicit durability downgrade, said out loud.
- *"How does this square with Chapter 9?"* ACID lives inside the coordination boundary; CRDTs outside it; choose per datum. The interviewer's favorite systems answer in 2026 is this sentence.

Checklist for the whiteboard. (1) Define each letter as a contract with its mechanism: log (A, D), locks/versions (I), constraints (C). (2) Draw the commit path through the WAL and say "commit = flush". (3) Recite three anomalies and the level that kills each. (4) Know SI's blind spot and both cures. (5) Sketch deadlock detection as a graph problem. (6) For distributed, refuse 2PC across services and offer sagas. (7) Mention one operational war story: snapshot age, log-full, or lock convoy.

11.20 Summary & Further Reading

A transaction layer is two disciplines answering two enemies. Against **crashes**, the write-ahead log: describe before you write, commit means flushed, recovery is replay — redo the committed, discard the rest, and let steal/no-force buffer management keep commits fast while checkpoints keep recovery short. Against **concurrency**, a legal-interleaving discipline: strict two-phase locking with deadlock detection, optimistic validation when conflicts are rare, or MVCC version chains that let readers and writers pass without blocking — each paying its tax in waits, aborts, or space. Isolation is sold in levels defined by the anomalies they forbid, and the map has exactly one trap door left open by the industry's favorite level: snapshot isolation admits write skew, cured by materializing the reads or by SSI's dangerous-structure detection. Past the single node, 2PC extends atomicity at the price of blocking, sagas replace it with compensation, and Chapter 9's boundary reappears on cue: ACID is what coordination buys; the art is knowing which data can afford it. Chapter 8 supplied the pages; this chapter supplied the promises.

Further reading.

- The source video: *"Intro to ACID Database Transactions — Systems Design Interview: 0 to 1 with Google Software Engineer"* (Jordan has no life): `https://www.youtube.com/watch?v=oGmxzUBCYtY`
- Gray & Reuter — *Transaction Processing: Concepts and Techniques* — the field's foundational text; the lock manager chapter alone justifies the shelf space.
- Mohan et al. — *"ARIES: A Transaction Recovery Method"* (1992) — the recovery protocol every serious engine implements.
- Adya — *"Weak Consistency: A Generalized Theory and Optimistic Implementations for Distributed Transactions"* (1999) — the anomaly taxonomy done rigorously.
- Berenson et al. — *"A Critique of ANSI SQL Isolation Levels"* (1995) — why the standard's names do not mean what engines ship.
- Cahill et al. — *"Serializable Snapshot Isolation"* (2008) — the PostgreSQL 9.1 implementation paper; serializable without the lockout.
- PostgreSQL and MySQL/InnoDB documentation on their isolation implementations — the same names, two different machines.

11.21 Chapter Glossary

Term	Meaning
ACID	Atomicity, Consistency, Isolation, Durability — the four contracts of a transaction
ARIES	The industrial recovery protocol: LSNs everywhere, repeat history, then undo
Atomicity	All-or-nothing execution; enforced by the log at recovery time
Checkpoint	Periodic dirty-page flush + log marker capping the redo window
Commit record	The log entry whose flush makes a transaction real
Consistency (C)	Invariants preserved; enforced by constraints, authored by the application
Deadlock	A cycle in the wait-for graph; detected, then broken by victim abort
Dirty read	Reading another transaction's uncommitted write
Durability	Committed survives crashes; a property of the log, not the pages
First-committer-wins	SI write-conflict rule: second committer on a contested row aborts
Group commit	One flush confirming a batch of commits; the throughput unlock
Isolation	Concurrency invisibility; sold in levels defined by forbidden anomalies
Lost update	Two read-modify-writes clobbering each other; increments vanish
MVCC	Multiversion concurrency control: writers append versions, readers walk snapshots
Non-repeatable read	Re-reading a row yields a new committed value mid-transaction
OCC	Optimistic control: run unlocked, validate at commit, abort on conflict
Phantom	A predicate query re-run sees newly committed matching rows

Term	Meaning
Read skew	A transaction sees pre-transfer A and post-transfer B — an impossible mix
Redo / undo	Replaying committed changes / erasing uncommitted ones, both from the log
Saga	Chained local transactions with compensating actions; atomicity traded for availability
Savepoint	An in-transaction marker allowing partial rollback
Snapshot isolation (SI)	Read a begin-time snapshot; first-committer-wins on writes; admits write skew
SSI	Serializable SI: detect dangerous read-write structures, abort a participant
Steal / no-force	Pages flushable while uncommitted / not forced at commit — the WAL's license
Strict 2PL	Two-phase locking with exclusive locks held to commit; no dirty reads, no cascading aborts
Two-phase commit (2PC)	Prepare + commit across nodes; atomic, blocking, coordinator-fragile
Vacuum	MVCC garbage collection of versions no live snapshot can see
WAL	Write-ahead log: the append-only stream every change is described in first
Wait-for graph	Edges waiter→holder; cycles are deadlocks
Write skew	Two txns read a shared set, write different rows, kill the invariant — SI's blind spot

— End of Chapter 11 · Next: Chapter 12 —

CHAPTER 12

12

Designing a Ride-Hailing Marketplace (Uber / Lyft)

> **NOTE — Problem source**
>
> This chapter solves the problem posed in the talk *"10: Design Uber/Lyft"* from the series *Systems Design Interview Questions with Ex-Google SWE* (channel: *Jordan has no life*). It is a full product design in the shape of Chapters 1–7 and 10, and a reunion tour of the fundamentals chapters: geospatial indexing from Chapter 2, idempotent payments from Chapters 10–11, real-time push from Chapter 1, and the per-city placement decisions from Chapter 9. The problem is the definitive **marketplace** interview: two sides (riders, drivers), one scarce resource (nearby idle drivers), and a clock that never stops. All terms are defined before use; all reference code is Rust with deterministic tests.

12.1 The Problem Statement

The interviewer drops a pin on a map and says:

"Design Uber. A rider requests a trip; the system finds a nearby driver, quotes a price and ETA, coordinates pickup and the ride itself, charges the rider, pays the driver — in real time, in hundreds of cities, while a million drivers move around updating their location every few seconds."

What makes this problem a classic is that it is **four** systems in a trenchcoat: a high-throughput location pipeline, a geospatial search engine over moving points, a real-time marketplace matcher, and a pedestrian transactional backend for trips and money. The interview is won by keeping those four cleanly separated — and by knowing which of them is actually hard (it is not the one candidates expect).

> **DEFINITION — Rider / driver / trip / marketplace**
>
> The two sides and the unit of work. *Riders* request transport; *drivers* supply it; a *trip* is one fulfilled request with a lifecycle (requested → matched → in progress → completed). The system is a *marketplace*: it does not own the supply, it **clears** it — matching demand to supply fast enough that neither side leaves, and pricing to keep the market balanced when it isn't (surge, Section 12.10).

> **DEFINITION — ETA / deadheading / utilization**
>
> *ETA* is the estimated time of arrival — of the driver to the pickup (pickup ETA) and of the trip (dropoff ETA) — produced by the routing engine of Chapter 2 over the live road graph. *Deadheading* is driving without a passenger: pure cost. *Utilization* is the fraction of driver hours with a rider in the car — the marketplace's core efficiency metric, and the number the matching engine (Section 12.8) silently optimizes.

12.2 Scope & Clarifying Questions

Candidate asks	Interviewer answers
Scale?	10^6 drivers online globally, 2×10^7 rides/day, one metro is the unit of operation
Geography?	Cities are independent shards; cross-city trips are rare and can be treated as an edge case
Matching quality?	Fast beats perfect: a good match in 2 s outperforms an optimal match in 30 s
Pricing?	Dynamic (surge) per area; quote shown up front and honored
Shared rides?	Out of scope for v1 — mention it as the natural extension (multi-stop matching)
Payments?	Card on file; charge at trip end; idempotency is non-negotiable (Chapters 10–11)
Real-time UX?	Rider sees the driver's pin move; push over WebSocket (Chapter 1's channel)

NOTE — Agreed scope

1. The **location pipeline**: ingest 2.5×10^5 driver position updates per second and keep a queryable, in-memory picture of "who is where".
2. The **geospatial index**: nearby-available-driver queries over points that never stop moving.
3. The **matching engine**: rider-to-driver assignment, offer/accept protocol, batched matching windows, idle-supply positioning.
4. The **trip backbone**: lifecycle state machine, idempotent payments, ratings — the transactional 20% that must never be wrong.
5. **Surge pricing** as a market-balancing control loop, with its failure modes designed in, not discovered.
6. Out of scope: pooling/shared rides, routing engine internals (Chapter 2), maps data pipelines (Chapter 2), driver onboarding.

12.3 Functional Requirements

1. **Location tracking.** Drivers publish position every 4 s while online; the system maintains a fresh, queryable picture per city; stale positions expire automatically.
2. **Quotes.** Given pickup and destination, return an upfront price and a pickup/dropoff ETA before the rider commits.
3. **Request & match.** A ride request enters matching immediately; an offered driver has 15 s to accept; on decline or timeout the request re-enters matching without rider involvement.
4. **Trip lifecycle.** Requested → matched → arriving → in progress → completed/cancelled, as an explicit state machine; illegal transitions are impossible by construction (Section 12.13).
5. **Payment.** At trip end, charge the rider's card once — exactly once as far as the rider can ever observe — and credit the driver's balance.
6. **Real-time status.** Both apps stream trip state and the driver's live position; polling is the fallback, not the primary channel.
7. **Surge.** Prices respond to per-area supply/demand within a minute, with smoothing and caps.

12.4 Non-Functional Requirements

- **Match latency.** p99 ≤ 5 s from request to driver-accepted in a healthy market; the matching engine is never allowed to be the bottleneck that leaves riders staring at a spinner.
- **Location freshness.** The geo index reflects a driver's position within 5 s end-to-end; anything staler is treated as offline.
- **Availability asymmetry.** Losing **matching** for a minute is a degraded minute; losing **trip records or payments** is a breach — the transactional backbone out-availability-targets the real-time side.
- **City isolation.** A city is a failure domain: New York's New Year's Eve must not move Lagos's Tuesday morning.
- **Correctness under retries.** Every client-visible effect — requests, cancels, charges — is idempotent, because every mobile client retries on every elevator ride.

12.5 Back-of-the-Envelope: A Million Moving Dots

Assumptions: 10^6 drivers online at global peak; position update every 4 s; 2×10^7 rides/day; 5×10^6 concurrent riders at peak; trip record ≈ 2 KB; city-sharded.

Quantity	Estimate (with assumptions)
Location ingest	10^6 drivers / 4 s ≈ **2.5×10^5 updates/s**, 100 B each → 25 MB/s — a Chapter 4 append stream, city-partitioned
Nearby-driver queries	Rider app polls + matcher candidate fetches ≈ **5×10^4 reads/s** against the geo index — in-memory, so this is CPU, not I/O
Geo index memory	10^6 drivers × 48 B (id, cell, freshness) ≈ **50 MB per region** — fits in cache ten times over; the index is cheap, the **churn** is the story
Match rate	2×10^7 rides/day ≈ 230/s average → **1.2×10^3 matches/s at evening peak**; with 2 s windows, 2 400 riders and 10^4 idle drivers per window globally
Trip storage	2×10^7 trips/day × 2 KB ≈ **40 GB/day** → 3.6 TB for a 90-day hot window — Chapter 8's indexed store, partitioned by city and day
Push fanout	10^5 active trips × 2 devices × 1 update/5 s ≈ **4×10^4 msgs/s** — Chapter 1's WebSocket fanout, modest by its standards
Payment volume	Peak ≈ 1.2×10^3 charges/s — trivial for the payments stack; the requirement is correctness (idempotency), not throughput

KEY INSIGHT — Read the table and notice what is not hard

Nothing here is big data. Fifty megabytes of live driver positions, forty gigabytes of trip records a day, a few thousand matches a second — a single modern machine could nearly run the whole thing. The genuine difficulty is the **physics of the problem**: positions expire in seconds, a match decision locks a moving 15-minute resource, riders abandon after 10 s of spinner, and the two sides of the market react to each other with feedback loops. This is a **liveness and correctness** design, not a capacity design — budget your interview time accordingly.

12.6 The Core Challenge: A Marketplace That Refuses to Sit Still

Strip the product away and three structural difficulties remain — none of them capacity.

1. **The index never rests.** A geospatial index over 10^6 points that each move every 4 seconds is not a structure you build; it is a structure you **maintain at 2.5×10^5 mutations per second**. B-trees

and R-trees answer "where is this" beautifully and "it moved again" badly; the answer is a flat, mutable, in-memory cell index where moving is an O(1) re-bucketing (Section 12.7).
2. **Matching is a decision under expiry.** A rider waits seconds; an idle driver is claimed by the next request; the road network keeps score. The matcher must trade "best possible assignment" against "assignment before the rider abandons" — a classic online decision problem, solved by batching into short windows (Section 12.8).
3. **The transactional tail must be perfect.** Requests, cancels, charges: twenty percent of the traffic and a hundred percent of the trust. A double charge is a support ticket and a regulator; a lost trip record is a safety incident. This side of the system buys Chapter 11's guarantees and pays for them gladly.

> **KEY INSIGHT — One city, one authority**
>
> The cleanest architectural decision in the whole design: **a trip is owned by exactly one city's stack.** Geographically partitioned, region-local writes, no cross-region consensus on the hot path — the trip state machine and its store live where the trip physically is. Chapter 9's physics is respected by making the problem almost never cross the boundary: riders and drivers move within a city; the rare cross-border trip is handed off between city stacks like a roaming call. Strong consistency where it is cheap, no coordination where it would hurt.

12.7 Deep Dive: Geospatial Indexing for Moving Points

The geo index answers one question, 5×10^4 times a second: **which available drivers are within a few hundred meters of this pickup?** Two candidate families exist; the moving-point constraint picks the winner.

R-trees and friends (Chapter 2's structures) give exact spatial answers but punish mutation: every driver move is a delete-plus-insert through a balanced tree, and at 2.5×10^5 moves/s the maintenance cost dominates. **Grid cells** give up exactness for churn-tolerance: assign each driver to a cell; a query reads a handful of cells; a move updates two hash-map entries. Exactness is recovered by post-filtering the (small) candidate set by true distance — cells are for **recall**, distance math is for **precision**.

> **DEFINITION — Geohash**
>
> A string encoding of a grid cell made by interleaving longitude and latitude bits, five bits per base-32 character. Its superpower is the **prefix property**: a cell's code is a prefix of every cell inside it, so "within this area" becomes `starts_with` — string prefix matching as a spatial operator. Longer codes are smaller cells (6 characters ≈ a 1.2 km × 0.6 km cell; 7 ≈ 150 m × 150 m). The Rust listing in Section 12.13 implements encode/decode in fifty lines. Production systems often use hexagonal successors (Uber's H3, Google's S2) whose cells have uniform neighbor distances and no pole weirdness; the interview-level mechanics are identical.

The query algorithm is ring expansion over prefixes: look in the pickup's own cell; if fewer candidates than needed, look in the parent prefix (a cell 4× wider); expand again to a grandparent; then post-filter by straight-line distance and rank by ETA from the routing engine. Two refinement notes that earn interview points: a driver near a cell **boundary** can be missed by pure prefix expansion, so production reads the 8 neighboring cells at each level as well; and cells must be sized to the market — Manhattan at lunch needs 150 m cells, rural Wyoming needs 5 km ones, so precision adapts per city and per hour.

The churn side is where the design earns its keep: `upsert(driver, lon, lat)` computes the driver's cell, removes the id from the old cell's list if it changed, and appends to the new one — O(1), no tree surgery, no locks beyond a shard-local latch. A driver who stops updating for 15 s (the freshness TTL) is evicted

by a sweeper and treated as offline. Freshness is part of the index's contract, not a separate monitoring system: **a stale position is a wrong position.**

12.8 Deep Dive: The Matching Engine

Given candidates, who gets which rider? Two designs, one evolution.

Naive dispatch — offer each request to the single nearest available driver, then the next on decline — is where every prototype starts and where every prototype's metrics die: it serializes decisions (each offer burns a 15 s accept window), and greedy nearest-first assignment is provably suboptimal in aggregate (the Rust listing includes a case where it wastes minutes of pickup time).

Batched matching windows — the production answer: accumulate ride requests and idle drivers for a short window (2 s), then solve the whole window as one assignment problem: riders on one side, drivers on the other, edge costs = pickup ETA (plus small penalties for driver rating mismatches, vehicle-type mismatch, long deadhead). This is a **bipartite matching** instance; the optimal solution is the Hungarian algorithm's min-cost assignment, $O(n^3)$ on a window of thousands — and a well-ordered greedy pass (cheapest edge first, skip taken endpoints) lands within 10–20% of optimal at O(E log E), which is the usual production compromise at window scale.

> **DEFINITION — Bipartite matching / assignment problem**
>
> A graph whose vertices split into two sets (riders, drivers) with edges only across (a possible pairing, weighted by pickup cost). A *matching* is a set of edges sharing no endpoints: each rider, at most one driver. The *assignment problem* asks for the matching of minimum total cost — solved exactly by the Hungarian algorithm and approximately, at a tenth of the compute, by sorted greedy. Batching requests into windows is what turns a stream of myopic one-at-a-time decisions into a global optimization — the same trick as Chapter 7's re-ranking stage, applied to cars instead of videos.

The offer protocol rides on Chapter 11's machinery: an offer is a short-lived exclusive lock on a driver — `offer_token` with a 15 s TTL, one outstanding offer per driver, accept = compare-and-set on the token. Decline, timeout, or app death releases the driver into the next window; the rider's request re-enters with its original timestamp so fairness is preserved across retries.

Idle supply positioning is the matcher's quiet second job: when a city cell has surplus idle drivers and a neighbor has a deficit, the system **nudges** drivers toward the deficit (map suggestions, small bonuses), shrinking future pickup ETAs before any rider appears. Matching reacts to the market; positioning **shapes** it.

12.9 Deep Dive: The Trip Backbone — State Machine and Money

The real-time side may be fuzzy; this side may not. Every trip is a row in the city-sharded trip store whose `state` column moves through one explicit machine:

`REQUESTED → MATCHED → ARRIVING → IN_PROGRESS → COMPLETED` (or `CANCELLED` from any pre-completion state).

Each transition is a compare-and-set on the row inside a Chapter 11 transaction, triggered by an idempotency-keyed API call. Three rules make the machine trustworthy:

1. **The table, not the app, owns legality.** `driver_arrived` before `match` is not "handled"; it is rejected by the transition function (the Rust listing makes illegal transitions unrepresentable as successes).
2. **Every transition is idempotent.** Network retries, app restarts, and double-taps replay the same key and return the same outcome — Chapter 10's dedup discipline applied to state changes.

3. **Completion triggers payment exactly once.** The `COMPLETED` transition emits one `charge` command with key `(trip_id, "charge")` into the payments pipeline; capture is retried until acked, and the dedup store guarantees the rider's card sees it once no matter how many times the pipeline replays it (Chapter 10's effectively-once, end to end).

Driver payout is deliberately **not** synchronous: the trip completes, the rider charge settles through the payments pipeline, and the driver's balance updates via a ledger event — seconds later is fine, because nobody's card statement depends on the driver's app refreshing in real time. Decoupling the two money flows keeps the trip state machine small and the payments pipeline independently auditable.

12.10 Deep Dive: Surge Pricing as a Control Loop

When requests outnumber idle drivers in a cell, the market needs a price signal — more drivers should come, price-sensitive riders should wait. The mechanism: per-cell **surge multiplier** = a smoothed, capped function of the demand/supply ratio, recomputed every 30 s.

The control-theory traps, named so the interviewer knows you know:

- **Oscillation.** Price rises → drivers flood in → price collapses → drivers leave → repeat. Damping: exponential smoothing of the ratio, and asymmetric responsiveness — quick up (riders are abandoning **now**), slow down (don't whipsaw drivers mid-drive).
- **The feedback loophole.** Drivers learn to wait out the first minute of surge for a higher multiplier; riders learn to walk one cell over. Mitigations: personalized-but-regulated caps, and never publishing the exact per-cell map as a gameable API.
- **Ethics and law.** Emergency contexts cap or disable surge; regulators in several jurisdictions constrain the multiplier's ceiling and its display. A pricing system is a policy system with a math core.

> **COMMON PITFALL — Surge measured at the wrong granularity**
>
> Compute the ratio city-wide and a stadium emptying at midnight surges the whole city; compute it per-100 m-cell and a single rider plus-or-minus one driver oscillates the price every thirty seconds. The cell size must hold **enough actors for statistics** — tens of riders and drivers per cell per window — which is why production surge uses multi-resolution cells (H3's hierarchy) and falls back up the hierarchy when a fine cell is too quiet to measure.

12.11 API Design

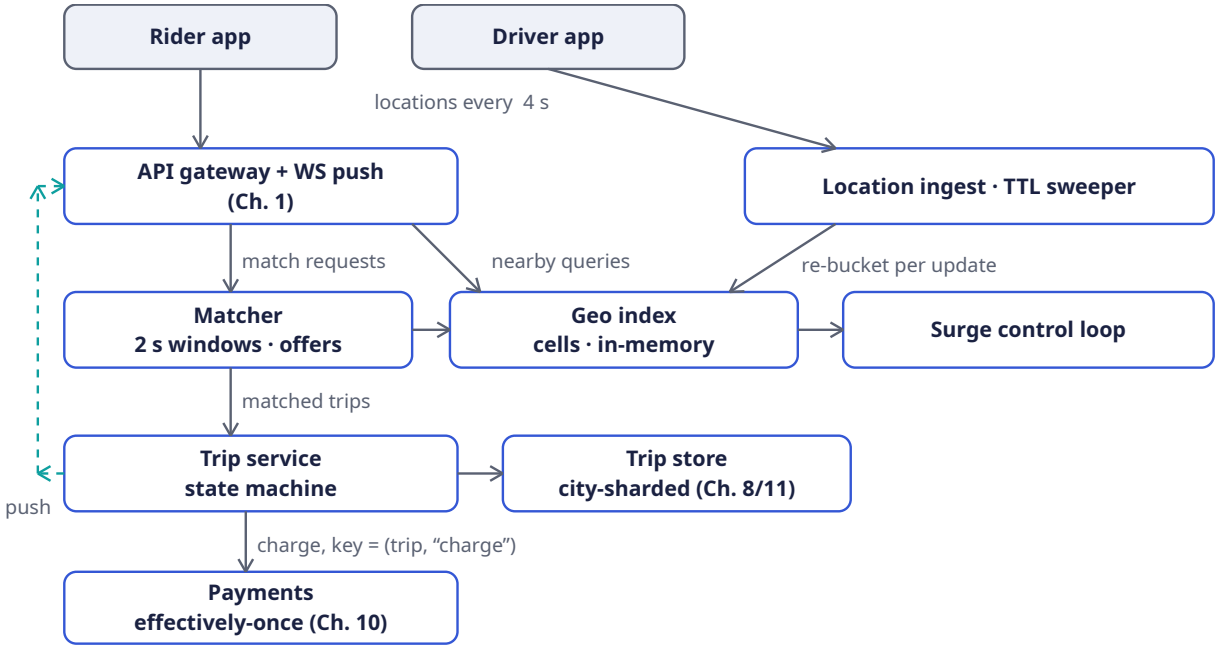
The API is deliberately small; every mutating call carries an idempotency key, and everything time-sensitive after the match rides the push channel rather than polling.

Verb & path	Purpose	Key fields	Guarantee
`POST /v1/location`	Driver heartbeat into the geo index	`driver_id, lon, lat, heading, seq`	`202`; stale `seq` dropped
`POST /v1/rides/quote`	Fare + pickup-ETA estimate before commitment	`pickup, dropoff` → `quote_id, price_band, eta, surge`	Read-only; quote TTL 60 s
`POST /v1/rides`	Create trip (`REQUESTED`) and enter matching	`Idempotency-Key, quote_id, rider_id`	Key-safe retry → same `trip_id`

Verb & path	Purpose	Key fields	Guarantee
`POST /v1/offers/{id}/accept`	Driver claims an offer within its 15 s TTL	`offer_id, driver_id, offer_token`	Compare-and-set; one winner
`POST /v1/trips/{id}/events`	Advance the state machine	`Idempotency-Key, event: arrived|start|finish|cancel`	Illegal transition → `409`
`GET /v1/rides/{id}`	Snapshot for reconnects and support tools	`trip_id` → full state + last known positions	Strongly consistent (city-primary read)
`WS /v1/stream`	Push: match found, driver en route, receipt	per-user channel, resume token	At-least-once + client dedup (Ch. 1)

Two design notes. First, the **quote before request** split matters: the quote is a cheap, cacheable, surge-aware estimate; the request is the committing write. Riders who balk at the price cost us one read, not one write-cancel-refund cycle. Second, `POST /v1/trips/{id}/events` is the only door into the state machine — apps never write `state` directly, so the transition table (Section 12.13) is enforced in exactly one place.

12.12 High-Level Design



Reading the diagram left to right, the system has three temperaments. The **top two rows are soft state**: heartbeats flow driver app → ingest → geo index, where a missed update simply expires at the TTL sweeper; the gateway asks "who is nearby" and forwards requests into the matcher. Nothing here is stored for long and nothing here may block on a disk. The **bottom half is hard state**: the matcher hands a mutually accepted pairing to the trip service, which walks the state machine against the city-sharded store, and exactly one charge command falls out the bottom into payments. The **dashed teal bus** is the return leg — every accepted transition is published to the gateway, which pushes it to both

apps over the long-lived connection of Chapter 1. The surge loop stands slightly apart: it reads the geo index's per-cell supply counts and the gateway's request rate, and answers quotes with multipliers — it never touches the trip backbone.

12.13 Rust Reference Implementations

Four small cores carry the chapter: the geohash codec that makes space prefix-searchable, the cell index that absorbs a quarter-million moves a second, the windowed matcher, and the state machine that guards the money. As in every chapter, the tests are the specification.

12.13.1 Geohash: space as a string

Bisect longitude and latitude alternately, five bits per character, and every coordinate becomes a string whose prefixes are its containing cells. `decode` returns the cell **center** — a geohash names a region, not a point.

```rust
const BASE32: &[u8; 32] = b"0123456789bcdefghjkmnpqrstuvwxyz";

pub fn encode(lon: f64, lat: f64, precision: usize) -> String {
    let (mut lon_lo, mut lon_hi) = (-180.0_f64, 180.0_f64);
    let (mut lat_lo, mut lat_hi) = (-90.0_f64, 90.0_f64);
    let mut out = String::with_capacity(precision);
    let mut even = true; // even bits refine longitude, odd bits latitude
    for _ in 0..precision {
        let mut ch: u8 = 0;
        for _ in 0..5 {
            ch <<= 1;
            if even {
                let mid = (lon_lo + lon_hi) / 2.0;
                if lon >= mid { ch |= 1; lon_lo = mid; } else { lon_hi = mid; }
            } else {
                let mid = (lat_lo + lat_hi) / 2.0;
                if lat >= mid { ch |= 1; lat_lo = mid; } else { lat_hi = mid; }
            }
            even = !even;
        }
        out.push(BASE32[ch as usize] as char);
    }
    out
}

pub fn decode(cell: &str) -> (f64, f64) {
    let (mut lon_lo, mut lon_hi) = (-180.0_f64, 180.0_f64);
    let (mut lat_lo, mut lat_hi) = (-90.0_f64, 90.0_f64);
    let mut even = true;
    for &b in cell.as_bytes() {
        let mut v = BASE32.iter().position(|&c| c == b).unwrap() as u8;
        for _ in 0..5 {
            let bit = v >> 4;              // consume bits, MSB first
            v = (v << 1) & 0b11111;
            if even {
                let mid = (lon_lo + lon_hi) / 2.0;
                if bit == 1 { lon_lo = mid; } else { lon_hi = mid; }
            } else {
                let mid = (lat_lo + lat_hi) / 2.0;
                if bit == 1 { lat_lo = mid; } else { lat_hi = mid; }
            }
            even = !even;
        }
    }
```

```
    ((lon_lo + lon_hi) / 2.0, (lat_lo + lat_hi) / 2.0) // cell center
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn roundtrip_lands_inside_the_cell() {
        let (lon, lat) = (-73.9857, 40.7484); // Manhattan
        let cell = encode(lon, lat, 7);       // ~150 m cells
        let (clon, clat) = decode(&cell);
        assert!((clon - lon).abs() < 0.01);
        assert!((clat - lat).abs() < 0.01);
    }

    #[test]
    fn prefixes_are_containment() {
        let (lon, lat) = (-73.9857, 40.7484);
        let fine   = encode(lon, lat, 8);
        let coarse = encode(lon, lat, 5);
        assert!(fine.starts_with(&coarse)); // "zoom out" = drop a char
    }
}
```

12.13.2 The geo index: built for churn

One hash map from cell to drivers, one from driver to cell, and a move becomes two O(1) updates. `nearby` widens the prefix one character at a time until enough candidates surface — recall from cells, precision from the distance sort that would follow in the routing engine.

```
use std::collections::{HashMap, HashSet};

pub struct GeoIndex {
    cells: HashMap<String, Vec<u64>>, // cell → drivers inside it
    where_: HashMap<u64, String>,     // driver → its current cell
    precision: usize,
}

impl GeoIndex {
    pub fn new(precision: usize) -> Self {
        GeoIndex { cells: HashMap::new(), where_: HashMap::new(), precision }
    }

    pub fn upsert(&mut self, driver: u64, lon: f64, lat: f64) {
        let cell = encode(lon, lat, self.precision);
        if self.where_.get(&driver) == Some(&cell) { return; } // no cell change
        if let Some(old) = self.where_.insert(driver, cell.clone()) {
            if let Some(list) = self.cells.get_mut(&old) {
                list.retain(|&d| d != driver);
            }
        }
        self.cells.entry(cell).or_default().push(driver);
    }

    pub fn remove(&mut self, driver: u64) { // offline, or TTL swept
        if let Some(old) = self.where_.remove(&driver) {
            if let Some(list) = self.cells.get_mut(&old) {
                list.retain(|&d| d != driver);
            }
```

```
        }
    }

    /// Ring expansion: exact cell, then parent prefix, then grandparent…
    pub fn nearby(&self, lon: f64, lat: f64, limit: usize) -> Vec<u64> {
        let mut found = Vec::new();
        let mut seen = HashSet::new();
        let mut prefix = encode(lon, lat, self.precision);
        while !prefix.is_empty() && found.len() < limit {
            for (cell, list) in &self.cells {
                if cell.starts_with(&prefix) {
                    for &d in list {
                        if seen.insert(d) { found.push(d); }
                    }
                }
            }
            prefix.pop(); // zoom out one ring
        }
        found.truncate(limit);
        found
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn nearby_finds_local_drivers_only() {
        let mut idx = GeoIndex::new(6);
        idx.upsert(1, -73.9857, 40.7484);   // beside the pickup
        idx.upsert(2, -73.9857, 40.7484);
        idx.upsert(3, -122.4194, 37.7749);  // San Francisco: never matches
        let mut got = idx.nearby(-73.9857, 40.7484, 10);
        got.sort_unstable();
        assert_eq!(got, vec![1, 2]);
    }

    #[test]
    fn moving_a_driver_rebuckets_them() {
        let mut idx = GeoIndex::new(6);
        idx.upsert(7, -73.9857, 40.7484);   // starts in Manhattan
        idx.upsert(7, -122.4194, 37.7749);  // teleports to SF
        assert!(idx.nearby(-73.9857, 40.7484, 10).is_empty());
        assert_eq!(idx.nearby(-122.4194, 37.7749, 10), vec![7]);
    }

    #[test]
    fn going_offline_removes_the_driver() {
        let mut idx = GeoIndex::new(6);
        idx.upsert(9, -73.9857, 40.7484);
        idx.remove(9);
        assert!(idx.nearby(-73.9857, 40.7484, 10).is_empty());
    }
}
```

The reference scans cell keys per ring — O(cells) per widening step, fine for one city shard; production H3 deployments precompute neighbor rings or keep per-resolution maps so widening touches only the ring's cells. The important invariant is unchanged either way: **no driver ever appears in two cells at once**, because `upsert` removes before it adds.

12.13.3 The matcher: a window, sorted edges, two sets

```rust
use std::collections::HashSet;

/// One batched window: pair riders with drivers, cheapest pickup first.
/// Positions are (id, lon, lat); squared distance is enough for ordering.
pub fn match_greedy(drivers: &[(u64, f64, f64)],
                    riders:  &[(u64, f64, f64)]) -> Vec<(u64, u64)> {
    let mut edges: Vec<(f64, u64, u64)> = Vec::new(); // (dist², driver, rider)
    for &(d, dlon, dlat) in drivers {
        for &(r, rlon, rlat) in riders {
            let (dx, dy) = (dlon - rlon, dlat - rlat);
            edges.push((dx * dx + dy * dy, d, r));
        }
    }
    edges.sort_by(|a, b| a.0.partial_cmp(&b.0).unwrap()
                          .then(a.1.cmp(&b.1))
                          .then(a.2.cmp(&b.2)));

    let mut used_drivers = HashSet::new();
    let mut used_riders = HashSet::new();
    let mut pairs = Vec::new();
    for &(_, d, r) in &edges {
        // Check BOTH endpoints before inserting either: inserting the
        // driver first would burn it on a rider who is already taken.
        if !used_drivers.contains(&d) && !used_riders.contains(&r) {
            used_drivers.insert(d);
            used_riders.insert(r);
            pairs.push((d, r));
        }
    }
    pairs
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn greedy_pairs_each_with_the_closest() {
        let drivers = vec![(1, 0.0, 0.0), (2, 1.0, 1.0)];
        let riders  = vec![(10, 0.05, 0.05), (20, 1.05, 1.05)];
        assert_eq!(match_greedy(&drivers, &riders), vec![(1, 10), (2, 20)]);
    }

    #[test]
    fn batching_beats_first_come_first_served() {
        // Serial dispatch serves rider 10 first: its nearest is driver 2
        // (24.01 < 26.01), leaving rider 20 to driver 1 — total 122.02.
        // The window sees both sides and pays 26.02 instead.
        let drivers = vec![(1, 0.0, 0.0), (2, 10.0, 0.0)];
        let riders  = vec![(10, 5.1, 0.0), (20, 9.9, 0.0)];
        assert_eq!(match_greedy(&drivers, &riders), vec![(2, 20), (1, 10)]);
    }

    #[test]
    fn imbalanced_market_leaves_a_rider_for_the_next_window() {
        let drivers = vec![(1, 0.0, 0.0)];
        let riders  = vec![(10, 0.1, 0.1), (20, 0.2, 0.2)];
        assert_eq!(match_greedy(&drivers, &riders), vec![(1, 10)]);
```

```
    }
}
```

12.13.4 The trip state machine: legality lives here

```
#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub enum State { Requested, Matched, Arriving, InProgress, Completed, Cancelled }

#[derive(Clone, Copy, PartialEq, Eq, Debug)]
pub enum Event { Match, DriverArrived, StartTrip, Finish, Cancel }

/// The only legal moves. `None` = reject (HTTP 409); the app never
/// writes state directly, so this function IS the trip's law.
pub fn next(state: State, event: Event) -> Option<State> {
    use Event::*;
    use State::*;
    match (state, event) {
        (Requested, Match)       => Some(Matched),
        (Matched, DriverArrived) => Some(Arriving),
        (Arriving, StartTrip)    => Some(InProgress),
        (InProgress, Finish)     => Some(Completed),
        // Cancellation is legal until completion; mid-trip cancels
        // still bill for distance covered.
        (Requested, Cancel) | (Matched, Cancel)
        | (Arriving, Cancel) | (InProgress, Cancel) => Some(Cancelled),
        _ => None, // terminal states, skipped steps, replayed events
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn happy_path_reaches_completed() {
        let mut s = State::Requested;
        for e in [Event::Match, Event::DriverArrived, Event::StartTrip, Event::Finish] {
            s = next(s, e).expect("legal transition");
        }
        assert_eq!(s, State::Completed);
    }

    #[test]
    fn illegal_transitions_are_rejected_not_handled() {
        assert_eq!(next(State::Requested, Event::Finish), None);    // no driver yet
        assert_eq!(next(State::Requested, Event::StartTrip), None); // skipped steps
        assert_eq!(next(State::Completed, Event::Cancel), None);    // terminal
        assert_eq!(next(State::Cancelled, Event::Match), None);     // terminal
    }

    #[test]
    fn cancellation_closes_at_completion() {
        assert_eq!(next(State::Matched, Event::Cancel), Some(State::Cancelled));
        assert_eq!(next(State::InProgress, Event::Cancel), Some(State::Cancelled));
        assert_eq!(next(State::Completed, Event::Cancel), None);
    }
}
```

Paired with an idempotency-keyed event API and a compare-and-set on the trip row, this function gives the marketplace its spine: replays return the stored outcome, races have one winner, and every charge the payments pipeline ever sees traces back to a single, legal `Finish`.

12.14 Scaling

The city shard is the unit of everything. Each city runs its own geo index, matcher, trip store, and surge loop; adding capacity means splitting a hot city into two stacks along a geographic seam and re-homing the cells — a config change, not a migration of state machines, because trips never straddle the seam (a cross-border pickup is handed off once, at request time, to the city that owns the pickup cell). This is Chapter 9's lesson applied deliberately: the coordination boundary is drawn where the physics already is.

Location ingest scales horizontally by driver-id hash; any ingest node can re-bucket any driver because the geo index is the system of record for "where," not the nodes in front of it. **Matching** scales by cell partition: disjoint pickup regions can run windows in parallel, with a thin arbitration layer for boundary cells. **The trip store** is ordinary Chapter 8 sharding — by `trip_id`, with a rider-id secondary index for history pages — and is deliberately the least exotic component in the building.

The one piece that resists scaling-by-addition is the **surge loop's global view** of a city: per-cell counts aggregate upward through a hierarchy (Chapter 6's top-k trick in geographic clothing), so even this reduces to a streaming aggregation problem we have solved before.

12.15 Failure Modes

Failure	Symptom	Response
Driver app dies mid-offer	Offer never answered	Offer TTL (15 s) expires; driver released to next window; rider re-entered with original timestamp
Driver stops sending location	Position goes stale in the index	TTL sweeper evicts after 15 s; driver marked offline; in-flight trips continue on rider-reported position
Matcher node crashes	One window's pairings lost	Requests are durable in the gateway queue; next window re-matches them — matching state is disposable **by design**
Trip store primary fails	Transitions stall in one shard	Failover to the shard replica (Ch. 11 durability); events queue on the API with their idempotency keys and replay after recovery
Payment capture times out	`COMPLETED` trip, no settled charge	The charge command retries with key `(trip_id, "charge")` until acked — effectively-once, never twice (Ch. 10)
City network partition	Riders and drivers split across two halves	Both halves keep working with degraded pools (availability asymmetry); the trip store accepts no writes it cannot confirm — no phantom trips, some unmatched riders

Failure	Symptom	Response
Surge loop goes haywire	Multiplier oscillates or spikes	Hard cap + floor, smoothing window, and a kill switch per city — pricing is a control loop and gets a circuit breaker

KEY INSIGHT — The asymmetry is the design

Notice what each row protects. The real-time side (offers, positions, windows) responds to failure by **discarding and retrying** — stale soft state is worthless anyway. The backbone (trips, charges) responds by **persisting and replaying** — a lost hard state is a lawsuit. A system that knows which half it is in at every line of code is a system that can be operated at 3 a.m.

12.16 Trade-offs

Decision	Chosen	Declined, and why
Geo structure	Flat cells, in-memory, O(1) rebucket	R-tree: exact but mutation-hostile at 2.5×10^5 moves/s; recall 1. post-filter beats exact indexing here
Dispatch	2 s batched windows, greedy assignment	Per-request nearest: serializes on offer TTLs and provably wastes aggregate pickup time; Hungarian optimum: 10× the compute for a sliver of quality at window scale
Consistency of trips	Strong, single city-primary, CAS transitions	Multi-leader across regions: trips are local physics; buying cross-region consensus (Ch. 11's price) for a ride across town is a bad trade
Location durability	Soft state with TTL; losses self-heal in 4 s	Persisting every heartbeat: 25 MB/s of WAL for data that is stale in 15 s — the textbook case **against** durability
Surge authority	Per-cell loop, capped, city kill switch	Global optimization: prettier equilibrium, ungovernable blast radius; regulators and drivers both prefer legible cells
Shared rides (pooling)	Out of scope v1	Matching becomes set-packing, ETAs become promises about other people's behavior — a whole second interview

12.17 Observability and SLOs

The four numbers on the on-call dashboard, and what each one **means**:

- **Match p99 latency ≤ 5 s** (request → `MATCHED`). The marketplace's heartbeat. Degradation means windows starved of supply — alert on per-cell supply/demand ratio, not on the latency itself, which is the symptom.
- **Location freshness: ≥ 99% of online drivers with a fix younger than 15 s.** Below this, the index is lying and every downstream decision inherits the lie.
- **Transition success ≥ 99.99%, double-charge count = 0.** The backbone SLOs. The second is not a percentage: it is a count with an alarm at one, fed by reconciling the payments ledger against `COMPLETED` trips nightly (Chapter 10's reconciliation pattern).
- **Push delivery p99 ≤ 2 s** from committed transition to app notification, measured with synthetic trips per city (Chapter 4's golden signals end-to-end).

Log every offer, every transition, every charge decision with `trip_id` as the join key — the trip's story must be replayable from logs alone, because when a rider and a driver disagree about what happened at 2 a.m., the log is the only witness.

12.18 Interview Wrap-Up

What the interviewer is listening for, roughly in order:

1. **Did you notice the two-systems problem?** Soft real-time state versus hard transactional state, with different durability, consistency, and failure semantics. Candidates who treat driver locations like rows in Postgres have announced their level.
2. **Is your geo story churn-first?** Cells and prefixes, not trees; recall plus post-filter, not exactness; TTL as a correctness mechanism.
3. **Does matching have a time axis?** Batched windows beat serial dispatch; offers are expiring locks; declined and timed-out offers must recycle cleanly. If your matcher never waits and never batches, it is leaving minutes of pickup time on the table.
4. **Is the money boring?** State machine, idempotency keys, one charge per completed trip, reconciliation. The highest compliment a payments design can receive is that there is nothing clever about it.
5. **Do you know surge is a control loop?** Smoothing, asymmetric responsiveness, caps, kill switch — and the humility to say "tuned in production" rather than deriving a multiplier on the whiteboard.

> **INTERVIEW TIP — The sixty-second answer**
>
> "City-sharded marketplace. Drivers heartbeat into an in-memory cell index; riders request; a matcher batches two-second windows and solves near-optimal assignment with expiring offers; an accepted match enters a strongly consistent trip state machine that ends in exactly one idempotent charge; a per-cell surge loop prices scarcity. Everything real-time is soft state with TTLs; everything financial is a logged, replayable, transactional record." That paragraph is the whole chapter — the rest is proving you can build it.

12.19 Summary and Further Reading

Ride-hailing compresses the whole playbook into one product: Chapter 1's push channel carries the match, Chapter 2's geography becomes a churn- tolerant cell index, Chapter 8's pages hold the trips, Chapter 9's physics dictates the city shard, Chapter 10's scheduler times out the offers, and Chapter 11's transactions guard the charge. The new muscle is **economic**: matching as an online optimization under expiry, and pricing as a feedback control loop. System design at this level stops being about components and starts being about **which guarantees each part can afford**.

Further reading: Uber Engineering's posts on H3 (hexagonal indexing) and on their marketplace matching and surge stack; the S2 geometry library docs for the spherical-geometry view; Kleinberg & Tardos, **Algorithm Design**, for bipartite matching and online algorithms; and any control- theory introduction — PID controllers in particular — before touching a production pricing loop.

12.20 Chapter Glossary

Term	Meaning in this chapter
Geohash	Interleaved lon/lat bits as a base-32 string; prefixes are containing cells, so spatial search becomes `starts_with`
Cell / H3 / S2	A fixed tile of the map used as an index bucket; hexagonal (H3) and spherical-cap (S2) successors to geohash's rectangles
Ring expansion	Widening a nearby-search by dropping prefix characters until enough candidates are found
TTL sweeper	The process that evicts drivers whose locations have gone stale; freshness as an index invariant
Matching window	The 2 s batching interval that turns serial dispatch into a joint assignment problem
Bipartite matching	Pairing two disjoint sets (riders, drivers) with no shared endpoints; minimum-cost version solved by Hungarian, approximated by sorted greedy
Offer token	A 15 s exclusive lock on a driver; accept is a compare-and-set, timeout recycles the driver
Deadheading	Driving without a passenger; the cost matching and positioning exist to minimize
Utilization	Fraction of driver-online time spent earning; the marketplace's supply-side health metric
Trip state machine	`REQUESTED → MATCHED → ARRIVING → IN_PROGRESS → COMPLETED` / `CANCELLED`; the only legal moves, enforced server-side
Terminal state	`COMPLETED` or `CANCELLED`; accepts no further events, which makes replays and late packets harmless
Idempotency key	Client-chosen request identity; retries return the stored outcome — the API-level twin of Chapter 10's dedup
Effectively-once	At-least-once delivery plus key-based dedup; how one `Finish` becomes one charge
Surge multiplier	Per-cell price factor from smoothed demand/supply; capped, damped, kill-switchable
Control loop	Measure → compare → actuate, repeated; surge priced this way oscillates unless damped
Feedback loophole	Actors gaming the loop (drivers waiting out surge); mitigated by caps and opacity of the per-cell map
City shard	The unit of scale and failure: one city's full stack, strong consistency inside, none needed across

Term	Meaning in this chapter
Soft state / hard state	Disposable, TTL-governed data (positions) versus durable, transactional data (trips, charges) — the chapter's organizing split
Compare-and-set (CAS)	Atomic "update only if unchanged"; how offers accept and states transition with exactly one winner
Reconciliation	Nightly ledger-versus-trips audit; the alarm that makes "double charges = 0" a fact instead of a hope

— End of Chapter 12 · Next: Chapter 13 —